

Szoftvertchnológia és -technikák

1. Előadás

Bevezetés, OOP ismételés, SOLID elvek



Automatizálási és
Alkalmazott
Informatikai Tanszék

Szoftvertechnológia és technikák

- Előadás
 - > Charaf Hassan,
Mezei Gergely, Benedek Zoltán, Albert István
 - > Kérdezni ér!
 - > Email: “[SzTT]”
- Gyakorlat
 - > Erősen ajánlott
 - > 70% részvétel kötelező (4 hiányzás megengedett)
 - > Laboronként +1 pont szerezhető a félév végi jegyhez

Számonkérések

- Házi feladat
 - > Első házi: tervezési (modellezési) feladat
 - kiadás: ~ 6. hét ; beadás: 10.hét
 - > Második házi: tervezési mintás, programozós feladat
 - kiadás: ~ 11. hét ; beadás 13-14. hét/póthét
 - > Kötelező!
- ZH
 - > Gyakorlati jellegű
 - > Minta ZH lesz idén is
- Vizsga
 - > Beugró (50% kell minimum), a vizsga jellege kicsit változik
 - > Minta vizsga lesz idén is
- Végső jegy: Vizsga 90p + Házi $2 \cdot 10p$ + ZH 40p = 150p (Változott!)
 - > + Gyakorlatokért max 12p!

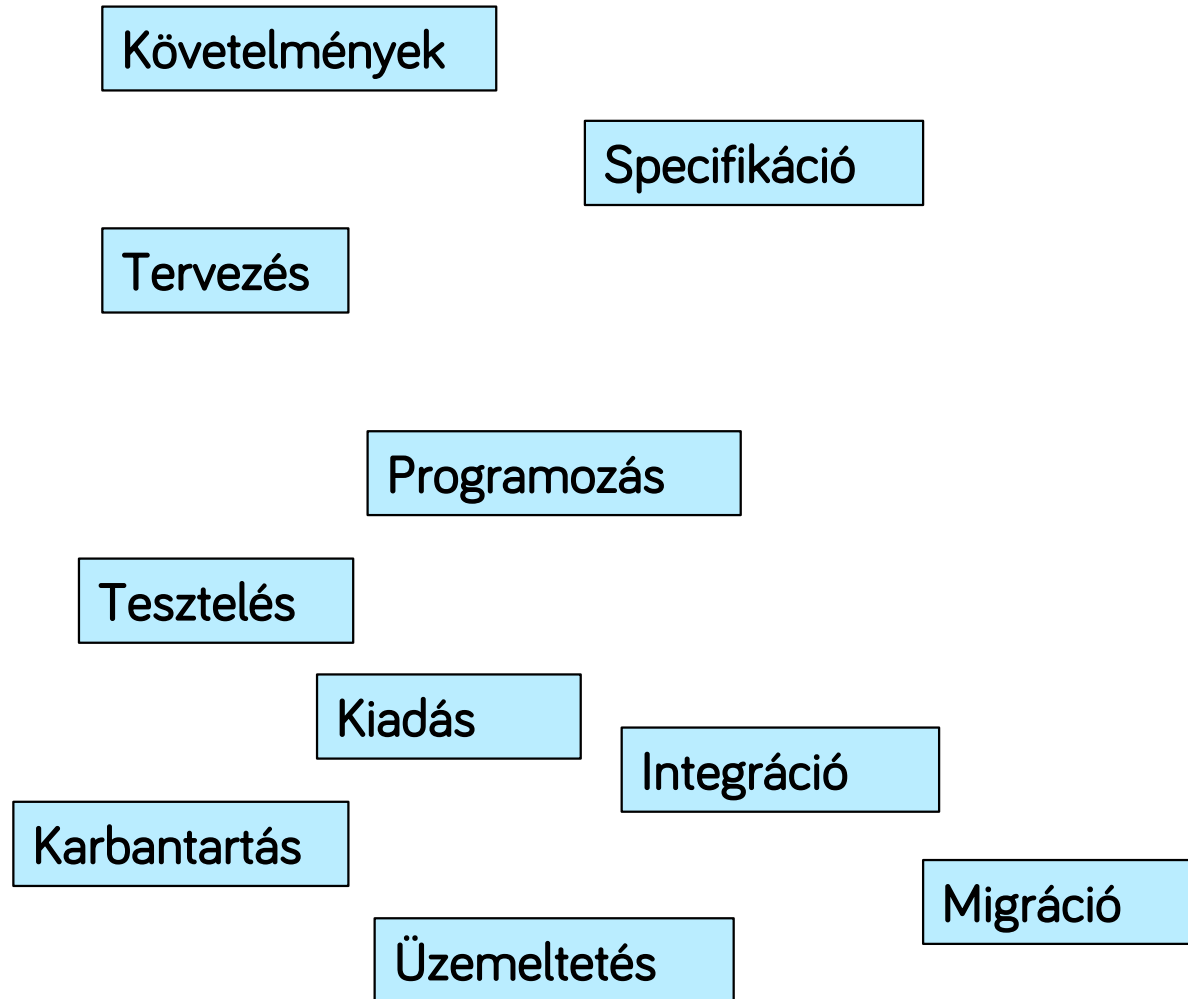
Miről szól a tárgy?

- Eddig: megtanultunk programozni
 - > 1. félév: A programozás alapjai
 - > 2. félév: Objektumorientált programozás
- Hogyan lesz ebből működő szoftver termék?
- A szoftvertervezés programozás nyelvtől független!
 - > Félév eleje: Java
 - > ~Félév közepétől: C#
 - Gyorstalpaló: <https://www.aut.bme.hu/Course/SzTT> 

A szoftverfejlesztés

Programozás

A szoftverfejlesztés



A tematika

- Bevezés és alapelvek
- Tervezés, UML
- Tervezési minták
- Tervezési módszertanok
- Tesztelés, üzemeltetés

OOP ismétlés

Clean code

- Olyan fókusszal vizsgáljuk, hogy átlátható, jól kezelhető kódot kapjunk
 - > Bővíthetőség, karbantartás
 - > Csapatmunka
 - > Félévre tesszük el, aztán vegyük elő...
 - > Uncle Bob: <https://cleancoders.com> 

Refactoring

- Olyan átalakítás, amelyekkel a programkód működése nem változik
- Ha nem tiszta a kód, akkor alkalmazzuk, IDE segít benne, hogy ne rontsuk el
- Bármilyen, ami a fentieknek megfelel, de vannak tipikusak:
 - > Átnevezés
 - > Kód kiszervezése új metódusba $\leftrightarrow$ inline
 - > Osztályhierarchián felfelé/lefelé mozgatás
 - > ...

Miért jó OO módon fejleszteni?

1. Közelebb áll az emberi gondolkodásmódhoz
 - > A szoftver való életbeli problémát old meg
 - > Domén: a szakterület, amelyhez a szoftver kapcsolódik → Domain class, entity class
 - > Dolgok, fogalmak kerülnek elő, ezeknek tulajdonságaik vannak, és műveletet végeznek, vagy mi végzünk rajtuk valamilyen műveletet → objektum, tagváltozó, tagfüggvény
 - > Főneveket és igék a követelményleírásban!

Domain class példa #1

```
public class Person {  
    private String name;  
  
    private LocalDate birthDate;  
  
    public Person() {}  
  
    public Person(String name, LocalDate birthDate) {  
        this.name = name;  
        this.birthDate = birthDate;  
    }  
  
    // ...  
  
}
```


Domain class példa #2

```
public LocalDate getBirthDate() {  
    return birthDate;  
}  
  
public void setBirthDate(LocalDate birthDate) {  
    this.birthDate = birthDate;  
}  
  
public long getAge() {  
    return Duration.between(birthDate,  
        LocalDate.now()).get(ChronoUnit.YEARS);  
}
```

Miért jó OO módon fejleszteni?

2. Jobban strukturálhatjuk vele a programunkat
 - > Nem tanultatok nem-OO nyelveket, ezért erre „igazi” példát nem tudunk mutatni
 - > De OO nélkül tapasztalhatjuk, hogy bizonyos műveletek megadott adatokhoz tartoznak
 - > Mindig paraméterként át kell adni az adatokat, és ez kényelmetlen, nem alkotnak egységet

Egységbe zárás példa

```
def fill_deck(deck):  
    for color in ["♣", "♦", "♥", "♠"]:  
        for shape in [2, 3, 4, 5, 6, 7, 8, 9,  
                       "T", "J", "Q", "K", "A"]:  
            deck.append((color, shape))  
    random.shuffle(deck)
```

```
def draw_from_deck(deck):  
    return deck.pop()
```

```
my_deck = []  
fill_deck(my_deck)  
card = draw_from_deck(my_deck)
```

Egységbe zárás példa

```
class Deck:
    def __init__(self):
        self.deck = []
        for color in ["♣", "♦", "♥", "♠"]:
            for shape in [2, 3, 4, 5, 6, 7,
                          8, 9, "T", "J",
                          "Q", "K", "A"]:
                self.deck.append((color, shape))
        random.shuffle(self.deck)

    def draw(self):
        return self.deck.pop()
```

```
deck = Deck()
card = deck.draw()
```

OO elvek

- Egységbe zárás: ld. előző példa, adatok és műveletek egy egységet alkotnak
- Adatrejtés: adatokat csak metódusokkal érjük el
- Öröklés
- Polimorfizmus

Adatrejtés példa

```
public class Product1 {  
    public int netPrice;  
    public int vatPercent;  
    public int grossPrice;  
  
    public static void main(String[] args) {  
        Product1 product = new Product1();  
        product.netPrice = 1000;  
        product.vatPercent = 25;  
        product.grossPrice = 1500; // 1250?  
        product.netPrice = -1000; // ??  
    }  
}
```

Adatrejtés példa

```
private int netPrice;  
private int vatPercent;  
private int grossPrice;
```

```
public int getNetPrice() {  
    return netPrice;  
}
```

```
public void setNetPrice(int netPrice) {  
    if (netPrice <= 0)  
        throw new IllegalArgumentException(  
            "Net price must be positive");  
    this.netPrice = netPrice;  
    this.grossPrice = (int) (this.netPrice *  
        ((100 + this.vatPercent) / 100.0));  
}
```

Adatrejtés példa

```
public int getGrossPrice() {  
    return (int) (this.netPrice *  
                  ((100 + this.vatPercent) / 100.0));  
}
```


Öröklés

- Szintén jól illeszkedik a hétköznapi gondolkodáshoz
- Modellezhetünk általános kategóriaként több dolgot, és az általános működés öröklődik
- Az öröklött működés kiegészíthető, felülírható

Öröklés példa

```
public class Employee extends Person {  
    private String officeNumber;  
  
    public Employee() {  
    }  
  
    public Employee(String name, LocalDate birthDate,  
                    String officeNumber) {  
        super(name, birthDate);  
        this.officeNumber = officeNumber;  
    }  
  
    public String getOfficeNumber() {  
        return officeNumber;  
    }  
  
    public void setOfficeNumber(String officeNumber) {  
        this.officeNumber = officeNumber;  
    }  
}
```

Polimorfizmus

- Az öröklés IS-A („AZ EGY”) kapcsolatot jelöl
 - > Pl. a bogár az egy rovar
 - > Tehát minden jellemzővel bír, amivel a rovarok, de esetleg másképp viselkedik
 - > Ahol egy rovar típusú objektumra van szükségünk, ott bogárt is adhatunk
 - > Deklarált (statikus) és futásidejű (dinamikus) típus
- Dinamikus kötés: nem a deklarált, hanem a futásidejű típus alapján dől el, hogyan viselkedik az objektum

ArrayList vs. LinkedList

- A polimorfizmus segítségével deklarálhathatunk List őstípust, és a felhasználás jellegétől függően más leszármazottat példányosítunk
- Mindig a választott futásidejű típusnak megfelelő algoritmus fut
- ArrayList: tömböt használ, indexelés, végére fűzés hatékonyabb, de beszúrás, törlés költségesebb
- LinkedList: láncolt listát használ, beszúrás, törlés könnyű, indexelés nehéz

ArrayList vs. LinkedList

```
start = Instant.now();  
for (int i = 0; i < 10_000; i++) {  
    nums.add(0, rnd.nextInt()); // LinkedList jobb  
    // nums.add(rnd.nextInt()); // ArrayList jobb  
}  
end = Instant.now();  
System.out.println("Filled list in "  
    + Duration.between(start, end));
```

Az öröklés további lehetőségei

- Lehet több őszosztály? ➔ Javában nem!
 - > Túl bonyolult, problémákat vet fel
 - > ld. Diamond problem
- Viszont néha praktikus lenne, ha egy osztály több különböző típusnak is meg tudna felelni.
- Ezért vezették be az interfészeket
 - > Csak előír metódust, nem implementál

Interfész példa

```
public class Employee extends Person
    implements Comparable<Employee> {

    // ...

    @Override
    public int compareTo(Employee o) {
        return this.getName().compareTo(o.getName());
    }
}
```

Az absztrakt osztályok

- Az absztrakt osztályok átmenetet jelentenek interfész és osztály közt
- Némely metódust csak előírnak, némelyhez adhatnak implementációt
- Közvetlen nem példányosíthatóak, csak olyan leszármazottjuk, amely minden előírt metódust megvalósít
- Sokszor adott a logika egy része, más pedig majd a leszármazottaktól függ

Absztrakt osztály példa

```
public class PersonTableModel extends AbstractTableModel {  
    private List<Person> personList;  
  
    public PersonTableModel(List<Person> personList) {  
        this.personList = personList;  
    }  
  
    @Override  
    public int getRowCount() {  
        return personList.size();  
    }  
  
    @Override  
    public int getColumnCount() {  
        return 2;  
    }  
  
    @Override  
    public Object getValueAt(int rowIndex, int columnIndex) {  
        Person person = personList.get(rowIndex);  
        return columnIndex ==  
            0 ? person.getName() : person.getBirthDate();  
    }  
}
```

Hierarchia tervezése

- Legyen interfész ➔ minden testre szabható
- Legyen absztrakt osztály ➔ amihez lehet, adhatunk alapértelmezett implementációt
- El lehet dönteni, melyikből öröklünk
- Pl.
 - > List interfész: teljesen általános saját lista
 - > AbstractList: pl. addAll() meg van írva az add()-re visszavezetve
- Pl.
 - > WindowListener interfész sok eseményhez
 - > WindowAdapter üres implementációkkal

Ha néhány dolog...

- ...csak paraméterekben tér el ➔ egyazon osztály példányai
- ...metódusokban is eltér, de sok a közös ➔ saját leszármazott osztályok

Az öröklés, mint kétélű fegyver

```
public class InstrumentedHashSet<E> extends HashSet<E> {  
    private int addCount = 0;  
  
    @Override  
    public boolean add(E e) {  
        addCount++;  
        return super.add(e);  
    }  
  
    @Override  
    public boolean addAll(Collection<? extends E> c) {  
        addCount += c.size();  
        return super.addAll(c);  
    }  
  
    public int getAddCount() {  
        return addCount;  
    }  
}
```

Az öröklés, mint kétélű fegyver

```
public class Prism extends Rectangle {  
    private int height;  
  
    public Prism() {}  
  
    public Prism(int a, int b, int height) {  
        super(a, b);  
        this.height = height;  
    }  
  
    public int getHeight() {  
        return height;  
    }  
  
    public void setHeight(int height) {  
        this.height = height;  
    }  
}
```

Öröklés vs. delegálás

- A fenti esetekben a delegációt kellene inkább használni
- Ez azt jelenti, hogy tagváltozóként tárolunk egy másik objektumot, és neki delegálunk hívásokat
- Szokás HAS-A relációként is hívni.

Delegálás

```
public class Prism {  
    private Rectangle base;  
    private int height;  
  
    public Prism(int a, int b, int height) {  
        this.base = new Rectangle(a, b);  
        this.height = height;  
    }  
  
    public int getA() {  
        return base.getA();  
    }  
  
    public void setA(int a) {  
        base.setA(a);  
    }  
  
    // ...  
}
```

A SOLID-elvek

Minták és „best practice”-ek

- Jót lopni itt nem bűn
- Később részletesebben lesznek a tárgyban

Architectural patterns

Design patterns

Idioms

A SOLID-elvek

- Single Responsibility Principle
- Open/Closed Principle
- Liskov Substitution Principle
- Interface Segregation Principle
- Dependency Inversion Principle

Single Responsibility Principle

- Egy osztálynak egy felelőssége legyen (egy ok miatt változhasson)
- Példa: riport összeállítása és nyomtatása
 - > Rokon dolgok, de mégis különböző
 - > Ok a változásra 1.: más formátum szükséges
 - > Ok a változásra 2.: másképp kell nyomtatni
- Előnyök:
 - > Robosztusság: egyik változtatással nem rontjuk el a másik komponenst
 - > Átláthatóság
 - > Jobb tesztelhetőség

Kohézió/csatolás

- Cohesion (kohézió): egy komponensbe tartozó logikák mennyire szorosan függenek össze ➔ erős/gyenge kohézió
- Coupling (csatolás): két komponensnek mennyire kell ismernie egymás belső működését ➔ szoros/laza csatolás
- Erős kohézió, laza csatolás az előnyös
 - > Egymást elősegítik
 - > A Single Responsibility is ezt segíti elő

SRP példa

```
public class PaymentService {  
  
    public void transferPayment(Employee initiator,  
                                Employee recipient,  
                                int amount) {  
        if (isAuthorizedToPay(initiator)) {  
            // ... transfer payment  
        } else {  
            // ... log unauthorized access  
        }  
    }  
  
    public boolean isAuthorizedToPay(Employee user)  
{  
    // ... check access  
  
    return false;  
}  
}
```

SRP példa javítva

```
public class PaymentService {  
  
    private AuthorizationService authorizationService;  
  
    public PaymentService(AuthorizationService  
                           authorizationService) {  
        this.authorizationService = authorizationService;  
    }  
  
    public void transferPayment(Employee initiator,  
                                Employee recipient,  
                                int amount) {  
        if (authorizationService.isAuthorizedToPay(initiator)) {  
            // ... transfer payment  
        } else {  
            // ... log unauthorized access  
        }  
    }  
}
```

Open/Closed Principle

- A komponensek legyenek nyitottak a bővítésre, de zártak a módosításra
- Másképpen: anélkül tudjuk őket bővíteni, hogy a meglévő működéshez hozzá kellene nyúlni

OCP példa

```
public class AreaCalculatorBad {  
  
    public double calculateArea(List<Shape> shapes) {  
        double area = 0;  
        for (Shape s : shapes) {  
            if (s instanceof Rectangle) {  
                Rectangle r = (Rectangle) s;  
                area += r.getA() * r.getB();  
            } else if (s instanceof Circle) {  
                Circle c = (Circle) s;  
                area += c.getRadius() * c.getRadius() * Math.PI;  
            } else  
                throw new UnsupportedOperationException(  
                    "Unsupported shape");  
        }  
        return area;  
    }  
}
```


OCP példa

```
public interface Shape {  
    double area();  
}  
  
public class AreaCalculator {  
    public double calculateArea(List<Shape> shapes) {  
        double area = 0;  
        for (Shape s : shapes) {  
            area += s.area();  
        }  
        return area;  
    }  
}
```

Liskov Substitution Principle

- Objektumokat lehessen leszármazott osztályával kicserélni, a helyes működés megsértése nélkül
- Ezt mondja a polimorfizmus is, de az csak a technikai megvalósíthatóság
- A lényeg: úgy is írjuk a kódot, hogy a szabálynak eleget tegyen
- Pl. lista listaként is viselkedjen, tárolja el, és adja is vissza, amit beletettünk
 - > <https://docs.oracle.com/javase/10/docs/api/java/util/List.html>

Interface Segregation Principle

- Az osztály ne függjön olyan interfészekről, amelyeknek a metódusait nem használja ki
- Gyakorlatban: használjunk specifikus, kicsi interfészeket a nagy interfészek helyett
- (Single Responsibility-vel és erős kohézióval analóg)

ISP példa

```
public interface Car {  
    void setSeatHeating(boolean value);  
    void setTurnSignalLeft(boolean value);  
    void setTurnSignalRight(boolean value);  
}
```

ISP példa

```
public class BmwX5 implements Car {  
    @Override  
    public void setSeatHeating(boolean value) {  
        // TODO  
    }  
  
    @Override  
    public void setTurnSignalLeft(boolean value) {  
        throw new UnsupportedOperationException("BMW's do"+  
            " not have turn signal");  
    }  
  
    @Override  
    public void setTurnSignalRight(boolean value) {  
        throw new UnsupportedOperationException("BMW's do"+  
            " not have turn signal");  
    }  
}
```

ISP példa javítva

```
public interface SeatHeating {  
    void setSeatHeating(boolean value);  
}
```

```
public class BmwX5 implements SeatHeating {  
    @Override  
    public void setSeatHeating(boolean value) {  
        // TODO  
    }  
}
```

//Not only for cars ...

```
public class WinterVehicleTesting {  
    public bool TestHeating(SeatHeating subjectVehicle) {  
        ...  
        subjectVehicle.setSeatHeating(true);  
        ...  
    }  
}
```

Dependency Inversion Principle

- Absztrakcióktól függünk, ne konkrét dolgoktól
- Nagy vonalakban:
 - > Használjunk interfészeket, absztrakt osztályokat a függőségeknek
 - > A függőségeket ne a függő hozza létre, hanem kívülről kapja
- Ld. későbbi előadások

A tervezés szintjei

- Főleg osztályokról beszéltünk, de sok dolog általánosítható kisebb/nagyobb egységekre is
- Az elvek megértése után adja magát
- Pl. metódus is csak egy dolgot csináljon
- Pl. package-ek is minél kevesebbet tudjanak a másik működéséről

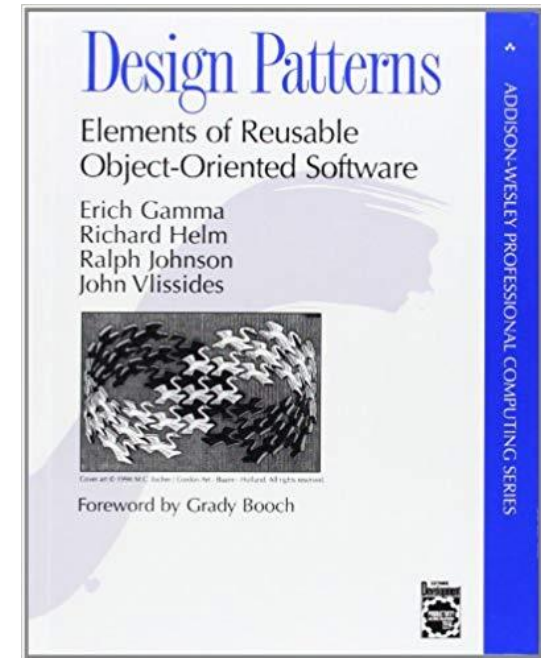
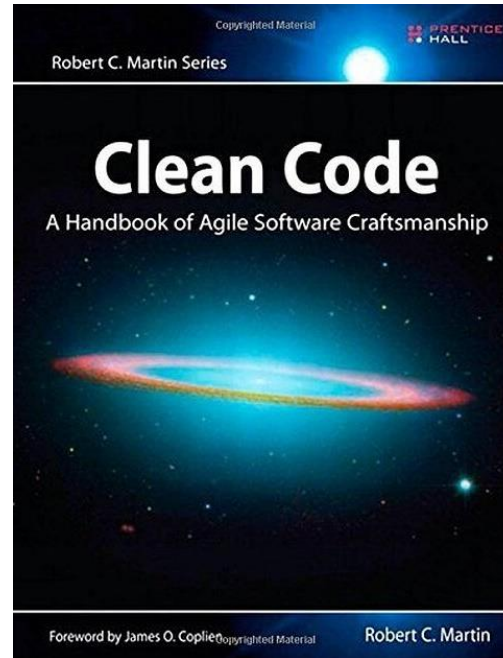
További tervezési elvek

- Keep it simple stupid (KISS):
 - > Feleslegesen ne bonyolítsuk a kódot
- You aren't gonna need it (YAGNI)
 - > Amíg nem bizonyos, hogy valami kell, addig ne fejlesszük ki
 - > Az idő pénz, a fejlesztésnek költsége van
 - > Az extra funkciók ráadásul a komplexitást is növelik

További tervezési elvek

- Do not repeat yourself (DRY)
 - > A kódduplikáció problémát okoz(hat)
 - > Minden feladatot egyszer oldjunk meg egy helyen
 - > Ha ismétlődés van, szervezzük ki egy helyre
 - > Az ismétlődés feleslegesen növeli a kód méretét, rontja az áttekinthetőséget
 - > És ha módosítani kell, inkonzisztenciát okozhat, ha nem írjuk át egységesen
 - > Drasztikusan nehezedik a kód karbantartása
 - > **Kivétel: biztonsági aspektusok!**
 - Mindenhol ellenőrizzünk
 - Ha valahol hiányos, a másik még rész még védhet
 - Ld. BKK e-jegy

Bibliográfia



Köszönöm a figyelmet!

Szoftvertechnológia és -technikák

2. Előadás

Bevezetés az UML világába, osztálydiagram



Tartalom

Modellezés - bevezetés

UML

Osztálydiagram



Modellezés

Szoftverfejlesztés

- Készítsünk egy programot, ami kiszámolja és kiírja egy adott szám faktoriálisát!



```
class Faktorialis
{
    public static void main(String args[])
    {
        int i;
        int fact = 1;

        int number = 5;
        for (i = 1; i <= number; i++)
        {
            fact = fact * i;
        }
        System.out.println("A(z) " + number + " faktoriálisa: " + fact);
    }
}
```



Szoftverfejlesztés

- Készítsünk programot, ami kiszámolja a binomiális tételt bekért paraméterekkel!
$$\sum_{k=0}^n \binom{n}{k} x^k a^{n-k}$$
- Készítsünk programot, ami kiszámol egy tetszőleges matematikai műveletsort!
(Az elérhető függvények listája előre adott)
- Készítsünk egy tőzsdei trendeket előrejelző, pénzügyi tanácsadó alkalmazást mobilra!



Szoftverfejlesztés

- Tervezés nélkül is lehet építeni házat
 - > Fatárolóhoz elég lehet
 - > Ha egyedül építi az ember
 - > Azonnal összeomlik...
 - > ... vagy csak később?
- Több szintes családi ház
 - > Többen építik
 - Sorrend: A tetőfedő csak az elején ér rá?
 - Értelmezés: Tűzfalra néző ablak
 - > Élni is szeretnék benne
 - > Sok évig



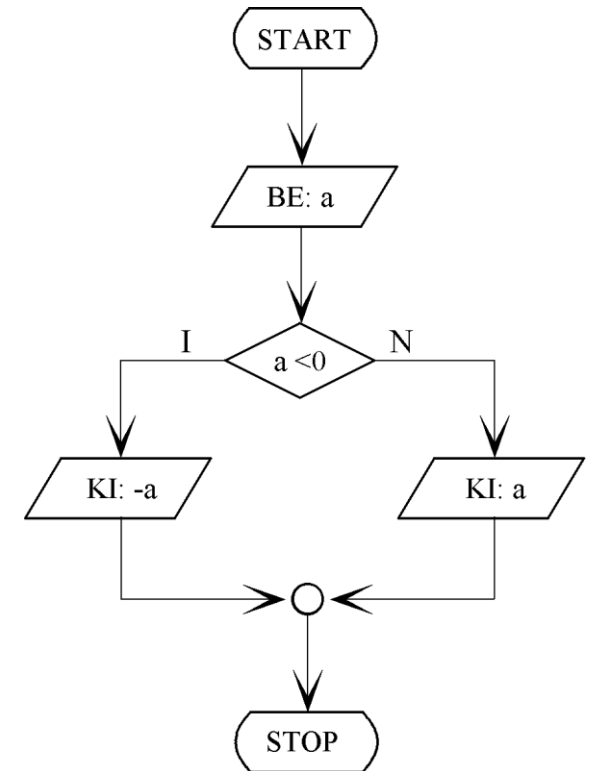
Szoftverfejlesztés nagyban

- Szoftver
 - > Kis feladat → “Csak meg kell írni”
 - > Komplex feladat → Érdemes megtervezni
- Tervezés
 - > Ha a kódméret/feladat komplex
 - > Ha egyeztetni kell a megrendelővel
 - > Ha meg kell becsülni a költségeket
 - > Ha több ember fejleszti, új belépők is lehetnek
 - > Ha segédmunkások is vannak
- Megoldás
 - > Modellezés



Modellezés

- Modell: a valóság egy egyszerűsített nézete
 - > Absztrakciós szint növekedés
 - > Hangsúly azon, ami fontos...
 - > ... elrejtve azt, ami nem
- Modell a szoftverfejlesztéshez
 - > A struktúrához?
 - > A folyamatokhoz?
 - > A viselkedéshez?
 - > ... Vagy mindenhez ezek közül?



Unified Modeling Language

Bevezetés

Szoftvermodellezés

- Igény a szoftverek tervezésére
 - > Magas absztrakciós szint, vizuális ábrázolás
 - > '90-es évektől több modellező nyelv: kaotikus helyzet
 - > 1997. – Unified Modeling Language (UML)
- Unified Modeling Language
 - > Cél: szoftverfejlesztés és dokumentáció modellezése
 - > Általános modellező nyelv
 - > A szabvány modellezés terén
 - > Jelenlegi verzió: 2.5.1.
<https://www.omg.org/spec/UML/2.5.1/PDF>
 - > Jó magyarázatok és példák:
<https://www.uml-diagrams.org>
 - > Modellezőeszköz a félév során:
Modelio (<https://www.modelio.org/>)
 - Részletek gyakorlaton!



UML

- UML használatának előnyei

- > Vázlat

- Implementáció előtt:

- Probléma jobb megértése
 - Hibák/hiányosságok feltárása

- Implementáció után

- Dokumentálás

- Kommunikáció

- Megrendelővel
 - Fejlesztőkkel

- > Tervrajz

- Terv alapján könnyebb dolgozni
(architect vs. programozó)



UML diagrammok

- Struktúra

- > Osztálydiagram (Class diagram)
- > Komponensdiagram (Component diagram)
- > Telepítési diagram (Deployment diagram)
- > Objektumdiagram (Object diagram)
- > Csomagdiagram (Package diagram)
- > Összetett struktúradiagram (Composite Structure diagram)
- > Profildiagram (Profile diagram)



UML diagramok II.

- Viselkedés
 - > Aktivitásdiagram (Activity diagram)
 - > Használati eset diagram (Use case diagram)
 - > Állapotgép (State machine)
 - > Interaktivitás
 - Szekvenciadiagram (Sequence diagram)
 - Kommunikációs diagram (Communication diagram)
 - Interakció áttekintő diagram (Interaction overview diagram)
 - Időzítő diagram (Timing diagram)



UML

- Mit ad az UML?
 - > Jelölésrendszer, közös nyelv
 - > Magas absztrakciós szint, tömör leírás
 - > Szabványos keret
 - > Segít szemléltetni a szoftvertervezési módszereket
- Mit nem ad az UML?
 - > Nem oldja meg a problémát, csak segít leírni azt
 - > Nem mutatja meg, *hogyan* tervezzünk jól
 - Ezért tanulunk a Clean code-ról, a SOLID elvekről,...
 - Ezért kellenek a tervezési minták



Osztálydiagram

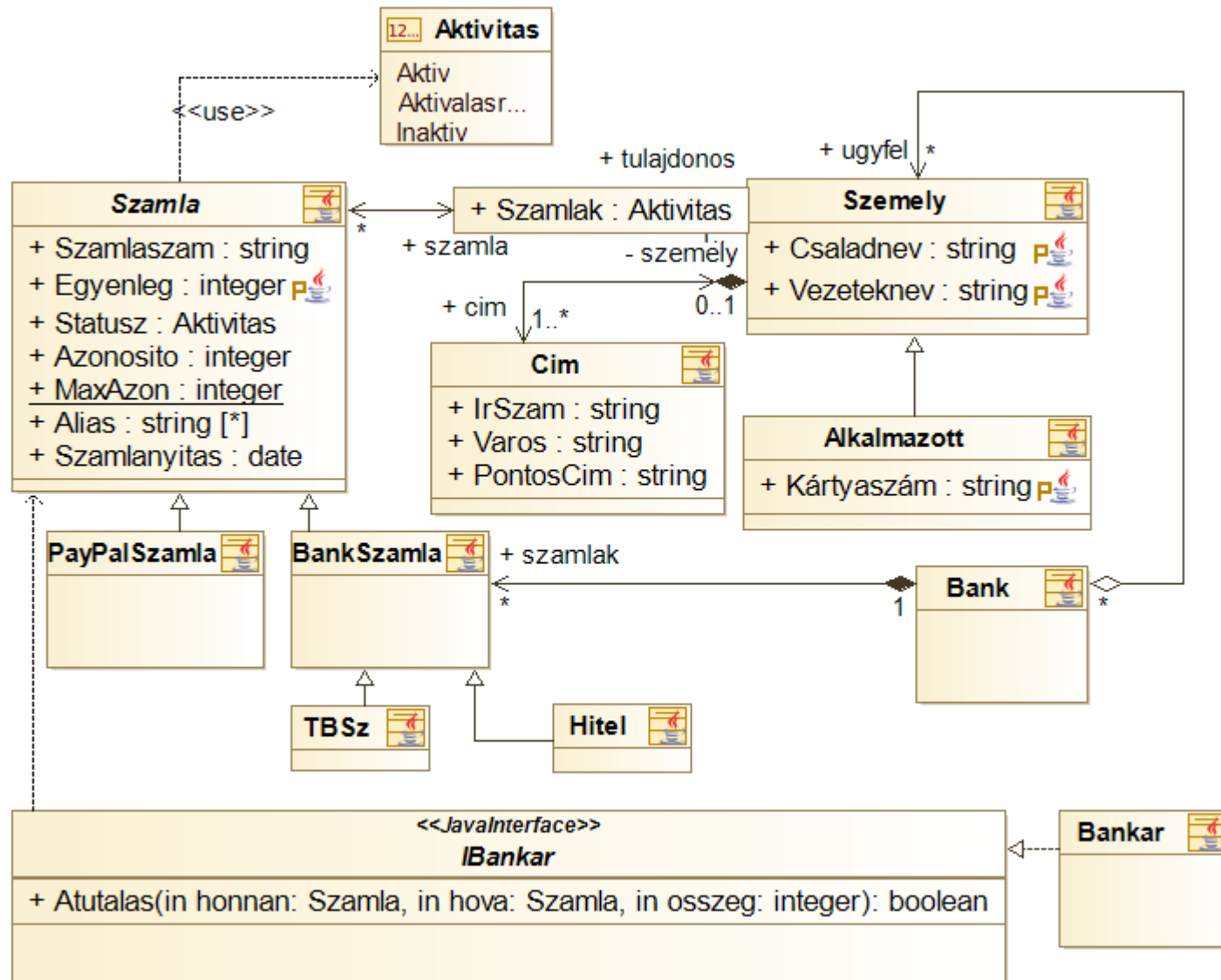
Class diagram

Osztálydiagram

- A leggyakrabban használt, legfontosabb UML diagram
- OOP-alapon készült rendszerekhez
- Struktúra megadására (osztályok, tagváltozók, metódusok, kapcsolatok)
- Programozási nyelvtől független!
 - > Generálható belőle forráskód (metódusoknál csak a váz)



Osztálydiagram: Banki adminisztrációs rendszer



Modellelemek

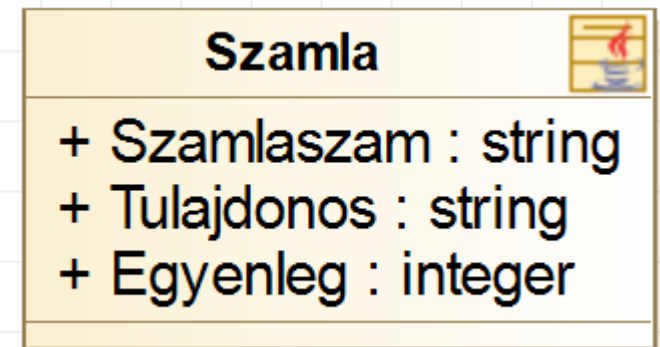
Osztályok



Számlaszám	Tulajdonos neve	Egyenleg (Ft)
11742232-15393276	Nagy Lajos	12 000 343
11742232-75343245	Hunyadi Mátyás	4 556
11742232-35393127	Károly Róbert	443 227 786

- Készítsünk hozzá egy osztályt és tagváltozókat!
 - > Osztály (class) és tagváltozó (attribute)

```
1 public class Szamla {  
2  
3     public String Szamlaszam;  
4     public String Tulajdonos;  
5     public int Egyenleg;  
6  
7 }
```



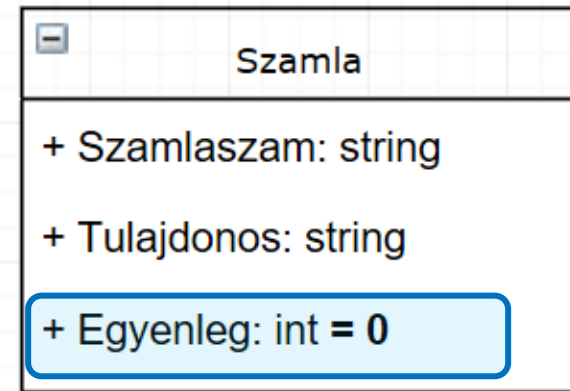
Kezdőérték

- Legyen az egyenleg alapesetben 0 Ft!
 - > Sajnos a Modelio eszköz nem jeleníti meg az ábrán!



> A generált kód:

```
1  import com.modeliosoft.modelio.javadesigner.annota
2
3  @objid ("84fe0cc8-8a45-4874-9b29-6c491370c38b")
4  public class Szamla {
5      @objid ("e2c21051-b938-40b4-b271-48f932146246")
6      public String Szamlaszam;
7
8      @objid ("f15659a4-cc45-45be-bc79-597568508df9")
9      public String Tulajdonos;
10
11      @objid ("d662343e-735f-44fa-8c68-1db19fdf66c2")
12      public int Egyenleg = 0;
13
14  }
15
```



Láthatóság

+ -> **public**
- -> **private**
-> **protected**

- Az egyenleg megváltoztatását és lekérdezését válasszuk szét!



- > Metódus (method)
- > Láthatóság (visibility/member access modifier)
- > Privát tagváltozó + publikus metódusok
- > Tagváltozó → property

Szamla	
+ Szamlaszam : string	
+ Tulajdonos : string	
- Egyenleg : integer	
+ getEgyenleg(): integer	00
+ setEgyenleg(in value: integer)	00

Szamla	
+ Szamlaszam : string	
+ Tulajdonos : string	
+ Egyenleg : integer	P

```
12  
13  
14  
15  
16  
17  
18  
19
```

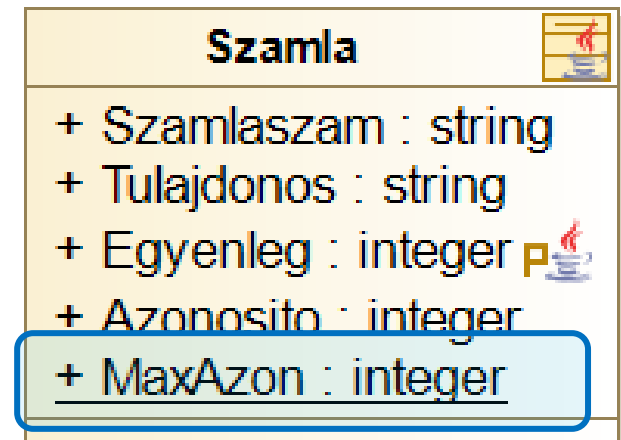
```
private int Egyenleg = 0;  
  
public int getEgyenleg() {  
    return this.Egyenleg;  
}  
  
public void setEgyenleg(int value) {  
    this.Egyenleg = value;  
}
```

Statikus tagváltozók

- Minden számlának legyen egy egyedi, belső azonosítója. Számla létrehozásakor az eddigi maximum azonosítóból számoljuk az új számla azonosítóját!
 - > A számlaazonosító példány szintű adat (minden számlának más)
 - > A max számlaazonosító viszont *osztály* szintű!
 - > Statikus tagváltozó kell (aláhúzás!)
 - > A számolási logika nem a modell része!



```
public class Szamla {  
    ...  
  
    public int Azonosito;  
    public static int MaxAzon;  
  
    ...  
}
```



Multiplicitás

[*] -> korlátlan elemű lista
[1] -> Pontosan 1 elem
[0..*] -> 0 vagy több elem (lista)

- Lehesse megadni a számlához több alias nevet!

- > Multiplicitás (multiplicity)

- > Attribútum többes multiplicitással

- 0..1
 - 1..1 (alapértelmezett)
 - 0..* (lista generálódik belőle!)
 - n..m

Szamla	
+ Szamlaszam :	string
+ Egyenleg :	integer 
+ Statusz :	Aktivitas
+ Azonosito :	integer
+ MaxAzon :	integer
+ Alias :	string [*]



Interfész

- Készítsünk egy interfészt, ami támogatja a számlák közti átutalás műveletét!
 - > **Művelet (operation)**
 - Művelet - definíció vs. Metódus - implementáció
 - > A paraméter típusa nem beépített típus!
 - > Szignatúra: `+ Atutal( honnan: Szamla, hova: Szamla, osszeg: integer): bool`



<<JavaInterface>>

IBankar

`+ Atutalas(in honnan: Szamla, in hova: Szamla, in osszeg: integer): boolean`

Osztályok és interfészek

- Osztályok és interfészek

- > Fejléc

- > Tagváltozók

- [Láthatóság] [Név]: [Típus][Multiplicitás] [= kezdőérték]
 - Statikus: aláhúzás

- > Metódusok/Műveletek

- [Láthatóság] [Név]([paraméterek])([: visszatérési érték])

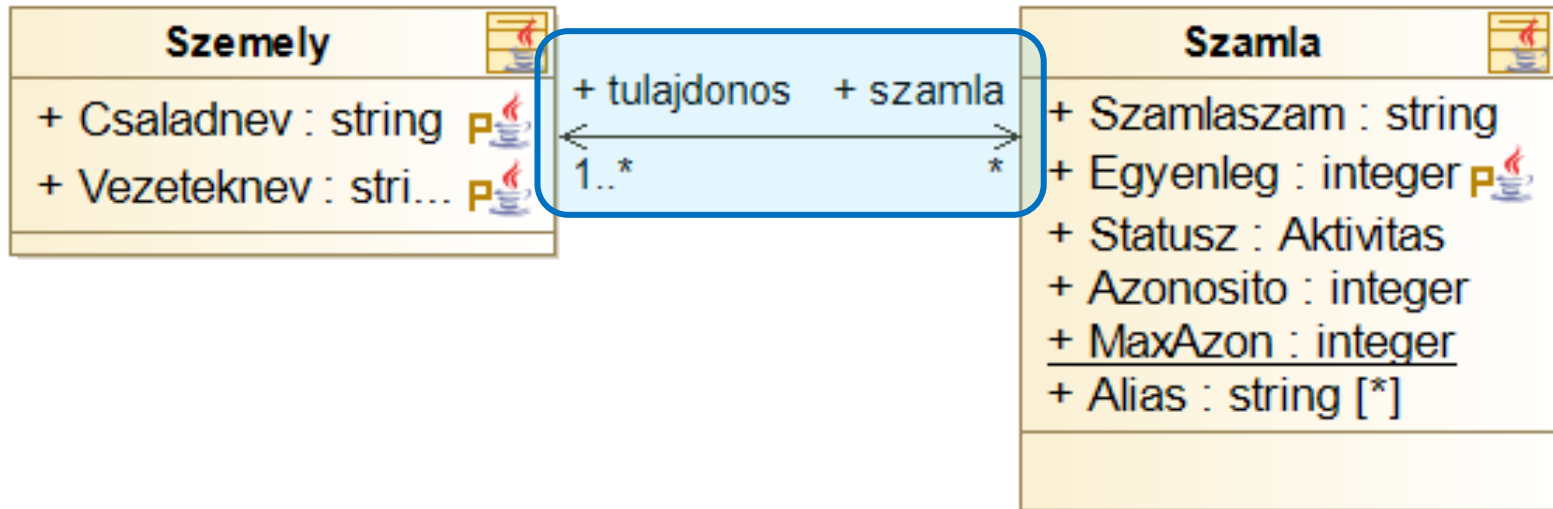
- > Láthatóság

- Public: +
 - Private: -
 - Protected: #
 - Package: ~

Kapcsolatok

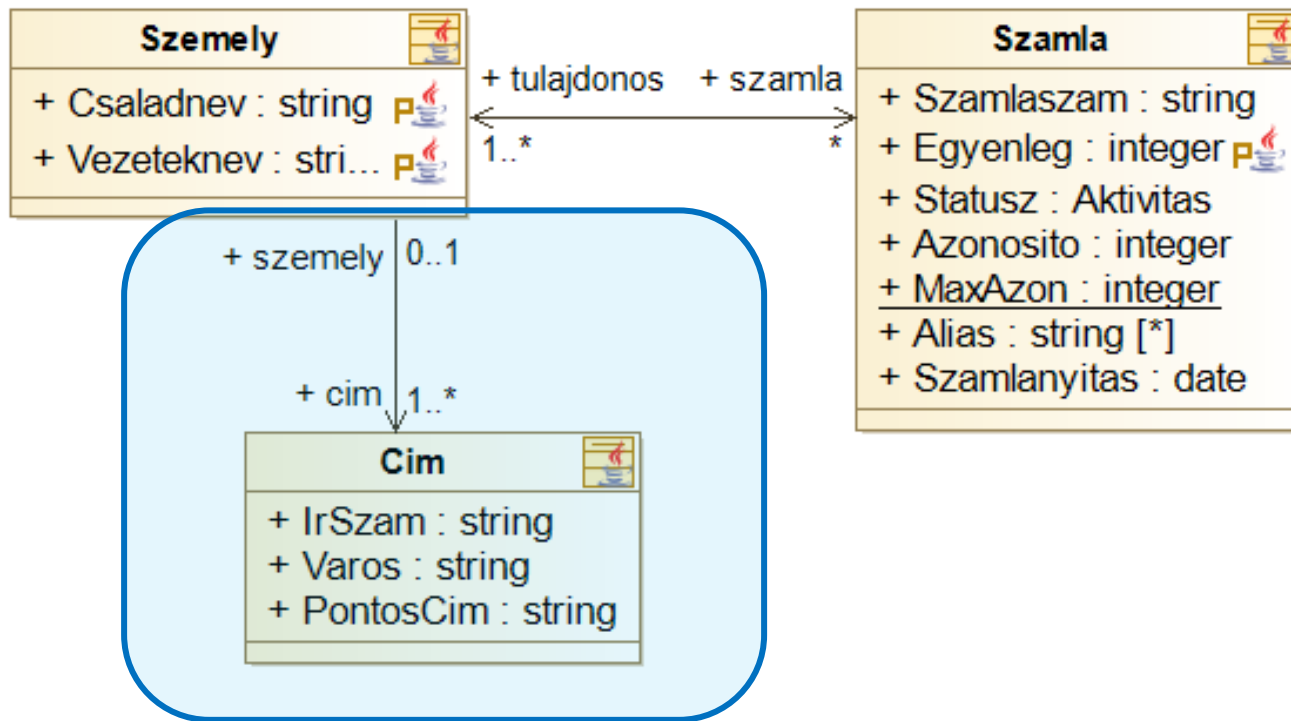
Asszociáció

- Támogassuk, hogy egy személynek több számlája is lehessen!
 - > Asszociáció (association)
 - > Ki kell emelni a személyt az osztályból!
 - > Szerepnév (role name) és multiplicitás (multiplicity)
 - > Kódban: tagváltozó



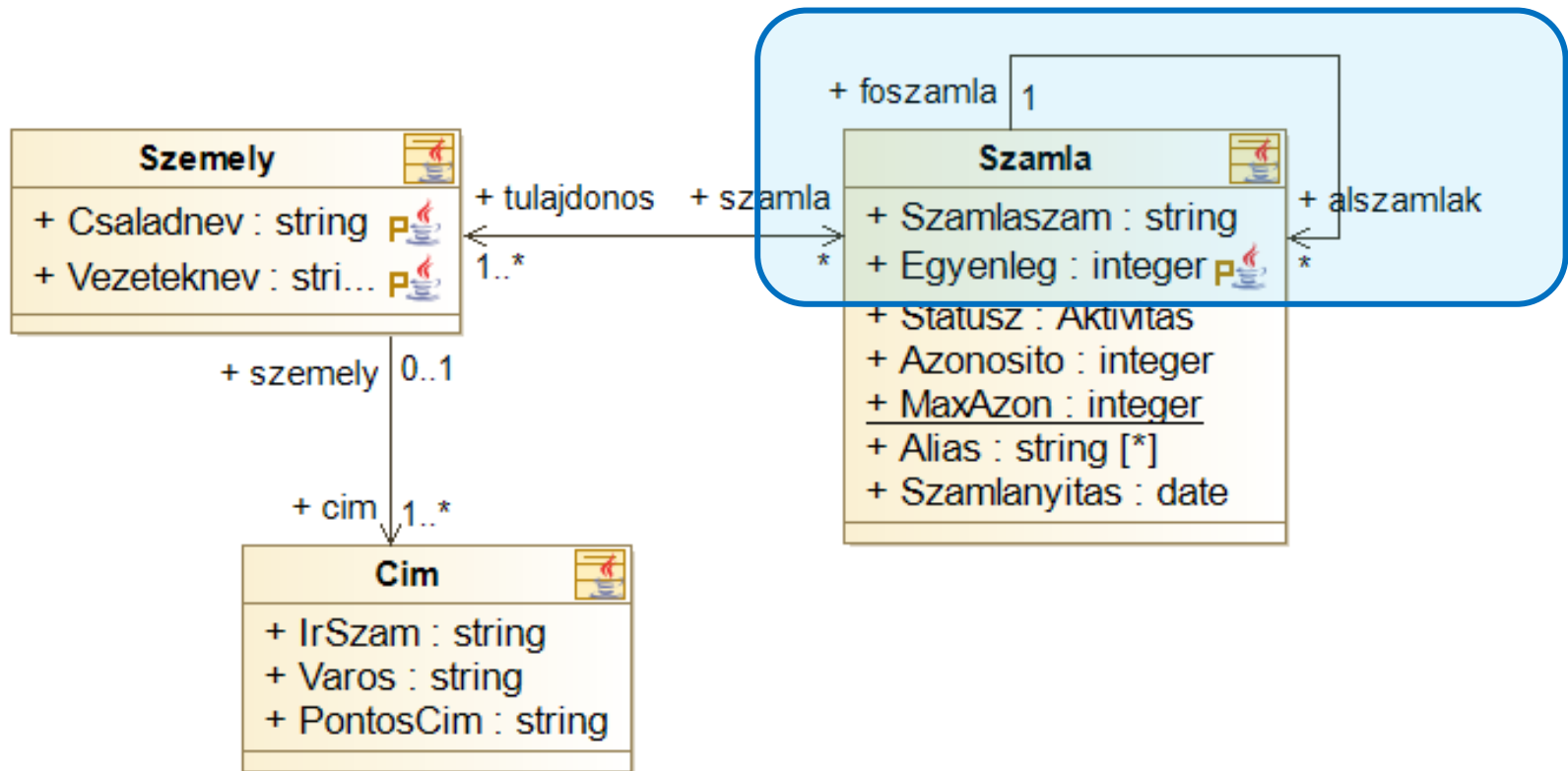
Asszociáció

- Lehessen megadni az egyes emberek lakcímét!
 - > Egy irányban navigálható kapcsolat



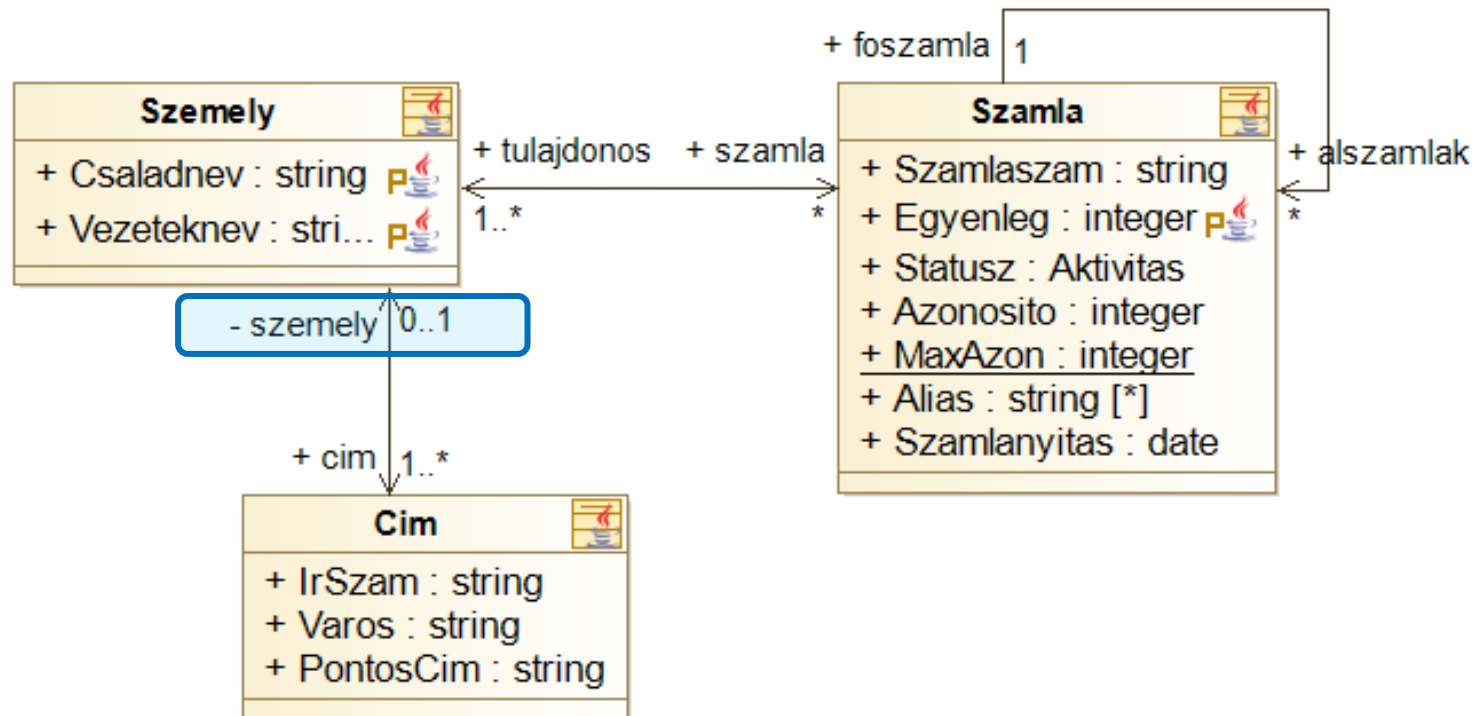
Asszociáció

- Lehessen megadni alszámlákat!



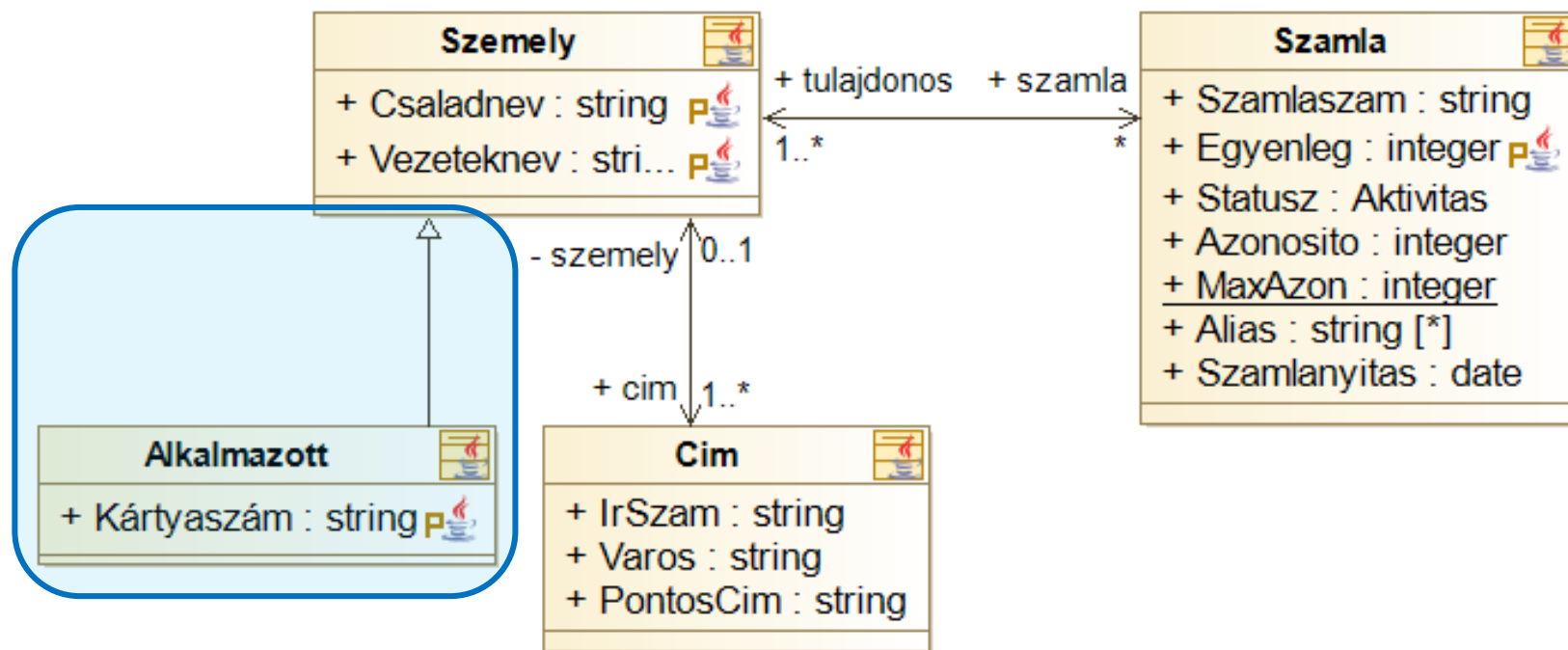
Asszociáció

- A címtől lehessen lekérdezni, melyik személyhez tartozik, de mindezt csak a cím entitáson belülről!
 - > Láthatóság a navigáción!



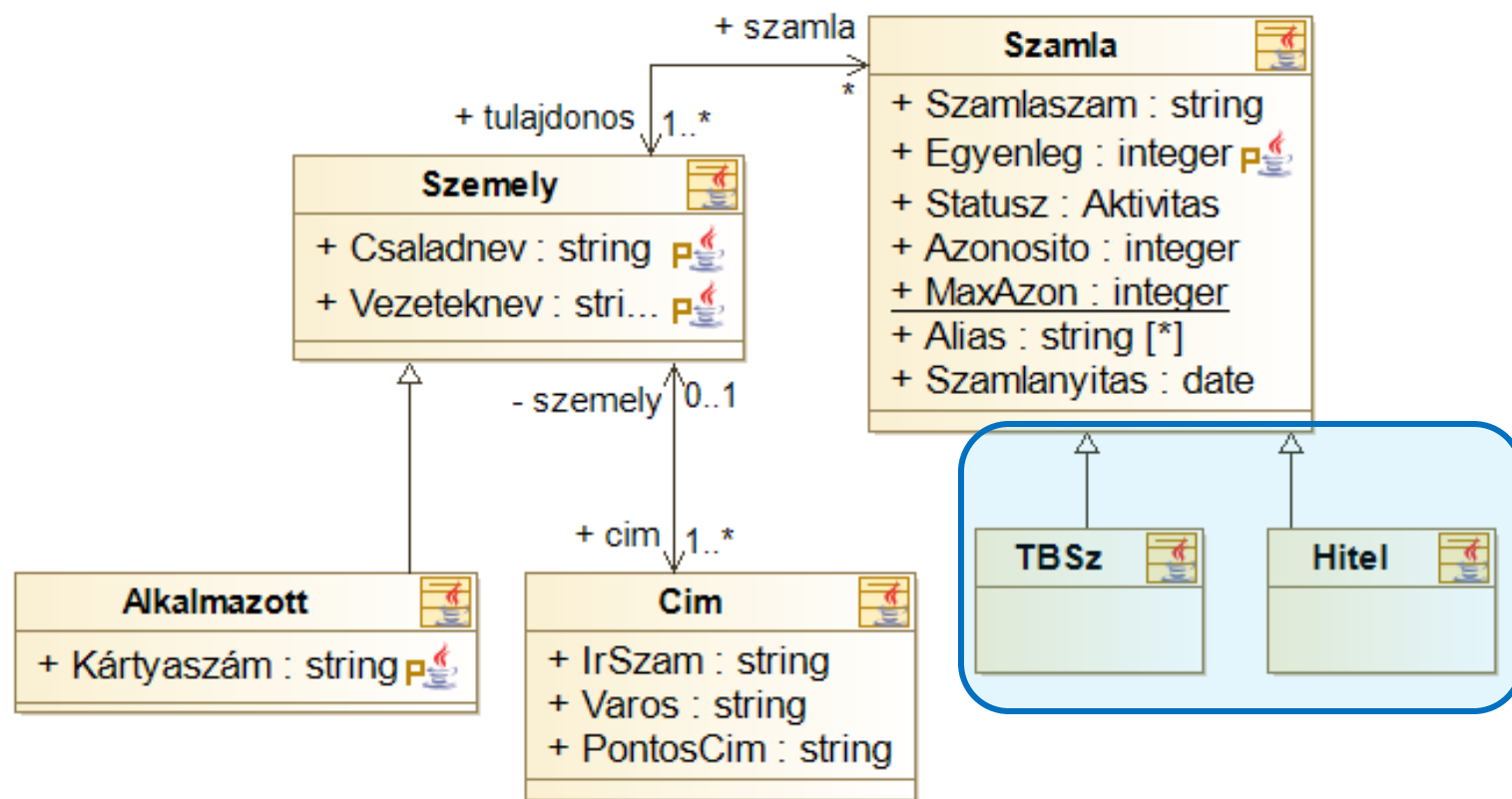
Általánosítás/öröklés

- Tartsuk nyilván a banki dolgozókat is! Nekik is lehet számlájuk, címük, de van azonosítókártyájuk is!
 - > Általánosítás/öröklés (generalization/inheritance)
 - > Kódban: öröklés a két osztály közt



Általánosítás/öröklés

- Legyenek külön számlatípusaink! (TBSz, Hitel)
 - > Általánosítás/öröklés?



Tartalmazás

- Minden számla tartozzon egy bankhoz!
 - > Egy számla nem tartozhat két bankhoz
 - > Ha a bankot töröljük, a számlát is törölni kell!
 - > Kompozíció
(composition)
 - > Kódban: tagváltozó



ERŐS kapcsolat!

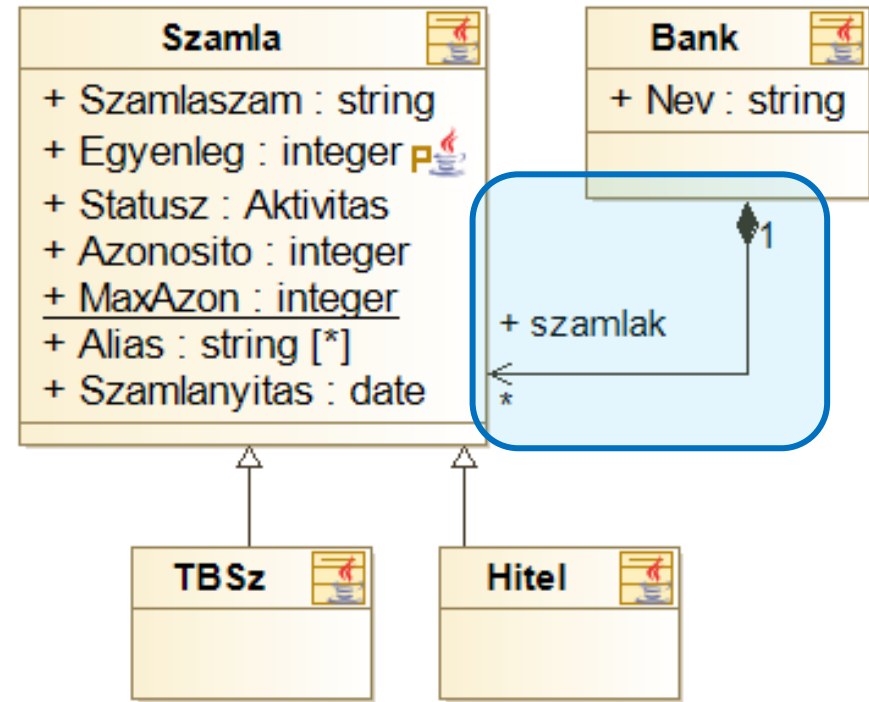
A rész (Számla) nem létezhet az egész nélkül (Bank).

Másik pl: Ház —♦— Szoba

-> Ha a ház megszűnik a szoba is.

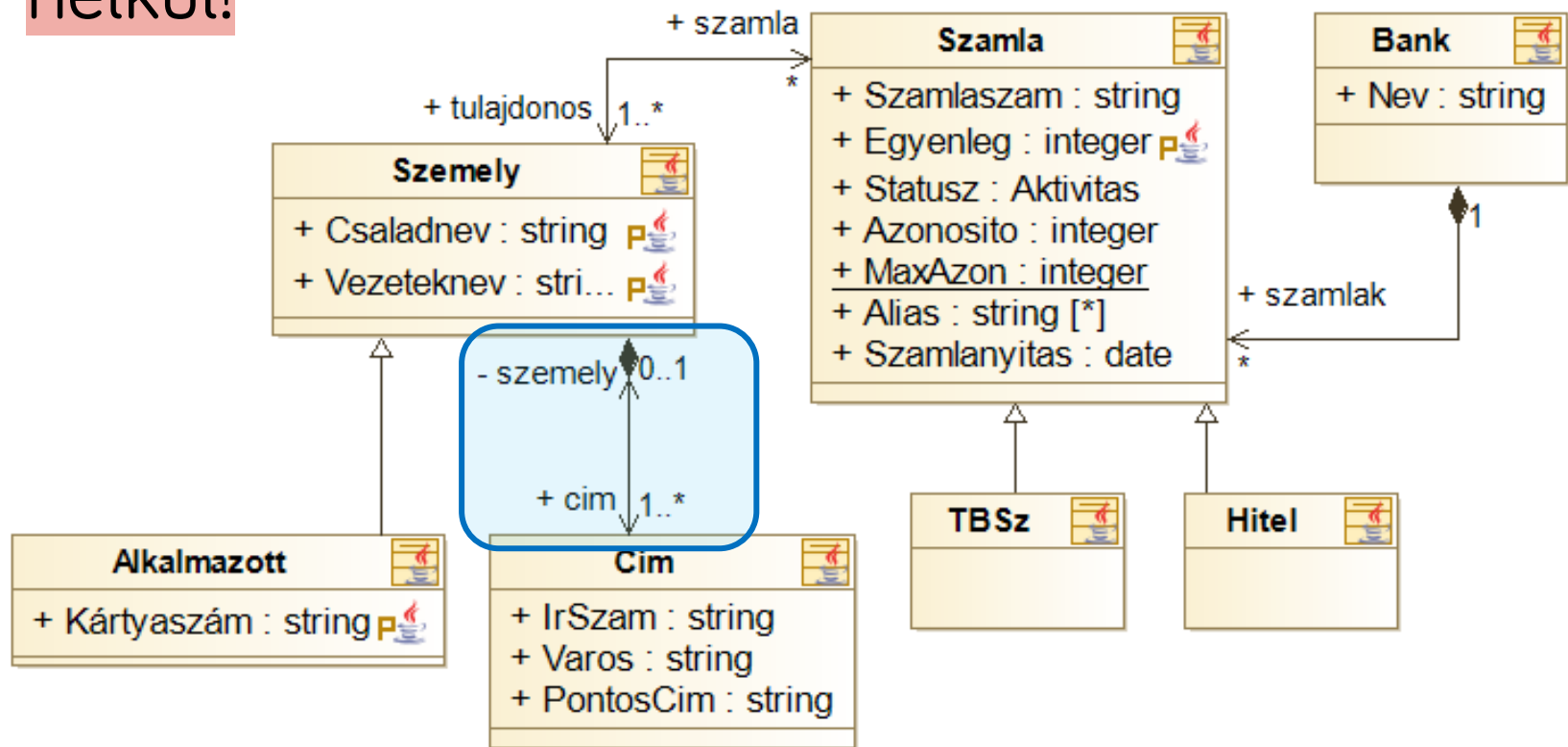
-> A rombusz ott van, ahol a TULAJDONOS!

EGÉSZ ♦ — RÉSZ | eset



Tartalmazás

- Egy cím csak egy személyhez tartozhasson és ne tároljunk feleslegesen címeket, személy nélkül!



Tartalmazás

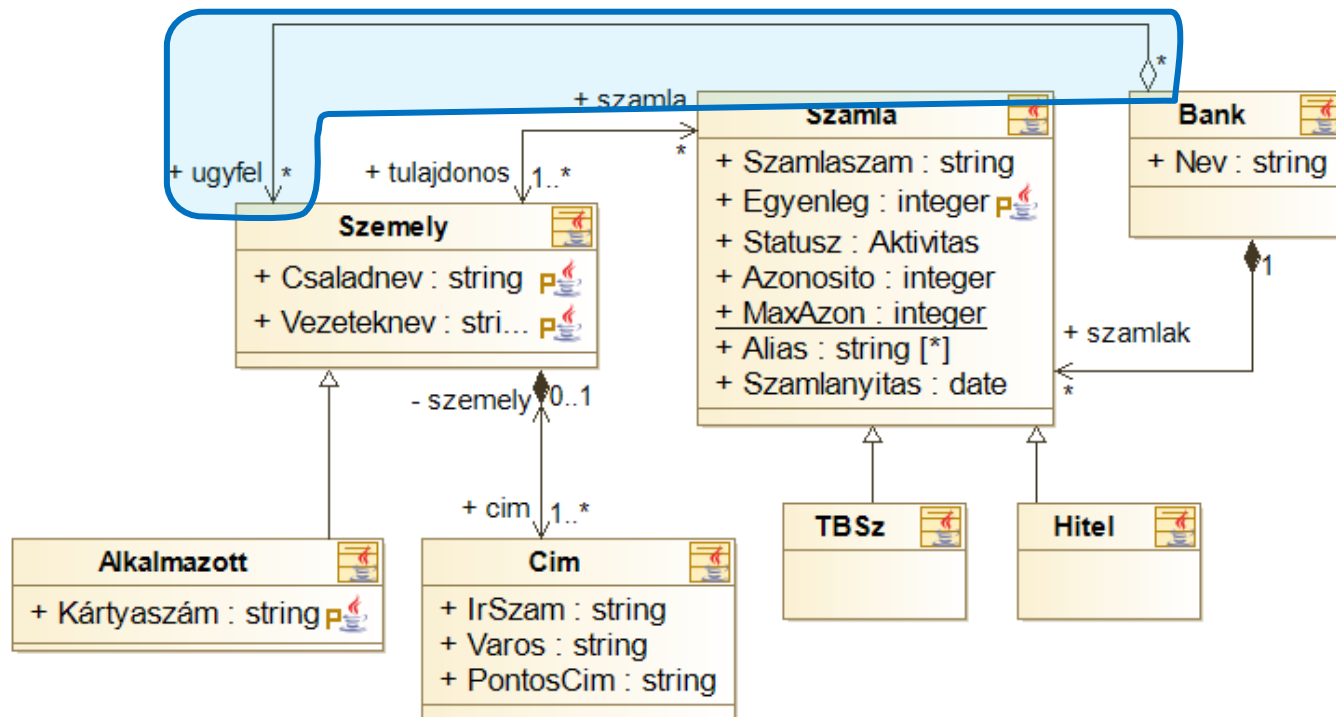
Aggregáció - gyenge tartalmazás
A RÉSZ létezhet az EGÉSZ nélkül is!

Tanszék ◇ — Tanár

- A bank tartsa nyilván az ügyfeleit, akkor is, ha azok megszűntetik a számlájukat! Egy ügyfél több bank ügyfele is lehet.



- > Megosztott tartalmazás (aggregation)
- > Kódban: tagváltozó

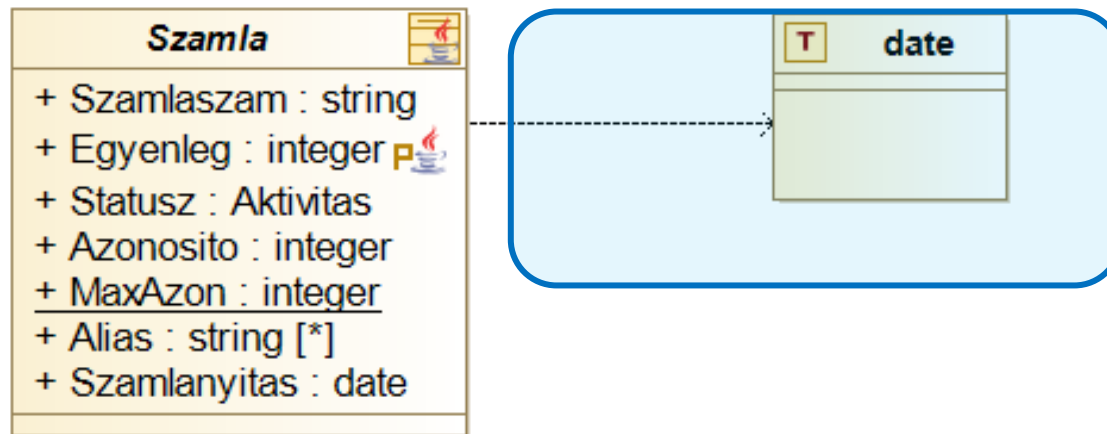


Függőség

Gondoljunk itt a Dependency Injection-re (DI)
Az osztály HASZNÁLJA a másikat, de nem TÁROLJA el!

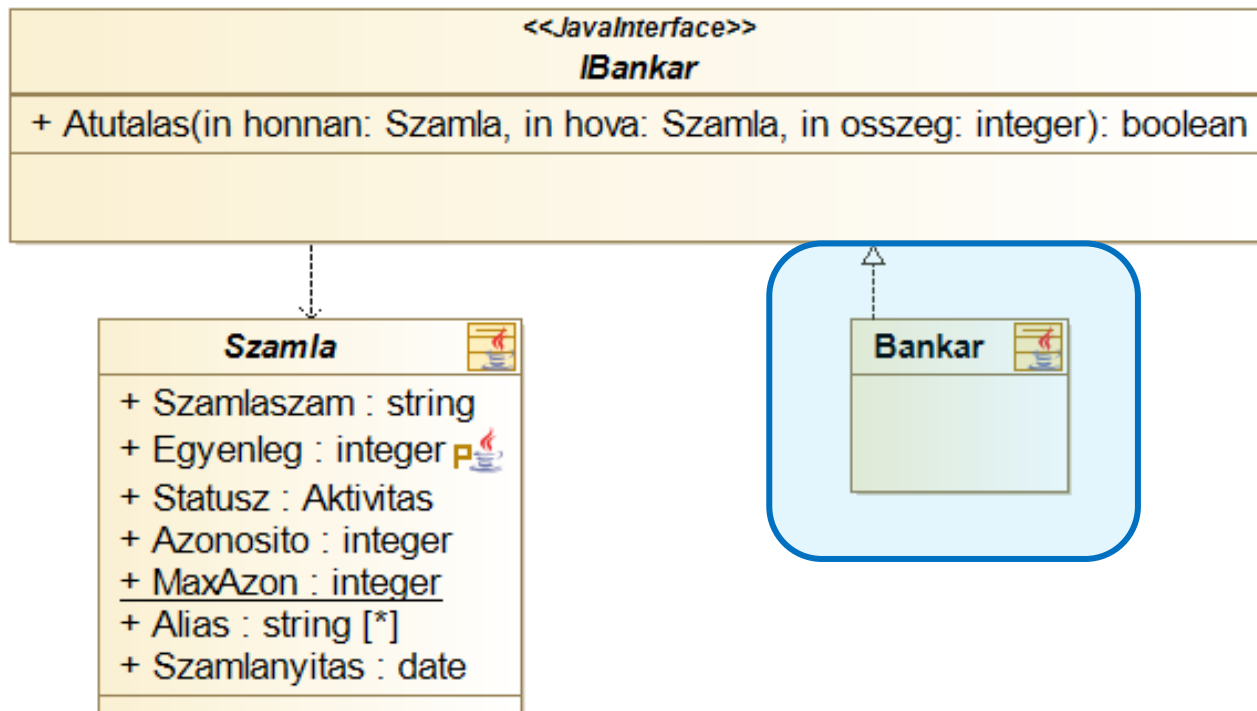
```
class garage {  
    void megszerel(Auto auto)  
        auto.indit();  
}
```

- Lehesse megadni a számlanyitás dátumát!
 - > A típus itt egy saját típus (modellelem)
 - > Függőség (dependency) kapcsolat
 - > Jellemzően nem szoros, hanem ideiglenes kapcsolat
 - > A függőből mutat afelé, amitől függünk
 - > Kódban: import/using



Interfészek II.

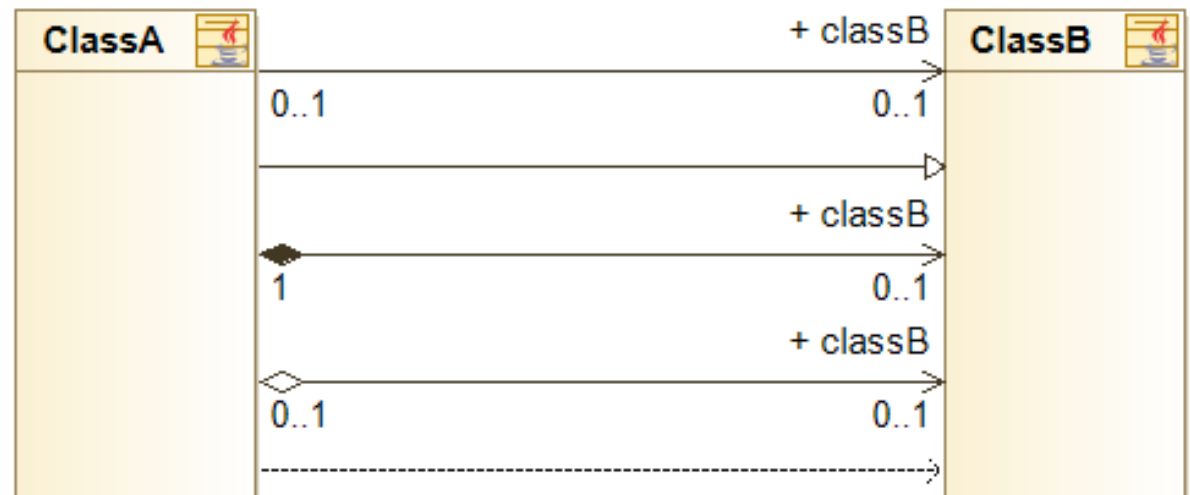
- Térjünk vissza az IBankar interfészhez:
 - > Jelöljük a kapcsolatot a számlával!
 - > Valósítsuk meg egy osztályban az interfészt!
 - > **Interfész megvalósítás (interface realization)**
 - > **Kódban: implements/”:**



Kapcsolatok

- **Kapcsolat típusok**

- > Asszociáció
- > Általánosítás/
Öröklés
- > Kompozíció
- > Aggregáció
- > Függőség
- > Interfész
megvalósítás



- Multiplicitás
- Navigálás
 - > Szerepnév
 - > Navigálhatóság

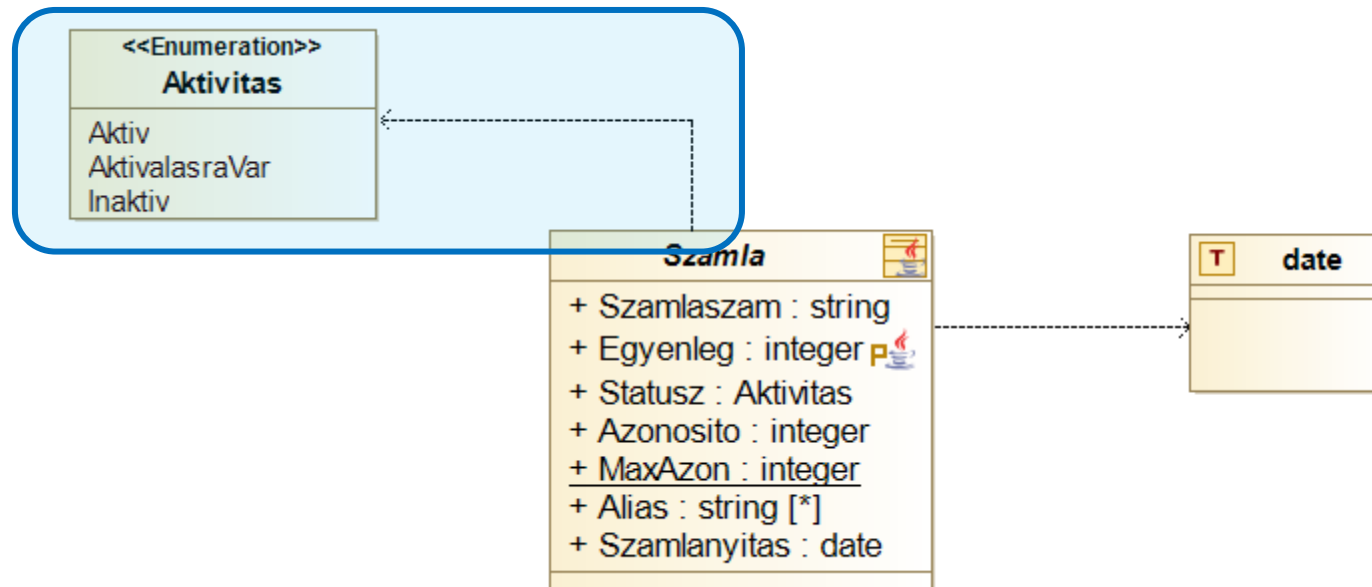
A kapcsolatok szemantikája

- „Is-a” kapcsolat A gyerekosztály egy speciális esete a szülőosztálynak. **PI: Kutya is-a Állat**
 - > Általánosítás/interfész megvalósítás (generalization/interface realization)
 - > Közös funkcionalitás
 - > Ősosztállyal felcserélhetőség (Liskov)
 - > Kibővített működés
- „Has-a” kapcsolat Az egyik osztály tartalmazza a másikat. **PI: Kompozíció / Aggregáció**
 - > Kizárólagos: kompozíció (containment)
 - Tartalmazó törlése törli a tartalmazottat
 - Tartalmazó egyedi
 - > Megosztott: aggregáció (aggregation)
- Nem tartalmazás, nem öröklés, de (részben) navigálható
 - > Asszociáció (association) CSAK IRÁNYT MUTAT!!! NEM KIZÁRÓLAGOS ÉS NEM IS MEGOSZOTT!
 - > Kódban hasonlít a tartalmazáshoz, szemantikai különbség
- Laza/ideiglenes kapcsolat
 - > Függőség (dependency)

További elemek

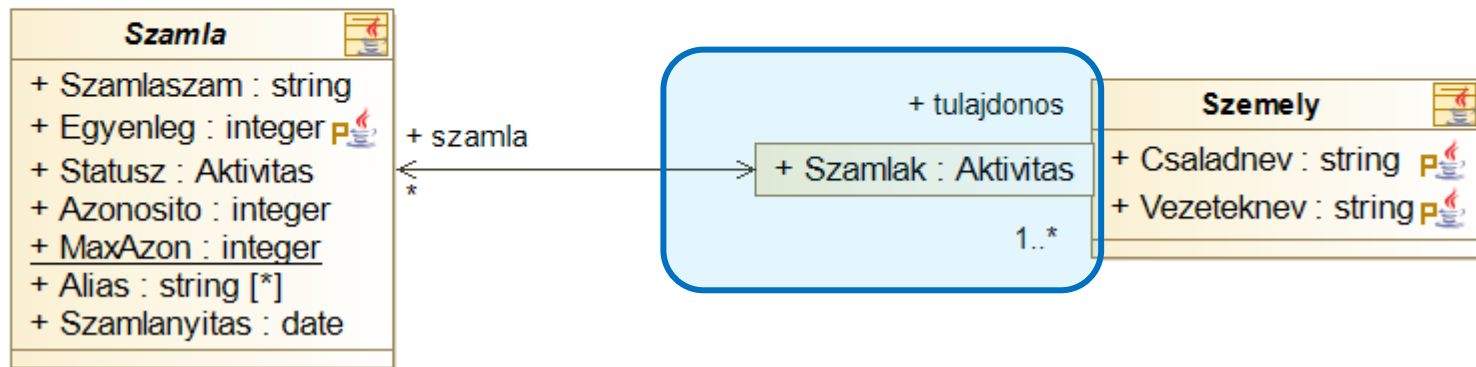
Enumerációk

- Tároljuk el a számla státuszát!
(aktív/aktiválásra vár/inaktív)
 - > Enumeráció (enumeration)



Minősítő (Qualifier)

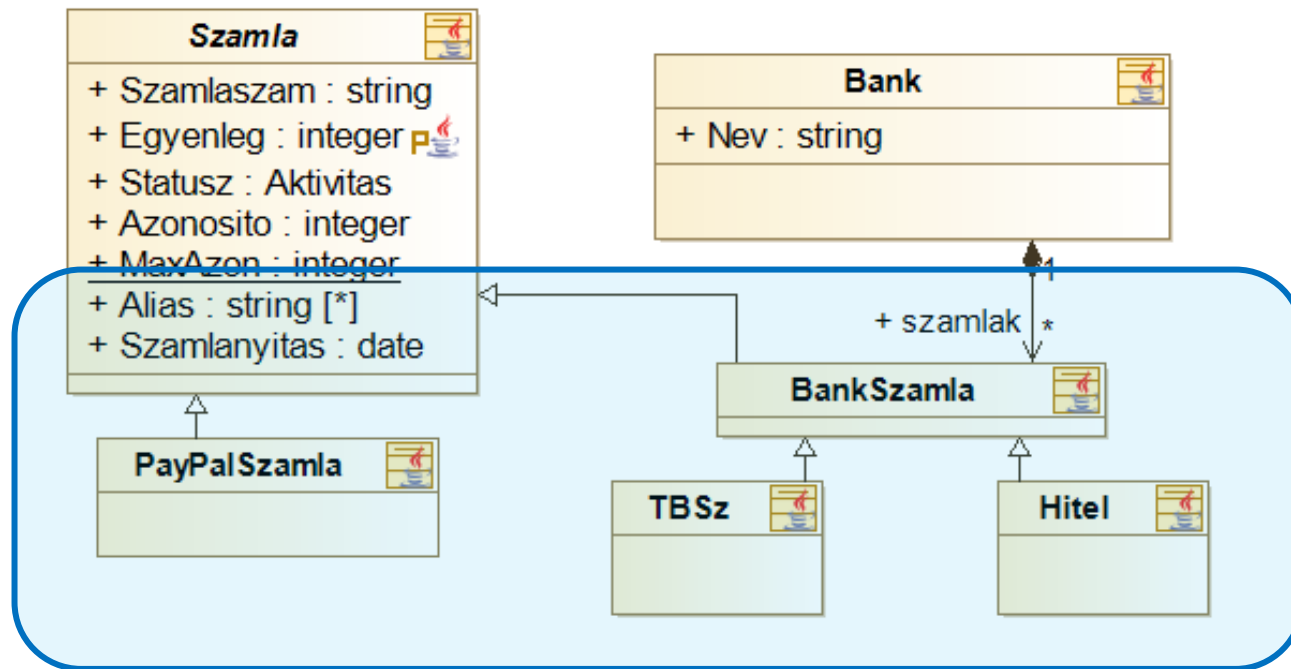
- Csoportosítsuk a számlákat aktivitás szerint!
 - > Minősített asszociáció (qualified association)
 - > Általános lista helyett egyedileg kiválasztott érték



```
public class Szemely {  
  
    ...  
    public Map<Aktivitas, Szamla> szamla =  
        new HashMap<Aktivitas, Szamla> ();  
    ...  
}
```

Absztrakt őszosztály

- Vezessük be a PayPal számlákat! Ezeknél nincs bank, nincsenek speciális számlatípusok, de egyébként bankszámlaként viselkedik minden tekintetben!
 - > Absztrakt őszosztály (abstract base class) (dőlt betű!)
 - > Számla + öröklés



Sztereotípiák

- Sztereotípia(stereotype)
 - > Meglévő elem jelentésének kiegészítése
 - > Jelölés: `<<stereotype_name>>`
 - > Példák
 - `<<interface>>`
 - `<<enumeration>>`
 - Dependency esetén `<<use>>`, `<<create>>`, `<<destroy>>`, ...
 - > Alkalmazástól/szakterülettől függő is lehet

Gondolatok: közös viselkedés?

- A Paypal és a hagyományos bankszámlák közt kell tudni átutalni bármilyen kombinációban.
 - > Mind a négy (2x2) fajta kombináció kell...
 - > ...de valójában ugyanazt ellenőrizzük!
- Megoldás 1. – Közös őosztály (Számla)
- Mi van, ha nem vonható közös őosztály alá?
 - > Megoldás 2. – Közös interfész



Gondolatok: asszociáció, vagy tagváltozó?

- Asszociáció – tartalmazás – attribútum
 - > Mindegyikből tagváltozót generálunk
 - > Mi a különbség?
 - Navigálhatóság, szerepnevek (pl. hurokél)
 - Minősítés (qualifier)
 - Vizualitás, átláthatóság

Köszönöm a figyelmet!

Szoftvertchnológia és -technikák

3. Előadás

Szekvenciadiagram



Tartalom

Osztálydiagram (ismétlés)

Szekvenciadiagram



Modellezés – UML - Osztálydiagram

Ismétlés

Szoftverfejlesztés nagyban

- Szoftver
 - > Kis feladat → “Csak meg kell írni”
 - > Komplex feladat → Érdemes megtervezni
- Tervezés
 - > Ha a kódméret/feladat komplex
 - > Ha egyeztetni kell a megrendelővel
 - > Ha meg kell becsülni a költségeket
 - > Ha több ember fejleszti, új belépők is lehetnek
 - > Ha segédmunkások is vannak
- Megoldás
 - > Modellezés



Szoftvermodellezés

- Igény a szoftverek tervezésére
 - > Magas absztrakciós szint, vizuális ábrázolás
 - > '90-es évektől több modellező nyelv: kaotikus helyzet
 - > 1997. – Unified Modeling Language (UML)
- Unified Modeling Language
 - > Cél: szoftverfejlesztés és dokumentáció modellezése
 - > Általános modellező nyelv
 - > A szabvány modellezés terén



Osztályok és interfészek

- Osztályok és interfészek

- > Fejléc

- > Tagváltozók

- [Láthatóság] [Név]: [Típus][Multiplicitás] [= kezdőérték]
 - Statikus: aláhúzás

- > Metódusok/Műveletek

- [Láthatóság] [Név]([paraméterek])([: visszatérési érték])

- > Láthatóság

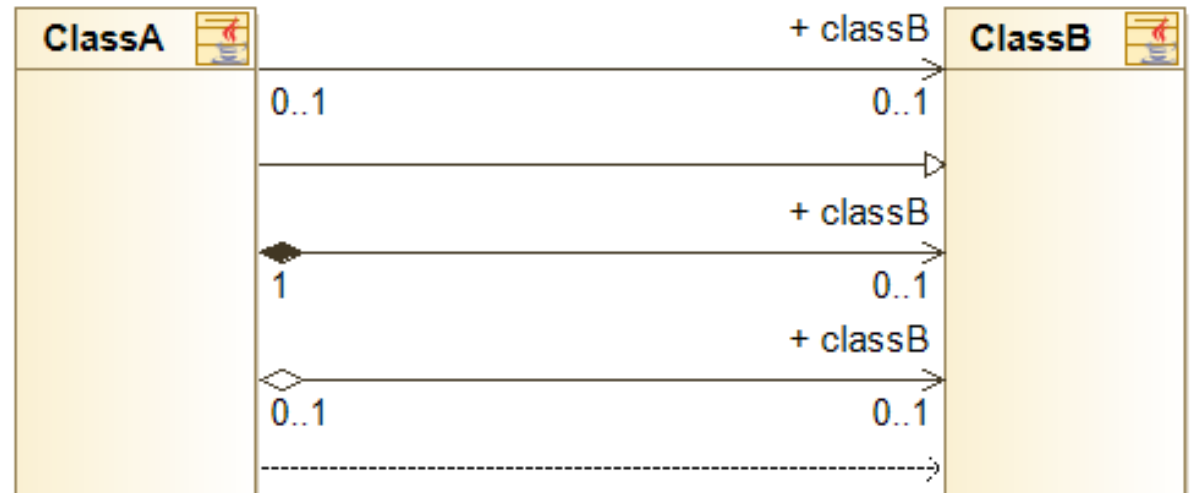
- Public: +
 - Private: -
 - Protected: #
 - Package: ~



Kapcsolatok

- **Kapcsolat típusok**

- > Asszociáció
- > Általánosítás/
Öröklés
- > Kompozíció
- > Aggregáció
- > Függőség
- > Interfész
megvalósítás



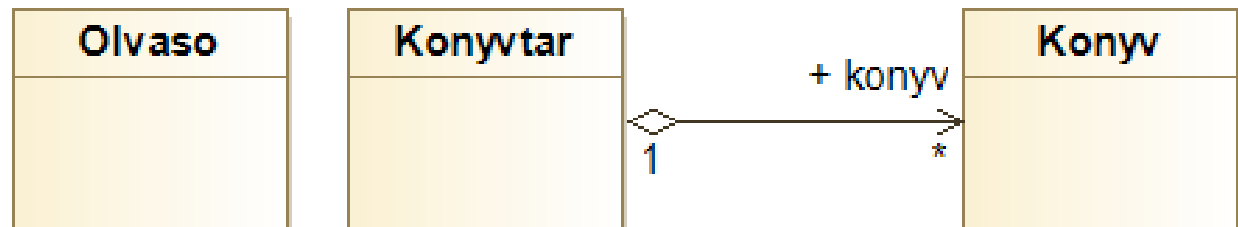
- Multiplicitás
- Navigálás
 - > Szerepnév
 - > Navigálhatóság



Osztálydiagram: könyvtár

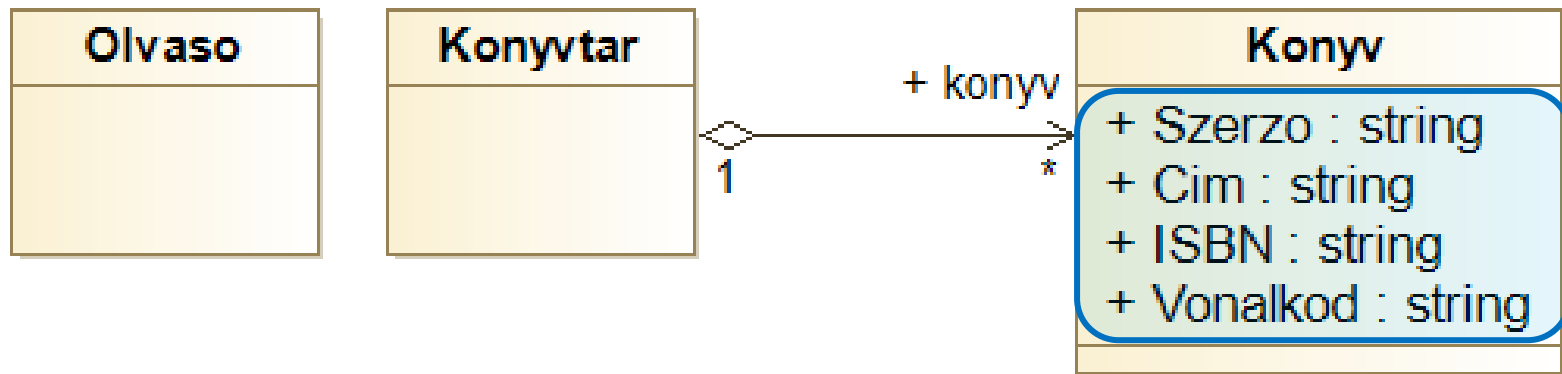
Könyvtár

- Hozzunk létre egy nyilvántartó rendszert könyvtárak számára!
 - > Legyenek benne olvasók és könyvek!
 - > ... természetesen a könyvtárat is modellezni kell
 - > ... a könyvtár tartalmaz könyveket



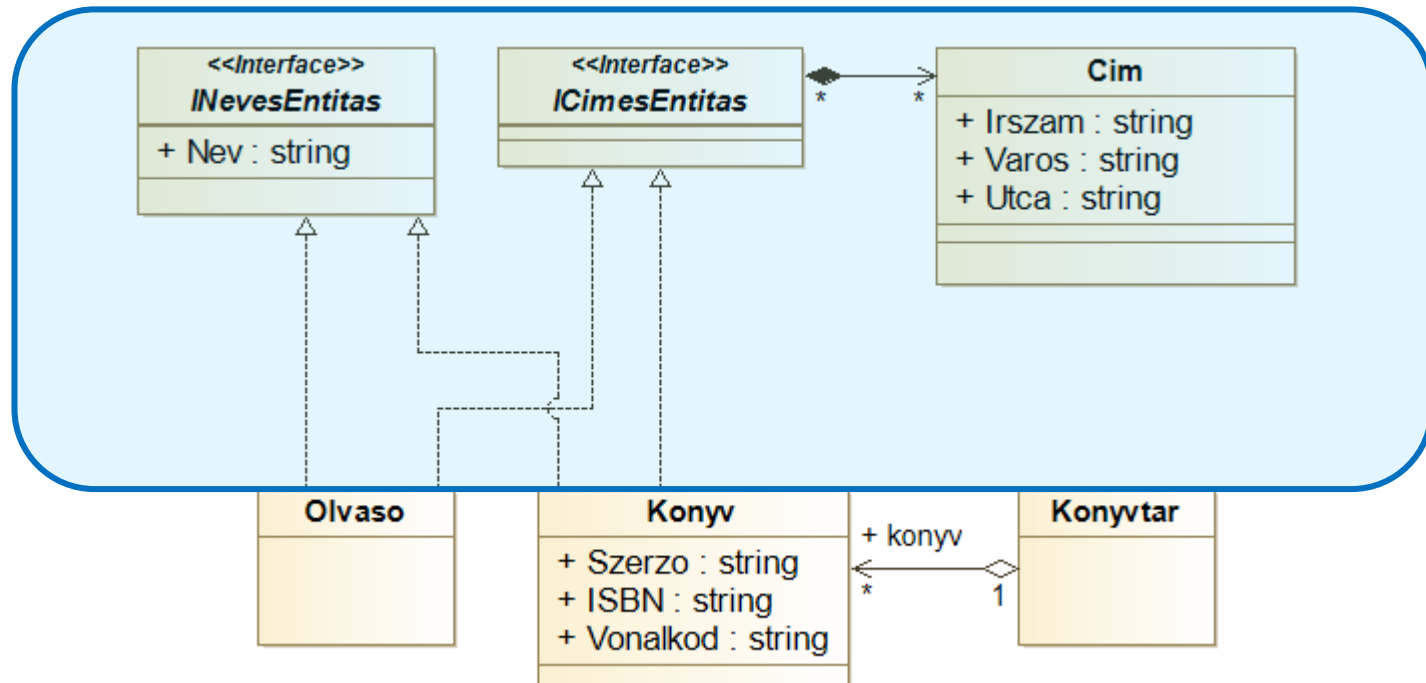
Könyvtár - adatok

- A könyveknek van szerzője, címe és ISBN száma
 - > Mi a könyv példány? Mi a könyv osztály?
 - > ISBN szám vs. egyedi azonosító
 - ISBN: Statikus?



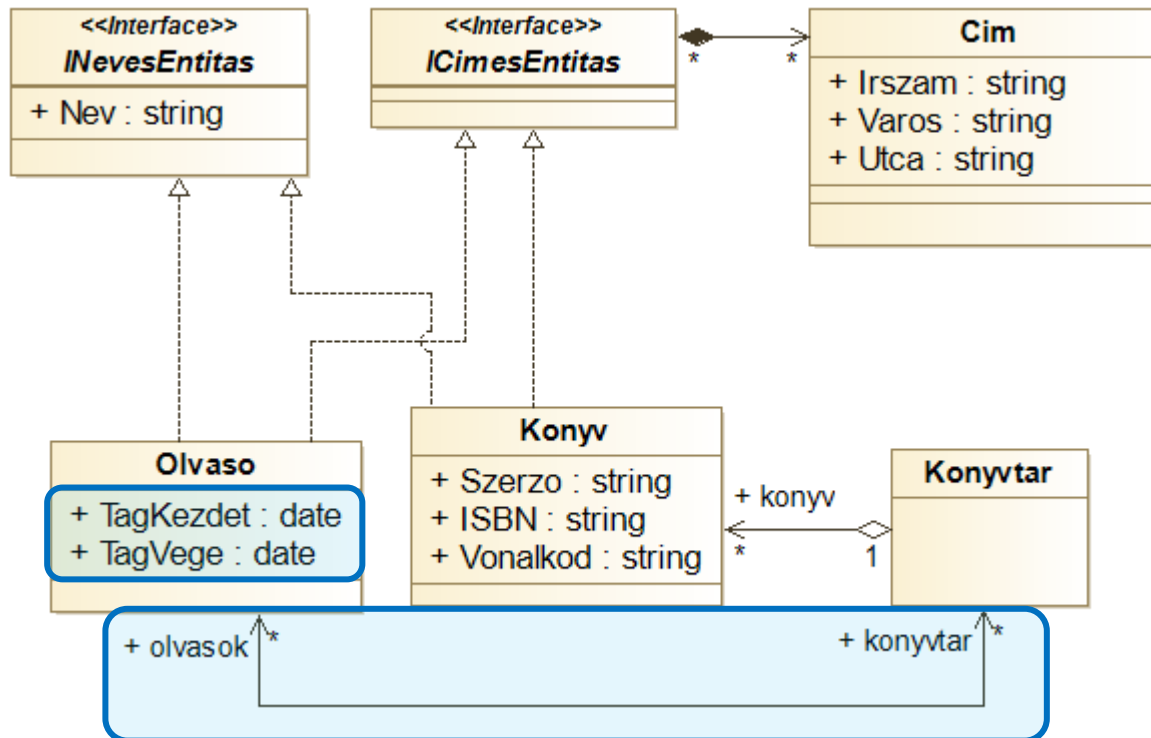
Könyvtár - adatok

- A könyvtárnak és az olvasónak van neve és címe
 - > Közös ős?
 - > Vagy... közös interfész? Egy interfész, vagy kettő?
 - > Vagy... csak egy saját, összetett típusú attribútum?
 - > Vagy... összetett attribútum + interfész?



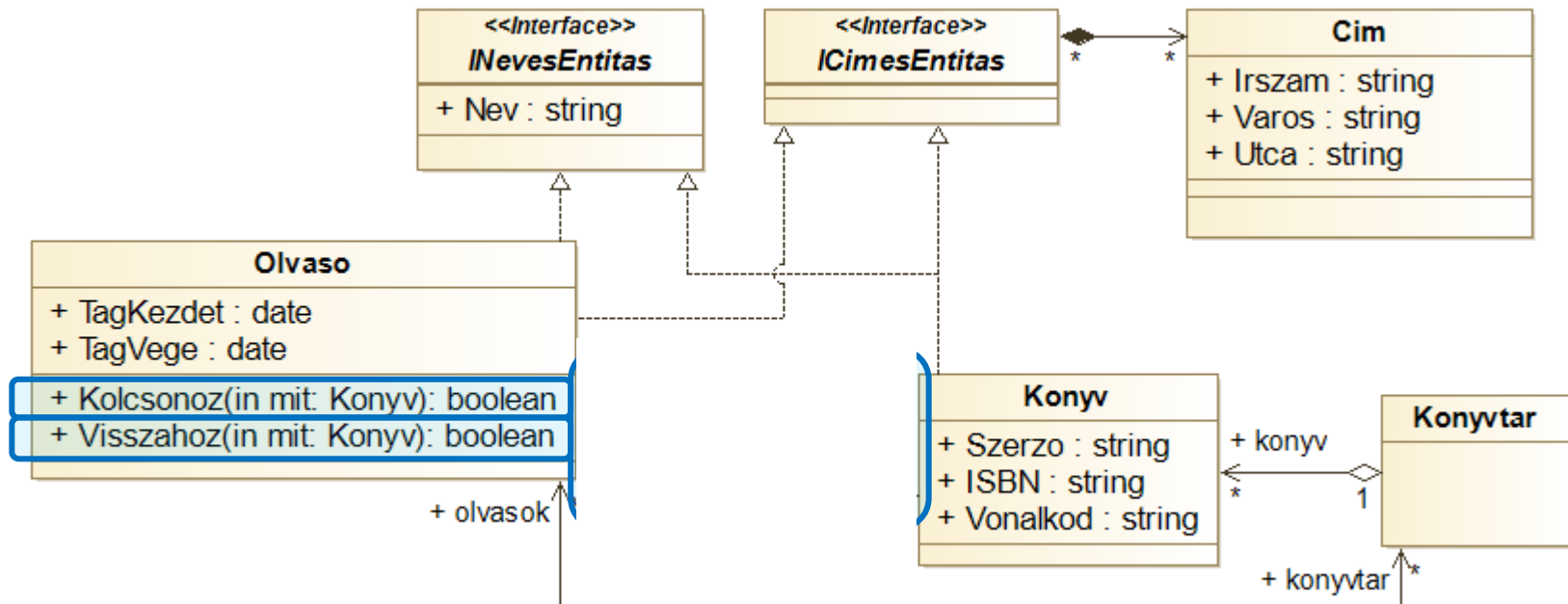
Könyvtár - adatok

- Az olvasó tagságának van kezdete és vége
 - > A könyvtár-olvasó kapcsolatot is érdemes lenne számontartani!



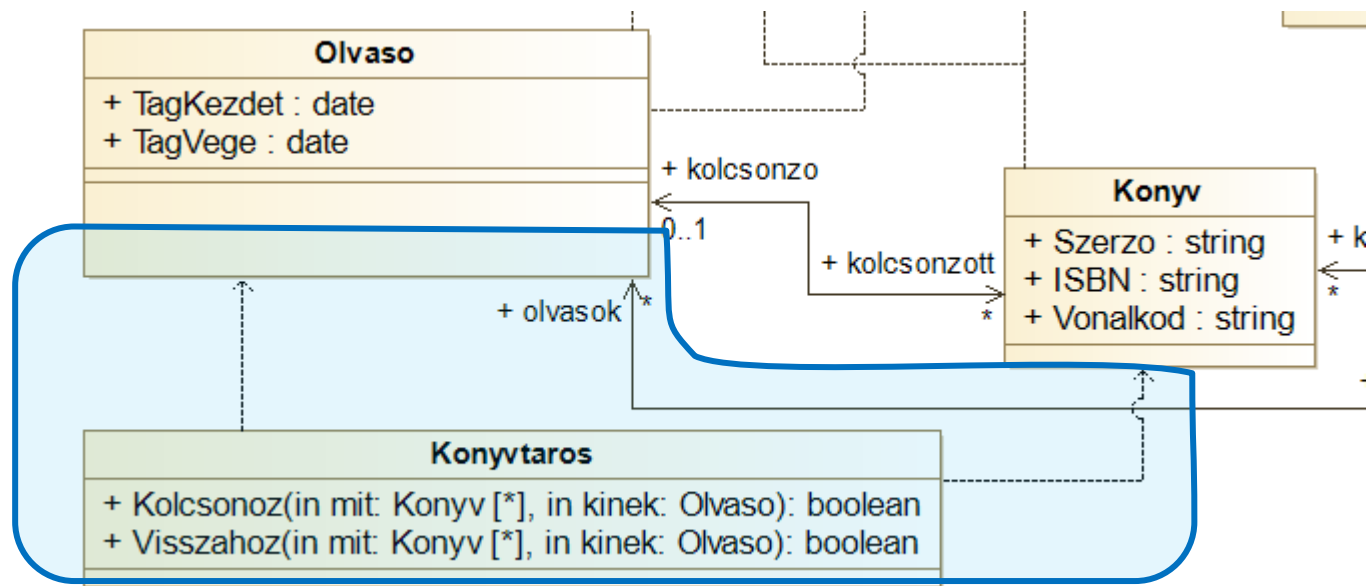
Könyvtár - kölcsönzés

- Az olvasó kölcsönözhet könyveket.
 - > Meg kell jegyezni, kinél van az adott könyv
 - > Tartalmazás vagy asszociáció?
 - > Vissza is kell tudni hozni



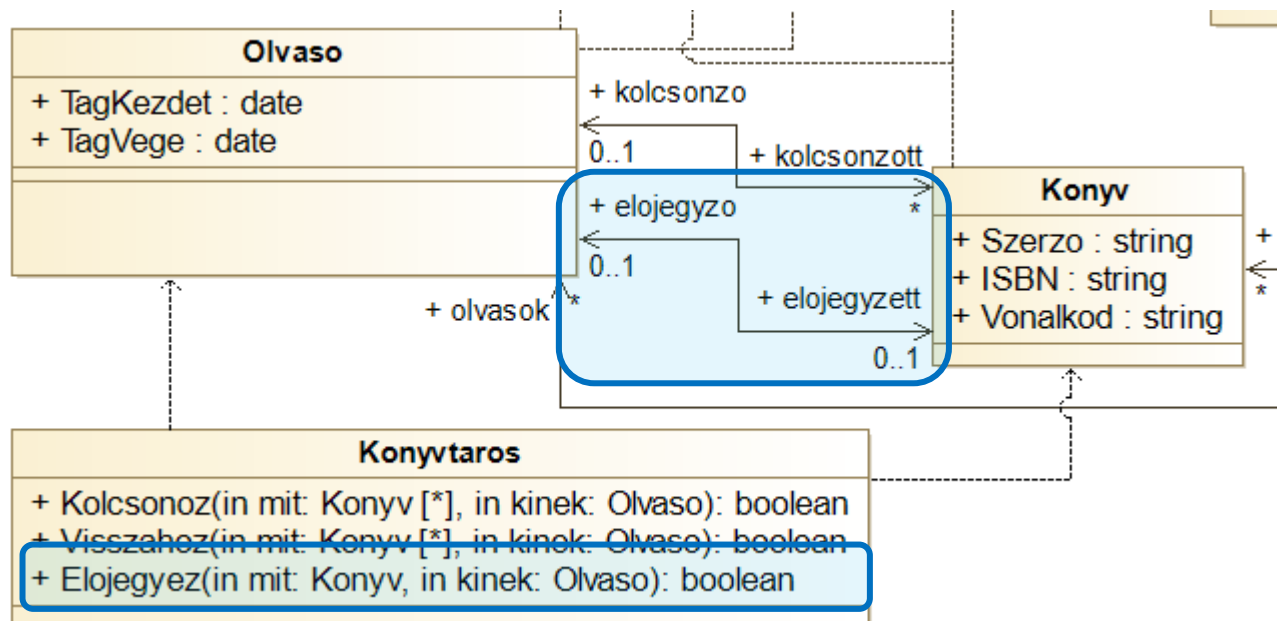
Könyvtár – könyvtáros

- Az olvasó kölcsönözhet könyveket.
 - > Az olvasó kölcsönzi?
 - > ... vagy a könyvtár?
 - > ... vagy a könyvtáros?



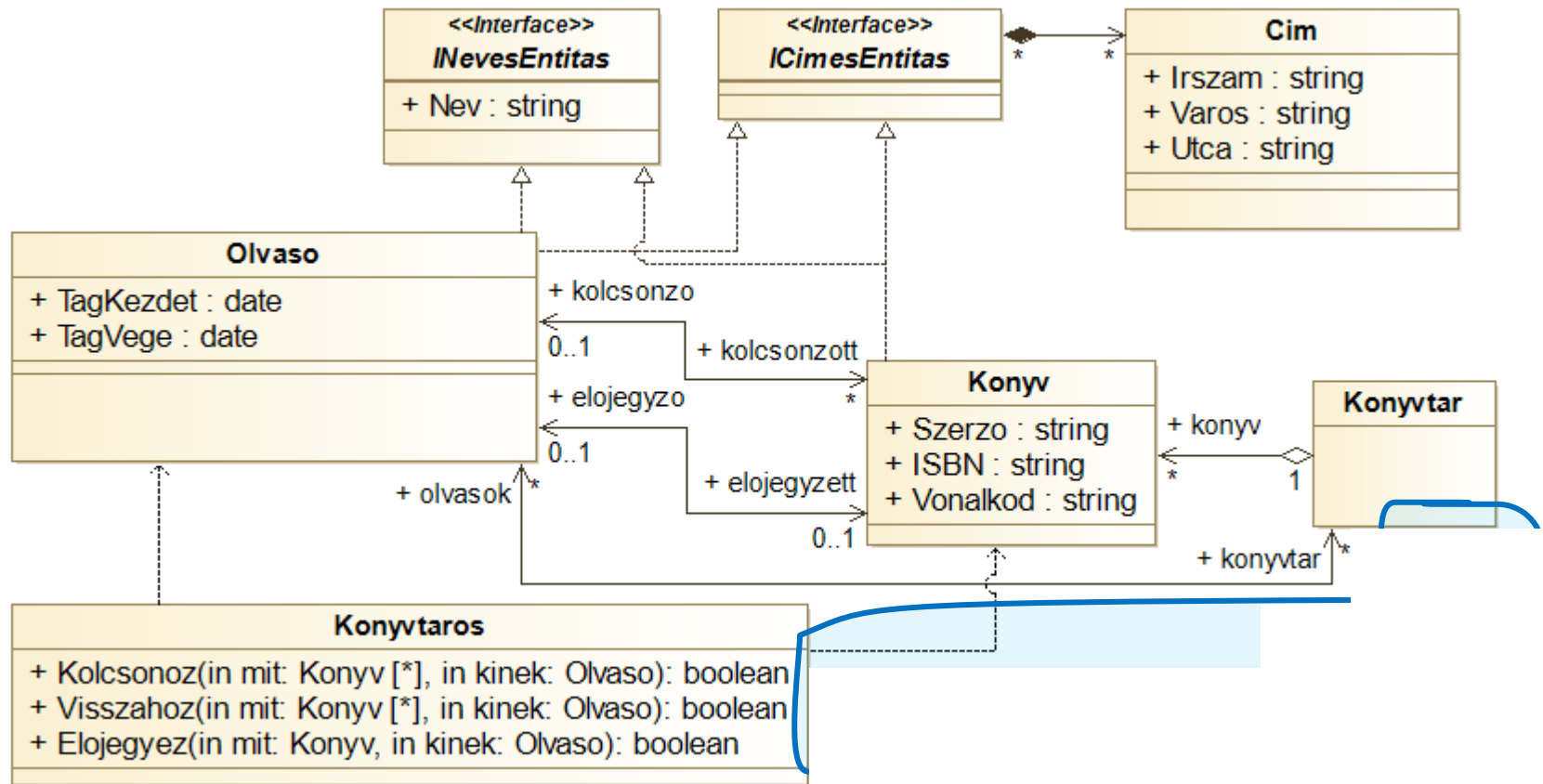
Könyvtár - előjegyzés

- Az olvasó elő is jegyezhet könyvet, de csak egyet!
 - > Az előjegyzés műveletet is támogatni kell
 - > Kiemeljük a könyv-olvasó adatpárost külön típusba?



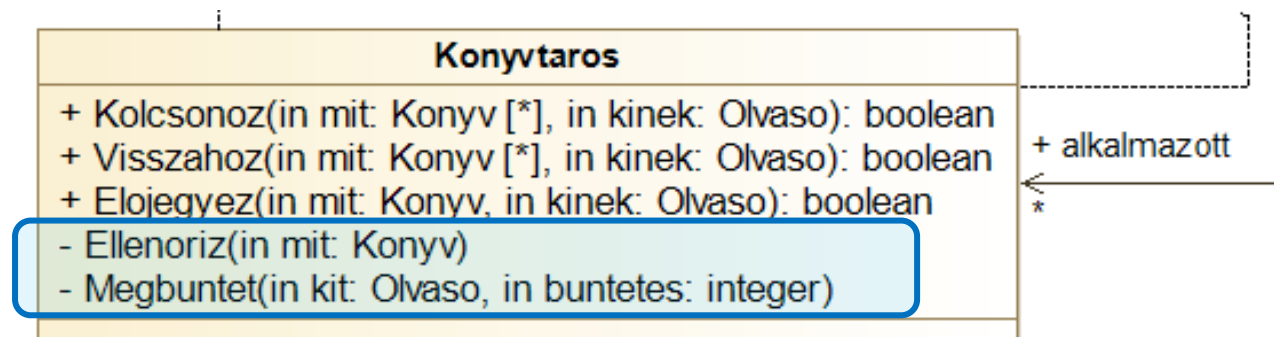
Könyvtár – könyvtáros

- A könyvtáros munkahelye a könyvtár



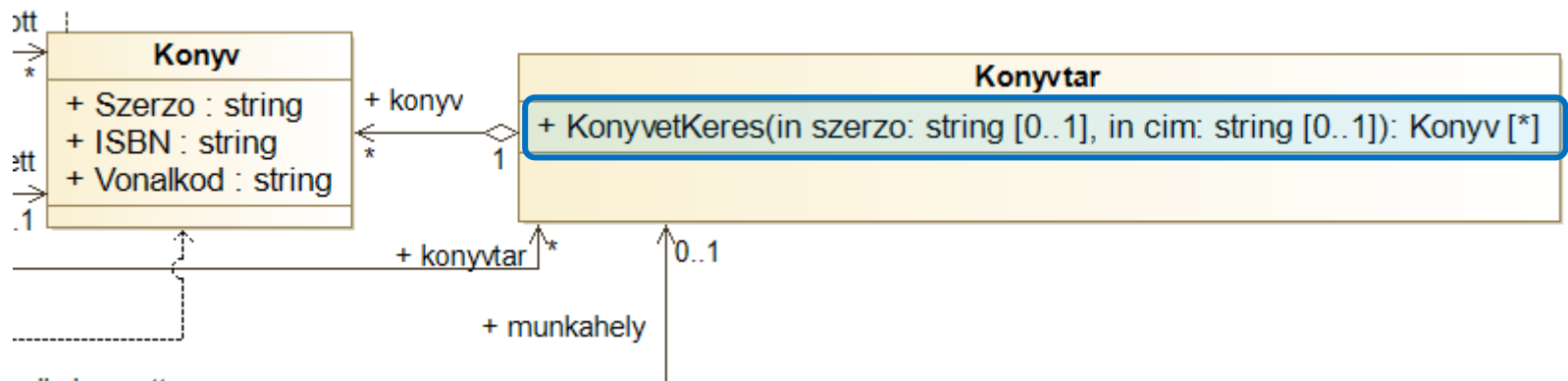
Könyvtár – könyvtáros

- A könyvtáros tudja a könyvet ellenőrizni és büntetést kiszabni, ha szükséges.
 - > Publikus művelet?



Könyvtár - keresés

- Lehessen könyvet keresni a könyvtárban szerző és cím szerint (mindkét parameter elhagyható)!
 - > Mi legyen a visszatérési érték?
 - > Az olvasó, a könyvtáros, vagy a könyvtár keres?



Szekvenciadiagram

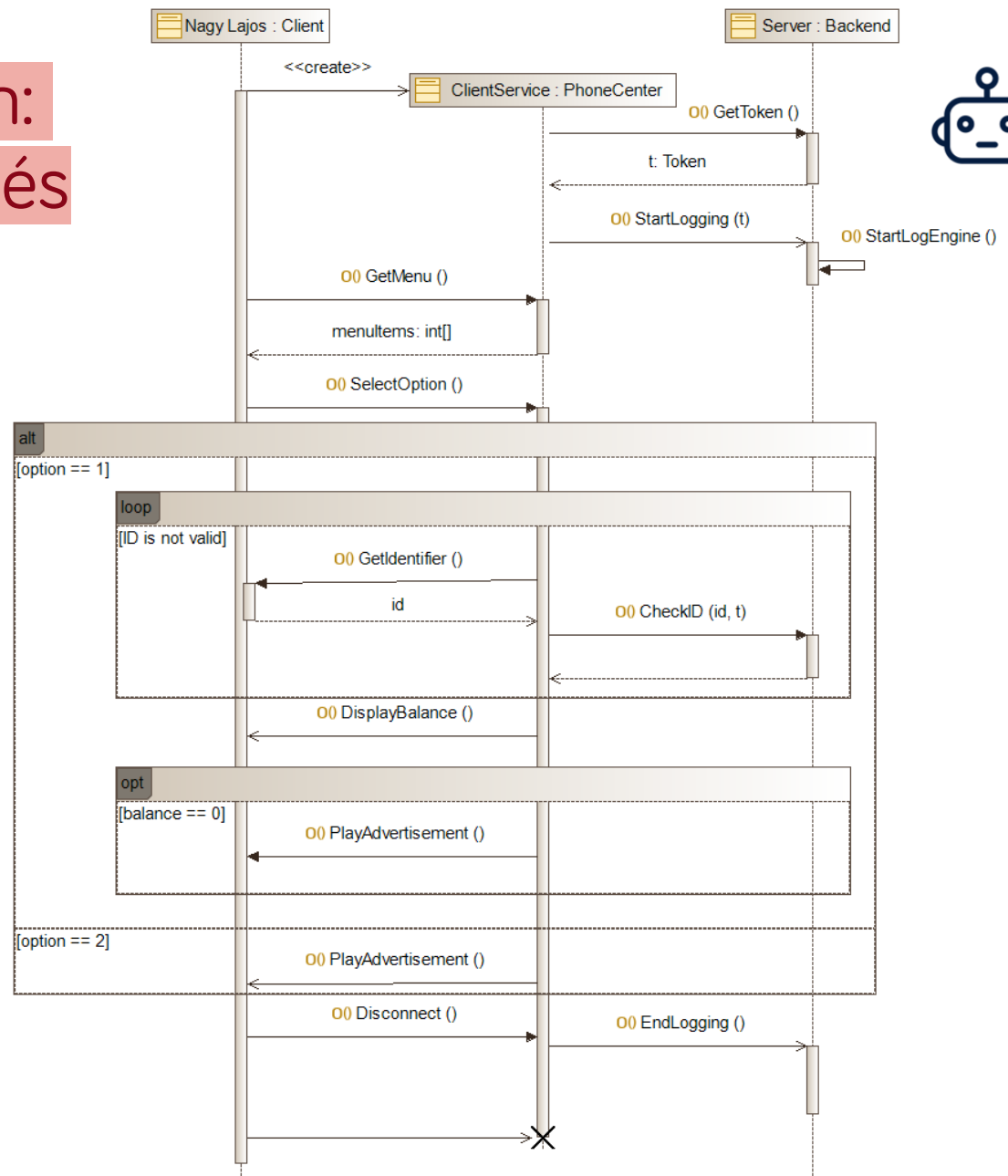
Struktúra vs. viselkedés

- Osztálydiagram
 - > Struktúra, adatszerkezet
 - > Metódusok
 - > Statikus
- Hogyan írjuk le a dinamikus működést?
 - > Szekvenciadiagram
 - > Állapotgép
 - > Aktivitásdiagram
 - > Használati eset diagram

Szekvenciadiagram

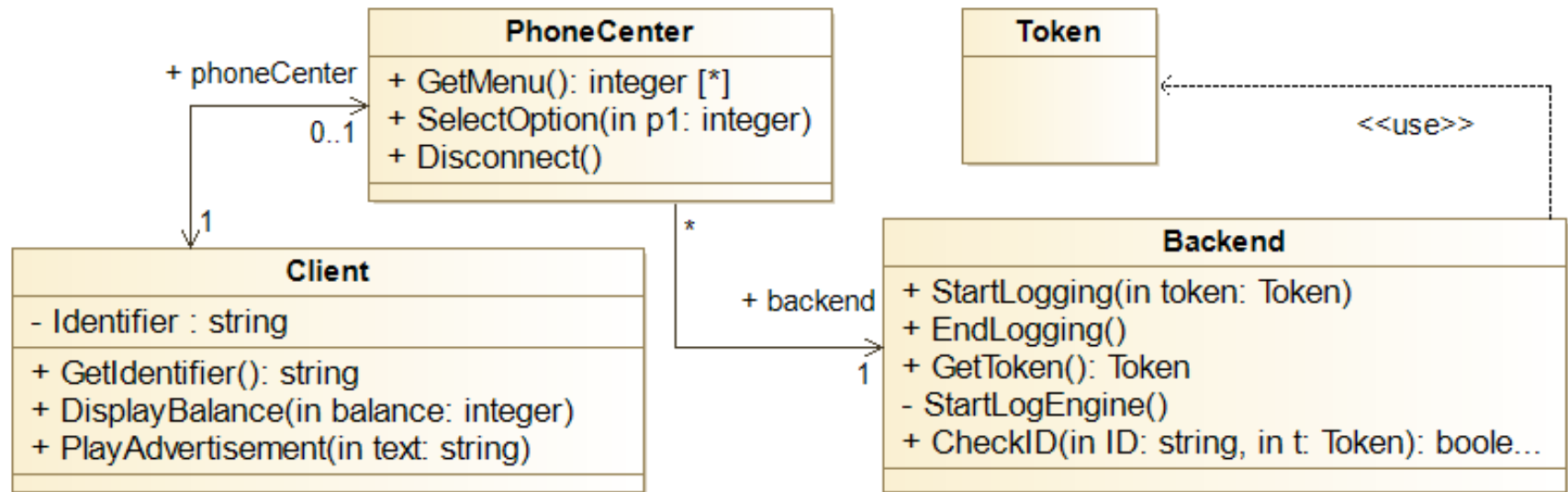
- A folyamatok, műveletek sorrendjére fókuszál
 - > Üzenetek, válaszüzenetek
 - > Interakciók
 - > Információcsere
- Rendszer-, és metódusszinten is alkalmazható
- Egy adott lefutást ábrázol
 - > Ugyanazon résztvevők közt több lefutás is lehetséges (több szekvenciadiagram ábrázolható)
- Résztvevők: osztályok példányai és *aktorok*
 - > Nem “független”

Szekvenciadiagram: Telefonos ügyintézés



Osztálydiagram

- Értelmezzük az alábbi struktúrát!



Életvonalak és üzenetek

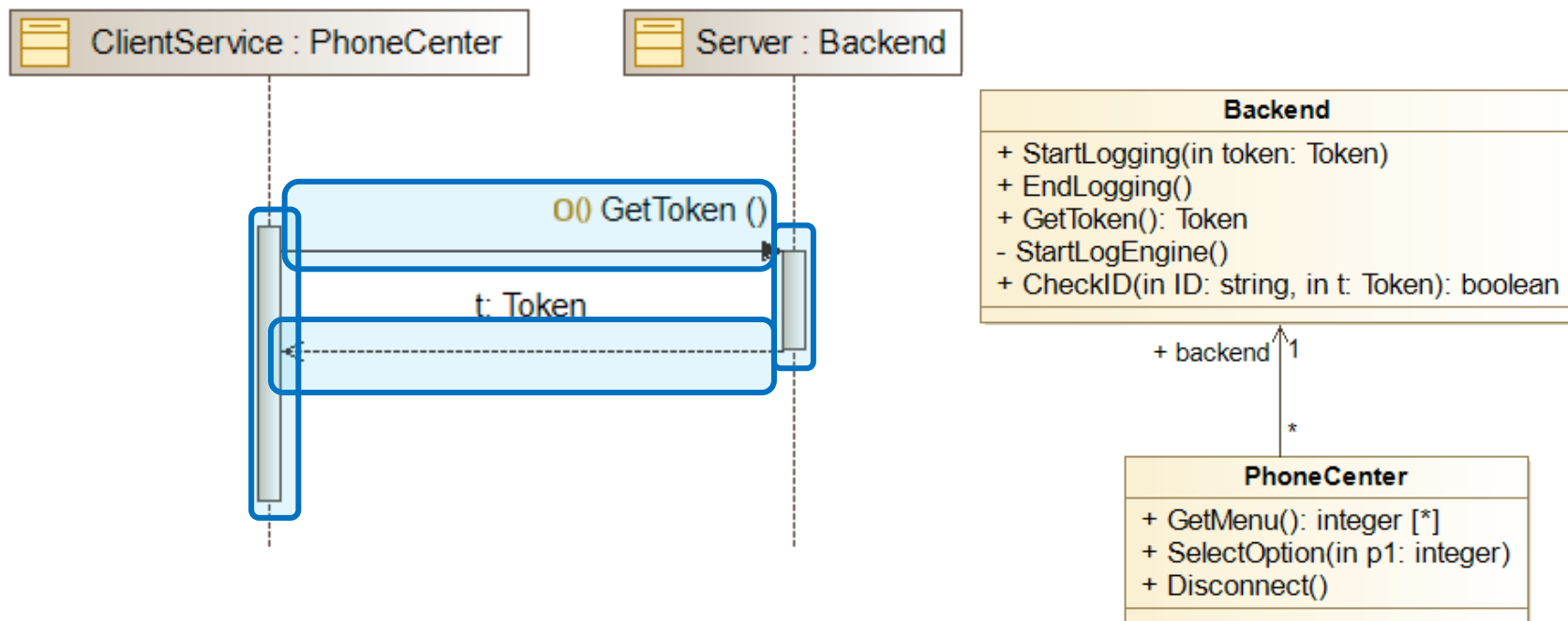
Életvonal

- Vegyünk egy konkrét ügyfelet, aki egy konkrét telefonos szervízzel kommunikál majd a server jóváhagyásával!
 - > Életvonal (Life line)
 - > Nem osztályok szerepelnek, hanem példányok!
 - > A diagram épít az osztálydiagramra!
 - > Idő fentről lefelé telik



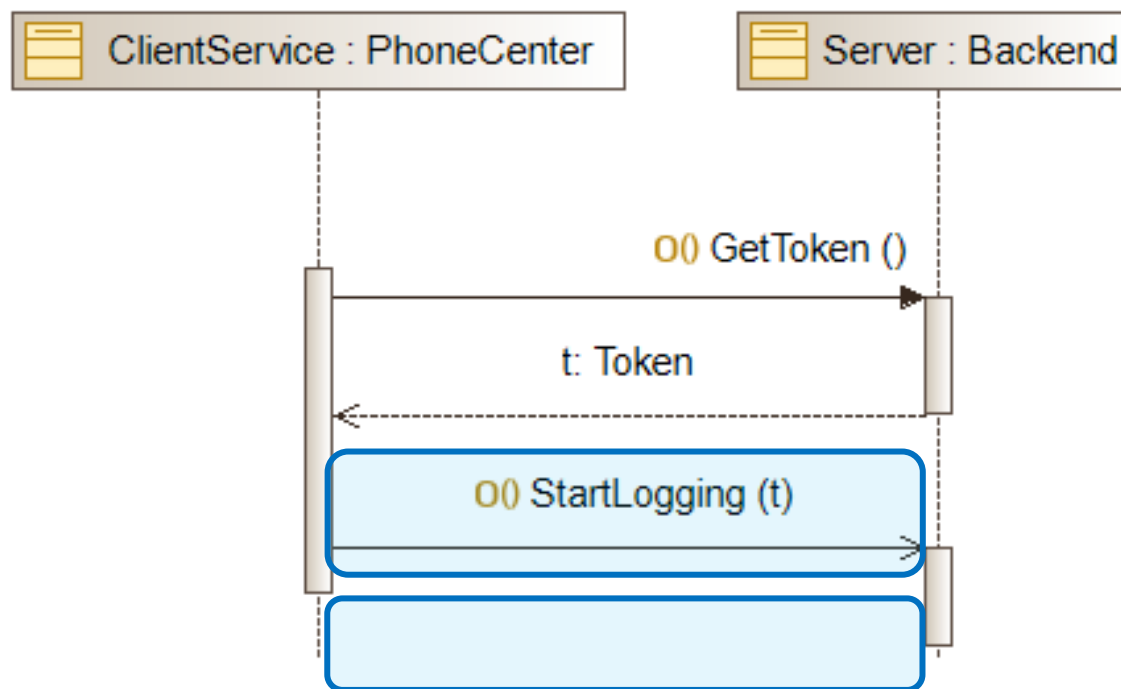
Üzenetek

- A szervíz kérjen el egy token a backendtől!
 - > Szinkron függvényhívás, visszatérési értékkel
 - > Futási specifikáció (execution specification)
 - > Üzenet (message)
 - > Válaszüzenet (return message)



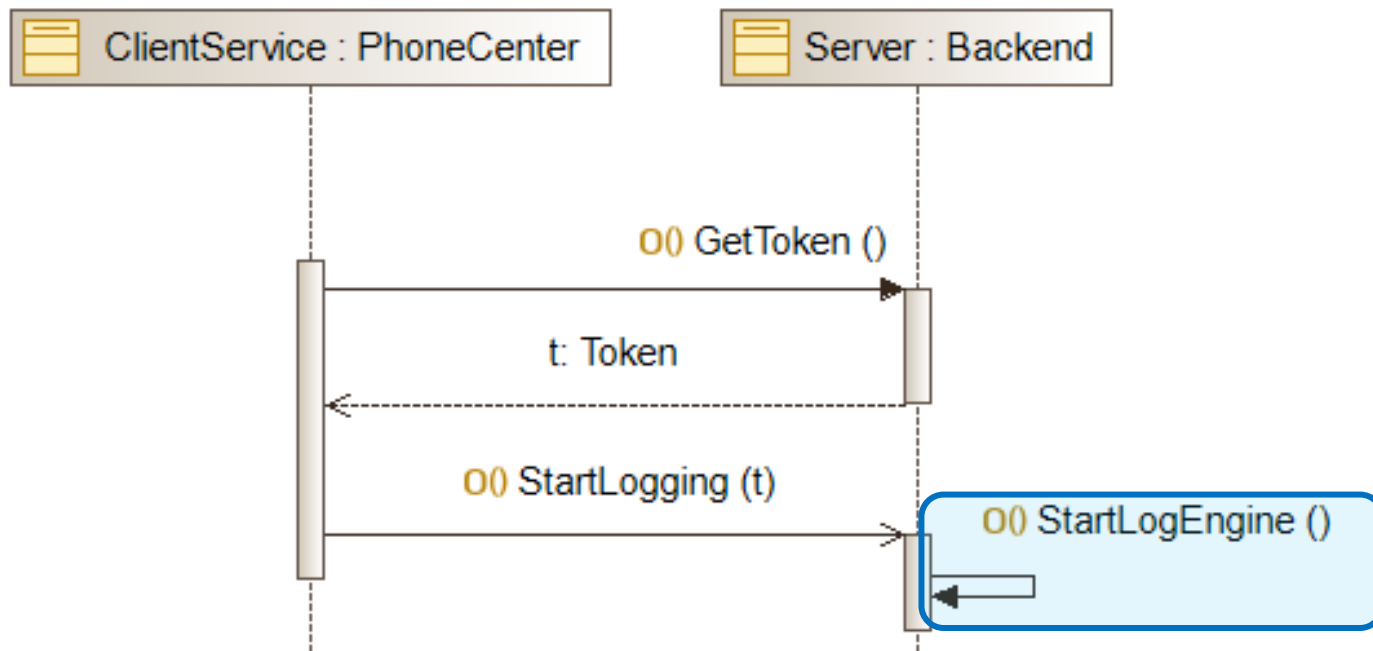
Üzenetek

- A szervíz indítsa el a loggolást a szerveren a token segítségével!
 - > Aszinkron függvényhívás, paraméterrel
 - > Nincs válaszüzenet!



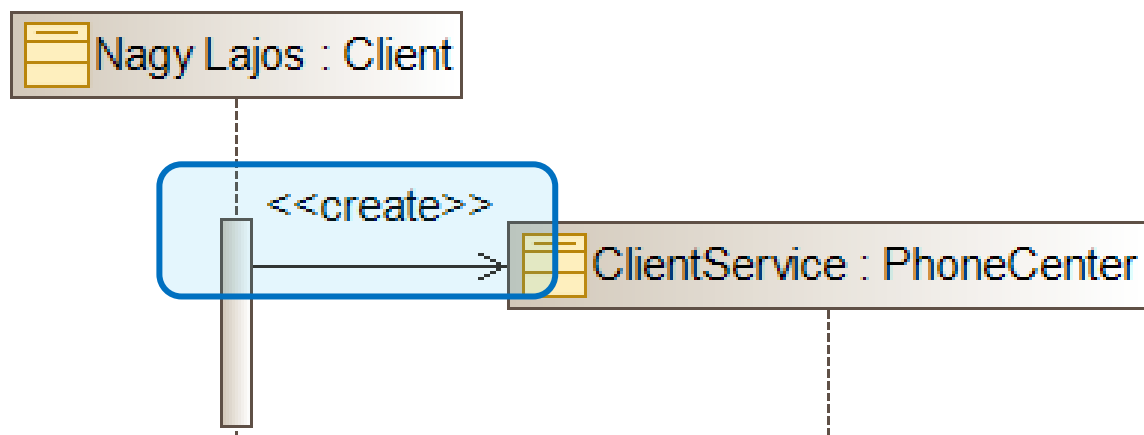
Üzenetek

- A szerver belső folyamatként indítsa el a loggoló metódusát, amikor felkérés jön a szervíztől!



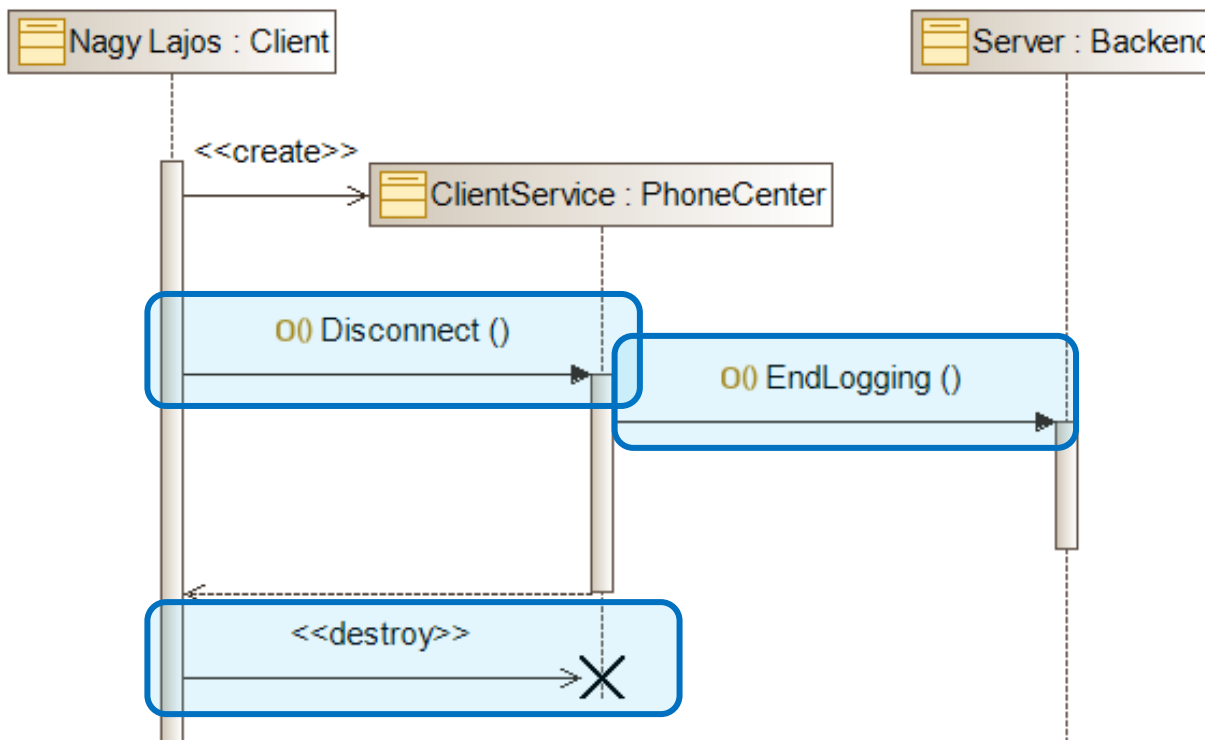
Üzenetek

- A szervíz csak a hívás elején jön létre!
 - > Létrehozási üzenet, konstruktor
 - > “Create” sztereotípia
 - > Később létrejövő életvonal



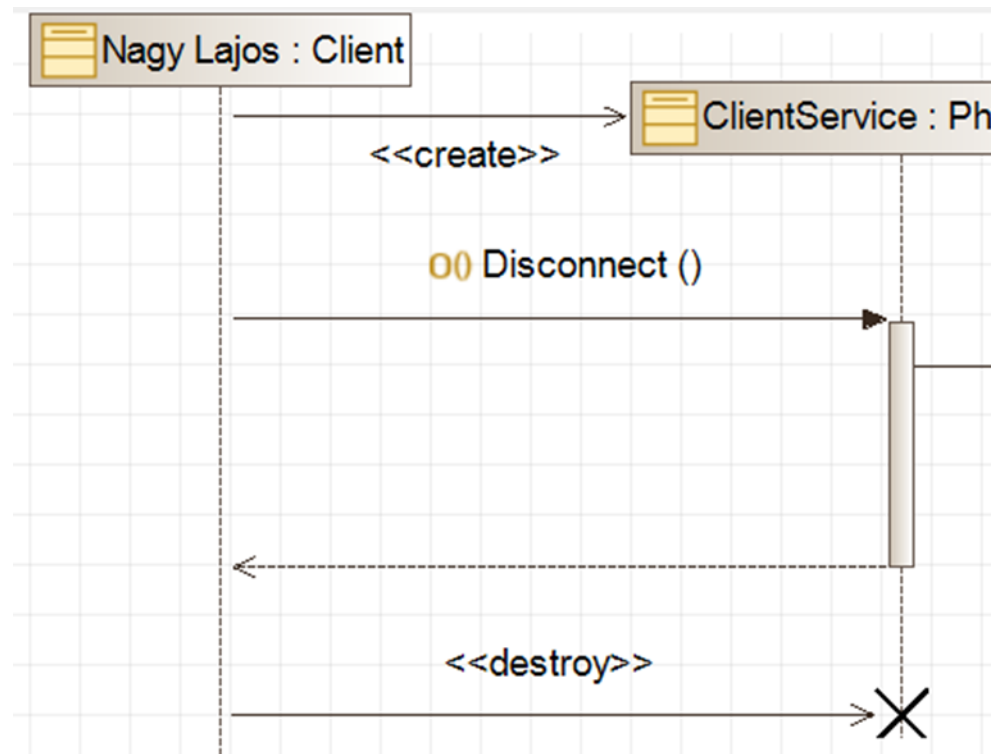
Üzenetek

- Modellezük le a hívás végét
 - > Hívó értesíti a szervízt, ami leállítja a loggolást
 - > Majd megszünteti a szervízt (destruktor)



Életvonalak és üzenetek

- **Életvonal**
 - > [Név]: [típus]
- **Üzenet**
 - > Név([params])
 - > Szinkron/aszinkron
 - > Létrehozás/
megszűntetés
- **Futási/végrehajtási
specifikáció** (execution specification)

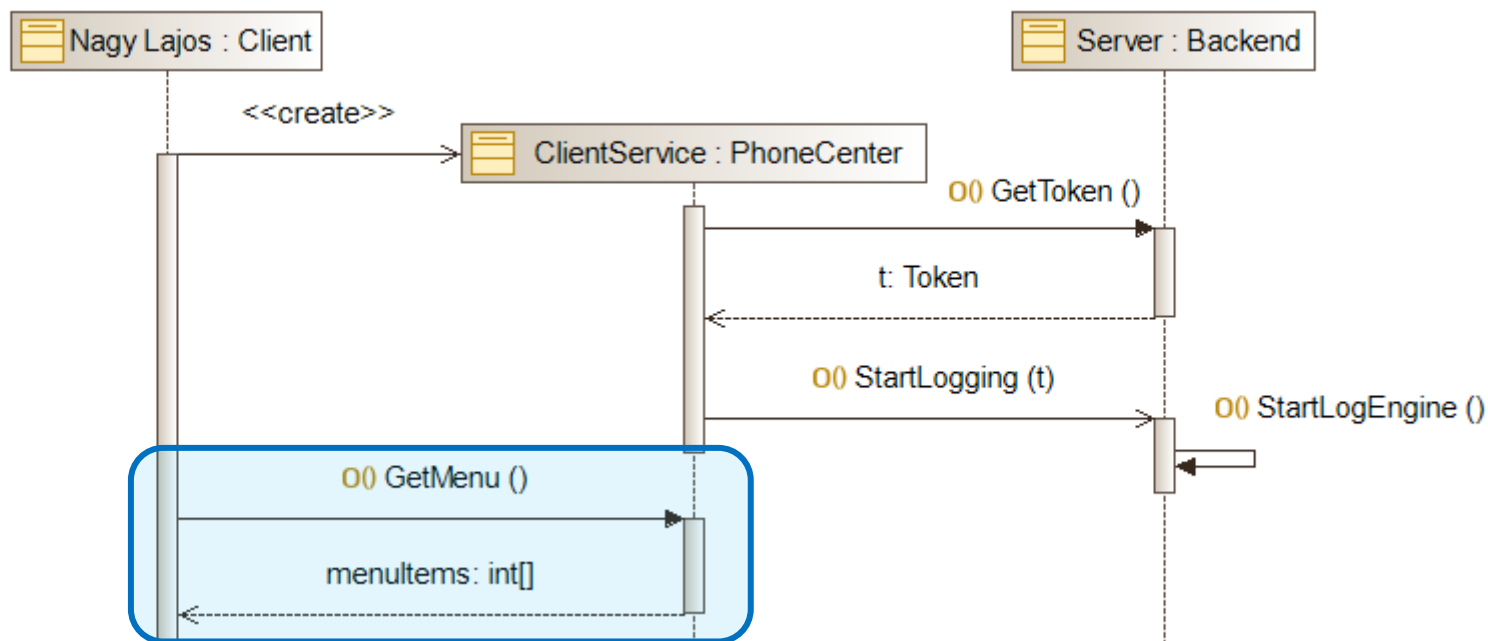


Blokkok

Combined fragments

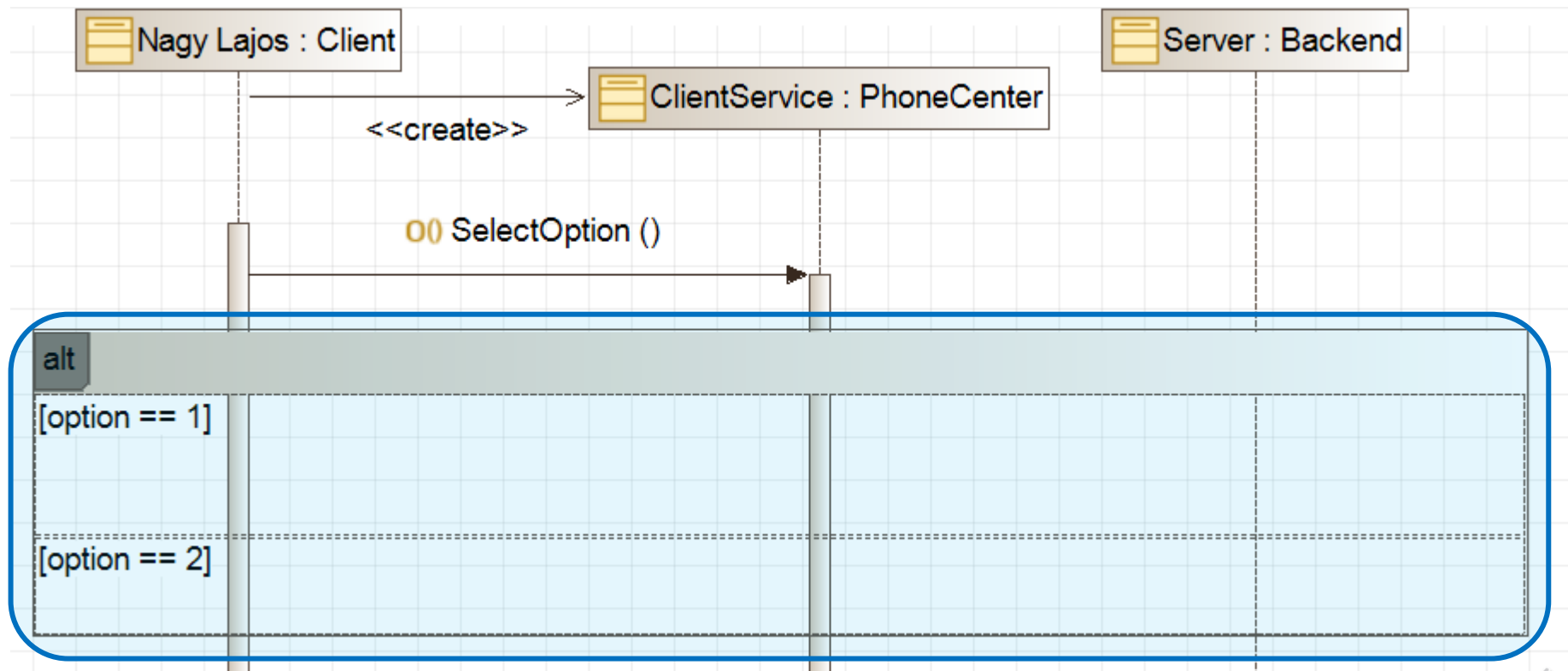
Üzenetek

- A hívó kérhesse le a menüpontokat a szervíztől!



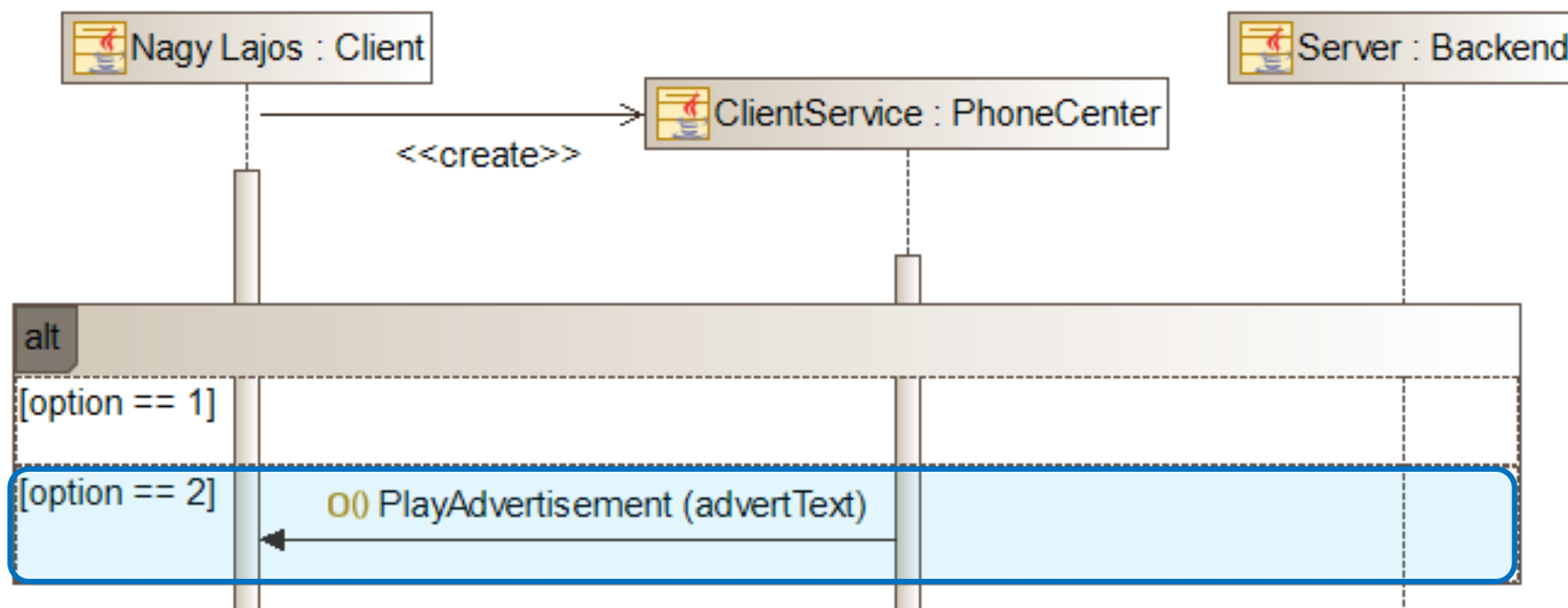
Üzenetek

- Válasszuk szét a műveleteket a hívó választásától függően!
 - > Összetett részlet (combined fragment)
 - > Alternatív blokk (alt) (if – then – else jellegű kompozíció)



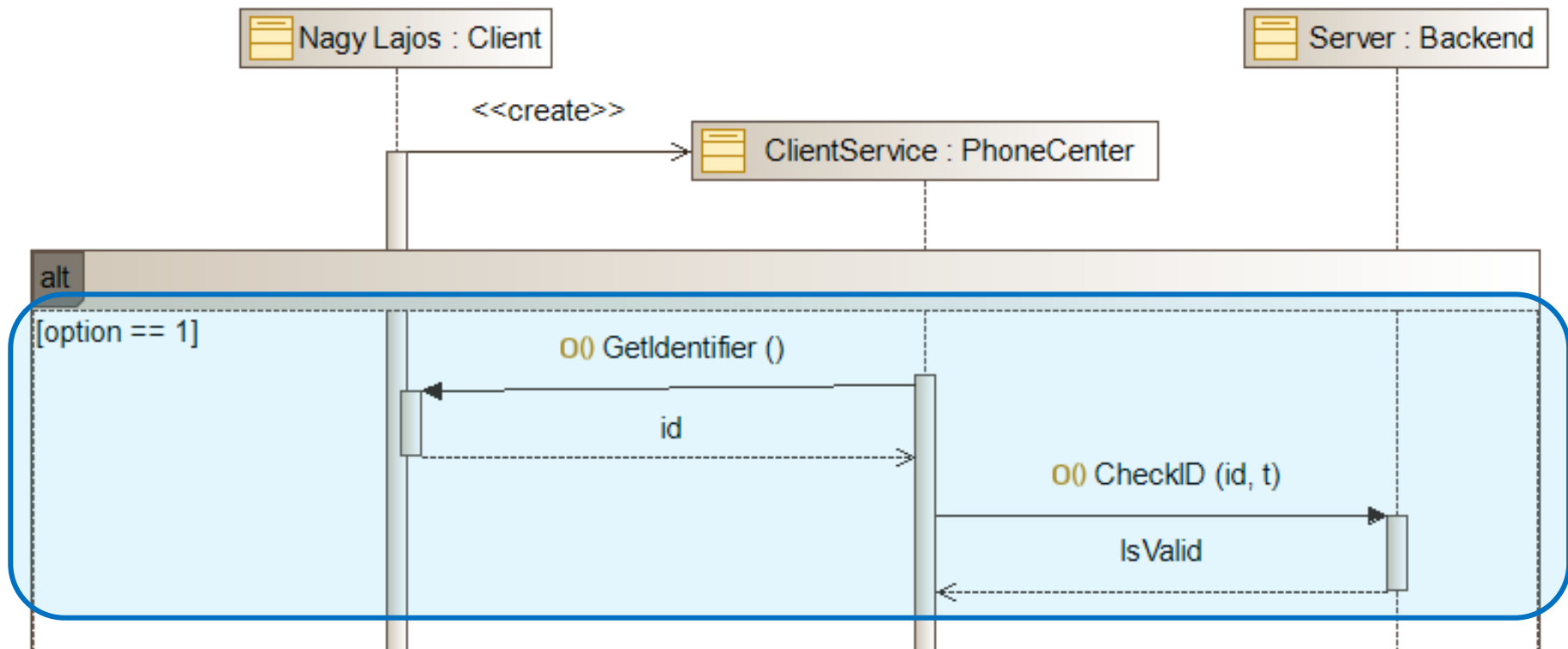
Üzenetek

- A “2-es” menüpont játszon be reklámokat!



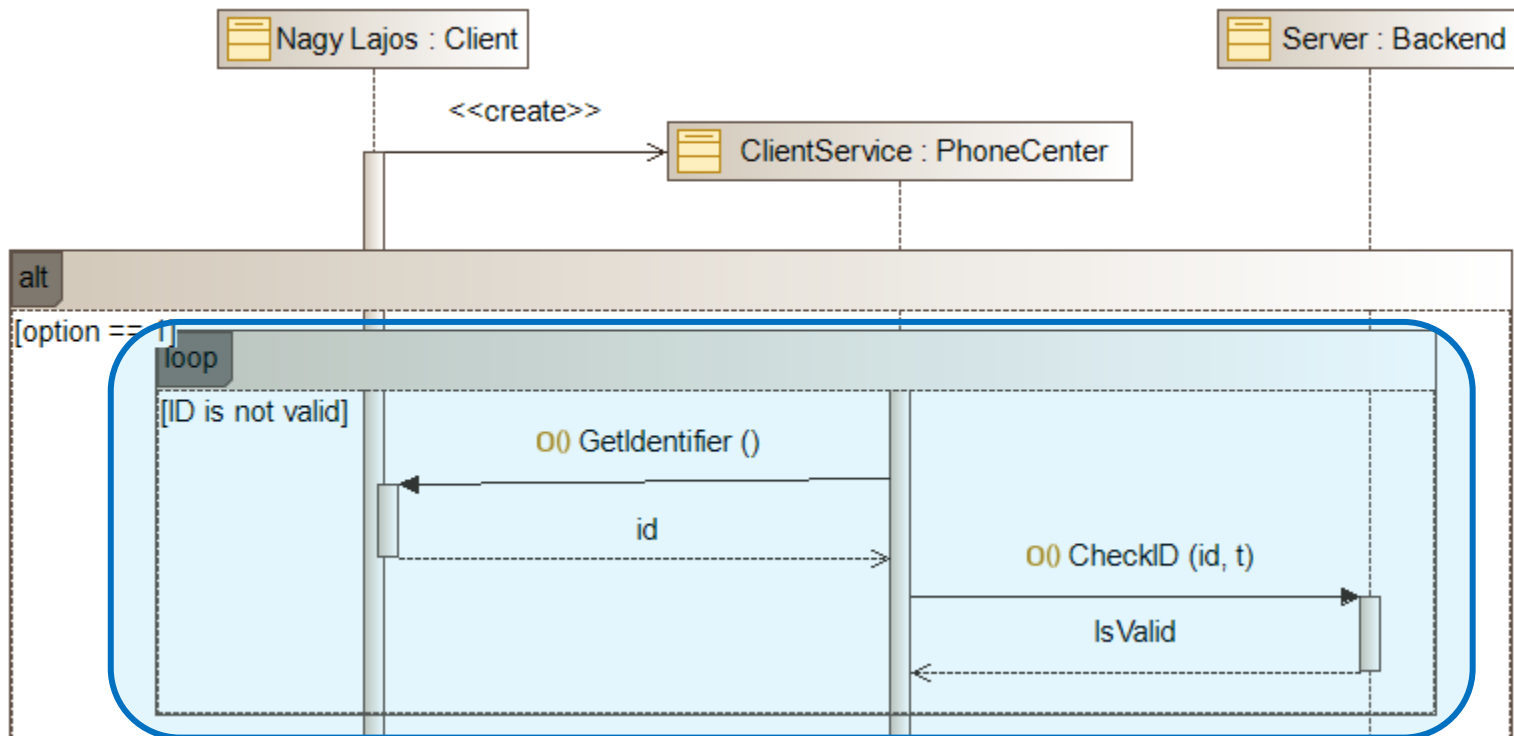
Üzenetek

- Az “1-es” menüpontban kérjük be a hívó azonosítóját és ellenőrizzük is le!



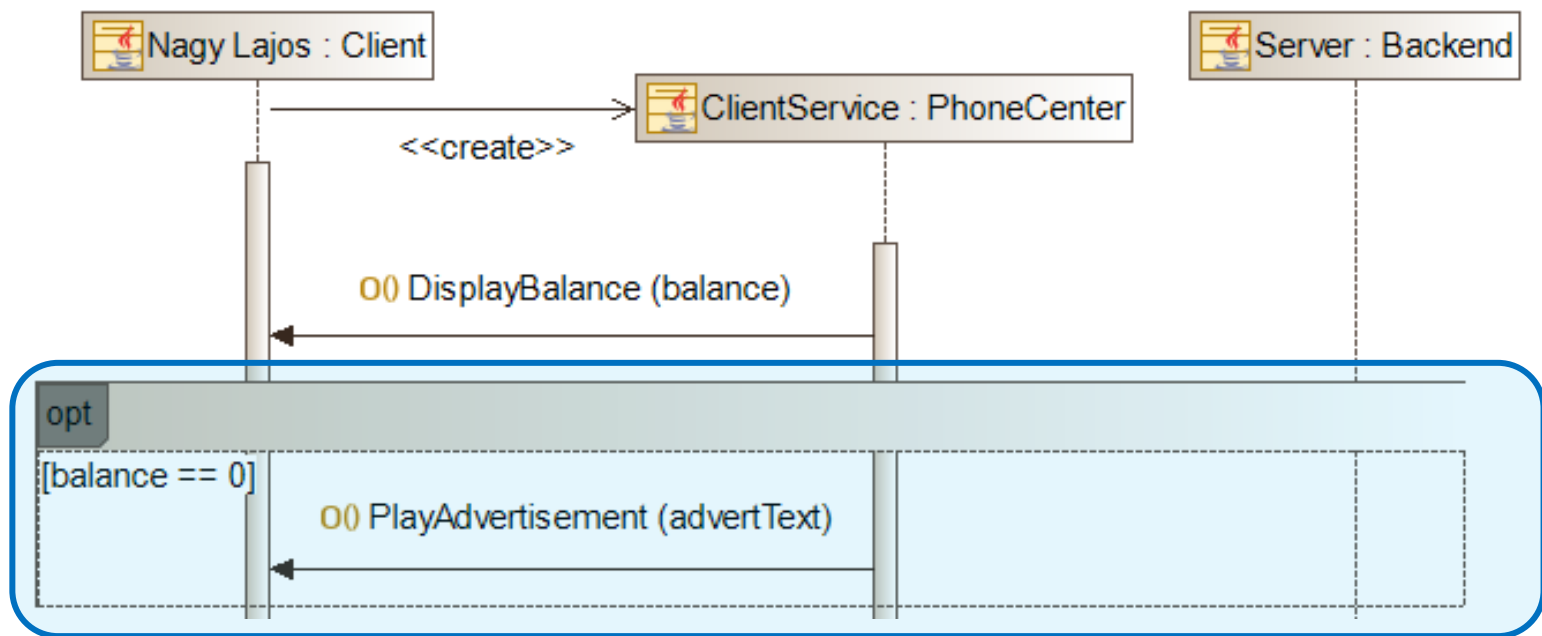
Üzenetek

- Az azonosítót mindaddig újra bekérjük, amíg az nem helyes!
 - > **Ciklus blokk (loop)** (feltétel szerinti ismételés)



Üzenetek

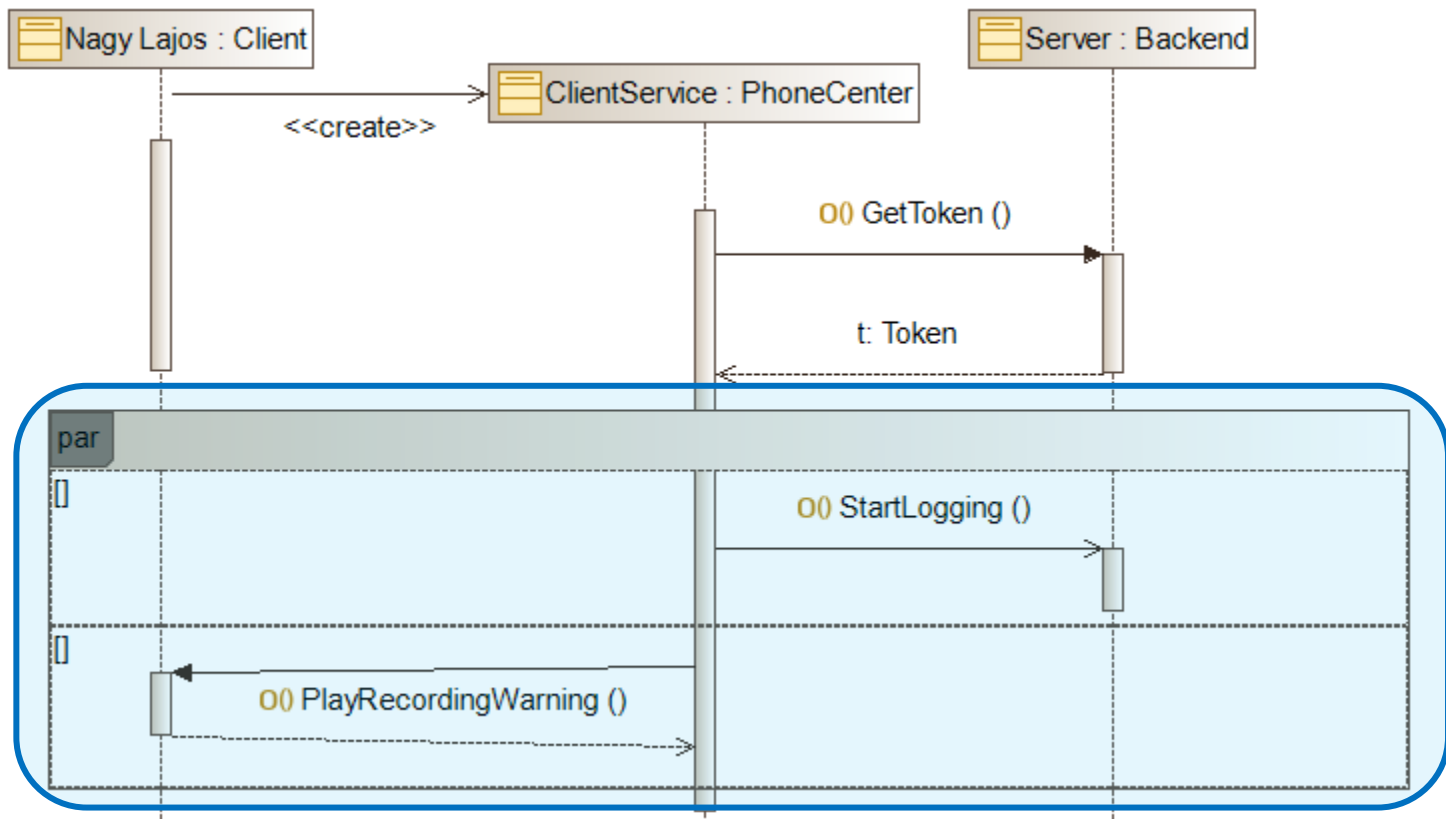
- Mutassuk meg a hívónak az egyenlegét és ha üres, automatikusan játszunk le neki reklámot!
 - > Opcionális blokk (opt) (if else nélkül)



Párhuzamosság

- A beszélgetés kezdetén a loggolás elindításával egyidőben játszunk le egy figyelmeztetést a hívónak, hogy rögzítjük majd a beszélgetést!

> Párhuzamos blokk (par)



Blokkok

- Kombinált részletek (Combined fragments)
 - > Alt: Feltételes elágazás (if – else if – else szerkezet)
 - > Loop: Feltétel szerinti előltesztelő ciklus
 - > Opt: Opcionális utasítások (else ág nélküli if)
 - > Par: Párhuzamosan végrehajtandó utasítások
 - > Seq, Critical, Assert, ... (nem tanuljuk)

Összefoglalás

- Modellezés
 - > Statikus nézet, szerkezet, interfészek -> Osztálydiagram
 - > Metódusok egymás utáni lefutása -> Szekvenciadiagram
- Szekvenciadiagram
 - > Életvonalak
 - > Üzenetek
 - > Blokkok

Köszönöm a figyelmet!

Szoftvertechnológia és -technikák

4. Előadás

Aktivitásdiagram, Állapotgép

Osztálydiagram és szekvenciadiagram

Osztályok és interfészek

- Osztályok és interfészek

- > Fejléc

- > Tagváltozók

- [Láthatóság] [Név]: [Típus][Multiplicitás] [= kezdőérték]
 - Statikus: aláhúzás

- > Metódusok/Műveletek

- [Láthatóság] [Név]([paraméterek])(: visszatérési érték)

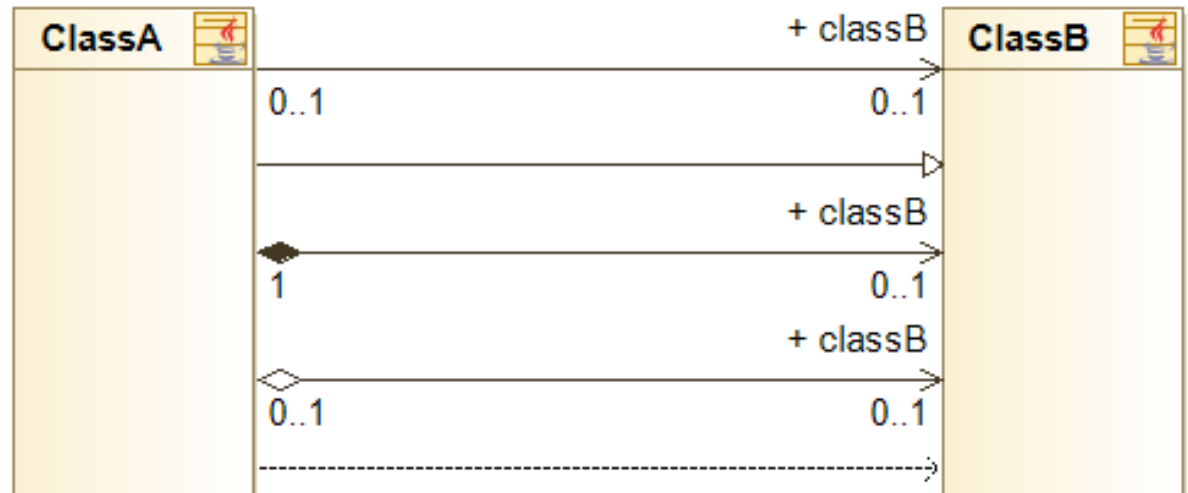
- > Láthatóság

- Public: +
 - Private: -
 - Protected: #
 - Package: ~

Kapcsolatok

- Kapcsolat típusok

- > Asszociáció
- > Általánosítás/
Öröklés
- > Kompozíció
- > Aggregáció
- > Függőség
- > Interfész
megvalósítás



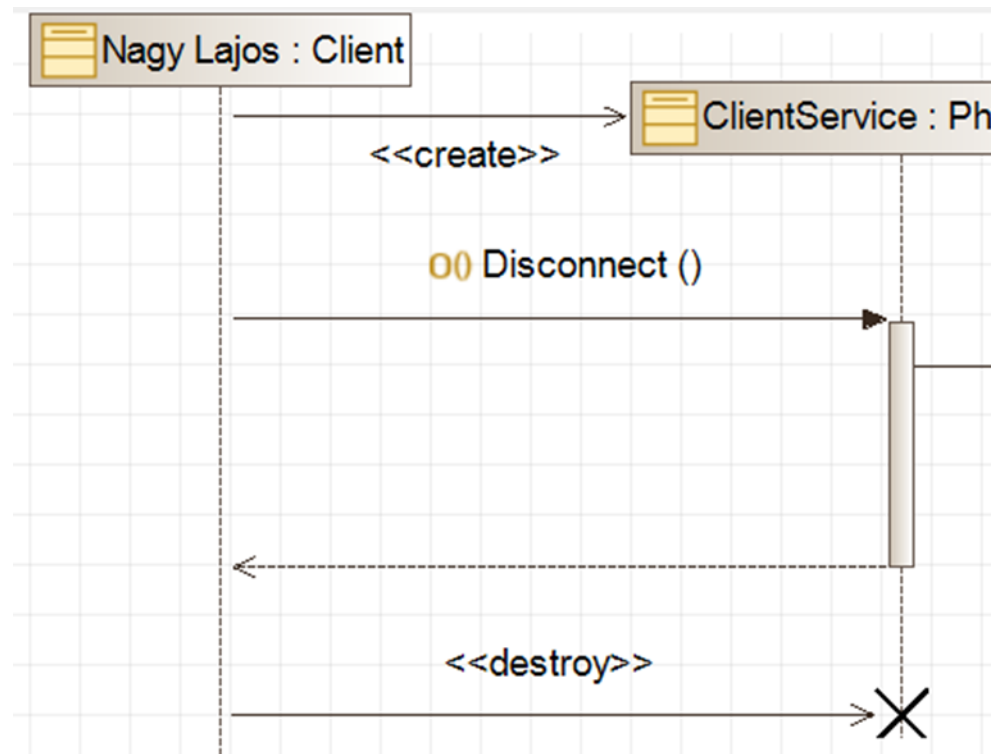
- Multiplicitás

- Navigálás

- > Szerepnév
- > Navigálhatóság

Életvonalak és üzenetek

- Életvonal
 - > [Név]: [típus]
- Üzenet
 - > Név([params])
 - > Szinkron/aszinkron
 - > Létrehozás/
megszűntetés
- Futási/végrehajtási
specifikáció (execution specification)



Blokkok

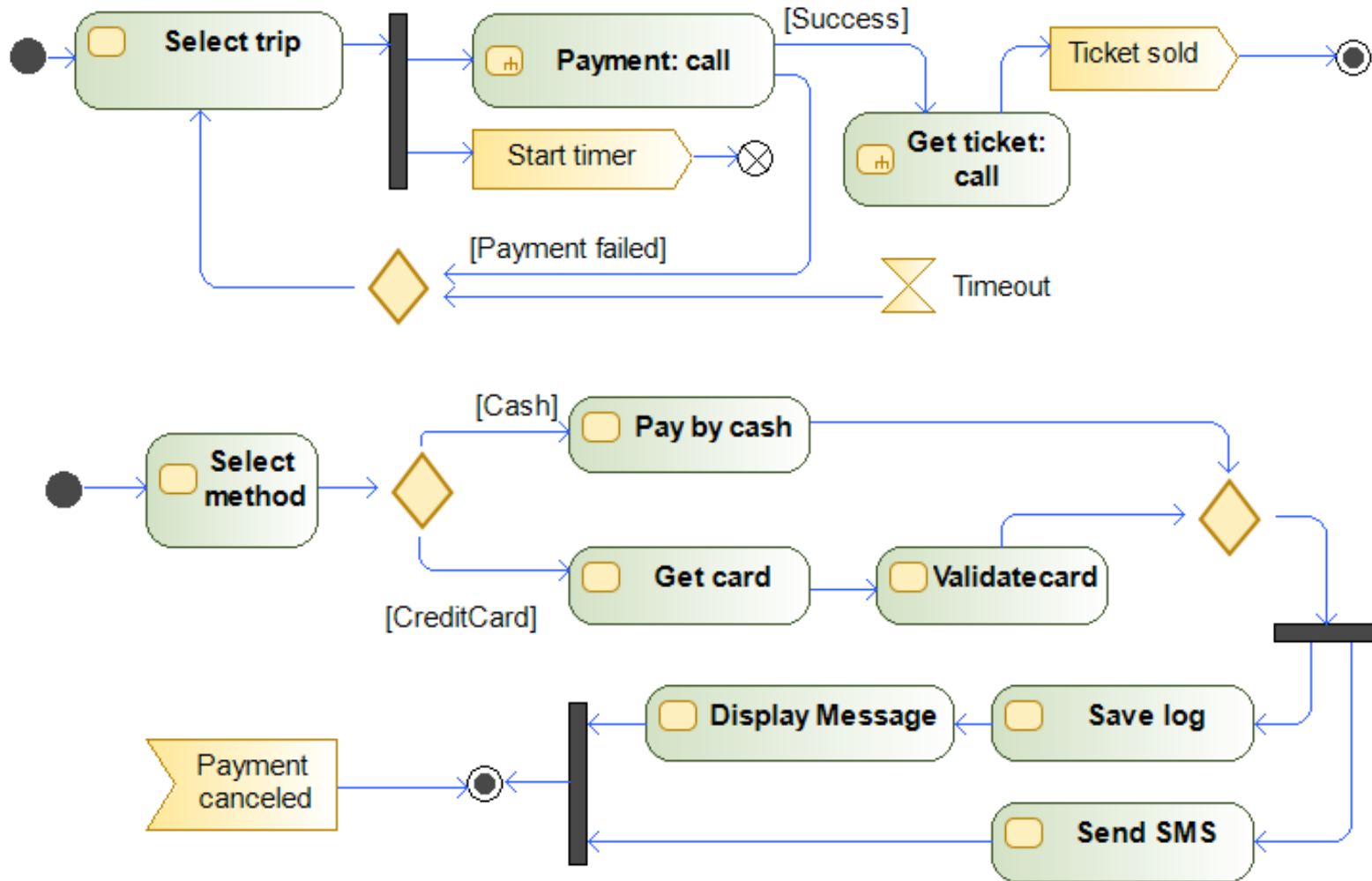
- Kombinált részletek (Combined fragments)
 - > Alt: Feltételes elágazás (if – else if – else szerkezet)
 - > Loop: Feltétel szerinti előltesztelős ciklus
 - > Opt: Opcionális utasítások (else ág nélküli if)
 - > Par: Párhuzamosan végrehajtandó utasítások
 - > Seq, Critical, Assert, ... (nem tanuljuk)

Aktivitás diagram

Aktivitásdiagram

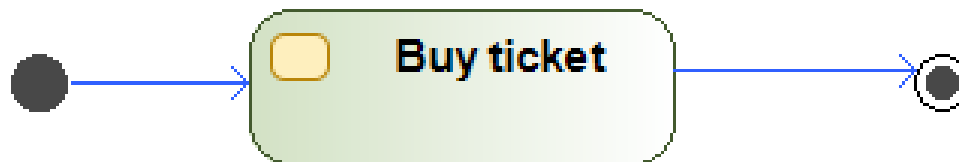
- Struktúra – osztálydiagram
- Műveletek sorrendje – szekvenciadiagram
 - > Szekvenciális végrehajtás
 - > Jellemzően függvényhívások
- Munkafolyamatok leírása – Aktivitás diagram
 - > Workflow, control flow
 - > Elágazások, párhuzamos ágak, feltételek
 - > Fókusz a funkciókon, a folyamatokon
 - > Magasabb szintű, több használati esetet is összefoghat
 - > <https://www.visual-paradigm.com/guide/uml-unified-modeling-language/what-is-activity-diagram/>

Aktivitásdiagram: BKV jegyvásárlás



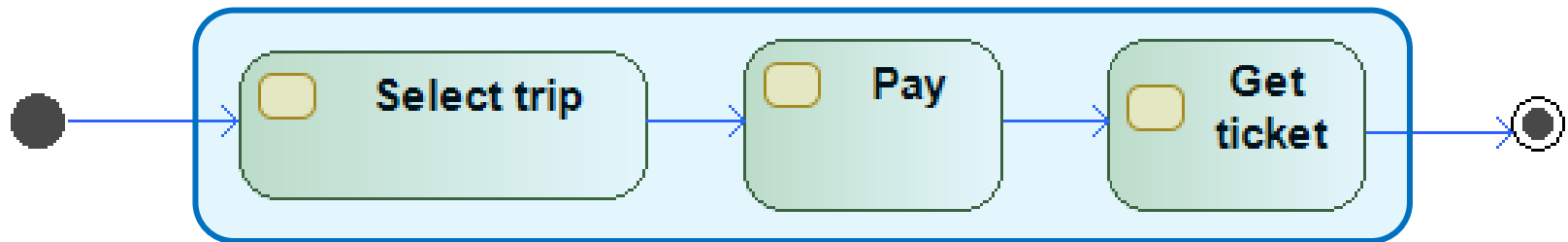
Jegyvásárlás

- Modellezzük a jegyvásárlást!
 - > **Kezdő csomópont (initial node)**
 - Folyamat kezdése
 - Nincs bemenő él
 - Több kezdőpont: párhuzamos kezdés
 - > **Aktivitás (Activity)**
 - > **Végső csomópont (final node)**
 - Folyamat befejezése
 - Nincs kimenő él
 - > **Vezérlési él (control flow)**
 - Elindítja a cél aktivitás futtatását



Több lépésben

- Bontsuk több lépésre a vásárlást!
 - > Lépésről-lépésre finomíthatjuk a folyamat leírását



Őrfeltétel és dekompozíció

- Csak sikeres fizetés esetén adjuk ki a jegyet!



- > Sikeres – sikertelen fizetés

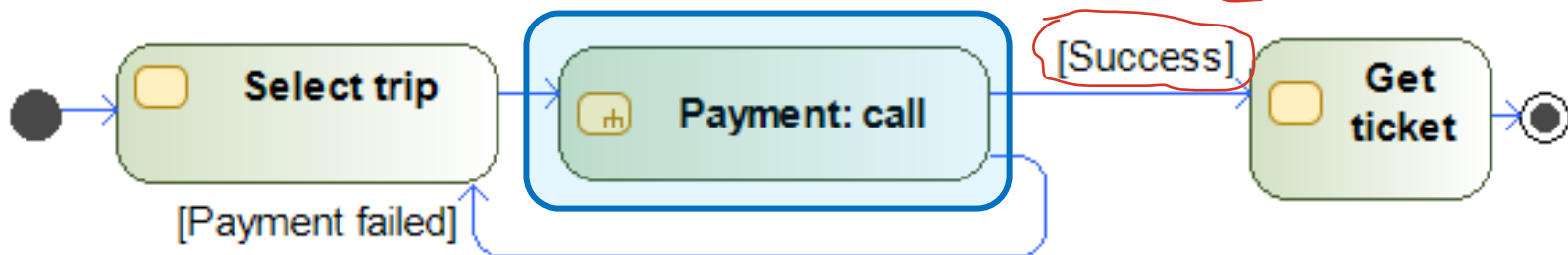
- Őrfeltétel (guard condition):

- Szabályozza melyik vezérlési él válhat aktívvá

- > A fizetési folyamatot bontsuk lépésekre!

- Hívás aktivitás (call activity):

- Hierarchikus dekompozíció



Fizetés

- Lehessen kártyával és készpénzzel fizetni!



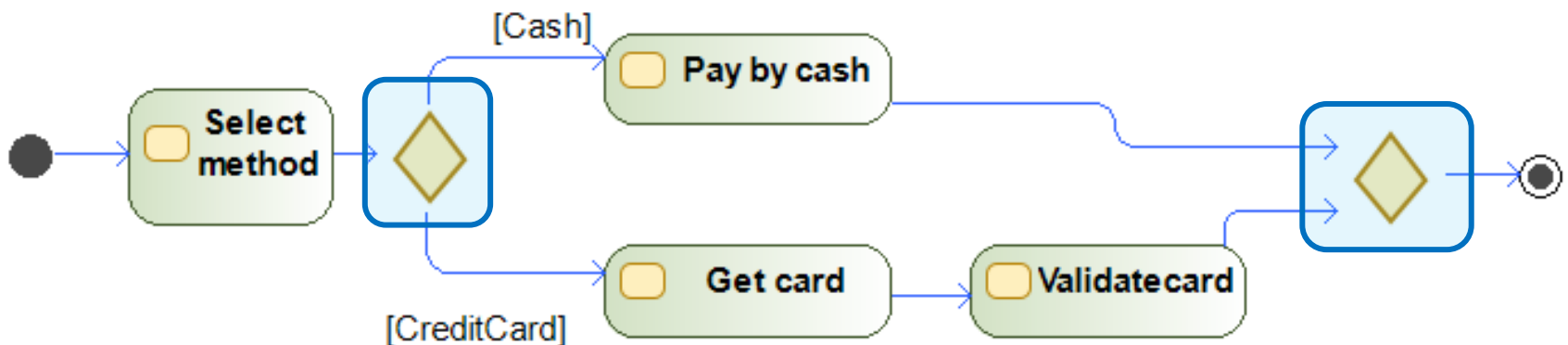
- > **Döntés (decision):**

- Legfeljebb egy kimenő él aktiválható

- > **Összevonás (merge):**

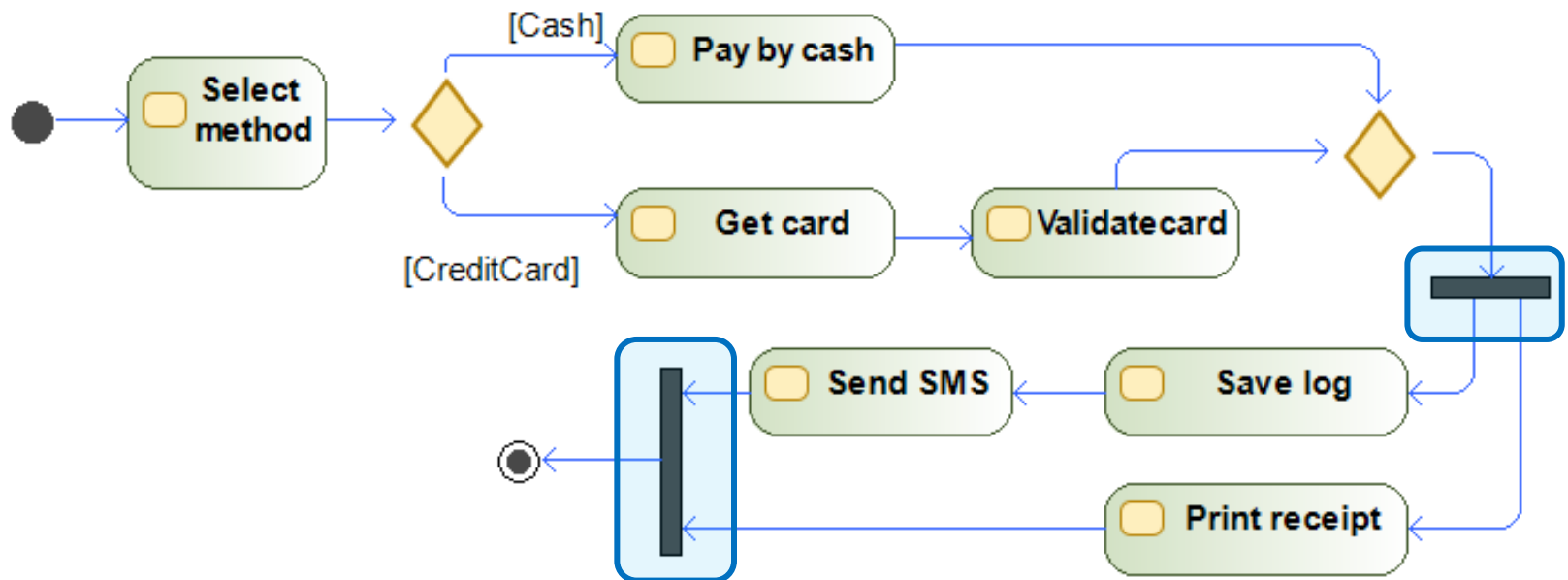
- Összefog több ágot, minden beérkező ágra lefut újra

- > Kártyás fizetésnél ellenőrizzük is le a kártyát!



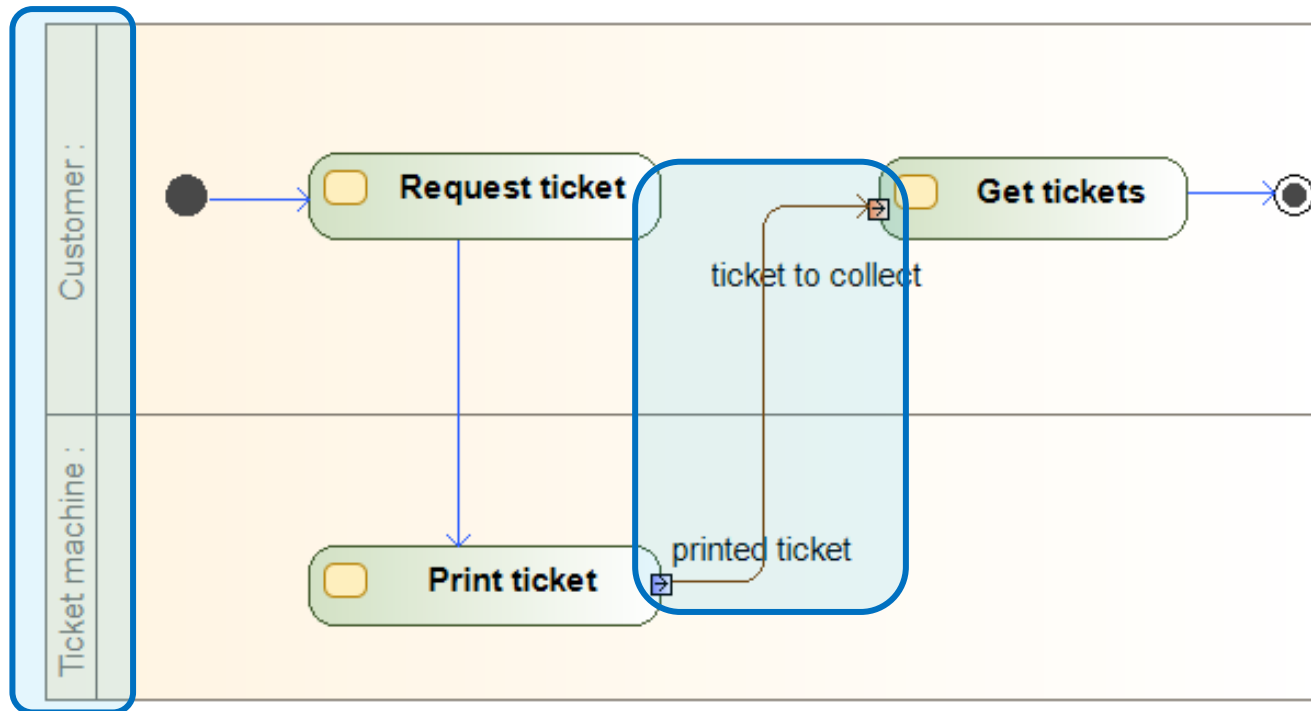
Fizetés

- A fizetés jóváhagyása után
 - > Párhuzamosan:
 - Jegyezzük be a fizetést, küldjünk sms-t!
 - Nyomtassunk ki bizonylatot!
 - > Elágazás (fork), ill. Egyesítés (join)
 - Párhuzamos futás, szinkronizáció



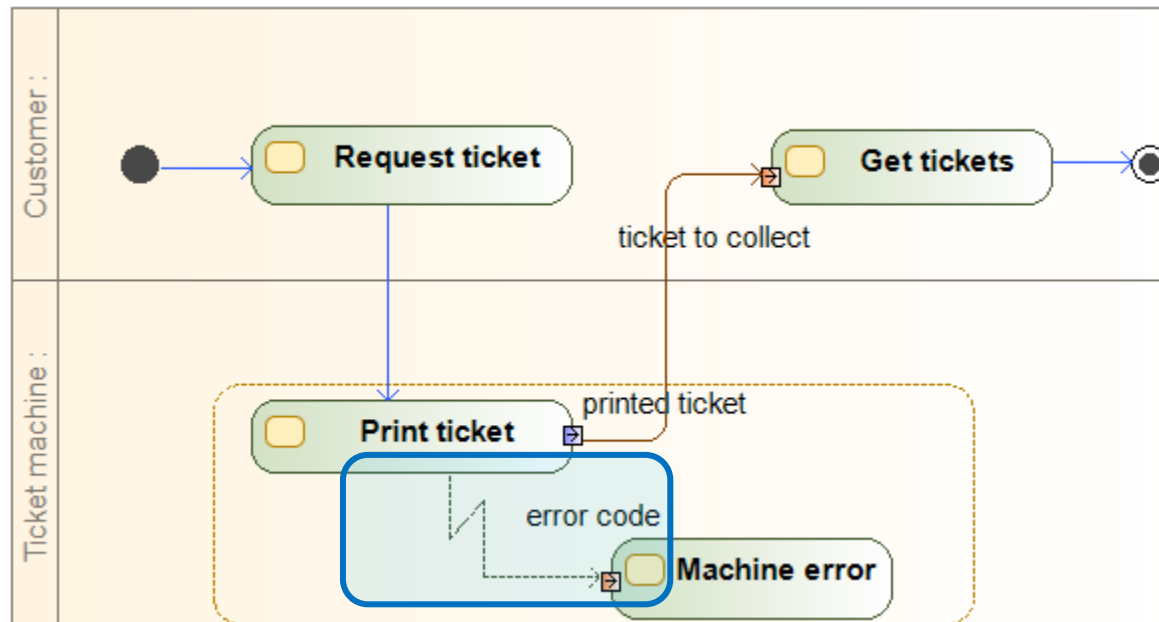
Jegy átvétele

- A jegyet az automata nyomtatja
 - > Két résztvevő rendszer (szerepkör) van a folyamatban!
 - Partíció (partition)
 - > A jegy átadása adat átvadás, nem vezérlés!
 - Objektum él (object flow)
 - Objektum csomópont (object node)
 - Tüske (pin) – az objektum csomópont átadásának jelölésére, egy fajta objektum



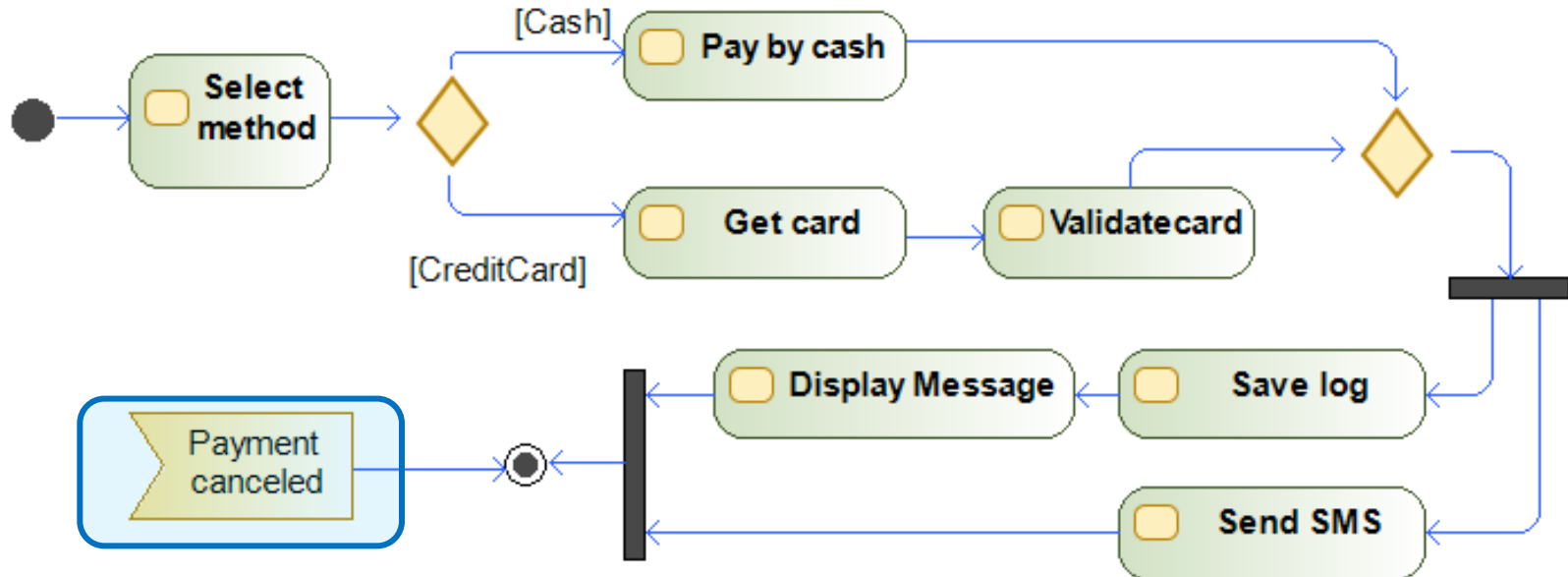
Hibakezelés

- Ha a nyomtatás során hiba adódik, kezelni kell
 - > Kivételkezelés, adatátadással
 - > **Kivétel (exception)**
 - Adott régióban történő váratlan megszakításhoz
 - (Modelio nem támogatja a „villám” jelölést)



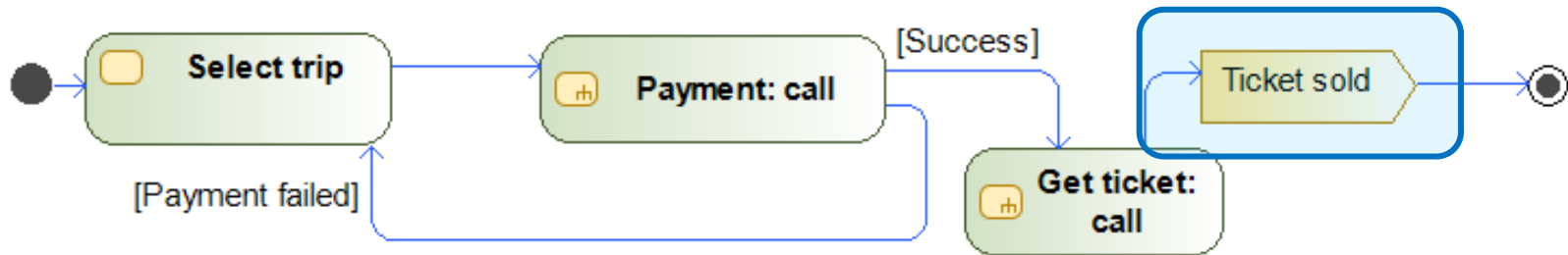
Események

- A fizetés legyen bármikor megszakítható!
 - > Esemény alapú vezérlés: akkor folytatódik a vezérlés, ha az esemény bekövetkezett
 - > Az esemény bárhol kiváltható (mi most nem modellezzük)
 - > Esemény fogadása akció (accept event action)



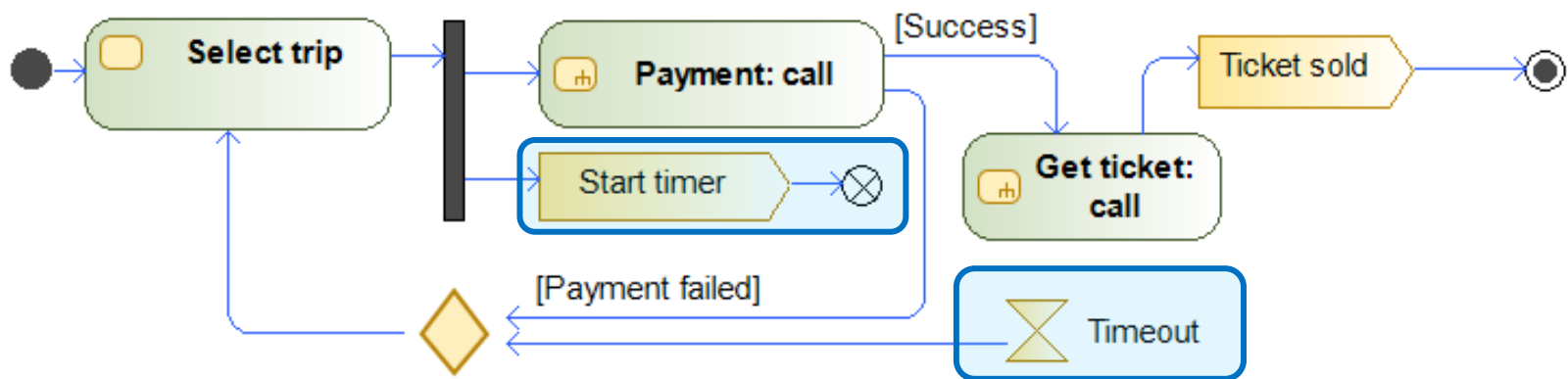
Események

- A sikeres tranzakciót eseményen keresztül publikáljuk!
> Esemény küldése akció (send signal action)



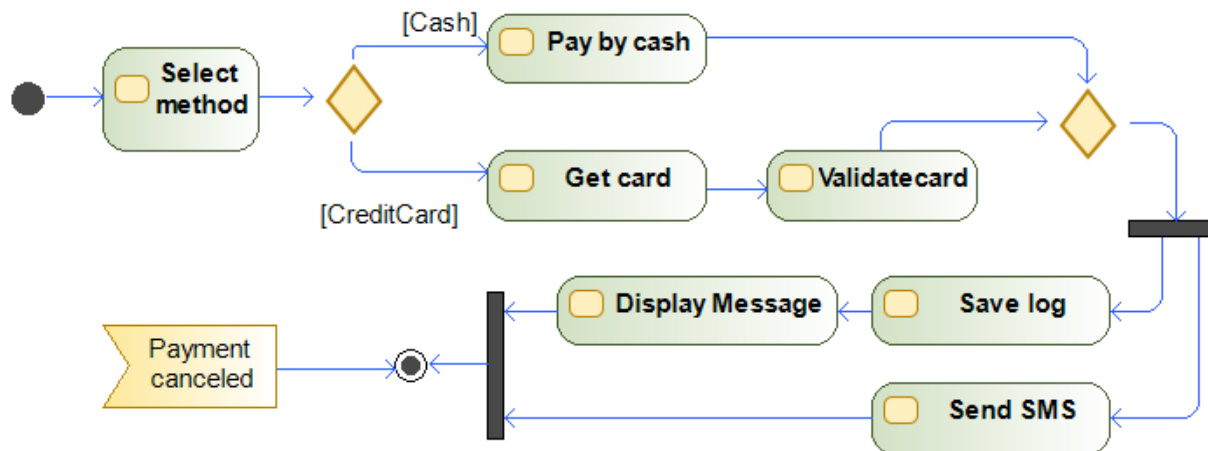
Események

- A tranzakciónak legyen időkerete! Ha az idő lejár, automatikusan váljon sikertelenné a tranzakció!
 - > Időesemény fogadása akció (accept time event action)
- Indítsuk el az időzítőt az elején!
 - > Az időzítő elindítása után az a folyamat ág véget ér, de a teljes folyamat nem!
 - > Folyam vége csomópont (flow final node)



Összefoglalás

- Aktivitás
- Kezdő
- Végcsomópont
 - > Folyam vége
 - > Végő csomópont
- Vezérlési (folyam) él
 - > Őrfeltétel
- Hívás aktivitás
- Döntés – összevonás
- Elágazás – egyesítés
- Partíció
- Objektum, objektum él
- Esemény – fogadás és küldés
- Időesemény



Állapotgép

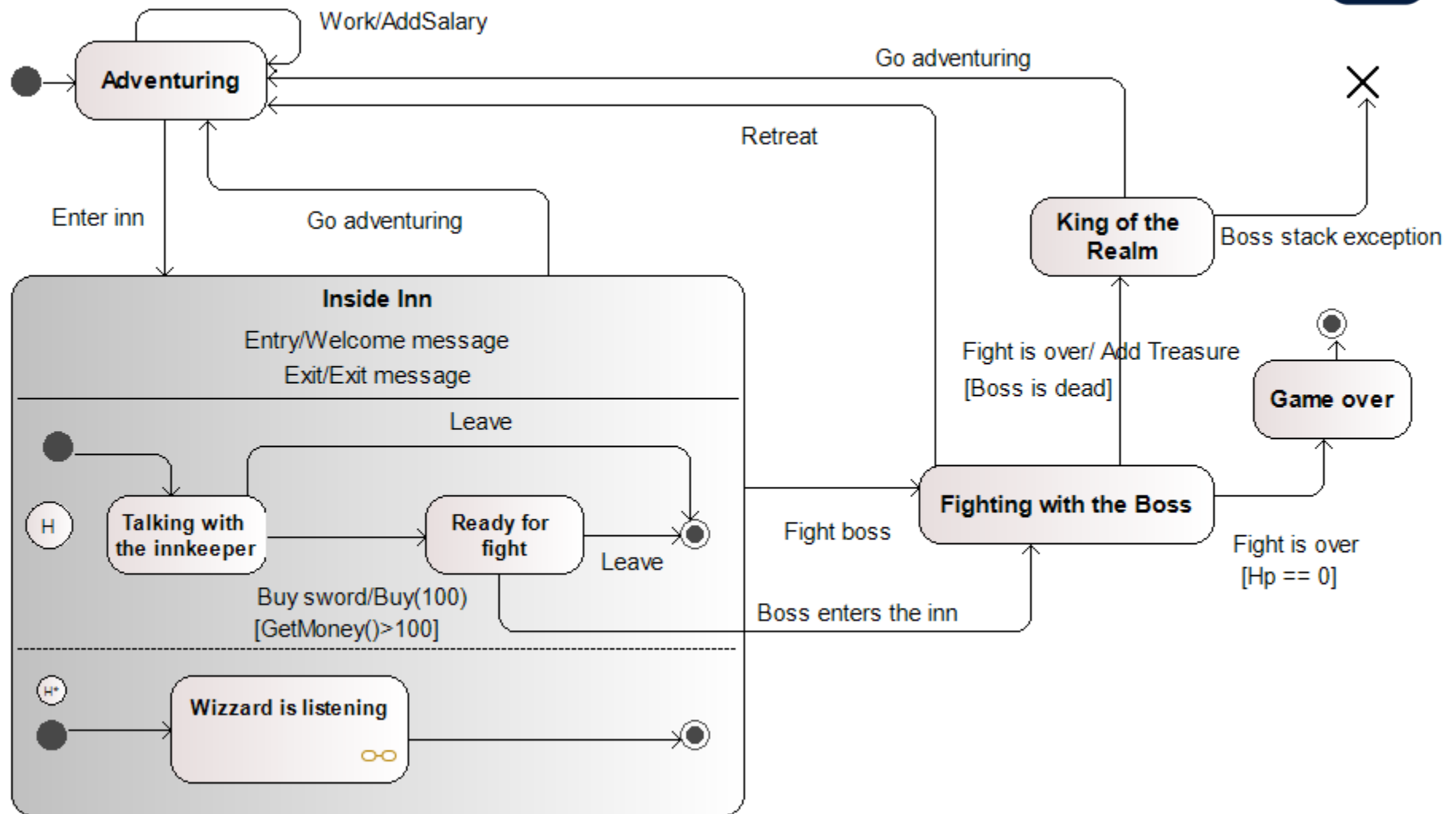
Állapotgépek

- **Állapotgép (Statemachine, statechart diagram)**
 - > **Objektum-orientált**, eseményvezérelt “véges automata”
 - > Fókusz:
 - A rendszer, vagy egy komponens állapotai
 - Állapotátmentek és feltételek
 - > Két fajta
 - Viselkedést leíró (Behavioral state machines)
 - Protokoll leíró (Protocol state machines)

Aktivitásdiagram: Folyamatot / munkafolyamatot modellez. (rendszer lépéseit írja le: megrendel -> feldolgoz -> kiszállít)

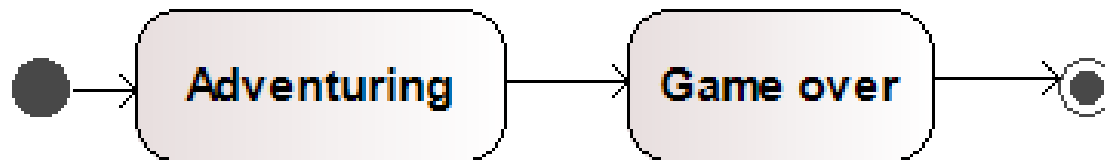
Állapotgép: Egy objektum életciklusát modellezi. (azt mutatja meg milyen állapotban van egy objektum: leadott -> feldolgozás alatt -> szállítás alatt)

Állapotgép: Kalandjáték



A kezdet és a vég

- Modellezzük egy kalandjátékot!
 - > A játék elkezdődik, kalandozunk, majd véget ér
 - > Kezdő állapot (initial state)
 - > Állapot (state)
 - > Átmenet (transition)
 - Ha több lehetséges átmenet van, nem-determinisztikusan választunk egyet
 - > Végő állapot (final state)



Főellenség

- Legyen egy főellenség, akit le kell győzni!

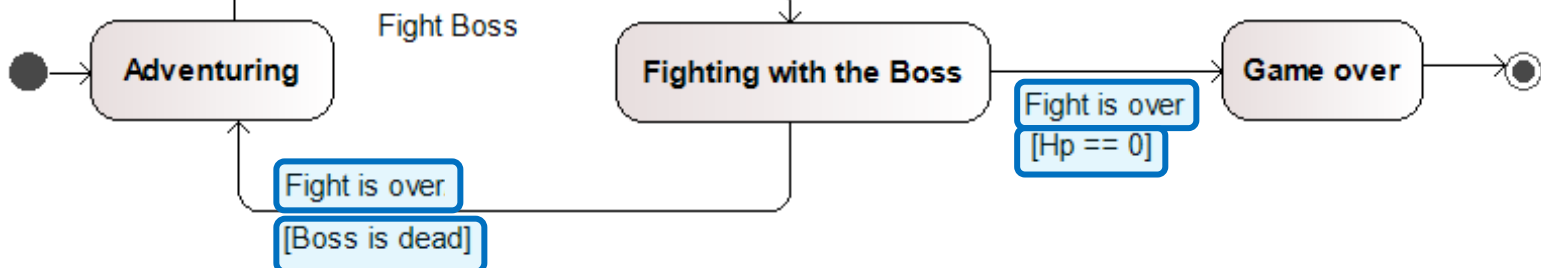
- > Trigger (trigger)

- Az állapotátmenet oka, leggyakrabban egy esemény
 - Kódban általában az objektumon kívülről hívott metódus

- Ne legyen automatikus a győzelem/vesztés!

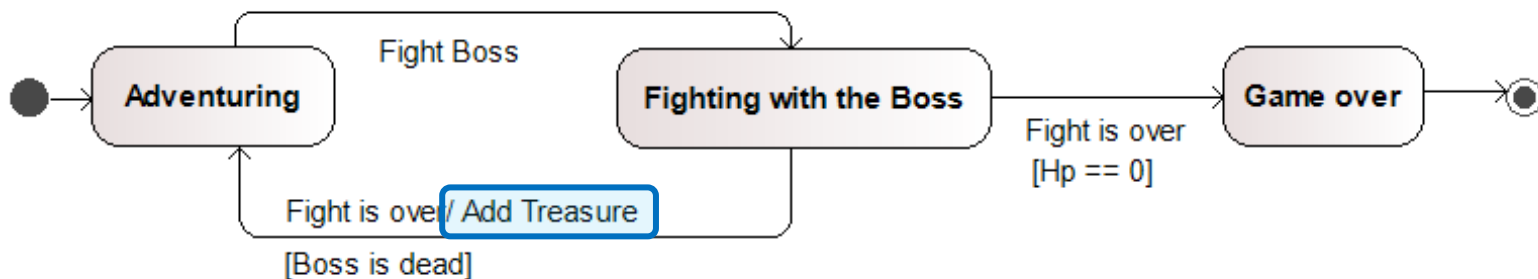
- > Őrfeltétel (guard condition)

- Lehetséges átmenetnél a feltételnek is teljesülnie kell
 - Triggerek finomhangolása



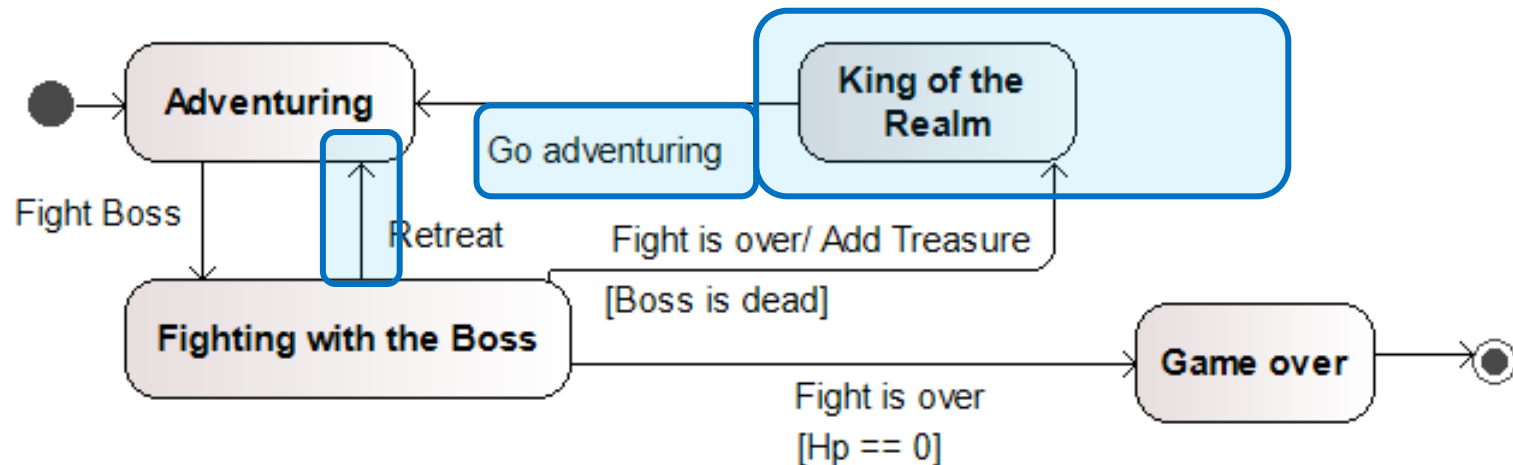
Jutalom

- Ha győztünk, kapjunk jutalmat!
 - > Akciók (action, hivatalosan: behavior expression)
 - Az átmenet során végrehajtandó akció
 - Kódban általában belső függvényhívás



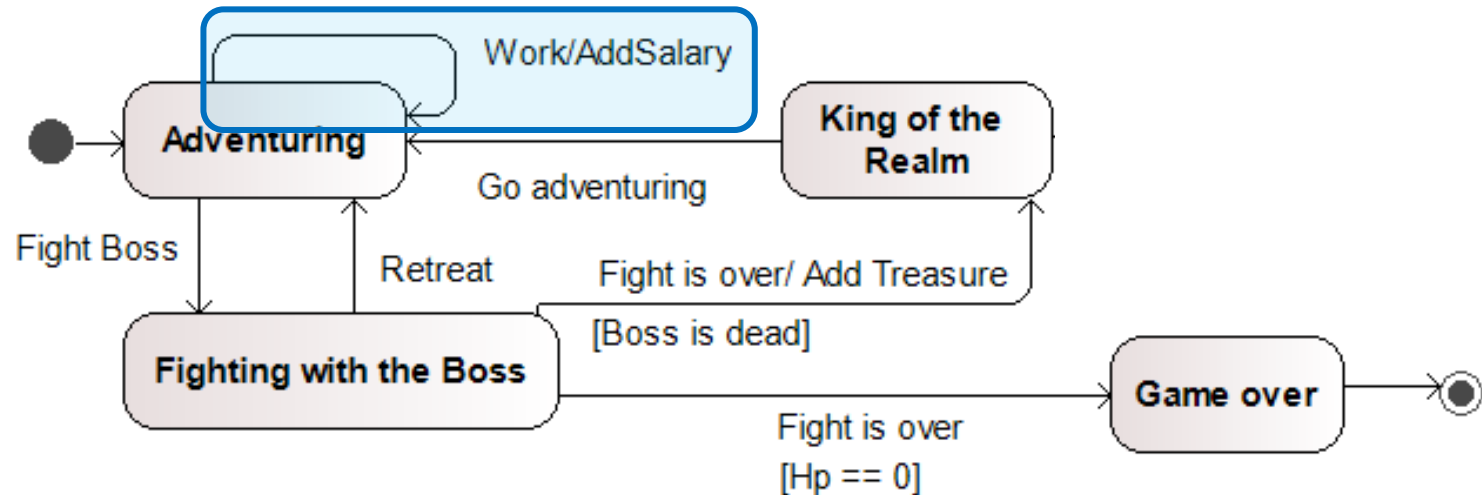
Visszavonulás és királyság

- Lehesse visszavonulni a csatából!
- Ha győztünk, kapjuk meg a királyságot!
 - > Utána még lehesse kalandozni!



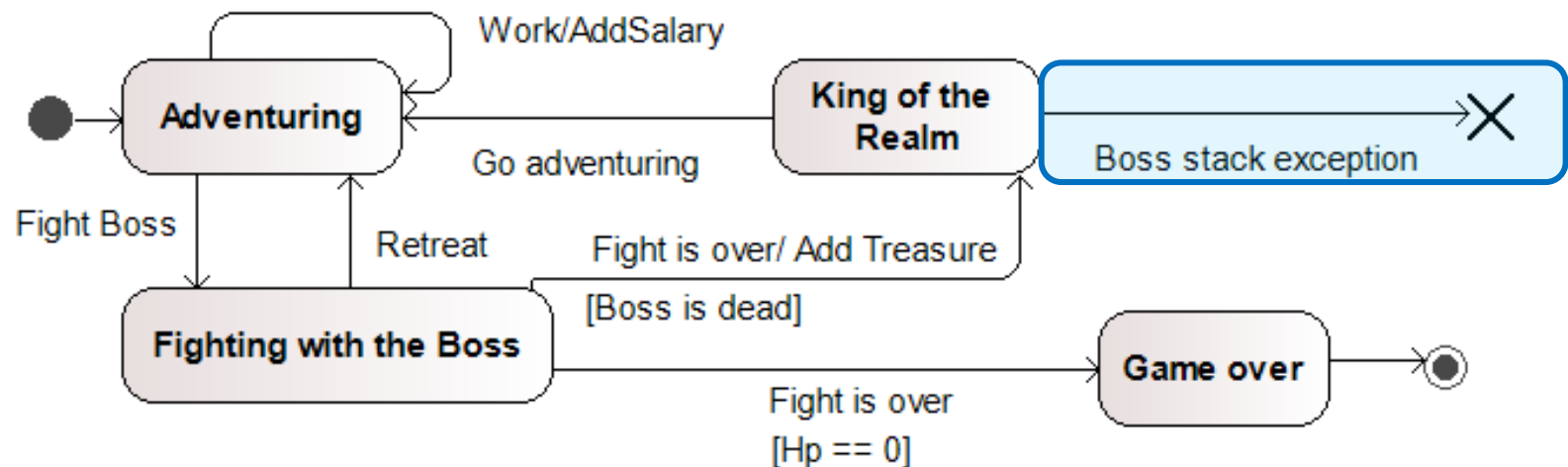
Munka, munka, munka

- Lehessen dolgozni (kalandozóként)!
 - > A munkáért kapjunk pénzt!



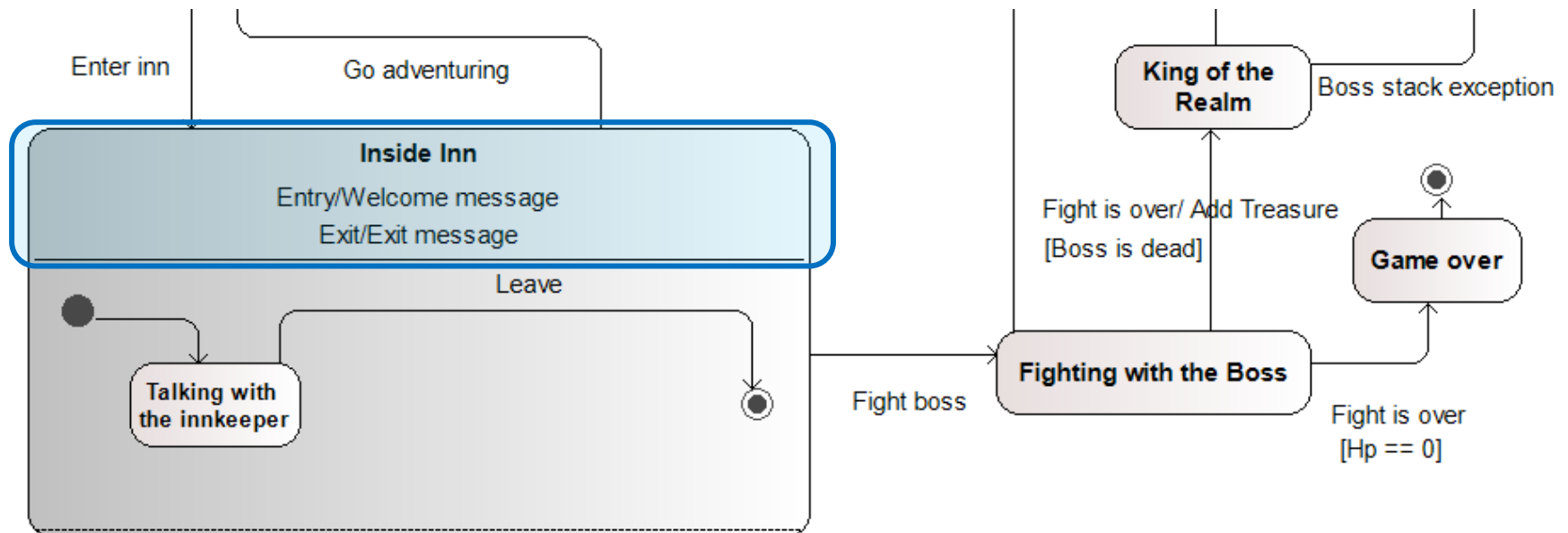
Hibakezelés

- Ha nincs több főellenség, kapjunk hibát!
 - > Terminálási állapot (terminate pseudo state)
 - > Kilépési akciók, állapotváltások nélkül véget ér a végrehajtás



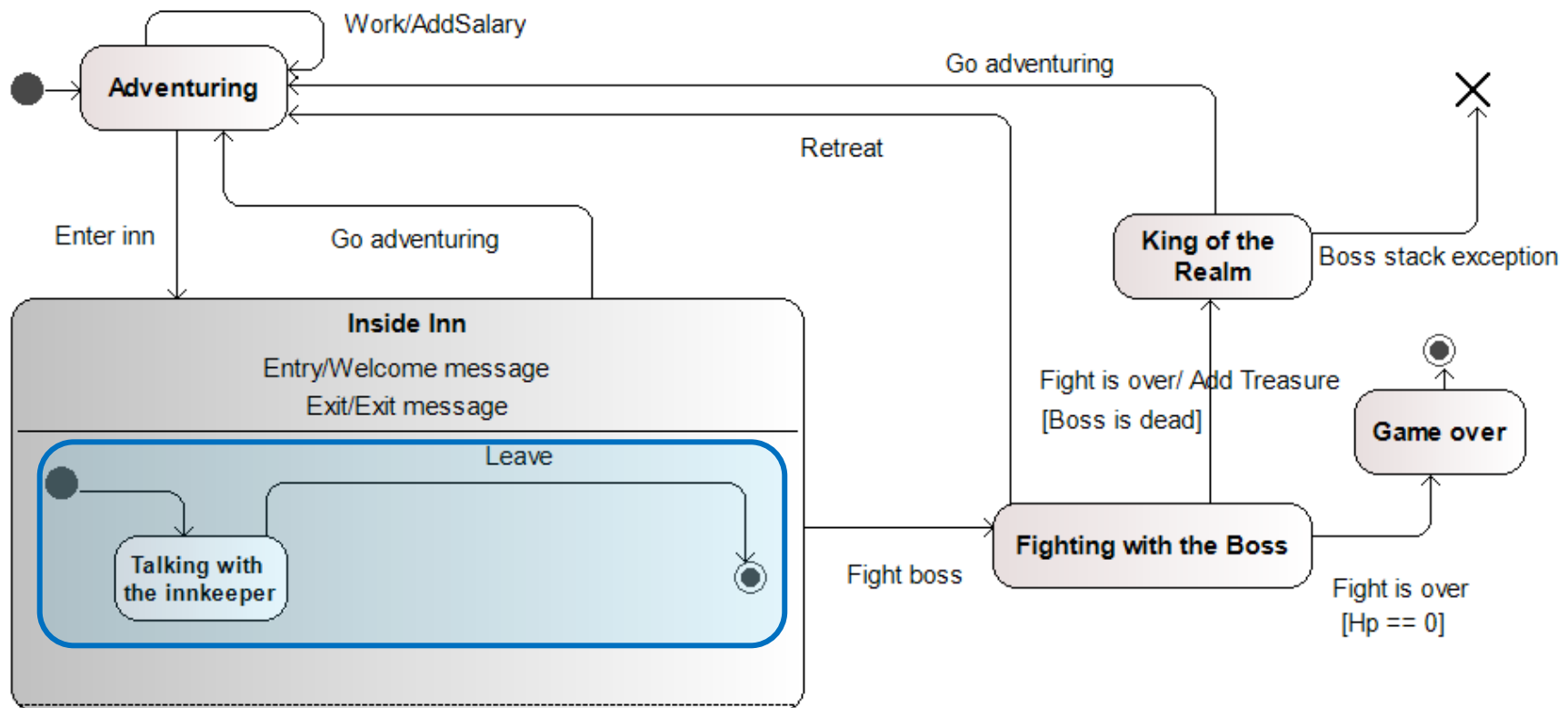
Fogadó

- Minden kaland egy fogadóban kezdődik...
 - > **Összetett állapot (composite state)**
 - > Kapjunk üdvözlést az elején, elköszönést a végén!
 - **Belépési akció (entry action)**: lefut, ha külső állapotból belépünk
 - **Kilépési akció (exit action)**: lefut, ha külső állapotba kilépünk
 - **Cselekvési akció (do action)**: folyamatosan fut a belépéstől a kilépésig (a példában ez nem szerepel)



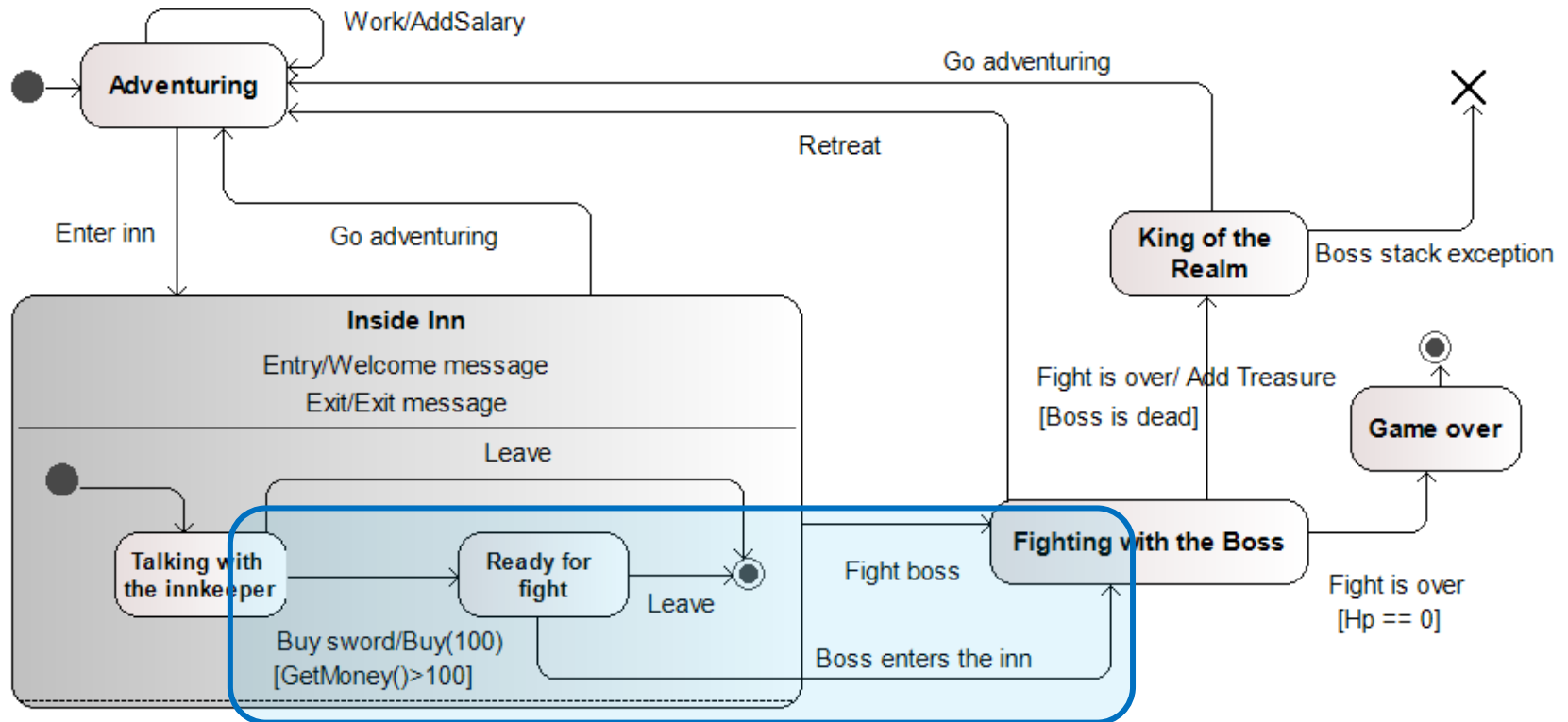
Fogadó (folytatás)

- Lehesse beszélgetni a fogadóssal!
 - > Belső állapotok/hierarchikus állapotgép



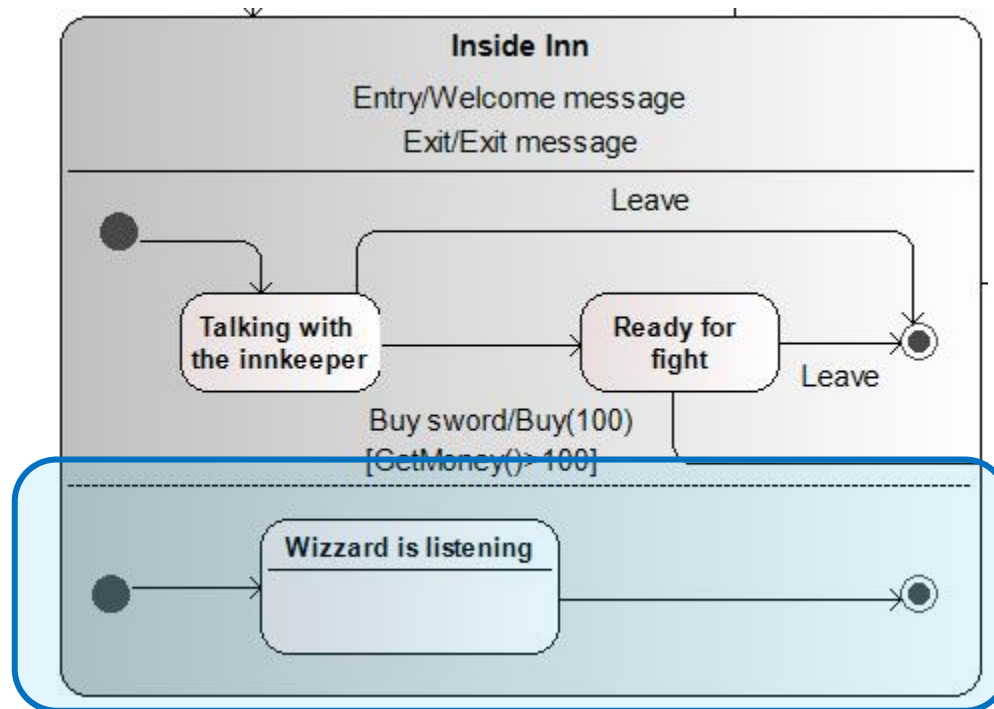
Kard

- Lehesse venni kardot a fogadóstól!
 - > A kard kerüljön pénzbe!
 - > Ha megvettük, készen vagyunk a csatára
 - > Ha a főellenség ilyenkor betoppan, azonnal harcolni kell
 - Közvetlen kilépés



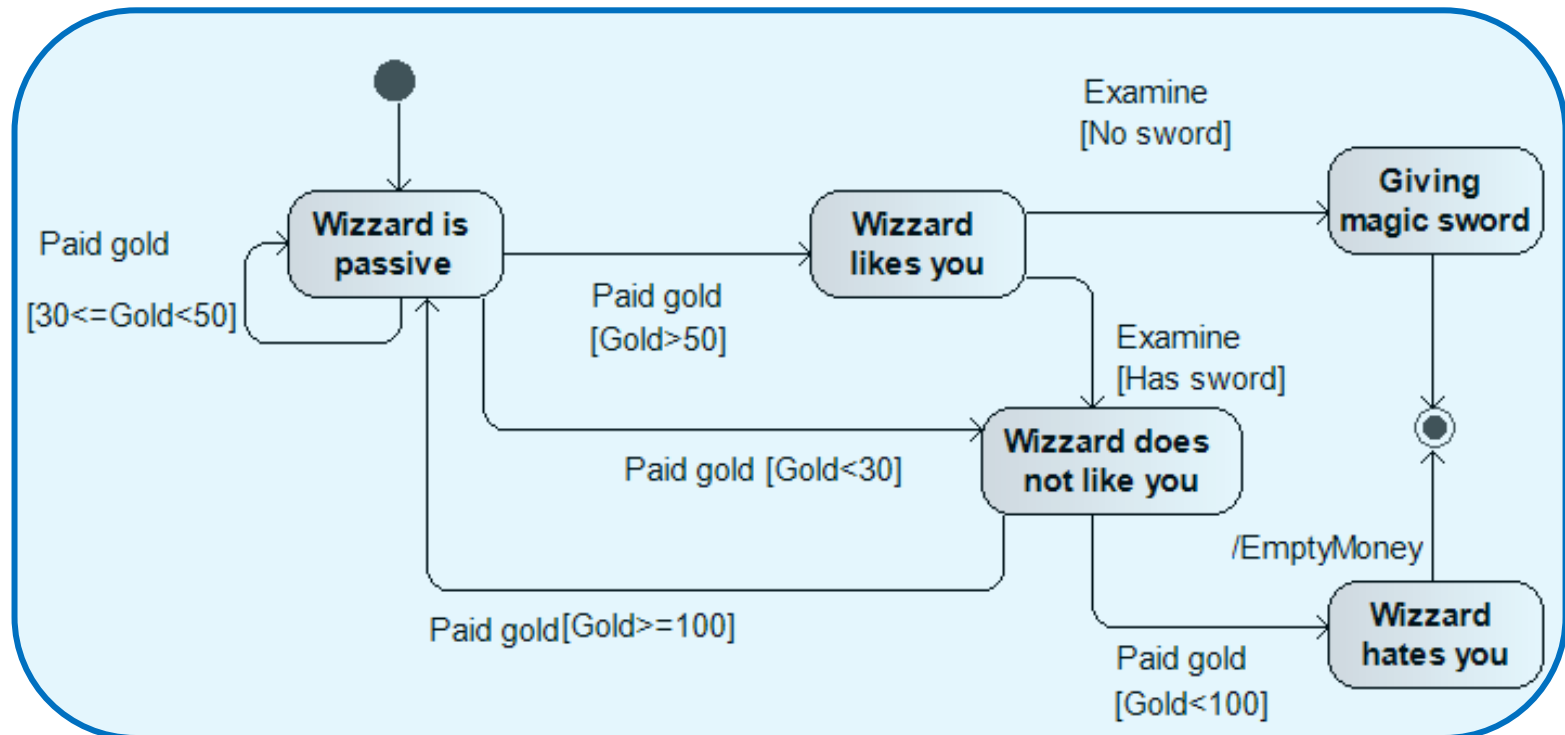
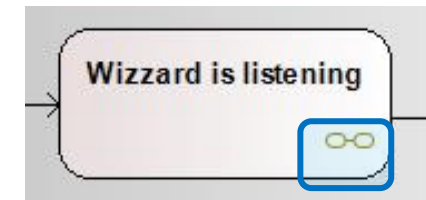
Varázsló

- Minden kalandban van egy varázsló...
 - > Az állapotváltás a fogadós szállal *párhuzamosan* zajlik
 - > Régió (region)
 - Összetett állapot ortogonális szétbontása



Varázsló

- Lehessen lefizetni a varázslót... de csak okosan!
 - > Állapoton belüli állapotgép
 - > **Alállapotgép állapot (submachine state)**
 - > Hivatkozás a másik összetett állapotra



Emlékezet

H^* -> Azaz ha kiléptél az **Inside Inn**-ből és visszatérsz, nem az **alapállapottól** indulsz újra, hanem onnan, **ahol abbahagytad**.

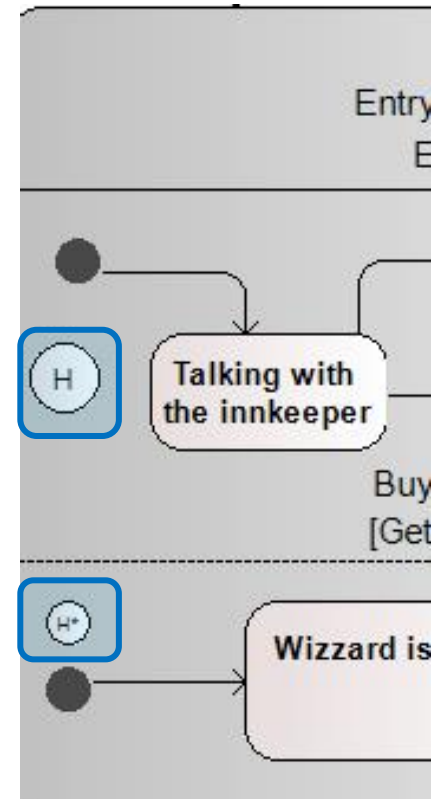
- Emlékezzen a rendszer, hol tartottunk a fogadóban, ha kimentünk és visszamentünk!

> Sekély történet (shallow history) (H)

- Visszaállítja a közvetlen tartalmazó állapotgép/régió állapotát visszatéréskor
- Csak egy hierarchia szinten keresztül működik

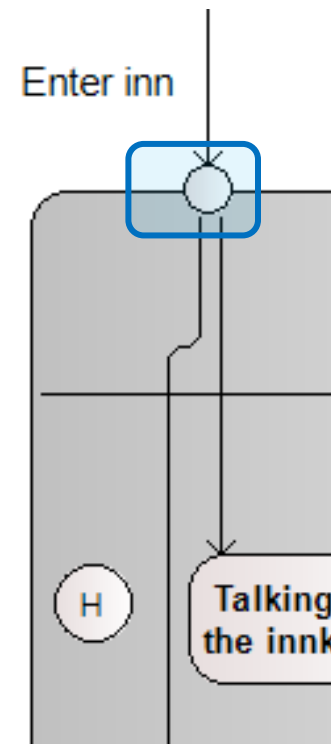
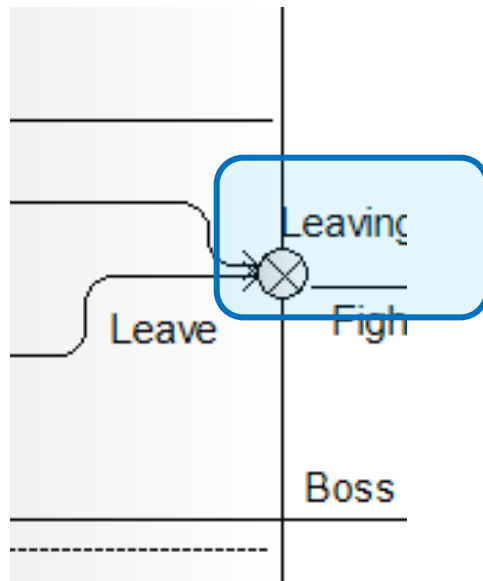
> Mély történet (deep history) (H^*)

- Visszaállítja a tartalmazó állapotgép/régió állapotát visszatéréskor
- Bármilyen mély hierarchiában működik



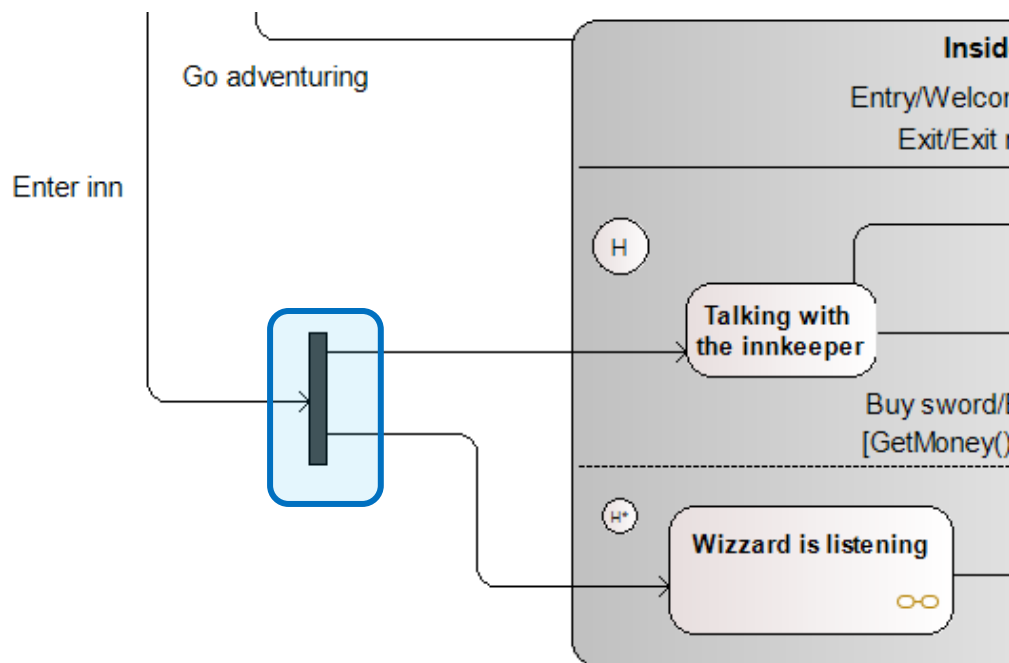
Belépési és kilépési pont

- Megadható külön nevesített be-, és kilépési pont
 - > **Belépési/kilépési pont (entry/exit point)**
 - Leaving / Enter inn
 - > Segíthet, ha különböző forgató- könyvekben különböző lépések kellenek
 - > Elrejtja a belső szerkezetet



Elágazás és egyesítés

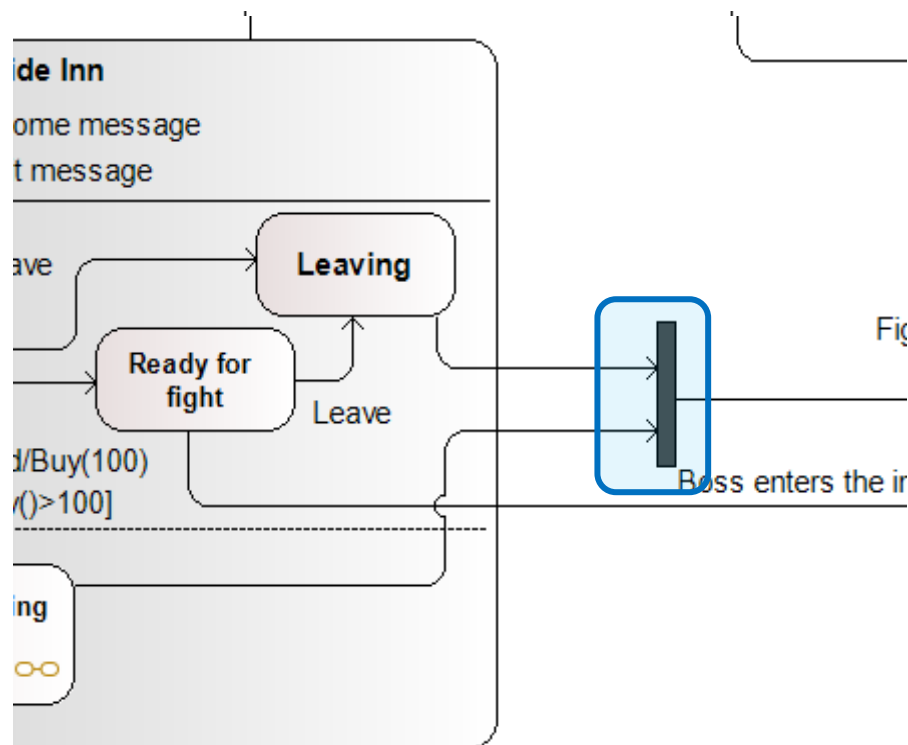
- Elágazó útvonalak
 - > Elágazás (fork)
 - > Egyszerre minden állapot aktiválódik
 - > Kimenő élek kötelezően esemény (trigger) és őrfeltétel nélkül!



Elágazás és egyesítés

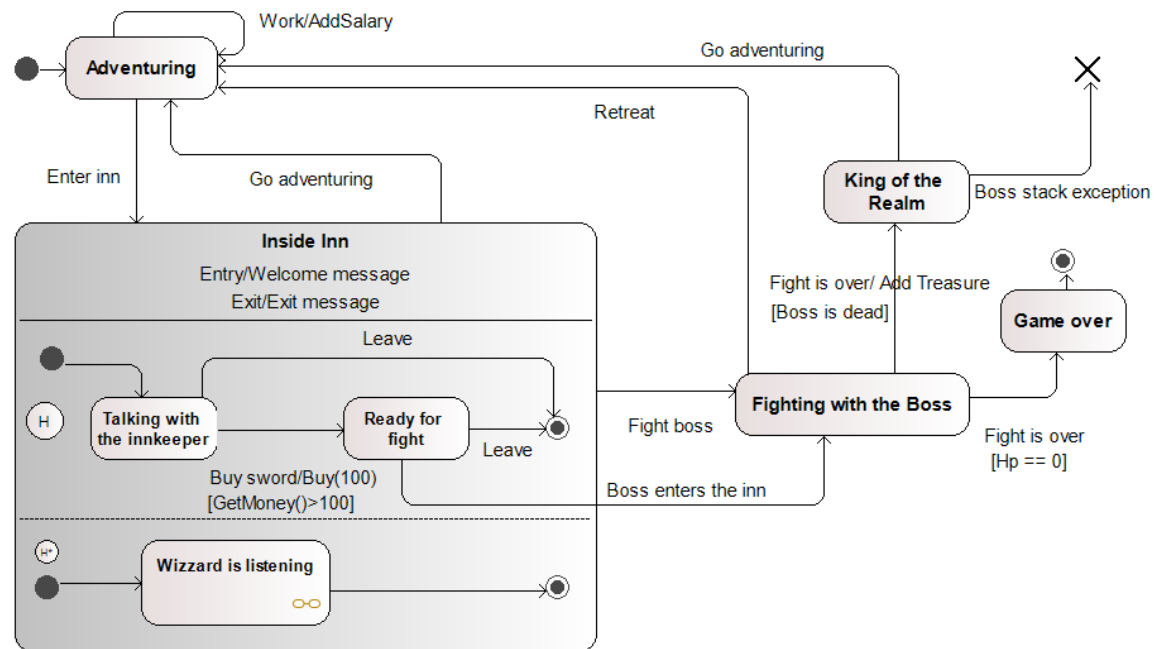
- Összefutó útvonalak

- > **Egyesítés (join)**
- > Megvárja a részállapotokat és egységesíti őket
- > Bemenő élek kötelezően esemény (trigger) és őrfeltétel nélkül!



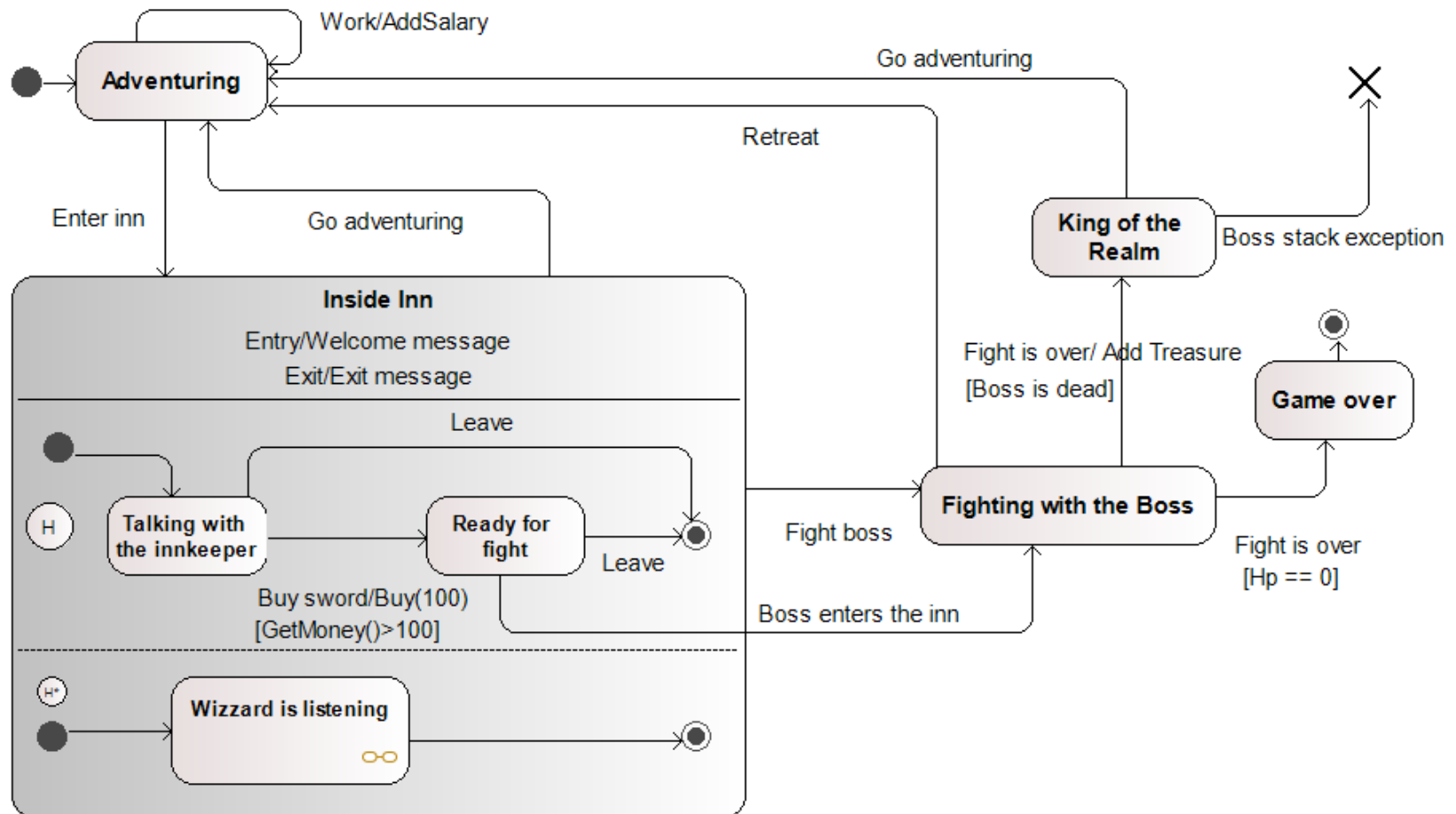
Állapotgép

- Állapotok
 - > Történet elem
 - > Explicit be-, és kilépési pont
 - > Elágazás, egyesítés
- Kezdő és végállapot
- Terminálás
- Átmenetek
 - > Esemény
 - > Örfeltétel
 - > Akció
- Összetett állapotok
 - > Régiók



Osztálydiagram

- Rajzoljuk meg az állapotgéphez tartozó osztálydiagrammot! 



Szekvencia-aktivitás-állapot?

- **Szekvenciadiagram**

- > Szekvenciális interakció objektumok közt
- > Metódushívások, üzenetek tényleges időbelisége

- **Aktivitásdiagram**

- > Vezérlési folyam: aktivitások/tevékenységek
- > Lehet szekvenciális, szétágazó, vagy párhuzamos
- > A vezérlés (flow) fontosabb az őrfeltételeknél

- **Állapotgép**

- > Állapotokat és állapotátmeneteket ír le
- > Események vezérlik az állapotátmenetet

Köszönöm a figyelmet!

Levélcsomag átvétele (+ pontért)

- MPL csomagponton szeretnénk a csomag átvételét modellezni
- Belépünk a Postára és odamegyünk a sorszám kiadó automatához
- Ha nem működik, akkor hazamegyünk, a folyamat véget ér
- Ha működik, akkor kérünk egy sorszámot
- Beállítjuk az óránkon, hogy fél óra múlva csipogjon. Ha csipog, akkor hazamegyünk, bárhol járunk is a folyamatban
- Leülünk, majd alszunk egy kicsit
- A rendszer eseménnyel jelzi, ha mi következünk ekkor odaadjuk a személyinket az ügyintézőnek, aki ezt ellenőrzi, majd elmegy a raktárba a csomagért és kihozza azt.
- Párhuzamosan, egy időben átvesszük a csomagot, valamint megköszönjük azt
- Hazamegyünk, a folyamat véget ér

Szoftvertechnológia és -technikák

5. Előadás

Használati eset diagram és kitekintés



I. Házi feladat

- A tanszéki weboldalon megtalálható jövő hét elejétől
- Modellezési feladat
 - > Nem kell kódot írni
- Szabad specifikáció
- Beadás: portálon keresztül
- Beadási határidő: 10. hét péntek (12:00)



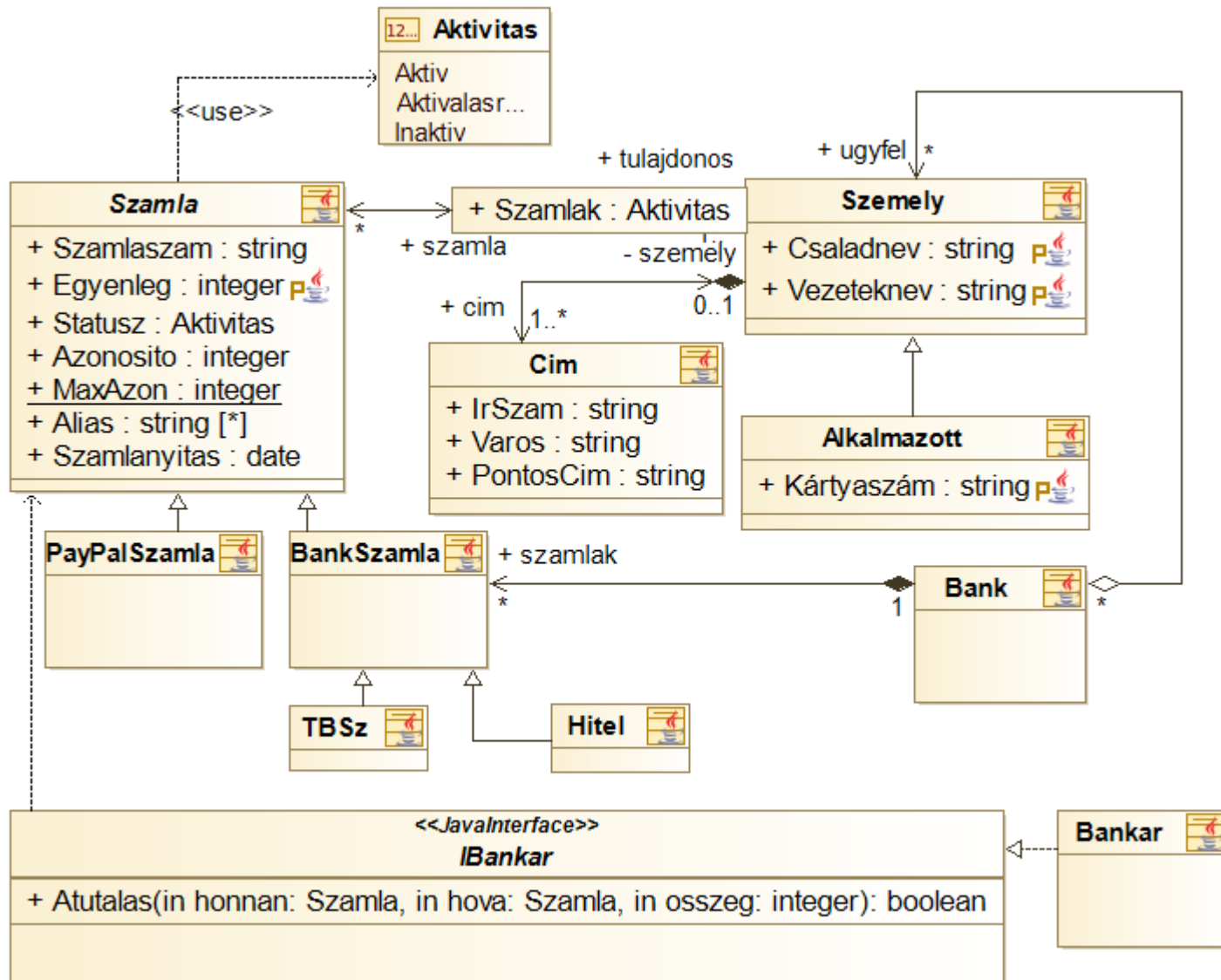
ZH

- Jelenléti ZH!
 - > Csak kedvezményes tanulmányi rendi kérvénnyel (022) lehet online teljesíteni!
- 2020. nov. 06. 08:15-
- Lesz minta ZH
- Számonkért anyag: ZH előtt 1-2 héttel

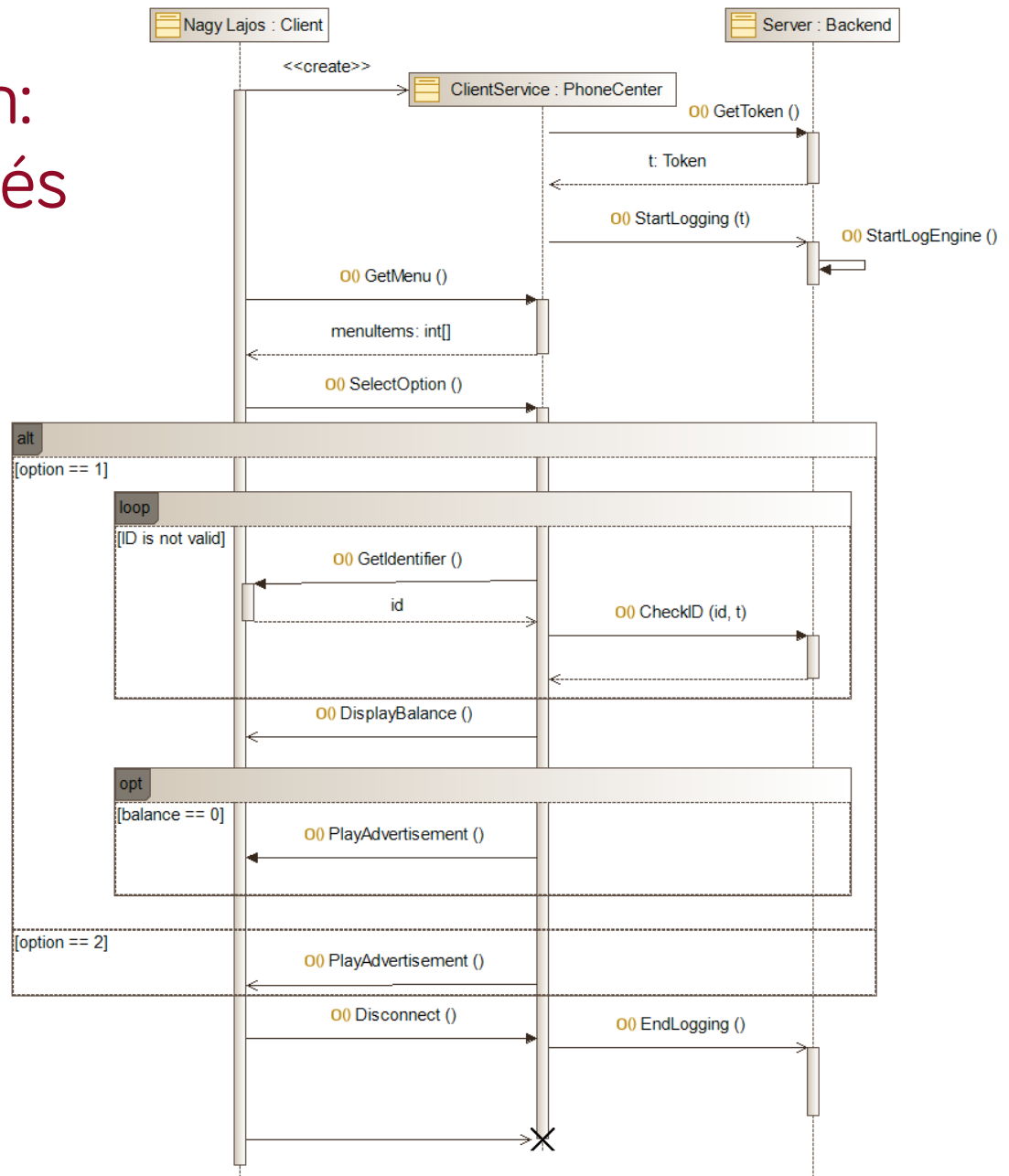


UML diagramok

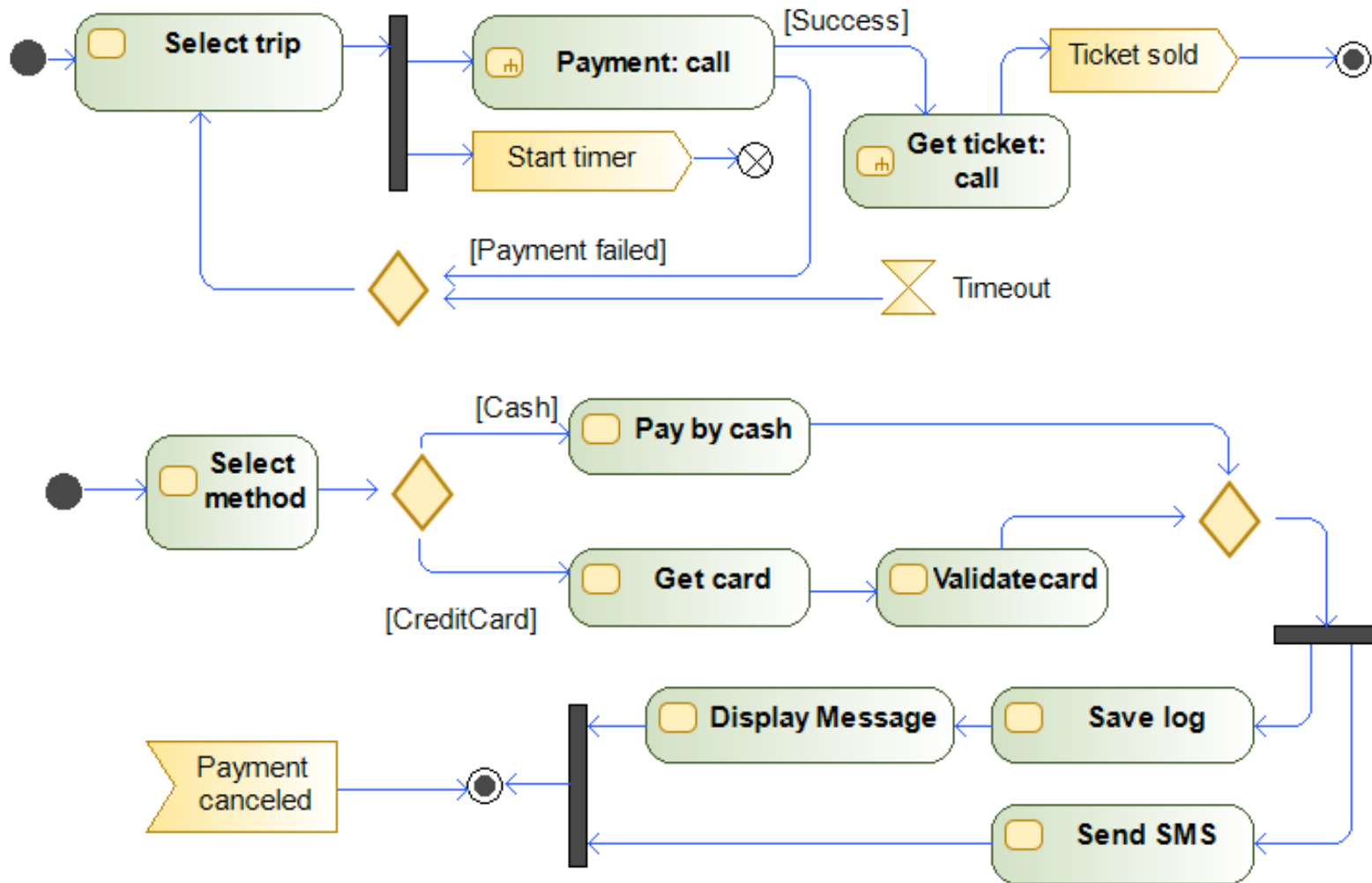
Osztálydiagram: Banki adminisztrációs rendszer



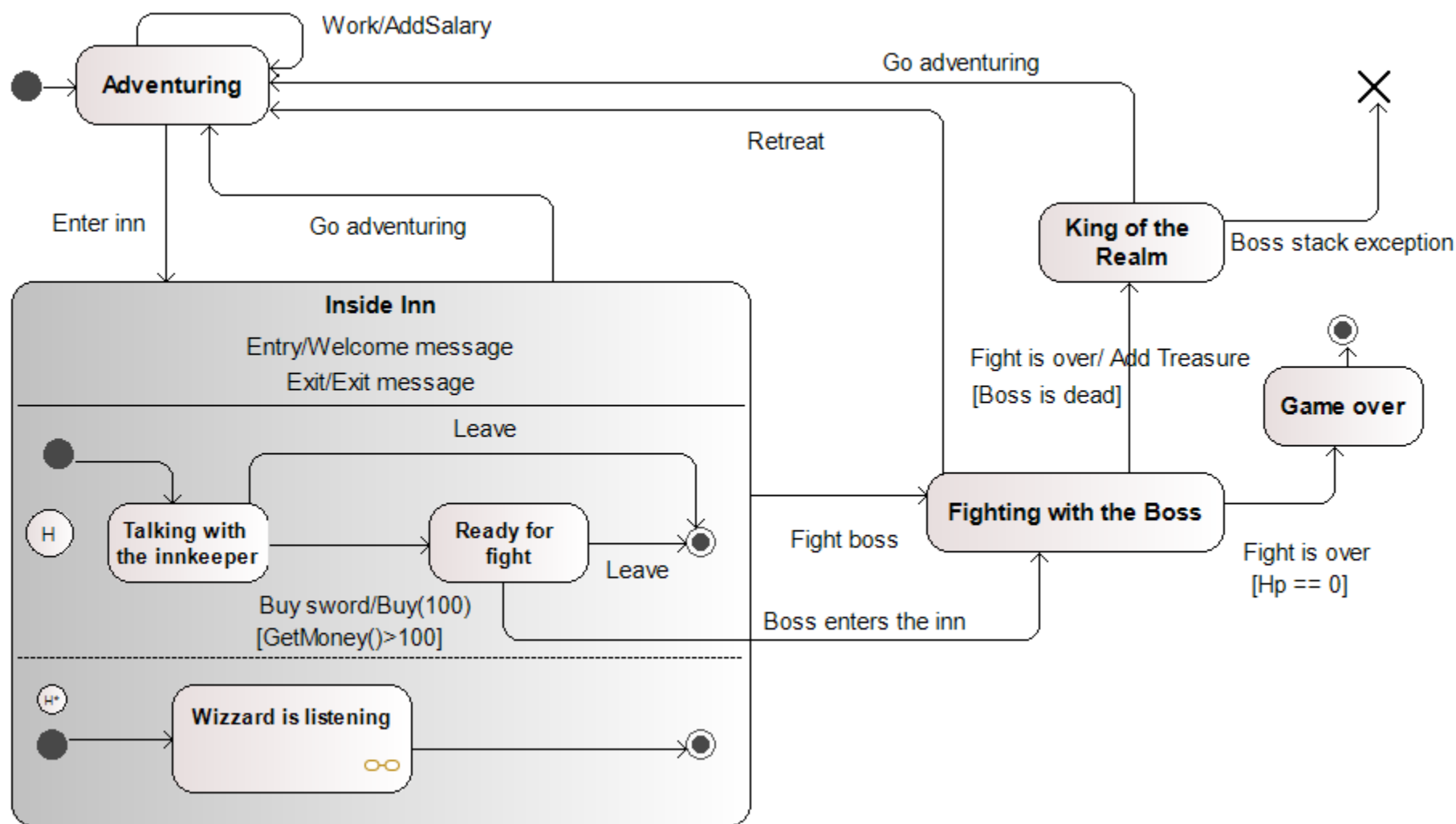
Szekvenciadiagram: Telefonos ügyintézés



Aktivitásdiagram: BKV jegyvásárlás



Állapotgép: Kalandjáték



UML diagramok

- A kódstruktúra: osztálydiagram
- A hívások sorrendje: szekvenciadiagram
- A folyamatok: aktivitásdiagram
- Az állapotátmenetek: állapotgép
- ... de hogyan fogjuk össze a teljes rendszert?



Használati eset diagram

Use case diagram

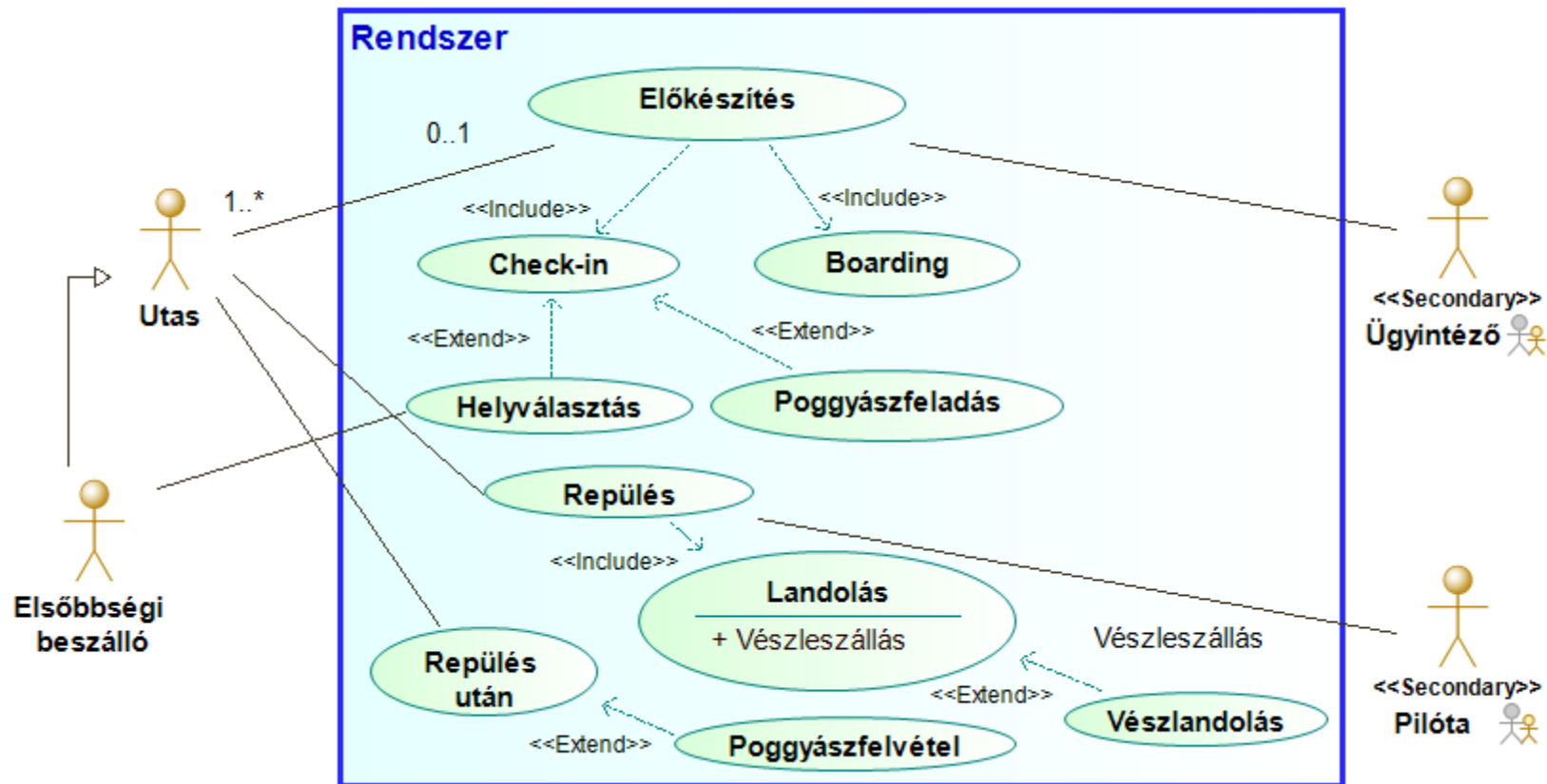
Használati eset diagram

- A teljes rendszer áttekintése
 - > Mi tartozik bele? Mi az, ami nem?
 - > Kik a szereplők?
 - > Milyen funkciókat kell megvalósítani?
- **Használati eset diagram**
 - > Rendszerszintű áttekintés
 - > Funkcionális követelmények
 - > Kommunikáció a megrendelővel

NEM viselkedést modellez, hanem azt mutatja meg, **ki mit tud csinálni** a rendszerrel!

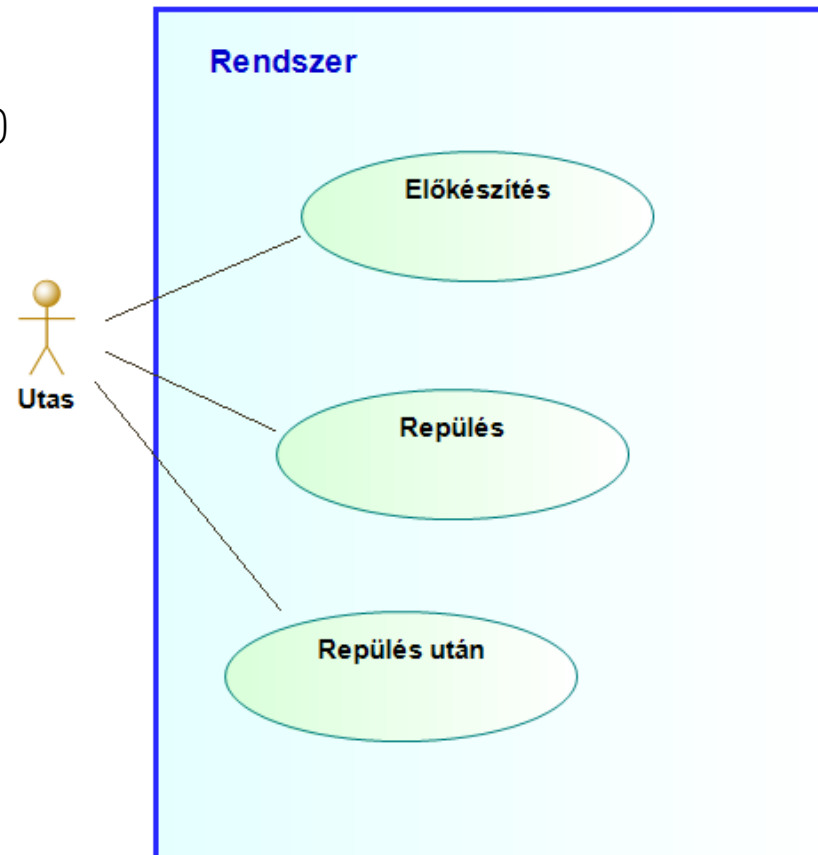


Használati eset diagram: Repülés



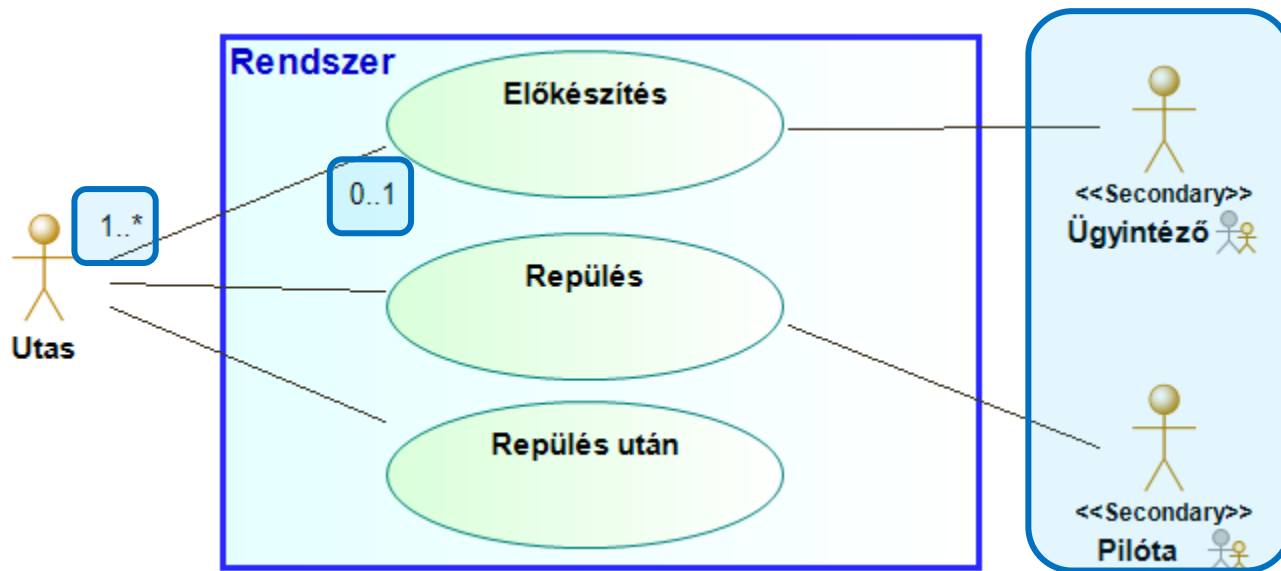
Funkciók

- A repülés három tevékenységből áll: előkészítés, repülés és befejezés. A főszereplő az utas.
 - > **Rendszer határ (system boundary)**
(a Modelio közvetlenül nem támogatja a jelölést)
 - > **Aktor (actor)**
 - Egy adott szerepkör (ember, gép, folyamat, ...)
 - > **Használati eset (use case)**
 - Actor – rendszer interakció reprezentálása
 - > **Asszociáció (association)**



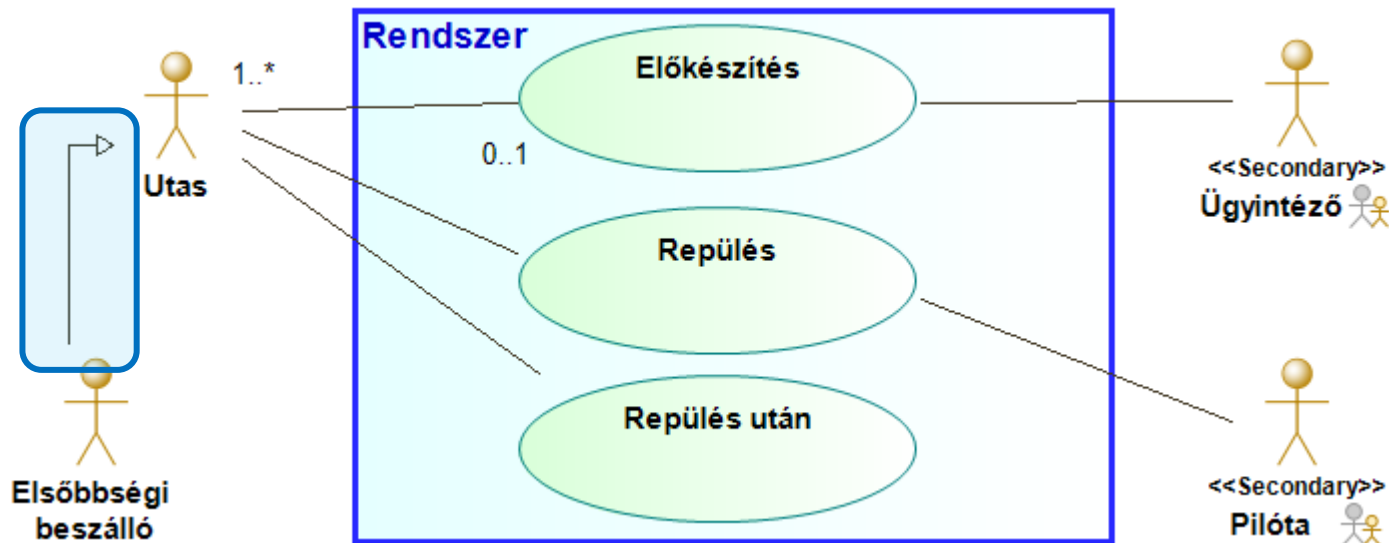
További szereplők

- A reptéri ügyintéző és a pilóta is fontos szereplő
 - > Más-más funkciókban kapnak szerepet
 - Elsődleges (primary) és másodlagos (secondary) aktorok
 - Sztereotípiával jelölt
 - > Multiplicitás megadható
 - Alapesetben 1..1



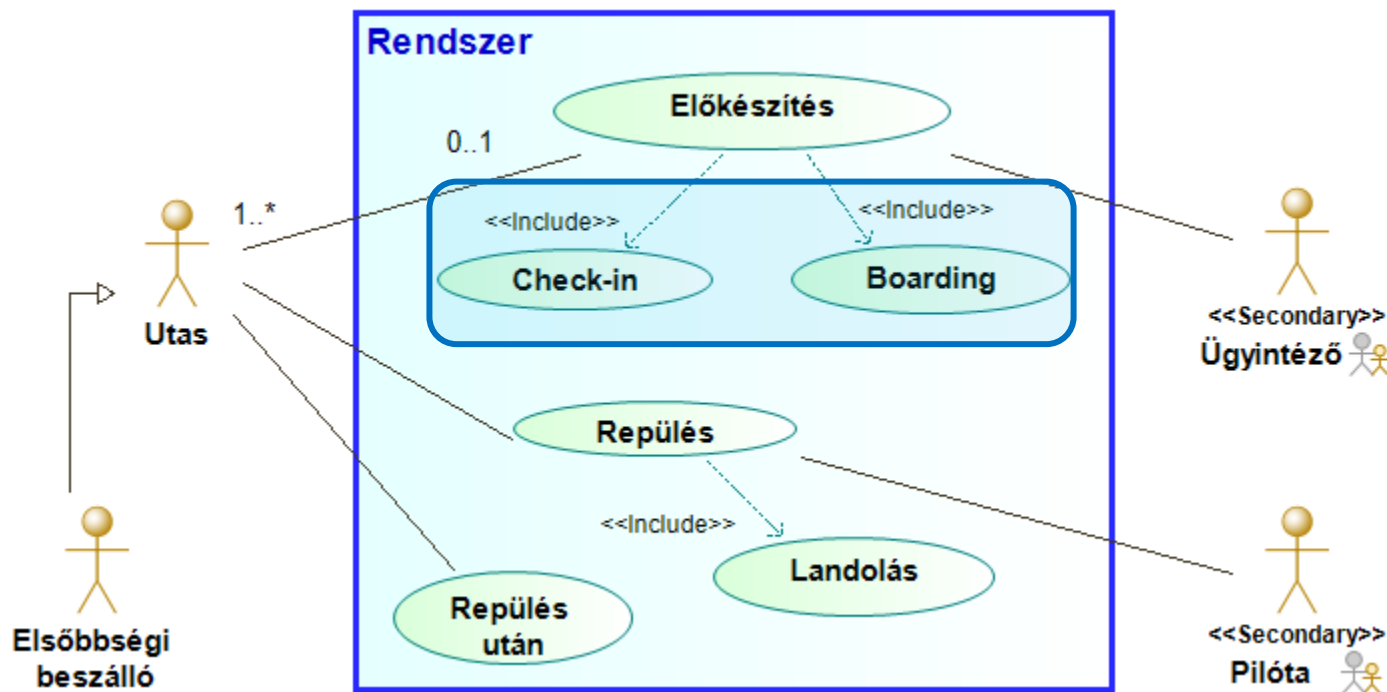
További szereplők

- Vannak elsőbbségi utasok
 - Mindent tehetnek, amit a többi utas, de jár nekik extra szolgáltatás is (l. később).
 - **Általánosítás (generalization)**
 - Aktorok és használati esetek esetében is meg lehet adni



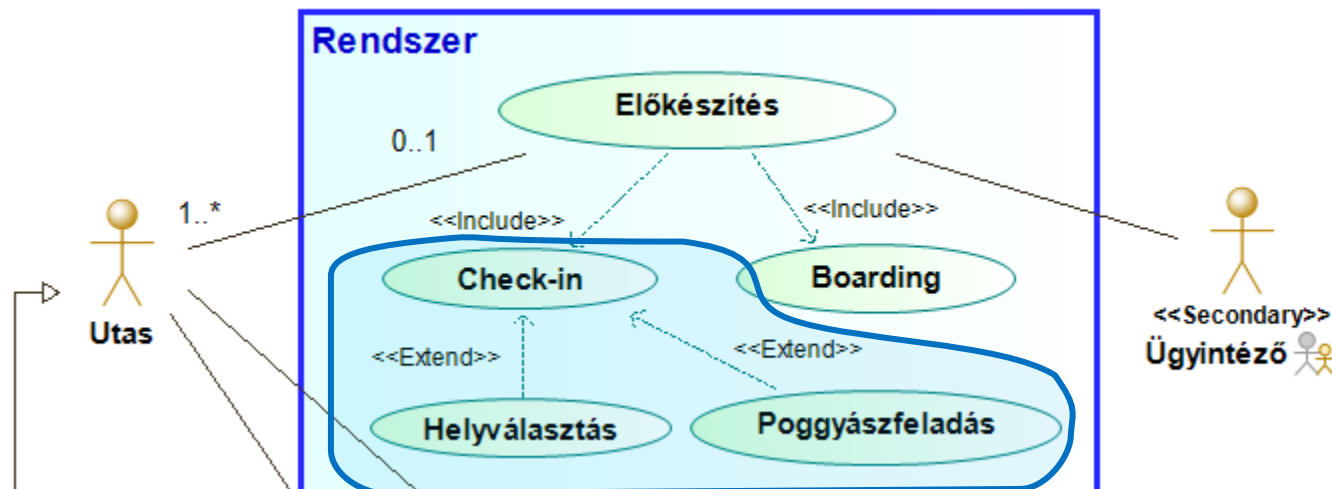
Részfunkciók

- Az előkészítés része a check-in és a boarding, a repülés része a landolás
 - > Tartalmazás (include) sztereotípa
 - > Részfunkció, rész use case (kompozíció)
 - > A tartalmazó use-case asszociációi a tartalmazottra is vonatkoznak



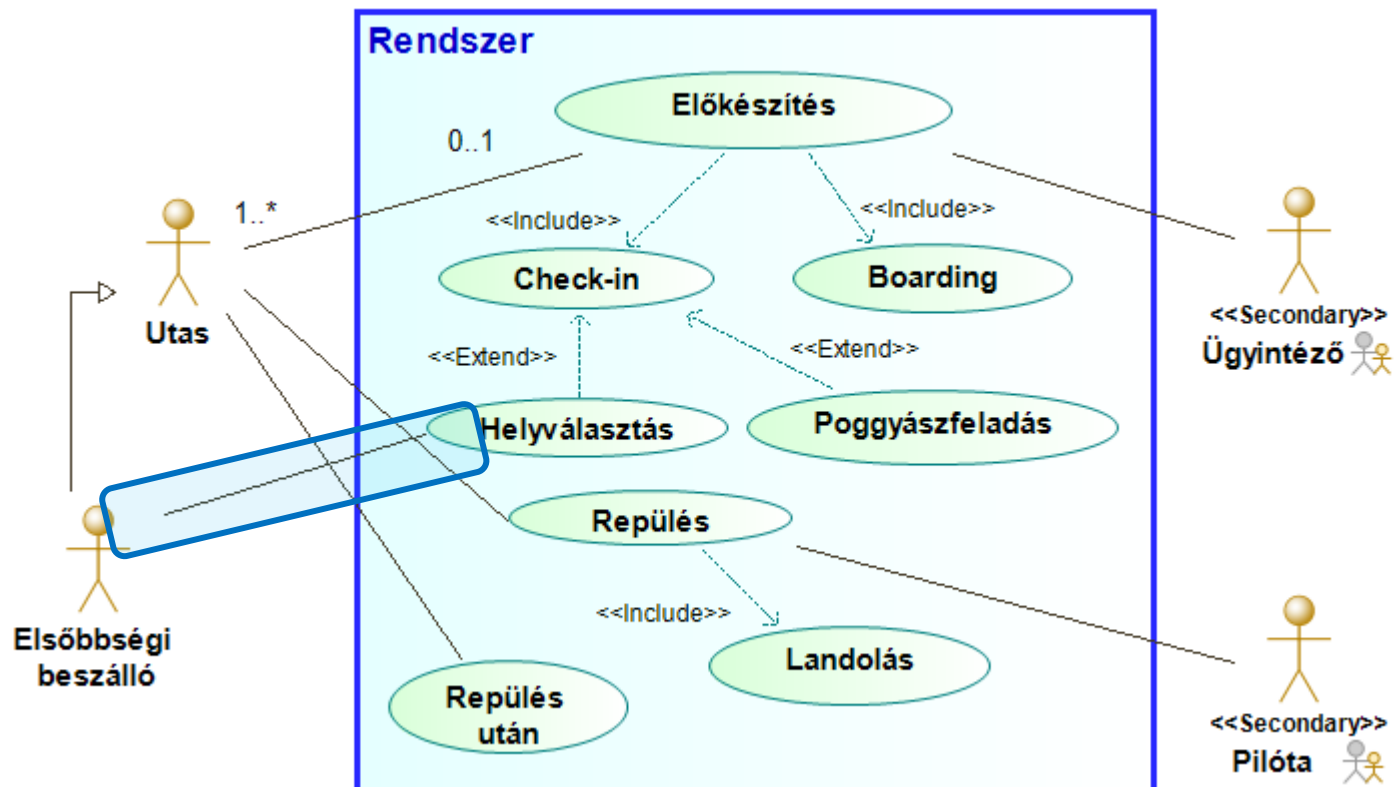
Kiegészítések

- A check-in folyamat néha kiegészül a poggyászfeladás és a helyválasztás folyamatával
 - > Nem mindig tartalmazza a funkciót, de a funkció önmagában nem értelmezett
 - > Kiegészítés (extend) sztereotípia
 - > Include vs extend vs inheritance
 - „Étteremben mindig fizetünk a végén” - include
 - „Étteremben fizetéskor néha adunk borraivalót” - extend
 - „Két fajta éttermi fizetés van: kártyás és készpénzes” - inheritance



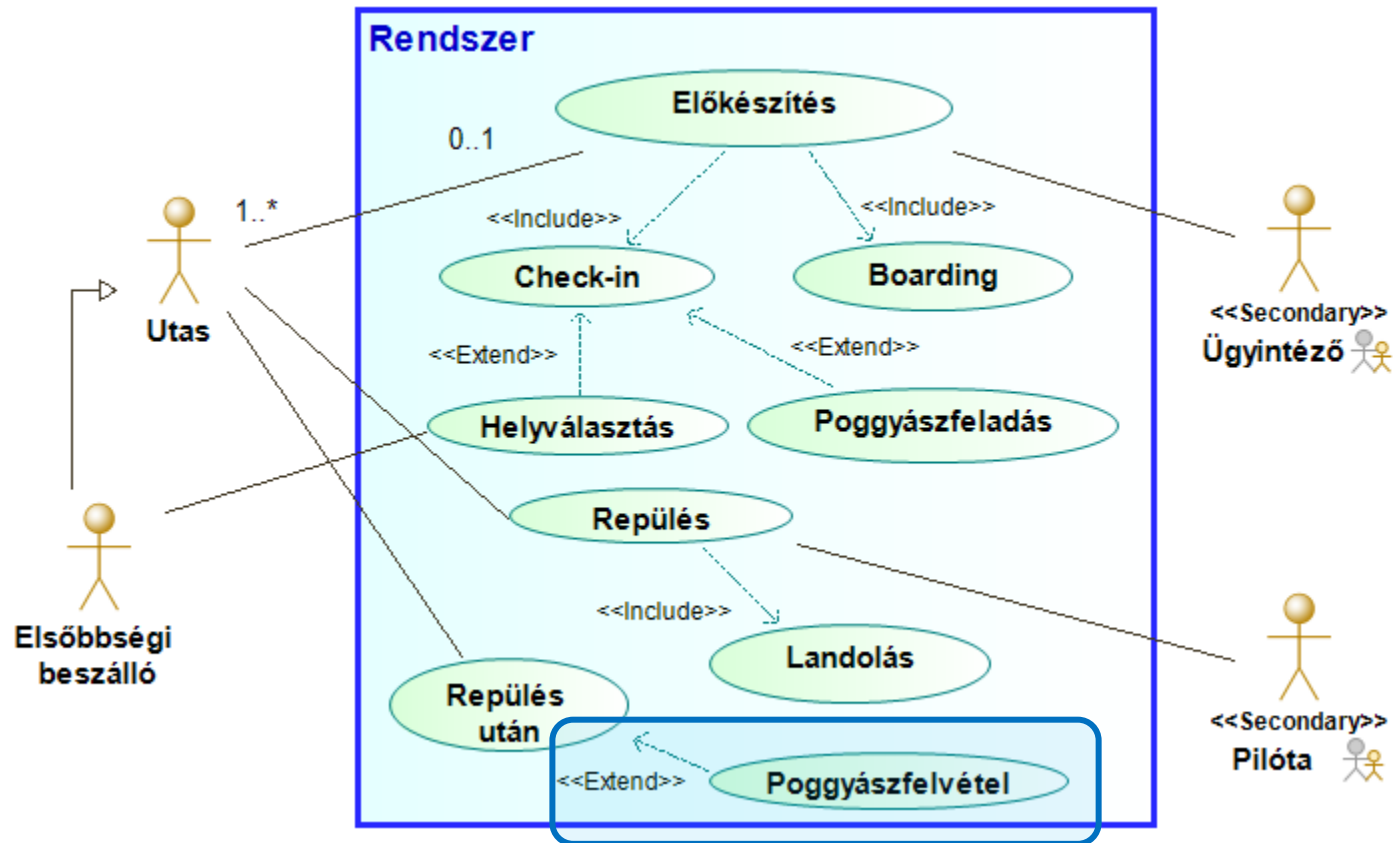
Helyválasztás

- A helyválasztás az elsőbbségi utasok számára elérhető
 - > Nem feltétlenül „öröklődik” úgy, mint az include esetén, külön jelöljük, hogy kik számára elérhető



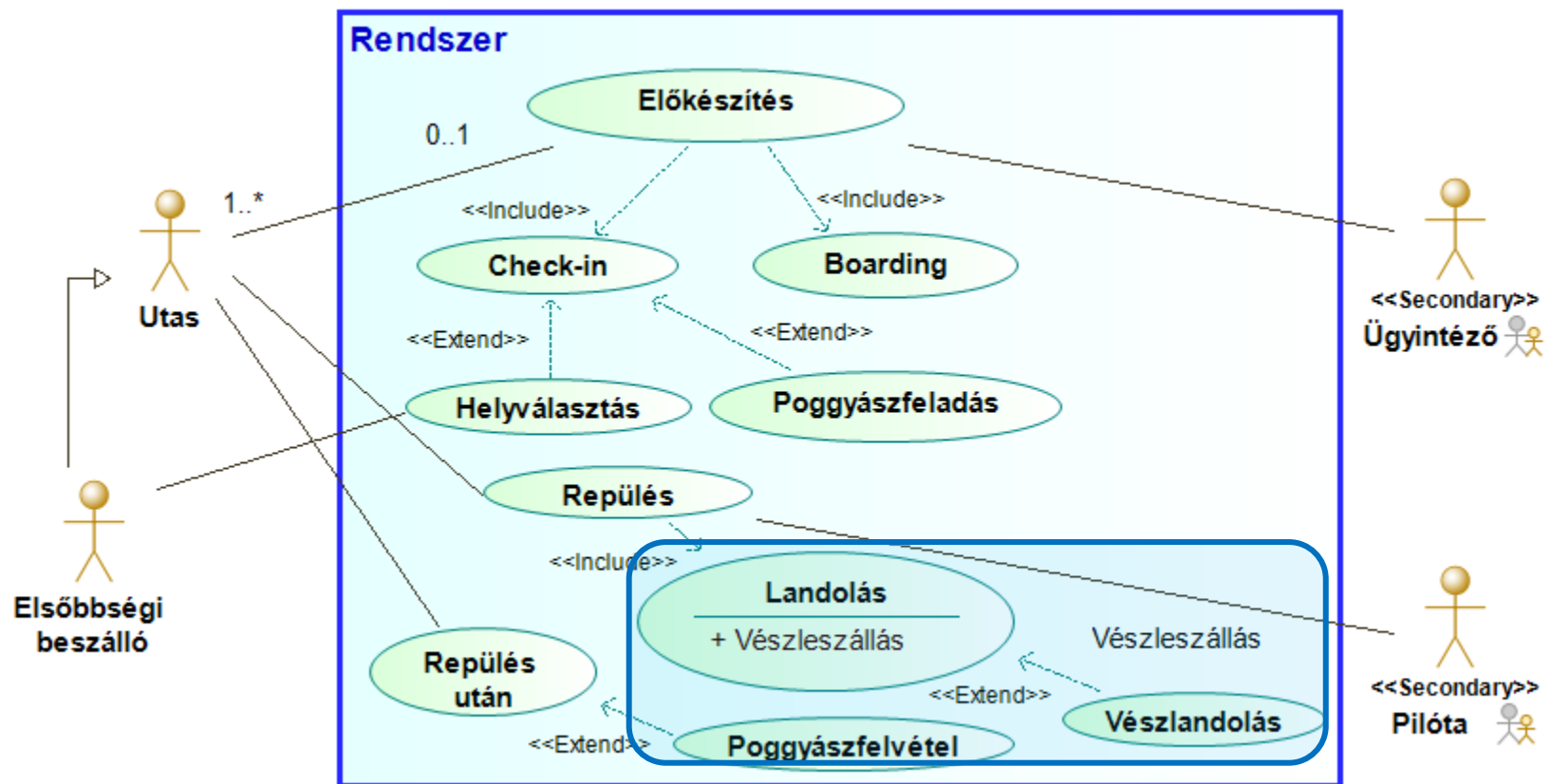
Poggyászfelvétel

- A poggyászfelvétel kiegészíthető a repülés utáni teendőket



Vészlandolás

- A landolás speciális esete a vészlandolás
 - > Kiterjesztési pont (extension point)



Forgatókönyv

- Szöveges forgatókönyv írható a diagramból
 1. Előkészítés
 - 1.1. Check-in elvégzése
 - 1.1.A. Helyválasztás
 - 1.1.B. Poggyászfeladás
 - 1.2. Boarding
 2. Repülés
 - 2.1. Landolás
 - 2.1.A. Vészleszállás
 3. Repülés után
 - 3.A. Poggyászfelvétel
- Nem része az UML szabványnak

Használati eset diagram

- Rendszer
- Aktor
- Használati eset (use-case)
- Asszociáció
- Általánosítás
- Tartalmazás (include)
- Kiterjesztés (extend)

Kitekintés része!

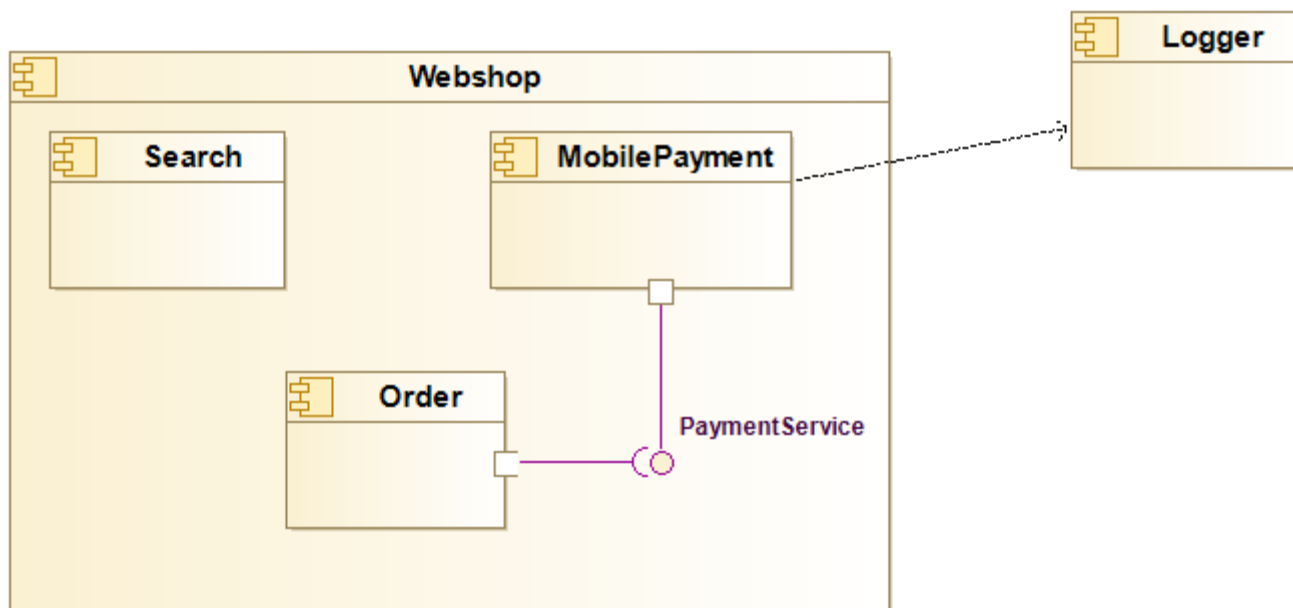
UML – diagrammok

UML diagrammok

- Rendszer modellezése
 1. Használati eset diagram
 - Követelmények, funkciók, résztvevők
 2. Aktivitás diagram
 - Folyamatok
 3. Állapotgép
 - A rendszer állapotai
 4. Osztálydiagram
 - Kódstruktúra
 5. Szekvencia diagram
 - Függvényhívások

További diagramok: komponens diagram

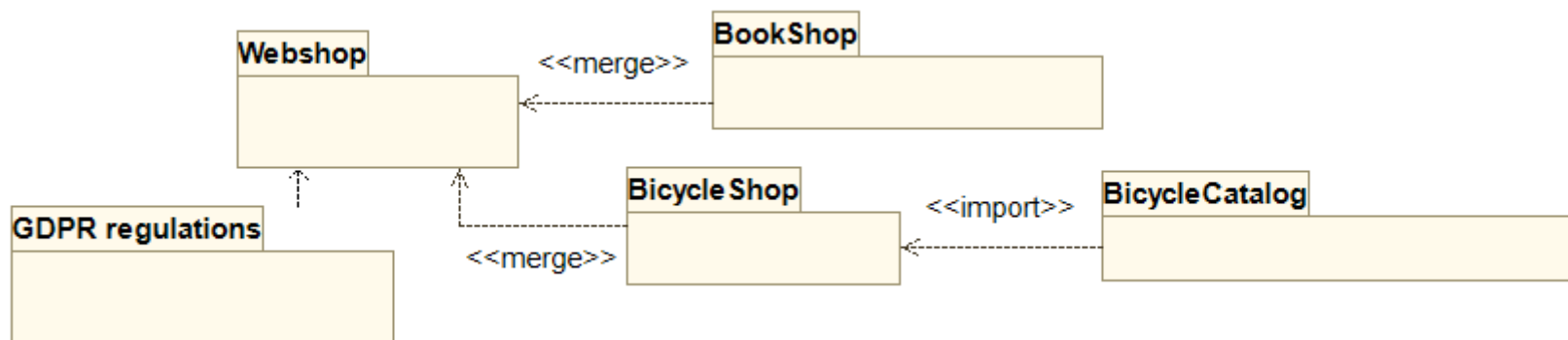
- Komponens diagram
 - > Komponens: összefüggő osztályok
 - > Hierarchikus felépítés
 - > Komponensek és függéseik
 - > Interfészek (biztosított ill. elvárt)



További diagrammok: csomagdiagram

- Csomagdiagram

- > Csomag: összefüggő információk (nem csak kód!)
- > Rendszer felbontása modell szinten
- > Csomagok közti függőségek ábrázolása
 - Függőség (dependency) – nem meghatározott kapcsolat
 - Importálás (include, import, using, ...) – újra felhasználás
 - Összevonás (merge) – egyesítés



UML diagramok

- Osztálydiagram (Class diagram)
- Szekvenciadiagram (Sequence diagram)
- Aktivitásdiagram (Activity diagram)
- Állapotgép (State machine)
- Használati eset diagram (Use case diagram)
- *Komponensdiagram (Component diagram)*
- *Csomagdiagram (Package diagram)*
- Objektumdiagram (Object diagram)
- Telepítési diagram (Deployment diagram)
- Összetett struktúradiagram (Composite Structure diagram)
- Profildiagram (Profile diagram)
- Kommunikációs diagram (Communication diagram)
- Interakció áttekintő diagram (Interaction overview diagram)
- Időzítő diagram (Timing diagram)

UML - Kényszerek

Kényszerek

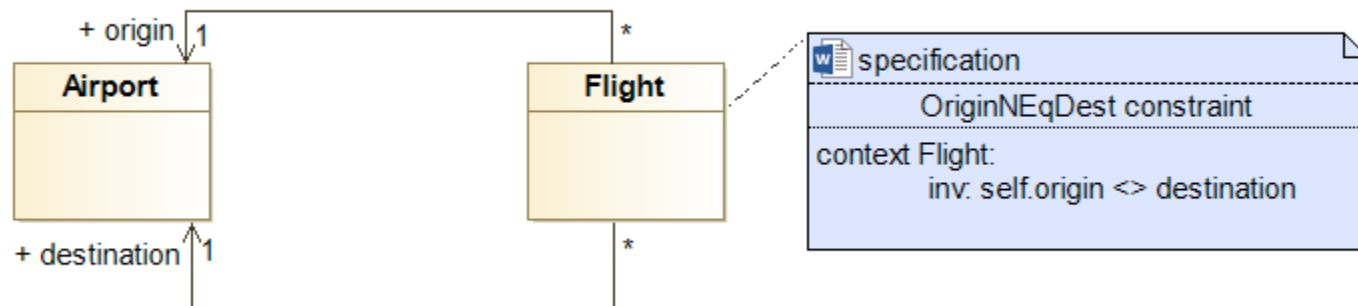
- UML: alapvetően grafikus leírás/modellek
- Grafikus leírás
 - > Könnyen átlátható
 - > Könnyen tanulható (megrendelő is érthetik)
 - > Megkötések/összefüggések nehezen írhatóak le vele
- Pédák
 - > “Mindenki fiatalabb a szüleinél.”
 - > “Senki nem lehet a saját szülője.”
 - > “Minden BME-n végzett üzemmérnök informatikus fizetése nagyobb 300 ezer Ft-nál.”

Kényszerek

- Megoldás: szöveges nyelv a grafikus modellhez illesztve
- Object Constraint Language (OCL)
 - > Deklaratív (“Mit?” a “Hogyan?” helyett)
 - > Programozási nyelvtől független
 - > UML modellekhez illeszkedik (pl. navigáció)
 - > Mellékhatás-mentes végrehajtás (lekérdez, nem változtat)
 - > Kényszertípusok
 - Invariánsok (mindig igaz feltétel)
 - Elő-, utófeltétel
 - Metódustörzs
 - Számított érték
 - Kiindulási/Alapértelmezett érték

Kényszerek

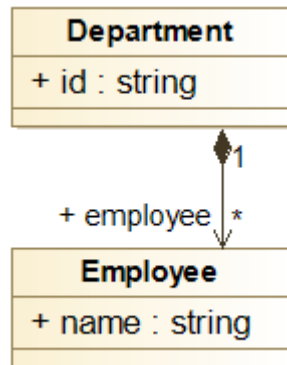
- Repülő esetén a kiindulási és célreptér nem lehet azonos!



UML – közös formátum

XMI

- UML – szabvány a modellező nyelvekhez
 - > Több UML- kompatibilis eszköz létezik
 - > Hogyan lehet adatokat cserélni köztük?
- Megoldás: XML Metadata Interchange
 - > XML-alapú
 - > Szabványos modell-leíró formátum



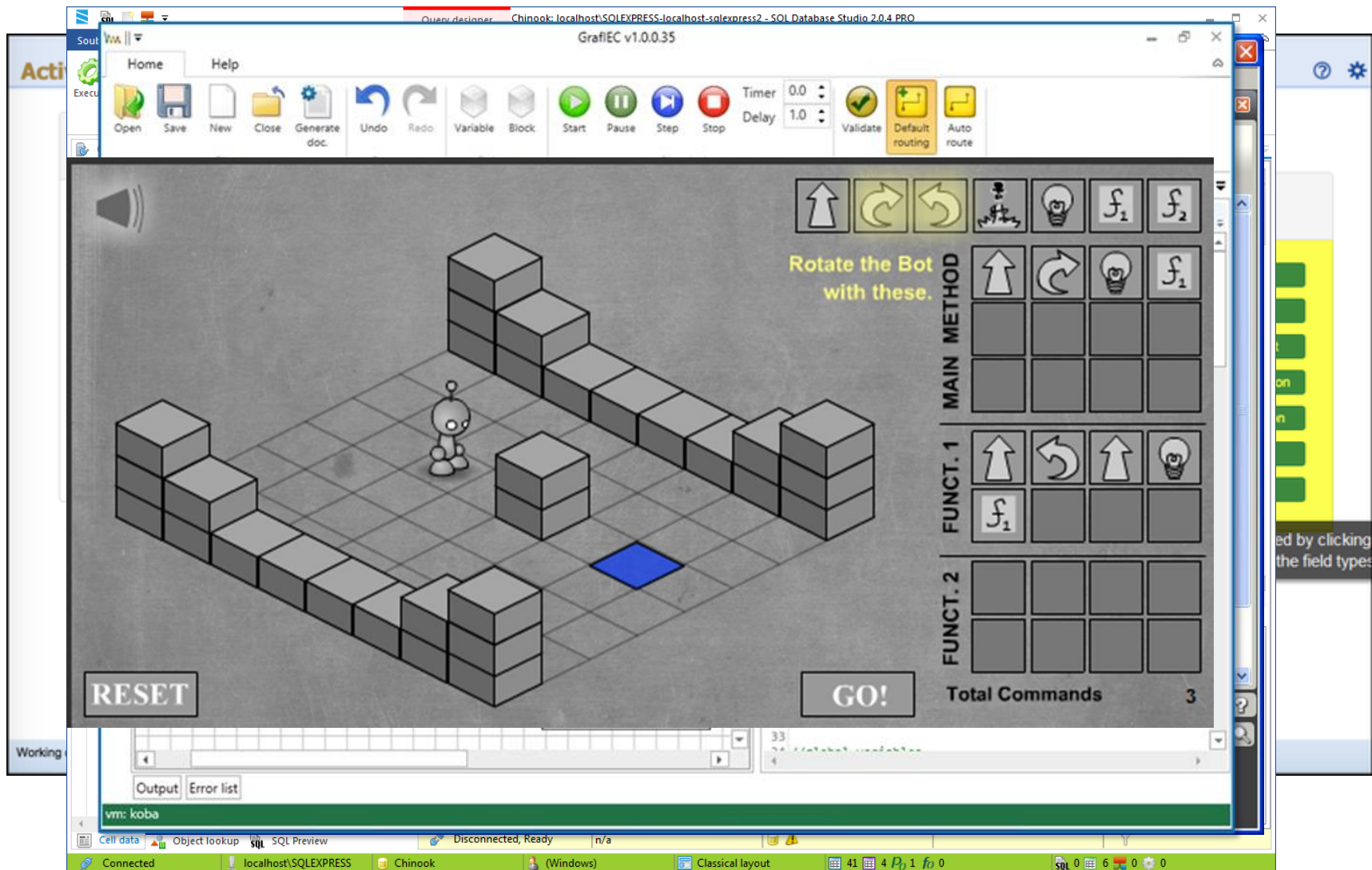
```
<Classifiers xsi:type="Class" name="Department">
  <StructuralFeatures xsi:type="Attribute" name="id" Type="String"/>
  <StructuralFeatures xsi:type="Reference" name="member" upperBound="-1"
    Type="#//Employee" containment="true"/>
</Classifiers>
<Classifiers xsi:type="Class" name="Employee">
  <StructuralFeatures xsi:type="Attribute" name="name" Type="String"/>
</Classifiers>
```

Túl az UML-en?

UML – egy nyelv(család) mind felett?

- Mire jó az UML?
 - > Szabványos
 - > Szoftvermérnökök közti közös nyelv
 - > Struktúrák, folyamatok ábrázolása
 - > Követelmények rögzítése a megrendelő felé
- Mire nem jó az UML?
 - > Nem szoftveres projektek
 - > Egyedi jelölések
 - > Szakterület-függő szabályok

Nem UML, mégis modell



UML?

- Az UML
 - > Elterjedt (mindenki ismeri a szoftver világban)
 - > Általános (minden modellezhető vele)
 - > Támogatott (rengeteg eszköz van hozzá)
 - > De nem az egyetlen modellező nyelv!



Szakterületi nyelvek

- Szakterületi nyelv
 - > Speciális nyelv egy adott szakterületre
 - > Korlátozott elemkészlet
 - > Erős, specializált szabályok és jelölés
 - > Egy adott termék(családhoz) készül
 - > Egyedi modellezőeszközök

UML vs. szakterületi nyelvek

UML

- Egységes, szabványos
- Minden szoftver-mérnök ismeri
- Jó eszköztámogatás
- Korlátozott kódgenerálás
- Megrendelő számára érthetetlen

Szakterületi nyelvek

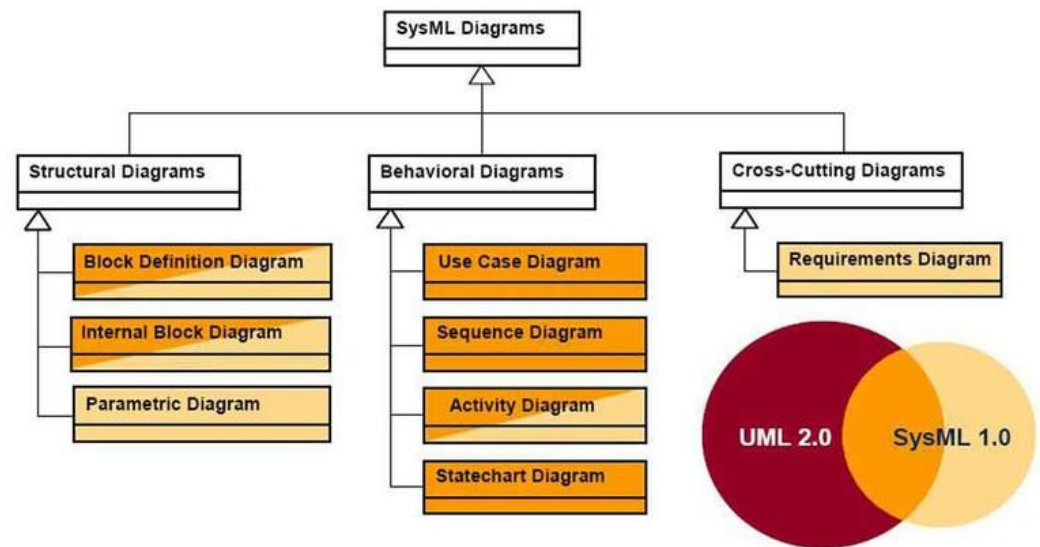
- Egyedi
- Csak a szakértők ismerik
- Egyedi eszközök kellenek
- Akár teljes kódgenerálás
- Megrendelő is jól érti

UML Profile

- Alapgondolat: terjesszük ki az UML-t
 - > Közös alapokon nyugszik
 - Könnyű megtanulni
 - Eszköztámogatás is lehetséges
 - > Szakterületi szabályok leírhatóak
- UML Profile
 - > OMG szabvány
 - > Jól formált UML részhalmoz, kiegészítésekkel
 - > Sztereotípiák, kényszerek, tag-ek
 - > Nem mond ellent az eredeti specifikációnak!

UML Profile

- UML profile for XML
- MARTE – valósidejű rendszerekhez
- SysML – rendszermodellezés
 - > Kevésbé szoftver-centrikus
 - > Elhagy és definiál diagram típusokat



Metamodellezés

- UML
 - > A nyelvek szabványban leírtak
 - > Nem lehet megváltoztatni őket
- Modellezzük a modellező nyelveket!
- Metamodellezés
 - > A nyelvek leírását is egy modellben adjuk meg!
 - > Rugalmasan összerakhatóak “saját” nyelvek
 - > Amire szükség van: nyelvleíró nyelv

Meta-Object Facility (MOF)

- OMG szabvány
- UML általánosítása
- Önleíró, négyszintű metamodellezési hierarchia
 - > 0. szint: objektumok
 - Konkrét személy objektum (pl. Nagy Lajos)
 - > 1. szint: UML modellek
 - A személy osztály definíciója (osztálydiagram)
 - > 2. szint: UML modellek modellje
 - Az osztálydiagram definíciója (és a többi diagrammé)
 - > 3. szint: MOF önleíró alapstruktúra
 - Általános modelldefiníció nyelv

MOF a gyakorlatban

- MOF – két változat
 - > Complete MOF – nem használt
 - > Essential MOF – gyakorlatban is használt
- Eclipse Ecore
 - > 95%-ban szabványos EMOF megvalósítás
 - > Az Eclipse Modeling Framework alapja
 - > Általános szakterület-, és metamodellezési keretrendszer
 - > XMI támogatással

Összefoglalás: Modellezés

Köszönöm a figyelmet!

Feladat: Use case diagram

- A könyvtárban a tagok könyveket tudnak kölcsönözni, visszahozni, valamint Internetezni tudnak
- A kölcsönzés a könyv kiválasztásával kezdődik, majd a könyvet a könyvtárhoz kell vinni, aki bejegyzi a kölcsönzést a rendszerbe
- Egyszerre maximum 5 könyvet lehet kölcsönözni
- A könyv visszahozatalakor a könyvtáros veszi vissza a könyvet és késedelem esetén büntetést szab ki a tag számára
- Az internetezéshez el kell kérni a Wifi kódot a könyvtárostól, de azt csak a főkönyvtáros tudja megadni
- A régi tagok számára lehetőség van előjegyzéseket is megadni, a normal tagoknak erre nincs lehetősége



Szoftvertchnológia és -technikák

6. Előadás – Benedek Zoltán
Tervezési minták 1



Automatizálási és
Alkalmazott
Informatikai Tanszék

Ez az oktatási segédanyag a Budapesti Műszaki és Gazdaságtudományi Egyetem oktatója által kidolgozott szerzői mű. Kifejezett felhasználási engedély nélküli felhasználása szerzői jogi jogsértésnek minősül.

Tartalom

Tervezési minták 1

- > Definíció
- > Áttekintés
- > Szervezésük
- > Miben segítenek a tervezési minták
- > Template Method
- > Strategy

Definíció

- Programtervezési minta v. tervezési minta v. design pattern
- Tervezési minta leír egy gyakran előforduló programtervezési problémát, annak környezetét és a megoldás magját, amit alkalmazva számos gyakorlati eset hatékonyan megoldható.

Irodalom

- Az interneten bármely minta nevére rákeresve számos leírást találunk, a legtöbb minta a wiki-n is megtalálható
 - > (de kezeljük némi fenntartással, ezek a wiki leírások lehetnek hiányosak, vagy nem feltétlen a leggyakorlatiasabb megközelítést mutatják be)
 - [1] Erich Gamma, Richard Helm, Ralph Johnson, John Vlissides: DESIGN PATTERNS, Elements of Reusable Object-oriented Software
 - > A nagy klasszikus, de ma már kicsit „régies”, ebben C++ és más, ma már ritkábban használt nyelven vannak a példák
 - https://sourcemaking.com/design_patterns
-
- Megjegyzés: előadáson és gyakorlatokon C# példák lesznek

Példakódok

- A legtöbb mintához tartozik példakód, kódmegjegyzésekkel ellátva
- GitHub-on érhető el
 - > A kód webböngészőben is nézhető/böngészhető a kód <https://github.com/bzolka/AUT-SZTT>
 - Ez alatt navigáljunk le a Előadás/Demo/DesPattCode mappába, az egyes minták demo kódja általában a mintának megfelelő nevű almappában található
 - > De célszerűbb az egészet a <https://github.com/bzolka/AUT-SZTT> alatt „Clone and download” gomb segítségével letölteni (vagy a “git clone https://github.com/bzolka/AUT-SZTT” paranccsal parancssorból klónozni), így Visual Studio-ban a solution megnyitható.
 - > A legtöbb mintához „futtatható” kód is tartozik: a mintához tartozó mappában levő Program.cs fájlban a Main2 függvényt előbb nevezzük át Main-re
 - > Ha más mintához tartozó kódot szeretnénk „futtatni”, nevezzük vissza a Main-t Main2-re, mert a VS projektünkben csak egy Main nevű statikus függvény lehet: ha több is van, fordítási hibát kapunk.

Bevezető példa

Példa

- Egy alkalmazást fejlesztünk, melyben adatokat kell tömöríteni
 - > Több tömörítési algoritmust szeretnénk támogatni: zip, rar, 7zip
 - > Futás közben is lecserélhető legyen az algoritmus
 - > Könnyen bővíthető legyen új algoritmusokkal

Példa – kiindulás

Csak zip algoritmust támogat (nézzük VS alatt)

```
class DataProcessor
{
    // Ciklusban beolvassa, tömöríti, majd feldolgozza a tömörített adatokat
    public void Run()
    {
        byte[] inputData;
        while ( (inputData = readData()) != null) {
            byte[] compressedData = zipData(inputData);
            processCompressedData(compressedData);
        }
    }

    byte[] readData() { /* adat olvasása hálózatról */ }

    byte[] zipData(byte[] data) { /* adattömörítés zip algoritmussal*/ }

    void processCompressedData(byte[] data) { /* tömörített adat feld.*/*; }
}
```

Példa –előző továbbfejlesztve

Több algoritmust támogat – 1. rész

```
class DataProcessor2
{
    // ...

    CompressionAlg compressionAlg;

    public DataProcessor(CcompressionAlg compressionAlg)
    {
        this.compressionAlg = compressionAlg;
    }

    public void Run() // változatlan
    {
        byte[] inputData;
        while ((inputData = readData()) != null)
        {
            byte[] compressedData = compressData(inputData);
            processCompressedData(compressedData);
        }
    }

    byte[] readData() { /* adat olvasása hálózatról */ }
    void processCompressedData(byte[] data) { /* töm. adat. feld. */; }
```


Példa –továbbfejlesztve

Több algoritmust támogat – 2. rész

```
byte[] compressData(byte[] data)
{
    switch (compressionAlg)
    {
        case CompressionAlg.Zip:
            return compressUsingZip(data);
        case CompressionAlg.Zip7:
            return compressUsingZip7(data);
        case CompressionAlg.Rar:
            return compressUsingRar(data);
    }
}

byte[] compressUsingZip(byte[] data) { /* adattömörítés zip alg. */ }
byte[] compressUsingZip7(byte[] data) { /* adattömörítés 7zip alg.*/ }
byte[] compressUsingRar(byte[] data) { /* adattömörítés rar alg. */ }

}
```

Példa –továbbfejlesztve

- Megoldás elve
 - > Konstruktorban átadjuk paraméterként (enum), milyen tömörítő algoritmust szeretnénk, ezt eltároljuk egy tagváltozóban (compressionAlg)
 - > Ezen tagváltozó alapján választjuk ki később a tömörítő algoritmust
- Problémák
 - > A megoldás nem bővíthető könnyen új algoritmussal
 - A DataProcessor2 osztályba be van építve, milyen algoritmusokat ismer.
 - Ha újat szeretnénk bevezetni, a DataProcessor2 kódját módosítani kell. Ezt követően pedig újra kell tesztelni az osztályt.
 - Olyan bővíthetőségi megoldást keresünk, mely NEM igényli a DataProcessor2 módosítását új algoritmus bevezetésekor.
 - A megoldás majd a **STRATEGY** tervezési minta alkalmazása lesz, rövidesen visszatérünk rá
- Sok tervezési minta létezik, pl. Strategy, Singleton, Composite, Observer, stb.
 - > Mind más programtervezési problémára nyújt megoldást

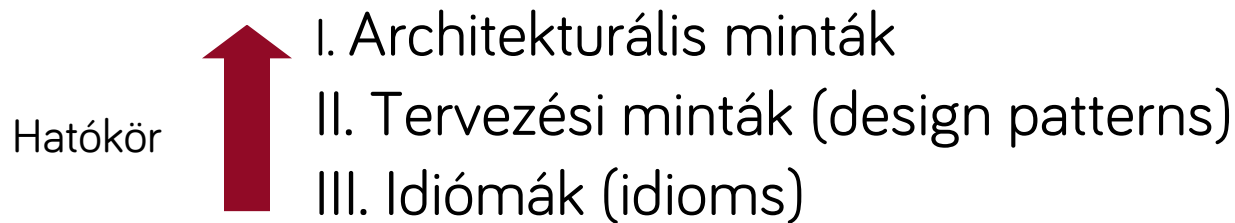
Tervezési minták jellemzői

Tervezési minták leírása

- Minden tervezési minta leírásához négy alapelem tartozik
 - > Pattern név (pattern name)
 - A minta neve, hatékonyan azonosítja a mintát.
 - **Fontos a kommunikáció miatt:** a fejlesztőknek elég a minta nevére hivatkozniuk, ebből már értik is, hogy milyen objektumok milyen szerepkörben szerepelnek az adott tervben/kódban (persze csak ha ismerik az adott mintát)
 - > Probléma (problem)
 - A **probléma** és a **környezet (context)** bemutatása, gyakran konkrét példán keresztül
 - > Megoldás (solution)
 - Leírja a megoldásban szereplő elemeket, kapcsolatukat, az egyes elemek felelősségét és együttműködését. Nem konkrét eset megoldása, inkább a megoldás absztrakt leírása.
 - > Következmények (consequences)
 - A minta alkalmazásának következményeit írja le. Fontos tapasztalatokat tartalmaz, és segít a minta kiválasztásban.

Minták csoportosítása hatókör szerint

- A szoftverrendszer milyen szintjén használhatók (architektúra → alrendszerek → ... → kódolás egy adott programnyelven)
- Három csoport



Minták csoportosítása hatókör szerint

- **Architektúra szint**

- > Alapvetően meghatározza az adott alkalmazás/alrendszer felépítését
- > Egy későbbi előadáson megnézünk párat, pl. MVVM, Document-View, Layers

- **Tervezési minta szint**

- > Egy alkalmazásban/alrendszerben több, sok is használható
- > Függetlenek a programozási nyelvektől pl: Singleton / Strategy

- **Idióma**

- > Egész alacsony szintűek, sokszor csak adott programozási nyelv kontextusában használhatók pl: Python - with open ("file.txt" as f

- Mostantól a középső, „**Tervezési minta**” szintre besorolható tervezési mintákról lesz szó

- > Ezt nem kell túl szigorúan venni, több design pattern jól alkalmazható architektúra vagy idióma szinten is

„Klasszikus”/GoF tervezési minták

- Alapirodalom
 - > [Erich Gamma](#), Richard Helm, Ralph Johnson, John Vlissides: DESIGN PATTERNS, Elements of Reusable Object-oriented Software
 - > Először nevesítettek és katalogizáltak tervezési mintákat
 - > Négyen írták: köznyelvben „[Gang of Four](#)” (röviden GoF), vagyis a „négyek bandája”ként szokás rájuk hivatkozni.
 - Nagyon elterjedt ez a hivatkozás, illik tudni!

GoF tervezési minták kategóriák

A kiemelteket nézzük majd (ezeket kell tudni)

Létrehozási (creational)	Strukturális (structural)	Viselkedési (behavioral)
Factory Method	Adapter	Template Method
Abstract Factory	Composite	Strategy
Singleton	Façade	Observer
Builder	Proxy	Command
Prototype	Bridge	Memento
	Decorator	Interpreter
		Chain of Responsibility
		Iterator
		Mediator
		Flyweight
		State
		Visitor

Hogyan segítenek a tervezési minták a probléma megoldásában

- A **tervezési** minták a fejlesztés **tervezés** fázisában segítenek (az analízis minták az analízis fázisában)
 - > Tervezés: amikor kódra „képezzük le” az analízis során kidolgozott fogalmi modellt
- Az alábbiak elérése nehéz, ezekben a magasszintű célokban segítenek a tervezési minták:
 - > I. Könnyen változtatható, kiterjeszthető kódot készíteni (design for change)
 - > II. Újrafelhasználható kódot készíteni (design for reuse)
 - > III. Könnyen unit (egység) tesztelhető kódot készíteni

Részletesebben



I. Változtathatóság, bővíthetőség

- Nem adódik magától, szem előtt kell tartani a tervezéskor. A tervezés egyik célja!
- Miért?
 - > Rendszert tervezni nehéz
 - > Sokszor nem is tudjuk az elején pontosan definiálni a feladatot, a megrendelő sem lát nagyon előre, nem is lehet elvárni tőle
 - Az utóbbi években előtérbe kerülő **agilis szemléletmód** esetén ez hatványozottan igaz
 - A szoftver termékek az első kiadás után általában hosszú éveken át, számos iterációban fejlődnek tovább
 - > A szoftverfejlesztésben csak egy dolog (biztos) és változatlan: maga a változás
 - Változnak a felhasználói követelmények
 - Változnak a technológiák

I. Változtathatóság, bővíthetőség

- Tervezési alapelv: tervezzünk úgy, hogy a rendszer nem végleges (design for change)!

Válasszuk külön az alkalmazás azon részeit, melyek változatlanok és melyek változnak (arra számítunk, hogy változni/bővülni fognak). Zárjuk egységbe ezen változó kódrészeket, hogy később úgy tudjuk bővíteni/változtatni az alkalmazást, hogy a nem változó funkciók/részek kódjához ne kelljen hozzányúlni.

- > Nem mindent tervezünk bővíhetőre, mert költséges!
 - Csak ahol számítunk arra, hogy bővíteni/módosítani kell majd

I. Változtathatóság, bővíthetőség

- Negatív példa: készítünk egy alkalmazást, melyben minden mindennel összefügg, az SRP elvekre sem ügyelünk, valamint hiányzik a kritikus pontokban a módosításra/bővíthetőségre való felkészítés
 - > A kollégák nem fognak szeretni minket, ha a kódunkhoz kell nyúlniuk
 - > Nekünk sem lesz kedvünk a kódhoz többet hozzányúlni
 - > Egy bug javításával/pici módosítással
 - Sok kódot kell átnézni, újra tesztelni
 - Több új bugot viszünk be a rendszerbe
- Pozitív példa
 - > A meglévő kód bővítése, módosítása is pont annyira élvezhető feladat, mintha teljesen új kódot kellene írni

II. Újrafelhasználhatóság (reuse)

- Nem adódik magától, szem előtt kell tartani a tervezéskor. A szoftvertervezés egyik célja!
- A költségcsökkentés leghatékonyabb módja a cégek számára
- Akár egy projekt különböző moduljaiban, akár projektek között is!
- Ha az alkalmazásban minden kódrészlet mindennel összefügg, akkor nem tudunk belőle részeket újra felhasználni, mert minden rész újrafelhasználásához szükség van az összes függőségre
 - > Lazán csatolt „részekre” van szükségünk
- Hosszú távon kifizetődő lehet: saját keretrendszer kifejlesztése

II. Újrafelhasználhatóság (reuse) *

- **Kitérő: Keretrendszer (framework) definíció***

- > Minden keretrendszer adott probléma területet céloz meg (pl. ablakozós rendszerek, dokumentumkezelő rendszerek, matematikai rendszerek, grafikus rendszerek, CAD rendszerek, játékok, stb.)
- > Meghatározza az alkalmazás architektúráját, ehhez alapvető építőköveket (pl. osztályok) biztosít.
- > OO környezetben: előre definiált, együttműködő osztályok vannak, pontosan meghatározott felelősséggel. Az alkalmazás fejlesztésekor gyakran:
 - le kell belőlük származtatni,
 - Virtuális/absztrakt függvényeket kell felüldefiniálni
- > **Cél: Adott szakterületen egy alkalmazás fejlesztésekor a lehető legkevesebbet kelljen kódolni (mert a keretrendszer már szolgáltatásként nyújtja)**
- > Konzisztens struktúrája lesz az alkalmazásainknak
- > Magas szinten már “tervezni sem kell”, hiszen az architektúra meghatározott - terv újrafelhasználás (design reuse)

pl:

Java - Spring framework

JavaScript - React (UI)

III. Unit (egység) tesztelhetőség

- Egy hosszú életű projekt esetén fontos, hogy a kód minél nagyobb részéhez automata módon futtatható **unit (egység) tesztek** készítsünk (legalábbis a logikai részekhez, felhasználói felületekhez kevésbé)
- **Unit tesztek esetén kóddal tesztelünk kódot**
- Egy unit teszt az osztályt önmagában, a függőségei (más logikai osztályok, melyeket használ) nélkül tesztel
 - > A unit tesztek futtatásához az osztály függőségeit le szeretnénk cserélni olyan implementációkra, melyek a tesztet szolgálják
 - > Ha egy osztályba be vannak építve a függőségei, akkor ez nem tehető meg, az osztály gyakorlatilag nem lesz unit tesztelhető
 - > A megoldás: az osztály a függőségeit lazán csatoltan kell kezelje (pl. interfészeken keresztül)
- Bizonyos tervezési minták abban is segítenek, hogy a kódunk könnyen unit tesztelhető legyen

System of Patterns *

- Mi a system of patterns?
 - > A rendszer tervezésekor több mintát is felhasználunk
- Általában egy (illetve alrendszerenként egy) architektúra szintűt
 - > Ez megadja az alapstruktúrát
- Az egyes részek megtervezésekor (detailed design) több tervezési mintát is felhasználunk, akár egymással kombinálva
- A kombinált megoldásba minden minta beleviszi a maga pozitív tulajdonságait, vagyis egy megoldást egy részproblémára

Néhány fontosabb, gyakrabban használt tervezési minta

- Létrehozási
 - > Factory method
 - > Abstract factory
 - > Singleton
 - > Dependency Injection*
- Strukturális és viselkedési
 - > Template method
 - > Strategy
 - > Observer
 - > Command
 - > Command Processor
 - > Memento
 - > Adapter
 - > Facade
 - > Proxy
 - > Composite

Mit kell tudni?

A minta neve alapján vagy példán, vagy általánosságában ismertetni a mintát (milyen környezetben, milyen problémát, hogyan old meg) . A bemutatáshoz a legtöbb mintához osztály, illetve néhány esetben szekvencia diagramot is kell rajzolni).

+

Szöveges feladatléírás alapján rá kell jönni, milyen mintákat célszerű bevetni, és ezeket alkalmazni is kell tudni.

Bővíthetőséghez, kiterjeszthetőséghez kapcsolódó alap tervezési minták

Template method (sablon metódus) minta

Strategy (stratégia) minta

Egyéb

Célok, elvek

- Azon osztályoknál, melyeknél fontos a bővíthetőség, építsük be ennek lehetőségét
 - > Absztrakt/virtuális függvények hívásával -> Template Method minta
 - > Delegáljuk más osztályoknak -> Strategy minta
- A cél, hogy ha bővíteni kell a funkcionalitást, az osztály kódját NE KELLJEN MÓDOSÍTANI

Template method (Sablonmetódus)

Korábbi példa némi kiegészítéssel

- Egy alkalmazást fejlesztünk, melyben adatokat kell tömöríteni
 - > Több tömörítési algoritmust szeretnénk támogatni: zip, rar, 7zip
 - > Könnyen legyen bővíthető új tömörítési algoritmusokkal
 - > Futás közben is legyen lecserélhető az algoritmus
 - > Legyen megszakítható (cancel) a művelet, de ne legyen beégetve a megszakítás módja, pl.:
 - Billentyű lenyomás
 - Gomb kattintás

Kiindulási példa áttekintése

- Lásd: DesPattCode\
StrategyAndTemplateMethod_Common\
DataProcessor_Initial mappa DataProcessor osztály
feletti megjegyzések !

https://github.com/bzolka/AUT-SZTT/blob/master/EI%C5%91ad%C3%A1s/Demo/DesPattCode/DesPattCode/StrategyAndTemplateMethod_Common/DataProcessor_Initial/DataProcessor.cs

DataProcessor

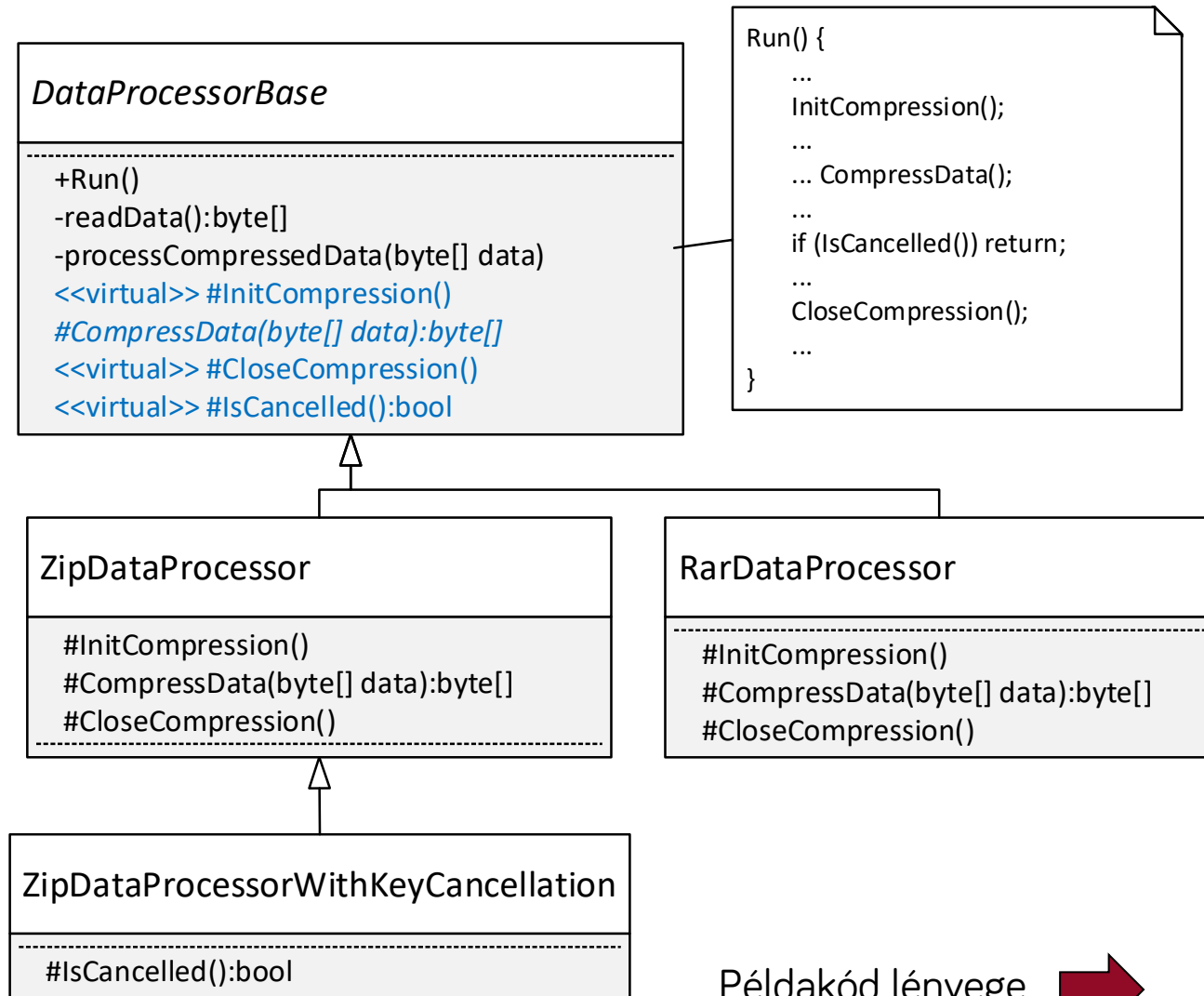
```
+Run()  
-readData():byte[]  
-processCompressedData(byte[] data)  
-initZip()  
-compressData(byte[] data):byte[]  
-closeZip  
-isCancelled
```

Template method minta alkalmazása

- A változatlan kódrészeket tegyük egy **ősosztályba**
- A változó/kiterjeszthető részeket nem drótozzuk az ős műveleteibe, hanem virtuális és/vagy absztrakt függvényekre bízunk
- A leszármazott osztályban ezen függvények felülírásával adjuk meg a specifikus viselkedést
 - > A példában:
 - Tömörítés módja
 - Cancel módja

Template Method példa áttekintése

- Lásd forráskód:
DesPattCode\
TemplateMethod
mappában fájlok
- **DataProcessorBase**
ősosztály a lényeg, a
különböző
implementációkra
közös kódot
tartalmazza. Az
implementációfüggő
részeket
virtuális/absztrakt
függvényekre bízva
(InitCompression,
CompressData,
CloseCompression,
IsCancelled)



Példakód lényege ➡

Template Method példa

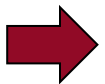
- **DataProcessorBase** őssosztály **Run** művelete a lényeg, a különböző implementációkra közös kódot tartalmazza. Az implementációfüggő részeket virtuális/absztrakt függvényekre bízza (**InitCompression**, **CompressData**, **CloseCompression**, **IsCancelled**)

```
...
public void Run()
{
    byte[] inputData;
    InitCompression(); // Tömörítés inicializás, impl. függő
    try
    {
        while ((inputData = readData()) != null)
        {
            // Tömörítés, impl. függő
            byte[] compressedData = CompressData(inputData);
            processCompressedData(compressedData);
            if (IsCancelled()) // Cancel vizsgálat, impl. függő
                return;
        }
    }
    finally
    {
        CloseCompression(); // Tömörítés lezárás, impl. függő
    }
}
...
```

Template Method célja

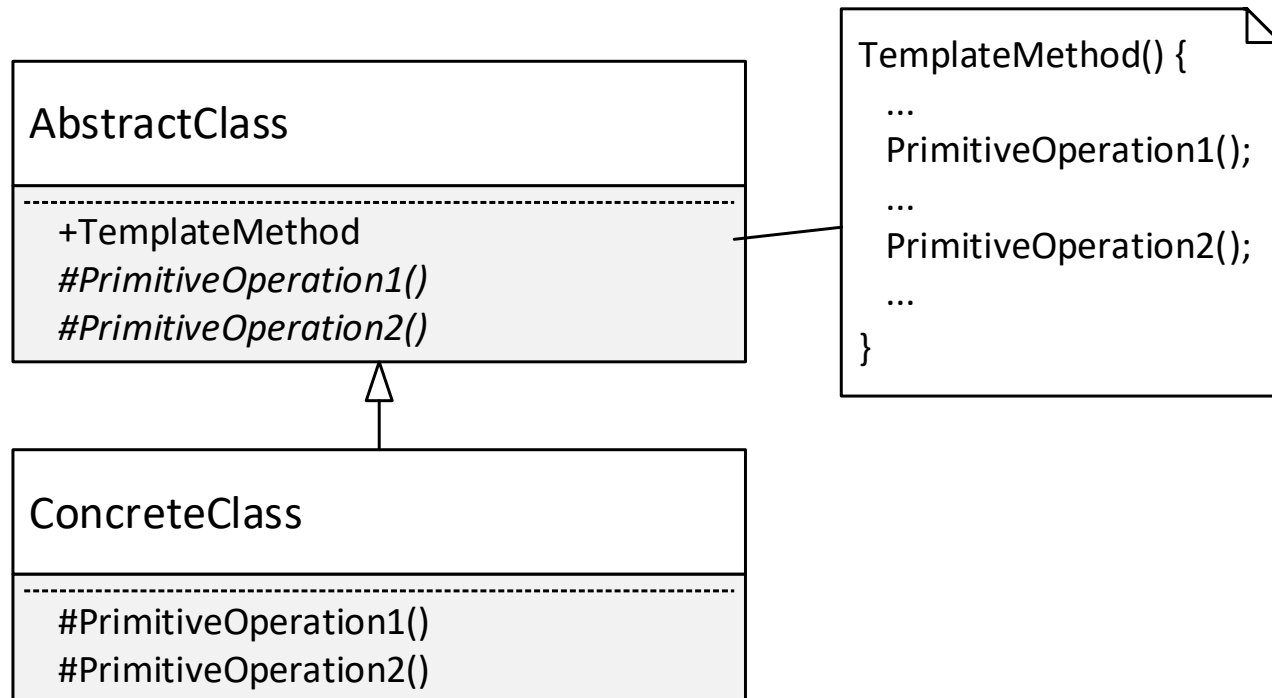
- Egy műveleten belül algoritmus vázat definiál, és az algoritmus bizonyos lépéseinek implementálását a leszármazott osztályra bízta.
 - > A példánkban az algoritmus váz a Run függvény volt, a leszármazottra bízott lépések az InitCompression, CompressData, CloseCompression, IsCancelled

Általánosítsuk



Template Method általánosan

- Amit eddig néztünk (DataProcessor), az csak egy **példa** volt a Template Method mintára. **Maga a minta teljesen általános:**



Template Method következmények

- Következmények

- > Lehetővé teszi, hogy az algoritmus/folyamat invariáns részeit egy helyen definiáljuk és a változó részeket a leszármazott osztályban adjuk meg.
- > Így megoldható a kód duplikálás elkerülése (DRY elv!): a hierarchiában a közös kódrészeket a szülő osztályban egy helyen adjuk meg (template method-ban), mely a különböző viselkedést megvalósító egyéb műveleteket hívja meg. Ezeket a “különböző viselkedést megvalósító egyéb műveleteket” a leszármazott osztályban felül kell/lehet definiálni.
- > Lehetővé teszi kiterjesztési pontok definiálását a kódban (ún. hook függvényeknek is szokás nevezni)

- Megjegyzés

- > Keretrendszerek esetében gyakori

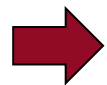
Kiértékelés

- Pozitívum
 - > Kódduplikáció elkerülése (lásd előző dia)
 - > **Új viselkedés könnyen bevezethető**, nem kell a meglévő kódot (lényegi helyen) változtatni
 - A példában DataProcessorBase kódjához nem kell hozzányúlni, csak le kell belőle származtatni egy új osztályt
- Negatívum
 - > Futás közben nem tudjuk egyszerűen cserélni viselkedést, rugalmatlan
 - A származtatási hierarchia fordításkor eldől
 - > Sok keresztkombinációra lehet szükség, lásd következő diák

Ha a viselkedésnek több aspektusa/dimenziója van

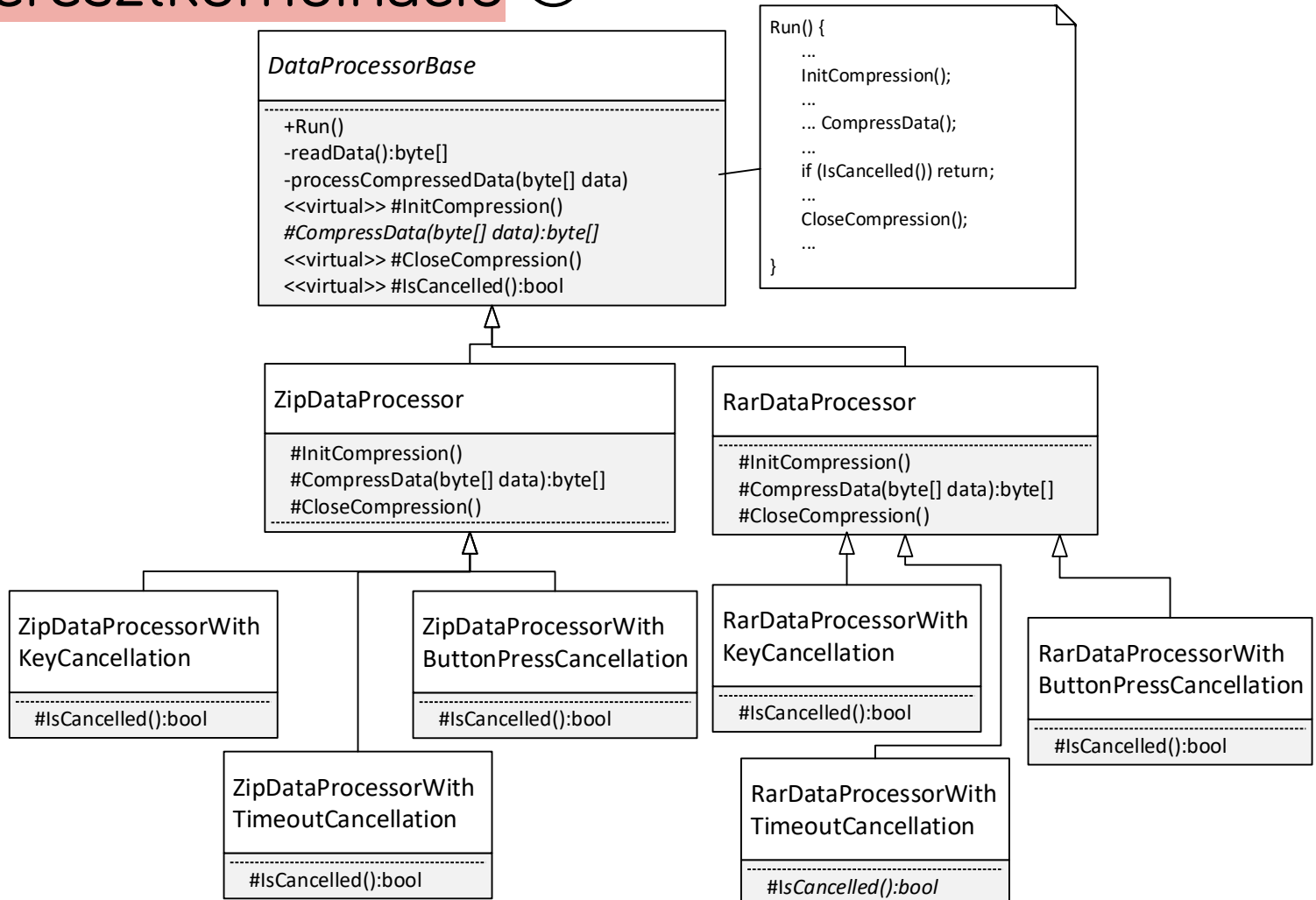
- DataProcessor példa aspektusok (kettő van)
 - > Tömörítés mód: Zip, Rar, 7Zip
 - > Cancel mód: billentyű lenyomás, timeout, gombkattintás (pl. UWP Cancel gomb)
- Minden használt kombinációnak egy külön leszármazott osztály kell
 - > Áttekinthetetlen és karbantarthatatlan hierarchiát eredményez, ha sok a kombináció

Példa



DataProcessor példa

Sok keresztkombináció ☹️



Sok keresztkombináció

A megoldást a Strategy minta alkalmazása jelenti
Rövidesen ...

Előfordulása

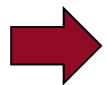
- Egyszerűbb esetekben gyakran használjuk
- Keretrendszeres esetén gyakori, pl. le kell származtatni a keretrendszer beépített Window, Page, Application, Control, stb. osztályából és virtuális függvényeket lehet/kell felüldefiniálni

Strategy (Stratégia)

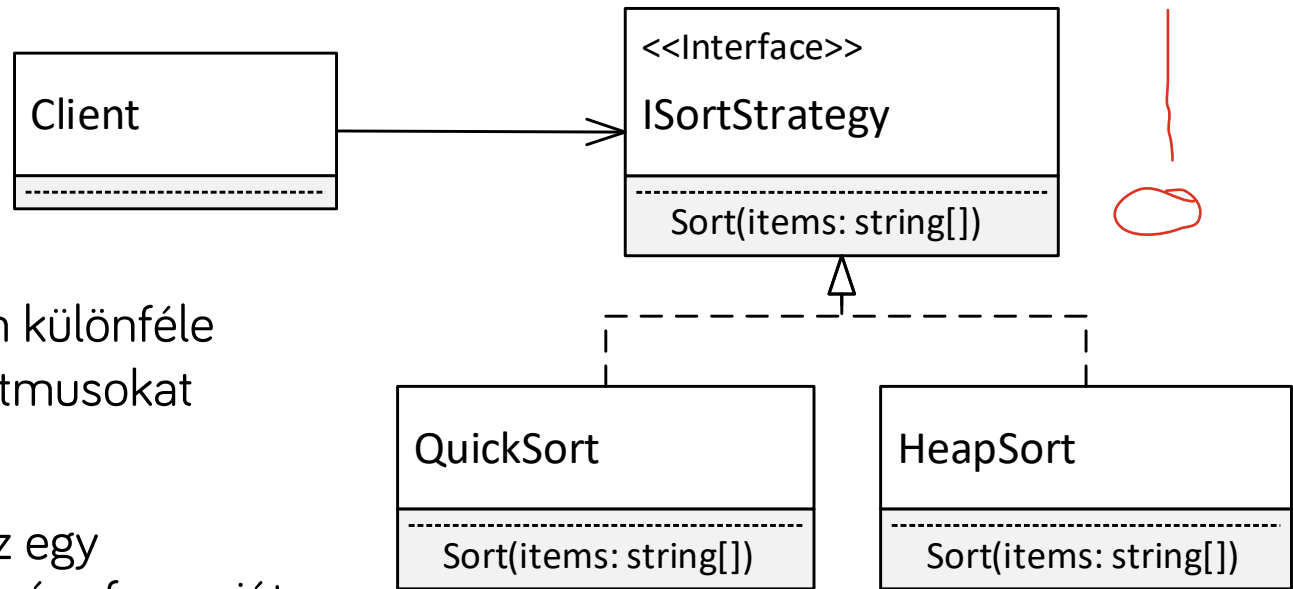
Strategy minta célja

- Célja az algoritmusok/viselkedések egy csoportján belül az egyes algoritmusok/viselkedések egységbe zárása és egymással kicserélhetővé tétele.
 - > A kliens szemszögéből az általa használt algoritmusok/viselkedések szabadon kicserélhetők.

Ez így nehezen érthető, nézzünk egy egyszerű példát



Strategy egyszerű példa - sorrendezés



- A kliens objektum különféle sorrendező algoritmusokat használhat
- A kliens tartalmaz egy **ISortStrategy** típusú referenciát egy konkrét (QuickSort v. HeapSort) objektumra.
- A referencia által hivatkozott implementációs objektumot könnyű kicserélni.

Példakód

- Kód, lásd „Strategy – Sort” mappa

```
// Strategy interfész
interface ISortStrategy
{
    void Sort(string[] items);
}

// QuickSort strategy implementáció
class QuickSort : ISortStrategy
{
    public void Sort(string[] items)
    {
        Console.WriteLine("Sorting items using quick sort algoritm ...");
    }
}

// HeapSort strategy implementáció
class HeapSort : ISortStrategy
{
    public void Sort(string[] items)
    {
        Console.WriteLine("Sorting items using heap sort algoritm ...");
    }
}
```

Példa folytatása

```
// Kliens, a Sort metódusa az aktuálisan beállított stratégiával rendez
class Client
{
    ISortStrategy sort; // Hivatkozás az aktuálisan beállított stratégia implementációra

    // Lehetőséget biztosít a stratégia beállítására
    public void SetSortStrategy(ISortStrategy sort)
    {
        this.sort = sort;
    }

    public void Sort(string[] items)
    {
        sort.Sort(items); // Sorrendezés az aktuális stratégiával
    }
}

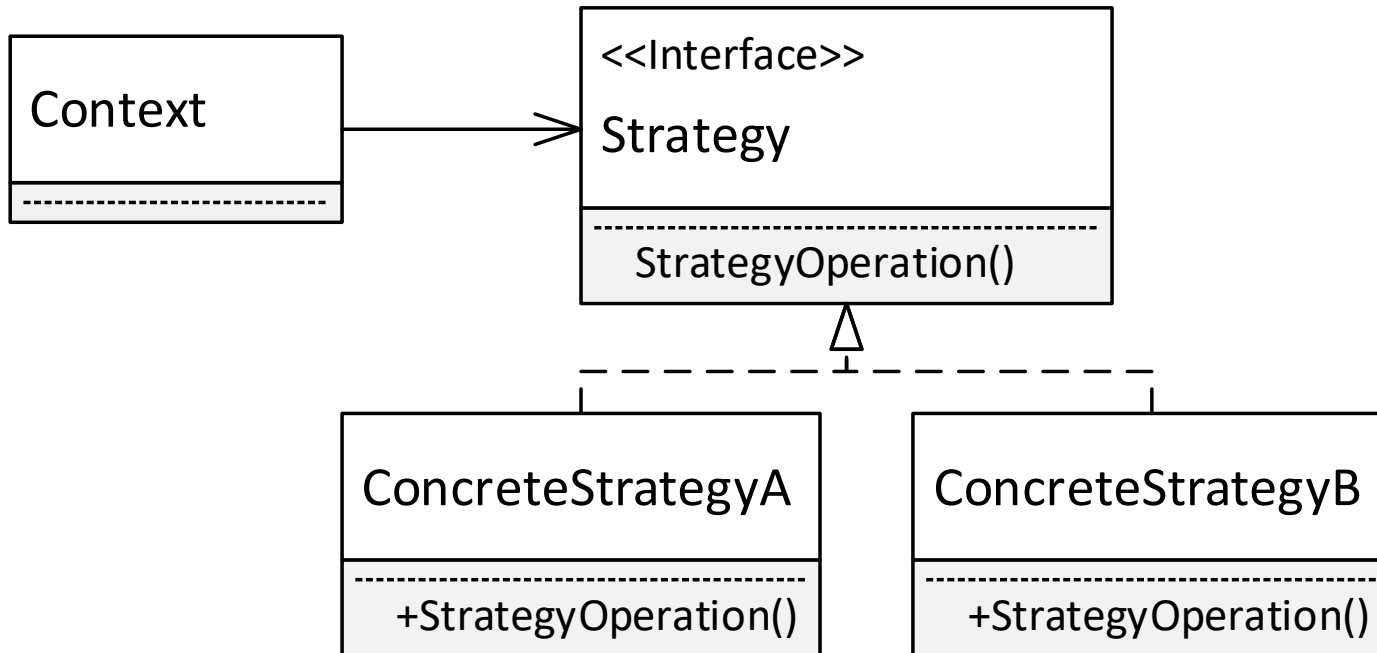
class Program
{
    static void Main(string[] args)
    {
        var items = new string[] { "körte", "alma", "szilva" };

        Client client = new Client();

        // Kezdetben használjuk a quick sort algoritmust
        client.SetSortStrategy(new QuickSort());
        client.Sort(items);

        // Mostantól a heap sort algoritmust használjuk
        client.SetSortStrategy(new HeapSort());
        client.Sort(items);
    }
}
```

Strategy - általános osztálydiagram



- **Strategy** interfész név mellett szokás a **Behavior** illetve **Policy** nevek használata is.
- Lehet több összetartozó művelet is (az ábrán csak egy szerepel)

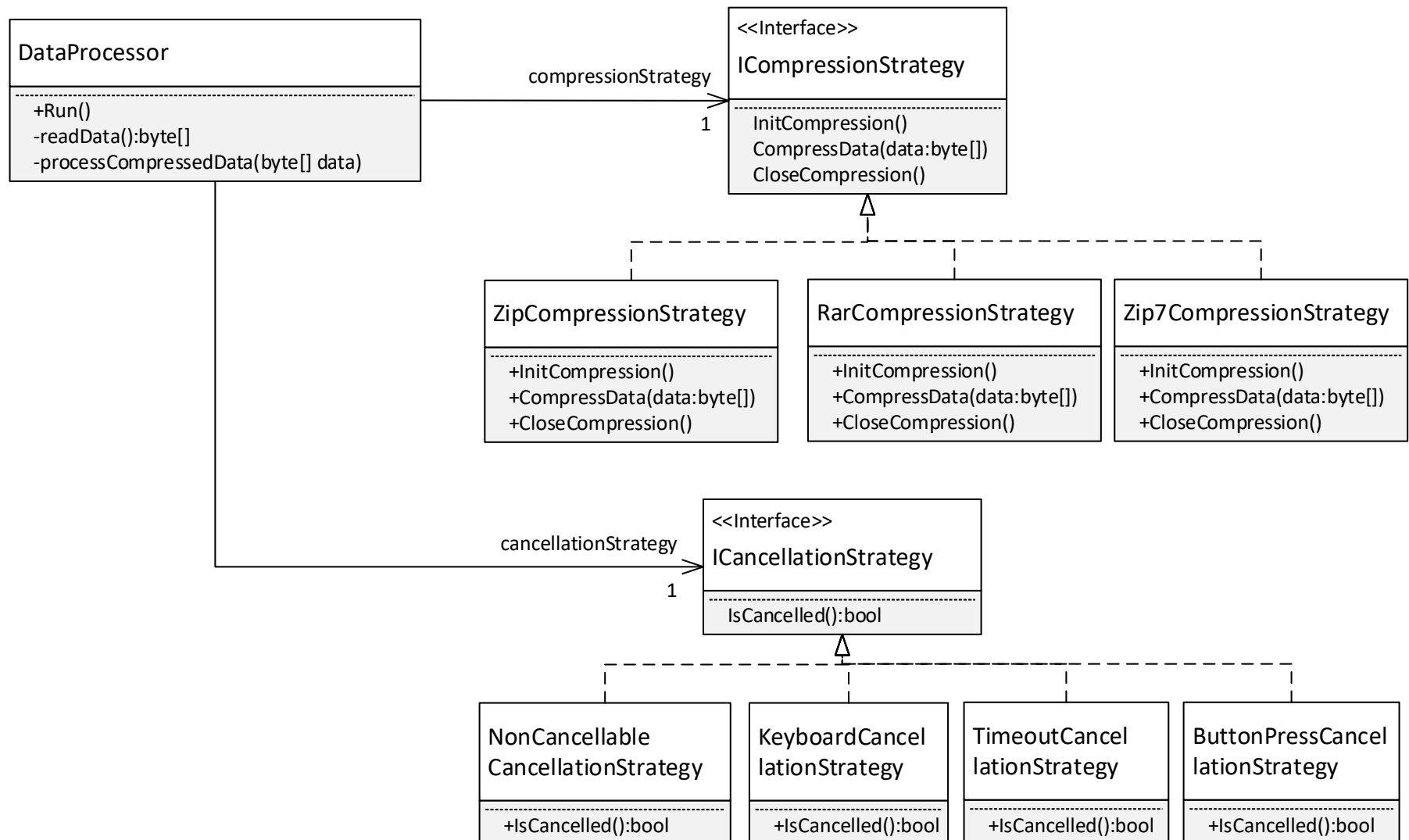
Strategy, DataProcessor példa

- Az osztály viselkedésének minden olyan aspektusára/dimenziójára, melyet lecserélhetővé/bővíthetővé szeretnénk tenni, külön strategy hierarchiát vezetünk be
 - > **Tömörítés mód variációk:** Zip, Rar, 7Zip
 - Interfész: ICompressionStrategy
 - Implementációk: ZipCompressionStrategy, RarCompressionStrategy, Zip7CompressionStrategy
 - > **Cancel mód variációk:** nem megszakítható, billentyű lenyomás, timeout, gombkattintás (pl. UWP Cancel gomb)
 - Interfész: ICancellationStrategy
 - Implementációk: NonCancellableCancellationStrategy, KeyboardCancellationStrategy, TimeoutCancellationStrategy, ButtonPressCancellationStrategy
- Az osztály külön (interfész típusú) hivatkozást tartalmaz minden viselkedés aspektusra/dimenzióra
 - > ICompressionStrategy tag
 - > ICancellationStrategy tag

Lássuk a gyakorlatban



DataProcessor osztálydiagram



DataProcessor példa, kód

- Kód, lásd DesPattCode\Strategy mappa
 - > Nincs minden osztály megvalósítva
 - > A DataProcessor konstruktorban kapja meg a függőségeit, egy ICompressionStrategy és ICancellationStrategy objektumot
 - Ha menet közben is meg akarjuk változtatni valamelyiket, egy megfelelő Setter függvényt vezessünk be (lásd korábbi sorrendező példa)

DataProcessor példa, kód

```
class DataProcessor
{
    ICompressionStrategy compressionStrategy; // Aktuális cancel stratégia/viselkedés
    ICancellationStrategy cancellationStrategy; // Aktuális tömörítési stratégia/viselkedés

    public DataProcessor(ICCompressionStrategy compressionStrategy,
        ICancellationStrategy cancellationStrategy)
    {
        ...
        // Eltároljuk a paraméterként kapott stratégiákat
        this.compressionStrategy = compressionStrategy;
        this.cancellationStrategy = cancellationStrategy;
    }

    // Ciklusban beolvassa, tömöríti, majd feldolgozza a tömörített adatokat
    public void Run()
    {
        byte[] inputData;
        compressionStrategy.InitCompression(); // Tömörítés inicializás, impl. függő
        try
        {
            while ((inputData = readData()) != null)
            {
                // Tömörítés, impl. függő
                byte[] compressedData = compressionStrategy.CompressData(inputData);
                processCompressedData(compressedData);
                if (cancellationStrategy.IsCancelled()) // Cancel vizsgálat, impl. függő
                    return;
            }
        }
        finally
        {
            compressionStrategy.CloseCompression(); // Tömörítés lezárás, impl. függő
        }
    }
}
```

DataProcessor példa, kód

- A stratégiák bármilyen kombinációban egyszerűen használhatók!

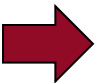
```
static void Main(string[] args)
{
    // A stratégiák bármilyen kombinációban egyszerűen használhatók. Nézzünk párat!
    // Jelen esetben a Zip algoritmust választjuk, és billentyű alapú Cancel lehetőséget
    var processor = new DataProcessor(
        new ZipCompressionStrategy(),
        new KeyboardCancellationStrategy(ConsoleKey.X)
    );
    processor.Run();

    // Második esetben a Rar algoritmust választjuk, és nem akarunk Cancel lehetőséget
    // biztosítani
    var processor2 = new DataProcessor(
        new RarCompressionStrategy(),
        new NonCancellableCancellationStrategy()
    );
    processor2.Run();
}
```

Kiértékelés - pozitívumok

- Bővíthetőség
 - > Új viselkedés könnyen bevezethető, nem kell a meglévő kódot (lényegi helyen) változtatni
 - A példában a meglévő DataProcessor kódjához nem kell hozzányúlni, csak új ICompressionStrategy vagy ICancellationStrategy implementációt kell bevezetni
 - > Több aspektus/dimenzió esetén nincs kombinatorikus robbanás az osztályhierarchiában
- Unit tesztelhetőséget segíti

Példa



Kiértékelés – unit tesztelhetőség

- Unit tesztek során az osztályt önmagában, a függőségei nélkül teszteljük (ezért is hívják UNIT tesztnek).
- Példa: a DataProcessor tesztelése során nem kívánjuk tesztelni azt, hogy az adatok tömörítése, vagy a cancel során a billentyű lenyomás hogyan történik.
 - > Pl.
 - ha hiba van a zip algoritmusban, és az elesik, akkor az **ne akassza meg** a DataProcessor unit tesztjeit
 - A tömörítés lassú, **ne lassítsa** ez feleslegesen a DataProcessor unit tesztjeit
 - > A tömörítés és cancel unit tesztelését más, ezekre dedikált unit tesztek kell végezzék.
 - > Ha ezek implementációja be van építve a DataProcessor osztályba, akkor a DataProcessor nem unit tesztelhető

Kiértékelés – unit tesztelhetőség

- A Strategy minta segíti, hogy a DataProcessor unit tesztelhető legyen. Hogyan? →
- Amikor a DataProcessor osztályt unit teszteljük, olyan „dummy” ICompressionStrategy és ICancellationStrategy implementációkat adunk át, melyek nem csinálnak semmit, vagy magát a tesztet segítik
 - Pl. DummyCompressionStrategy, mely nem tömörít, csak visszaadja az eredeti adatot
 - Pl. NonCancellableCancellationStrategy, mely nem Cancel-el, illetve egy AlwaysCancelCancellationStrategy, mely azonnal automatikusan cancel-el (attól függően, hogy mit tesztelünk éppen).
 - Konkrét példa a félév végén tesztelésről szóló gyakorlaton (?)

DataProcessor példa, SRP elv

- A DataProcessor legutolsó megvalósítása valóban teljesíti az SRP elvet, az osztálynak valóban egyetlen felelőssége van?
- Nem, még mindig több felelőssége van!
 - > A Run-ban a folyamat vezérlése
 - > Adatbeolvasás logikája (readData művelet)
 - > Adatfeldolgozás logikája (processCompressedData művelet)
- Az adatbeolvasás és adatfeldolgozás logikáit is ki lehetne faktorálni az osztályból, a compression és a cancel mintájára interfészek mögé lehet tenni
 - > Ez segíteni a unit tesztelhetőséget, és a bővíthetőséget
 - > Gyakorlásképpen megcsinálható

Strategy

- **Program to an interface** (and not to an implementation) elv egyik megtestesülése

Kiterjeszthetőség összefoglaló

Áttekintés

- SOLID Open/Closed elv
- Lehetőségek
 - > Template method
 - Leszármazottra bízva a kiterjesztést
 - Egyszerűbb
 - Kevésbé rugalmas
 - A leszármaztatás miatt (a leszármazási hierarchia fordításkor dől el, futás közben nem változtatható)
 - Több aspektus: áttekinthetetlen, sok keresztkombináció
 - > Strategy
 - Más osztályokra bízva (delegál), ez a legrugalmasabb
 - > Van más lehetőség is → metódusreferencia, lambda használata

Egyéb kiterjeszthetőségi technikák

- Metódusreferenciák (delegate-ek), lambda kifejezések használata
 - > Egyre inkább terjed
 - > C#, C++, JavaScript nyelvek is támogatják
 - > Java-ban „csak” lambda kifejezések
- Mostantól C#-ot nézünk
 - > Azt a kódot, amely a kiterjeszthetőséget szolgálja, egy függvény/delegate vagy lambda formájában adjuk át
 - > A részletek "Eseményvezérelt és vizuális programozás,, tárgyból

Metódusreferencia, lambda használata

- Lásd DesPattCode\Delegate példa

```
class DataProcessor
{
    // ...

    Func<bool> onCancelled; // Metódusreferencia, konstruktorban kapja meg, Run-ban hívjuk

    public DataProcessor(Func<bool> onCancelled)
    {
        this.onCancelled = onCancelled;
    }

    // Ciklusban beolvassa, tömöríti, majd feldolgozza a tömörített adatokat
    public void Run()
    {
        ...
        if (onCancelled != null && onCancelled()) // Meghívjuk a kódot
            return;
        ...
    }
}

...
// Konstruktorban adjuk át a DataProcessor által hívandó kódot, pl. lambda formájában
var processor1 = new DataProcessor(() => { return false });
```

Kiterjeszthetőség, módosíthatóság

- Számos más minta is (gyakorlatilag valamennyi) a könnyű kiterjeszthetőséget, módosíthatóságot szolgálja, csak kevésbé direkt módon!

Szoftvertchnológia és -technikák

7. Előadás – Benedek Zoltán
Tervezési minták 2



Automatizálási és
Alkalmazott
Informatikai Tanszék

Ez az oktatási segédanyag a Budapesti Műszaki és Gazdaságtudományi Egyetem oktatója által kidolgozott szerzői mű. Kifejezett felhasználási engedély nélküli felhasználása szerzői jogi jogsértésnek minősül.

Tartalom

Tervezési minták 2

- > Observer
- > Létrehozási minták
 - Dependency Injection
 - Singleton
 - Abstract factory
 - (Factory method – nem tananyag)

Observer (Megfigyelő)

Observer célja

- Cél (két megközelítésben)
 - > Lehetővé teszi, hogy egy objektum (subject/alany) értesítést küldjön más objektumoknak (observer/megfigyelő) az állapotának változásairól. Mindezt anélkül, hogy az alany függene a megfigyelők konkrét típusától.
 - > Lehetővé teszi, hogy objektumok értesíteni egymást állapotuk megváltozásáról anélkül, hogy függőség lenne köztük a konkrét osztályaiktól (ez az absztraktabb definíció, mélyebb célokat fogalmaz meg).

Még nem értjük, lássunk egy példát



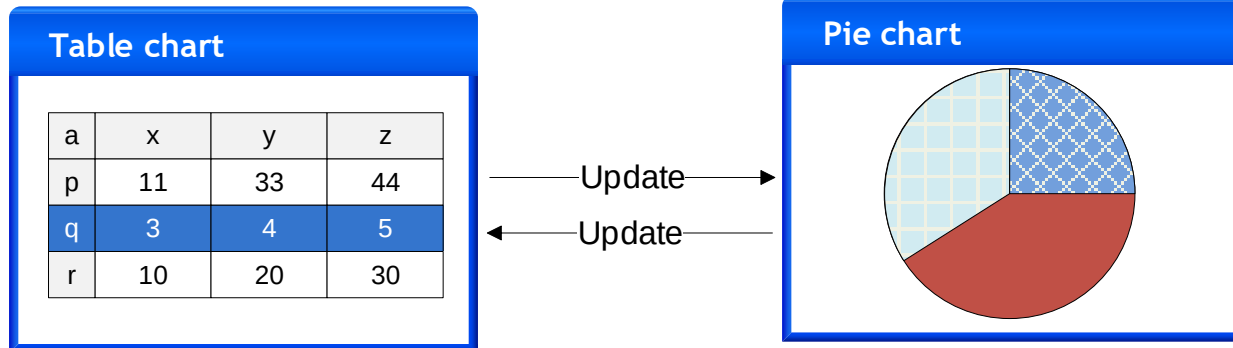
Példa

- Egy táblázatkezelő alkalmazásban támogatni kell az adatok különböző megjelenítését
 - > Első körben **TableView** és **PieChartView**
 - > T.f.h. a felhasználó mindkettőben tudja módosítani az adatokat
 - > Meg kell oldani, hogy a két nézet konzisztens nézetét mutassa az adatoknak. Bármelyikben módosul az adat, azt jeleznie kell a másiknak.
 - Megoldás első körben: mindkét osztály tartalmaz egy hivatkozást a másikra

Ábra és kód



Példa



```
class TableView
{
    // ...

    double[,] data;
    PieChartView pieChartView;

    public void Update(double[,] data) {
        /* this.data beállítása data param alapján */
    }
    public void SetCellData(int col, int row, double value)
    {
        this.data[col, row] = value;
        pieChartView.Update(data);
    }
    public void Draw() { /* ... */ }
}
```

A PieChartView kódja hasonló, csak a TableView-ra tartalmaz hivatkozást.

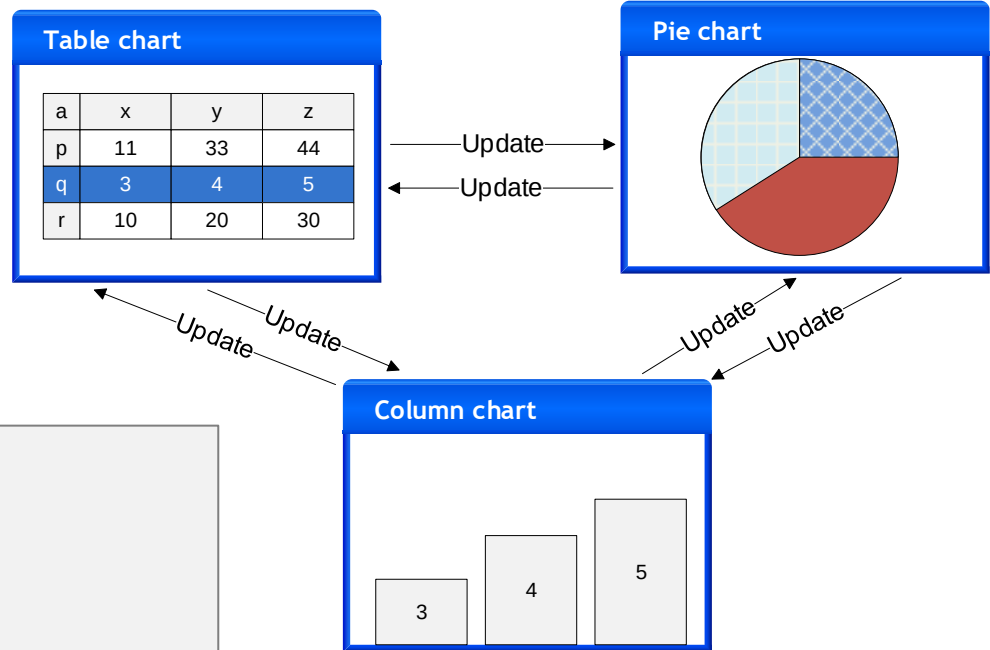
Példa

- T.f.h bővíteni kell a megoldásunkat
 - > Be kell vezetni az oszlopdiagramos megjelenítést (**ColumnChartView**)
 - > T.f.h. ebben is lehet módosítani az adatokat
 - > Konzisztens nézetek: bármelyik nézet módosul, frissíteni kell a másik kettőt is!
 - > Új nézetet szeretnék bevezetni, minden MEGLÉVŐ nézet osztályt módosítani kell ☹️☹️☹️
 - Fel kell vegyük a meglévő osztályokba a hivatkozást az új nézetre

Ábra és kód



Példa



```
class TableView
{
    // ...

    double[,] data;
    PieChartView pieChartView;
    ColumnChartView columnChartView;

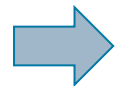
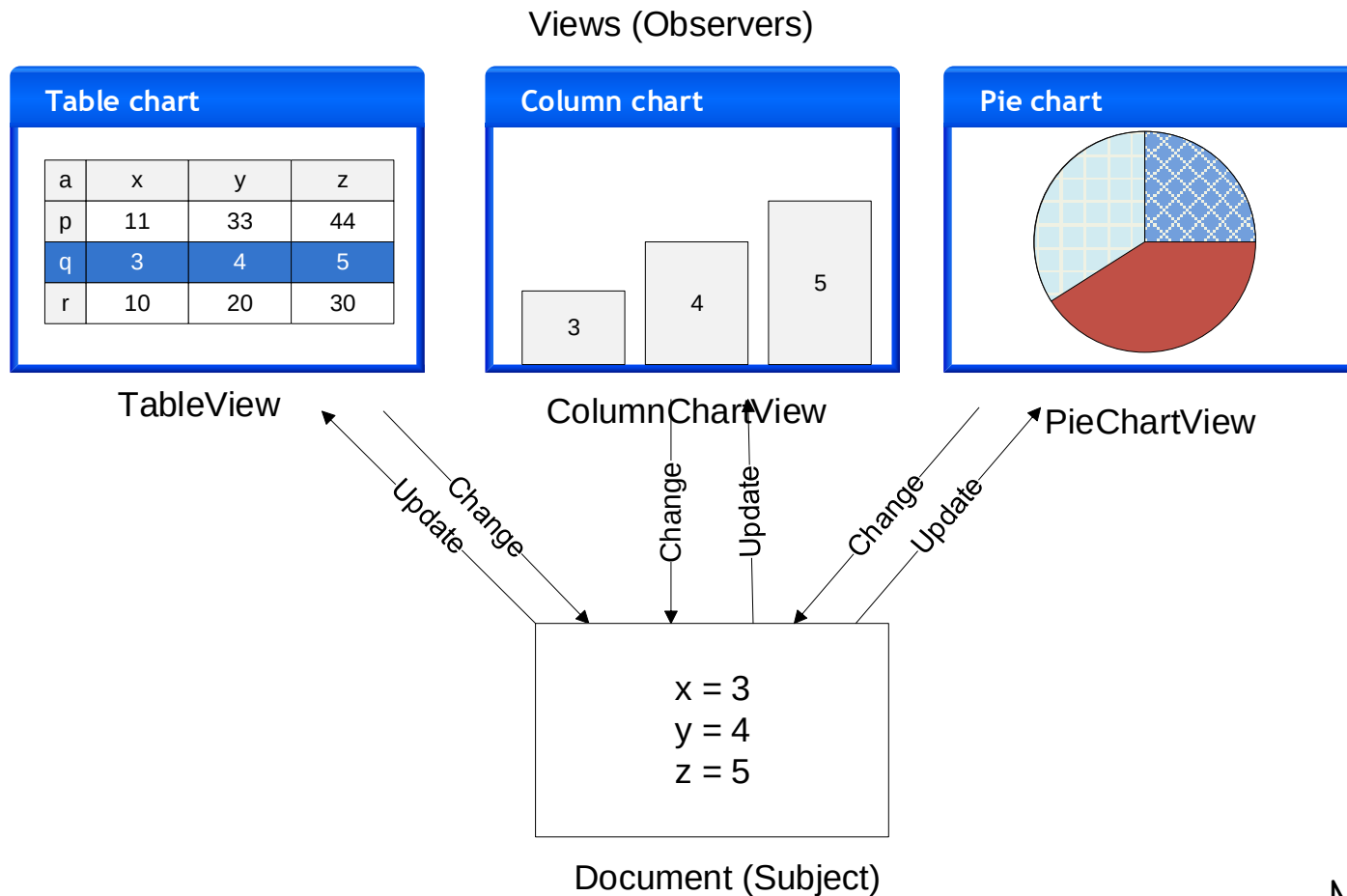
    public void Update(double[,] data) {
        /* this.data beállítása data param alapján */
    }
    public void SetCellData(int col, int row,
        double value)
    {
        this.data[col, row] = value;
        pieChartView.Update(data);
        columnChartView.Update(data);
    }
    public void Draw() { /* ... */ }
}
```

A PieChartView és a ColumnChartView kódja hasonló: a másik két nézetre tartalmaz hivatkozást

Példa kiértékelése

- A megoldásunk (mindenki-mindenkivel összekötve, közvetlen függvényhívással) eddig is nehezen volt bővíthető
 - > A további bővítés teljes rémálom
 - > Áttekinthetetlen + új nézet bevezetésekor mindig módosítani kell a meglévő nézeteket is
 - > Nehéz karbantartani, továbbfejleszteni, újra felhasználni részeket, mert túl szoros a csatolás az osztályok között (a nézetek függenek egymástól).
- Alkalmazzuk a példánkban az **Observer** mintát

Nézetek frissítése Observer mintával



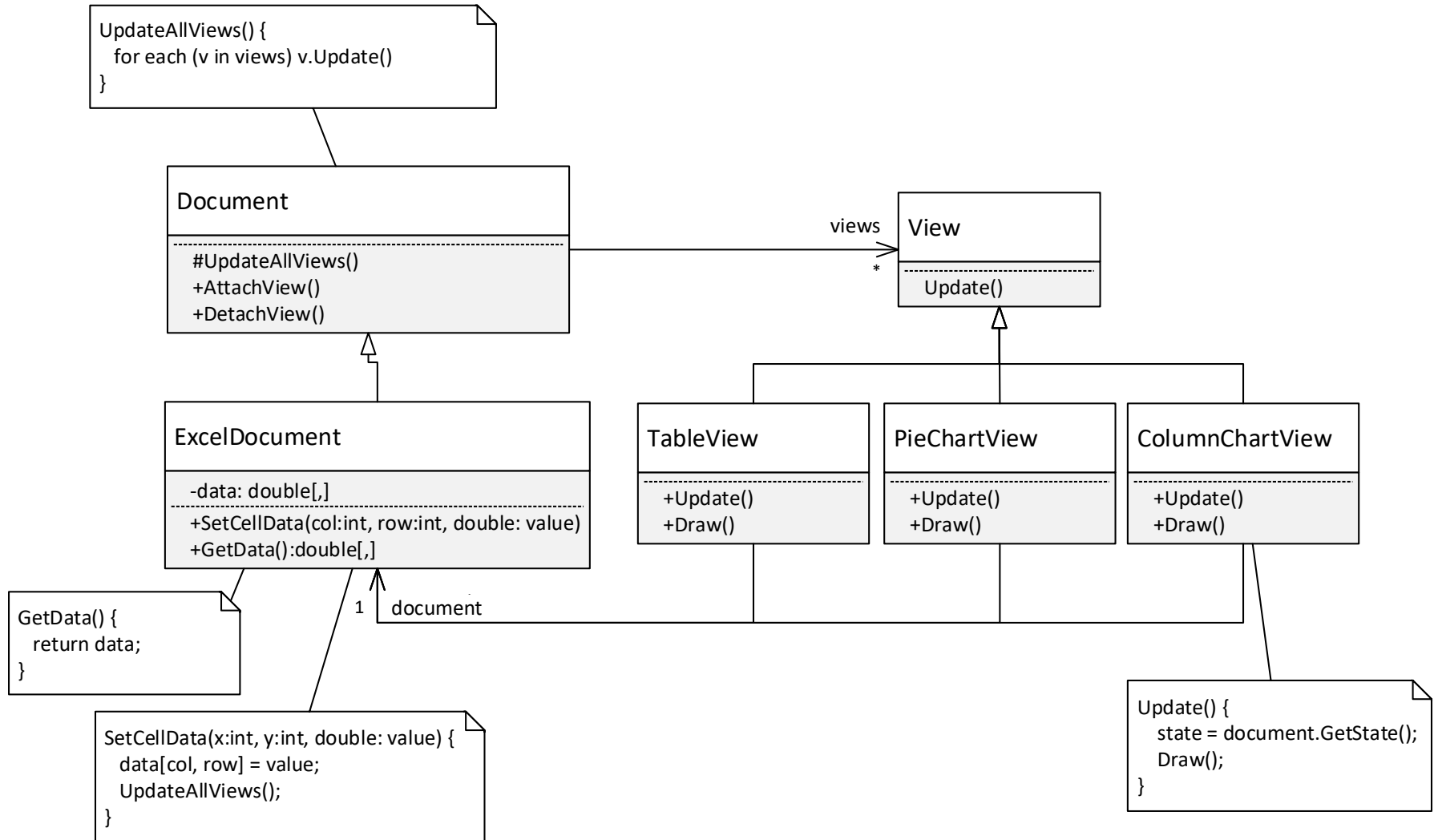
Magyarázat

Nézetek frissítése Observer mintával

- **Magyarázat, minden pont fontos!**

- > Az adatoknak egyetlen, közös „hiteles” forrása legyen: emeljük ki az adatokat és az azon értelmezett műveleteket egy osztályba, ez lesz a **dokumentum** (subject)
- > A dokumentumhoz különböző **view**-kat lehet beregisztrálni (observer)
- > Ha valamelyik view megváltoztatja a dokumentum adatait, a dokumentum értesíti az összes beregisztrált view-t a változásról
- > Az értesítés hatására a view-k lekérdezik a dokumentum állapotát és frissíti magát
- > A dokumentum csak egy közös View interfészen/őssosztályon keresztül tárolja a beregisztrált view-kat (nem függ az egyes nézet típusoktól)

Példa, osztálydiagram



Osztálydiagram magyarázat

- **Document**

- > Az osztály objektumai reprezentálnak egy-egy dokumentumot, és a *views* nevű listájában tárolja a beregisztrált nézeteket.
- > Tartalmazza a dokumentumokra közös kódot, ha több különböző típusú dokumentum is van (a példánkban nincs)

- **ExcelDocument**

- > Egy példa *Document* leszármazottra.
- > Tárolja a *data*, stb. tagváltozójában a dokumentum adatait (pl. cella értékek).
- > Publikus lekérdező függvényeket biztosít a többi osztály számára az adatokhoz (elsősorban a nézet számára), pl. *GetData*.
- > Publikus adatmódosító műveleteket biztosít a többi osztály számára, pl. *SetData*. Ezek módosítják a tagváltozókat, majd az *UpdateAllViews* hívásával értesítik a többi nézetet a változásról.
- > Az *UpdateAllViews* frissíti a beregisztrált nézeteket (Update-et hív mindre).

Osztálydiagram magyarázat

- **View**

- > Az egyes nézetek közös őse vagy interfésze, lehetővé teszi egységes kezelésüket.

- TableView, PieChartView, stb.

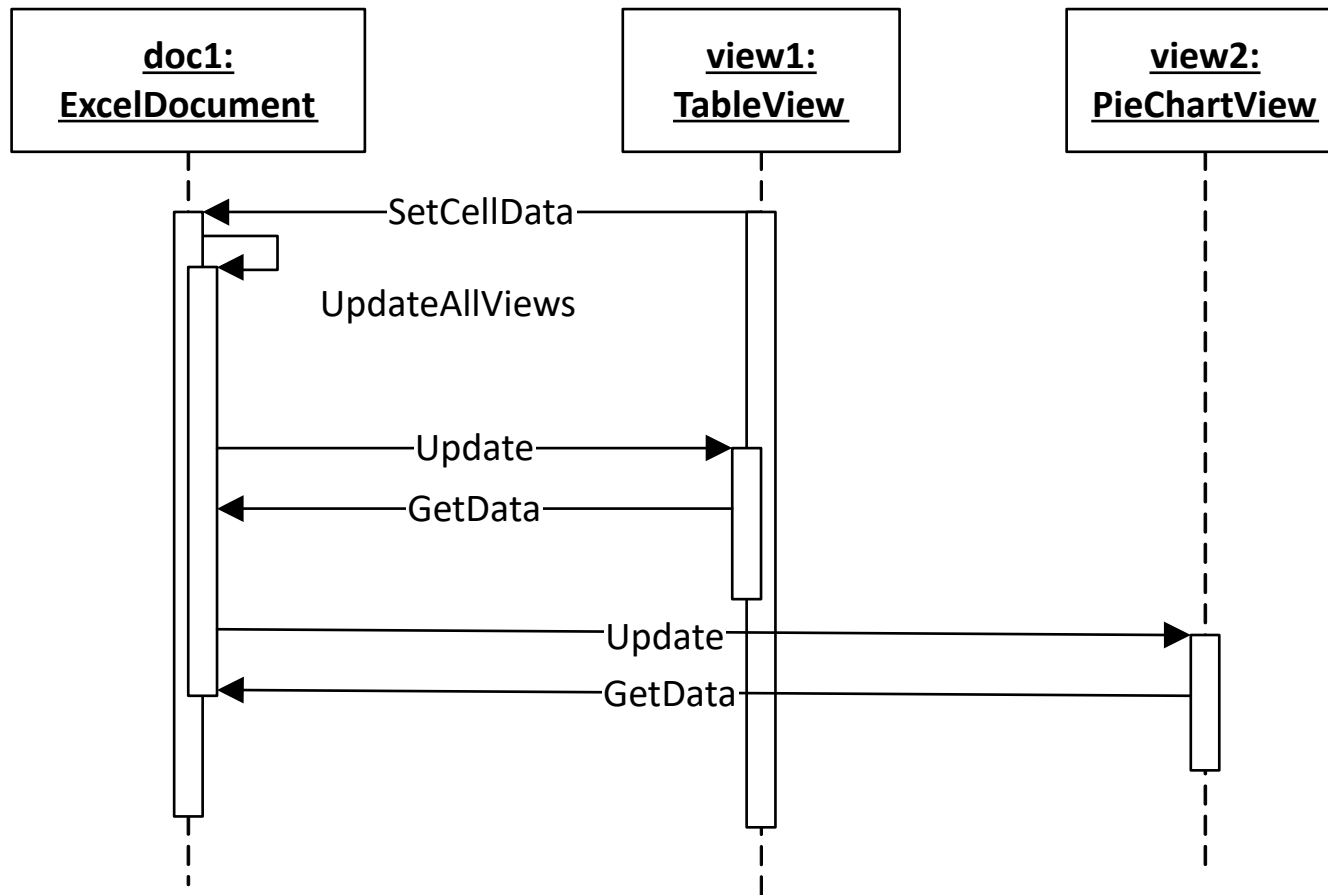
- > A dokumentum egyes nézeteit reprezentálják.
- > A *View*-ből származnak (vagy a *View* interfészt implementálják, ha a *View* interfész)
- > Felüldefiniálják vagy implementálják a *View Update* műveletét. Az *Update* műveletben a nézet a dokumentum aktuális állapota alapján frissíti magát.
- > Tartalmaznak egy referenciát a dokumentumra, amin keresztül a leszármazott nézetek elérhetik a dokumentumot, melynek adatait megjelenítik.
 - Alternatív megoldás lehet, ha a *View* közös őse tartalmazza ezt a referenciát a *Document* ősről

- Nézetben hivatkozás a *Document* (subject)-re, két lehetőség

- > A példában a *View* implementációkba tettük, feltételezve, hogy a *View* egy interfész
- > Ha a *View* osztály (nem interfész), akkor a *View* ősebe is tehető.
 - A gyakorlatban a *View*-kat a .Net, Java, stb. keretrendszer *Form/Window/Control*/egyéb osztályából származtatjuk

Példa szekvenciadiagram

- Pár diával korábban szövegesen már szerepeltek a lépések



Példa kód

- Kód: lásd *ObserverDocView* mappa
- Egy kicsit szofisztikáltabb változat:
 - > A nézet vonatkozásában van *IView* interfész, de egy *ViewBase* közös őosztály is: ez utóbbiba a nézetekre közös kód került.
- Az egyszerűség kedvéért egy ez konzol alkalmazás, a *ColumnChartView* és a *PieChartView* így természetesen nem rajzol

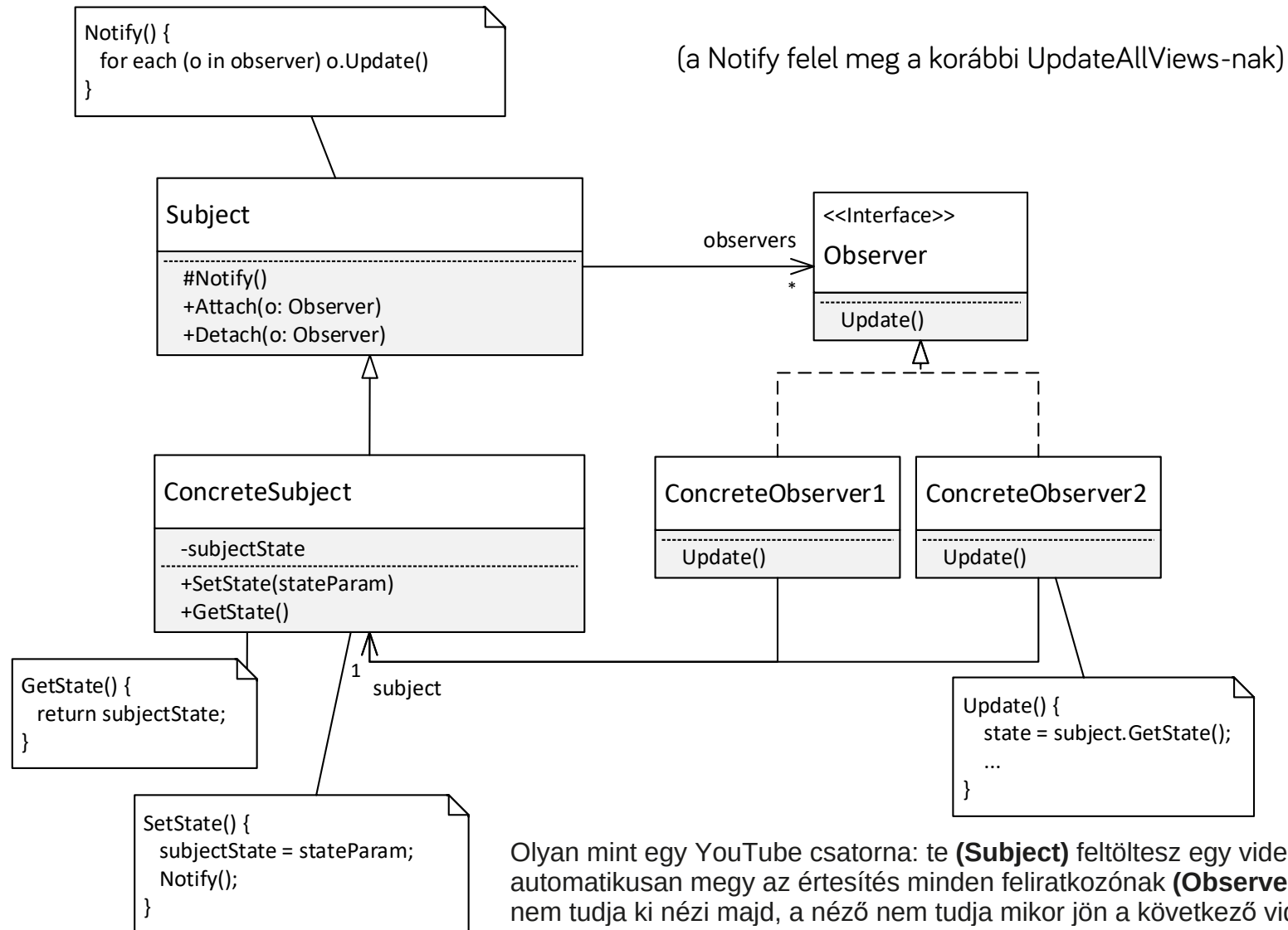
Példa összefoglaló

- Ez egy példa volt, az Observer minta alkalmazására egy speciális esetre, adatok megjelenítésére
 - > Ez maga a Document-View architektúra, még visszatérünk rá pár előadás múlva
- Observer általánosítva
 - > Lehetővé teszi, hogy egy objektum (subject/alany) értesítést küldjön más objektumoknak (observer/megfigyelő) az állapotának változásairól. Mindezt anélkül, hogy az alany függene a megfigyelők konkrét típusától.
- További példák gyakorlaton

Observer

- Előnyök
 - > A rendszer könnyen kiterjeszthető új view osztályokkal. Sem a document (subject), sem a többi view (observer) osztályt nem kell ehhez módosítani.
 - Próbáljuk ki, vezessünk be egy új nézet osztályt!
 - > Egy egyszerű mechanizmust kaptunk arra, hogy az összes view konzisztens nézetét mutassa az adatoknak.
 - > A document (subject) kódjában csak egy *View* (observer) lista van, így a modell független az egyes *View-t* implementáló osztályoktól. A document (subject) újrafelhasználható!
- Általánosítva, elvonatkoztatva a document-view esettől
 - > A fenti megközelítés lehetővé teszi, hogy egy objektum megváltozása esetén értesíteni tudjon tetszőleges más objektumokat anélkül, hogy bármit is tudna róluk
 - > Ez az ún. *observer* minta lényege

Observer osztálydiagram

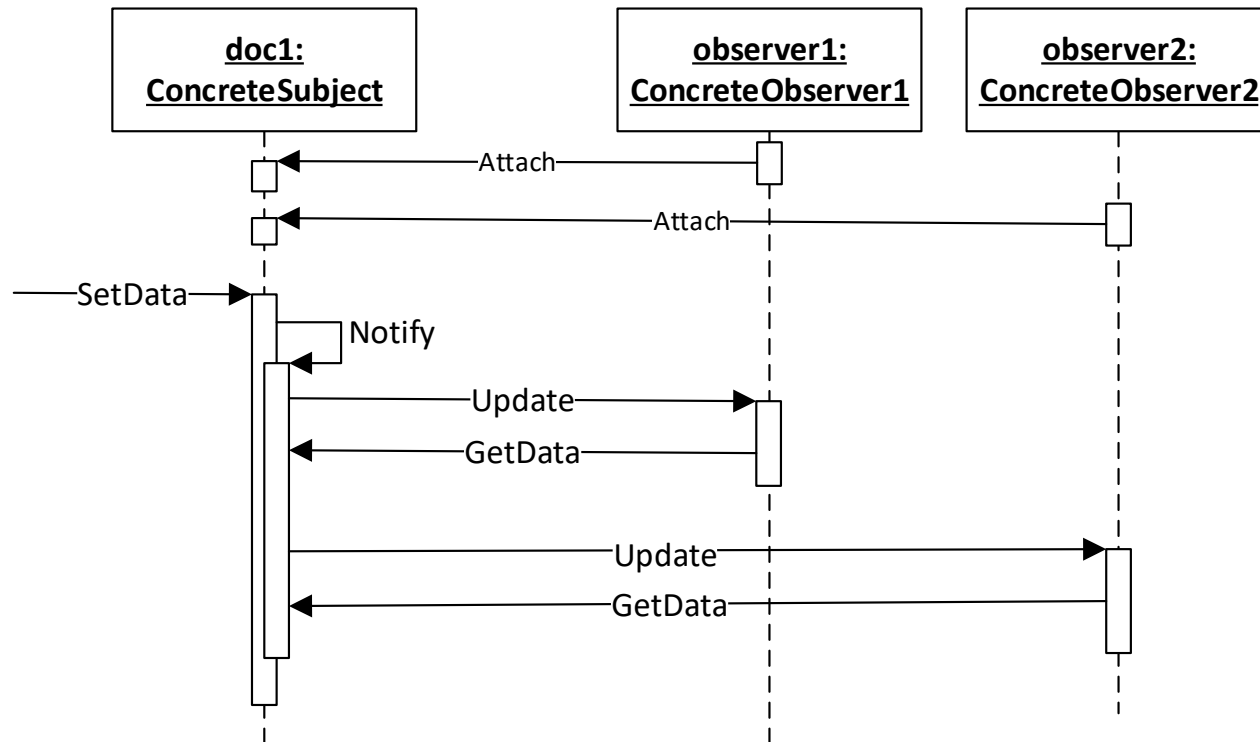


Olyan mint egy YouTube csatorna: te **(Subject)** feltöltesz egy videót → automatikusan megy az értesítés minden feliratkozónak **(Observer)**. A csatorna nem tudja ki nézi majd, a néző nem tudja mikor jön a következő videó.

Observer - szereplők

- **Subject**
 - > Tárolja a beregisztrált Observer-eket
 - > Lehetőséget ad Observer-ek be- és kiregisztrálására valamint értesítésére. Ezáltal „tartalmazza” a különböző ConcreteObserver-ek közös kódját (közös ős).
 - > A gyakorlatban nem jelenik meg mindig
- **Observer**
 - > Interfészt definiál azon objektumok számára, amelyek értesülni szeretnének a Subject-ben bekövetkezett változásról (Update művelet)
- **ConcreteSubject**
 - > Az observer-ek számára érdekes állapotot tárol
 - > Értesíti a beregisztrált Observer-eket, amikor az állapota megváltozik
- **ConcreteObserver**
 - > Referenciát tárol a megfigyelt ConcreteSubject objektumra
 - > Olyan állapotot tárol, amit a megfigyelt ConcreteSubject állapotával konzisztensen kell tartani
 - > Implementálja az Observer interfészt (Update művelet), ez az, amit a Subject meghív, amikor a ConcreteSubject állapota megváltozik. Ebben frissíti a saját állapotát a megfigyelt ConcreteSubject állapotának megfelelően

Observer dinamikus nézet



- A subject állapotváltozását (**SetData hívás**) nem csak observer válthatja ki! Ezért az ábrán ezt egy kívülről jövő művelettel jelöltük.
- Megjegyzés: „Sajnos” az UML szekvencia diagram csak objektumokat ábrázol, nem képes kifejezni, hogy milyen interfészen keresztül hivatkoznak egymásra az objektumok, így a függetlenséget nem tudja kifejezni

Observer

- Használjuk, ha
 - > Amikor egy objektum megváltoztatása maga után vonja más objektumok megváltoztatását, és nem tudjuk, hogy hány objektumról van szó
 - > Amikor egy objektumnak értesítenie kell más objektumokat az értesítendő objektum szerkezetére vonatkozó feltételezések nélkül

Observer – záró gondolatok

- Gyakran használt minta
- Laza csatolás megvalósításával segíti a könnyű bővíthetőséget
 - > Az observerek nem direktben kommunikálnak egymással, egymásról nem is tudnak, csak a subject-et ismerik
 - > A subject nem függ az egyes observerek konkrét típusától (csak egy observer interfésztől)
 - > Új observer bevezetésekor nem kell módosítani sem a subject-et, sem a többi observert, csak implementálni kell az observer interfészt
- A .NET esemény (event) is ennek egy speciális megjelenése: egy osztály minden event tagja egy „subject”, amire a += operátorral iratkoznak fel az observerek

Observer – záró gondolatok

- A példáinkban változás esetén a subject nem adja át az observernek az adatot. Csak a változás tényét jelzi, az observer egy plusz lépésben kéri le az adatot.
 - > Teljesen érvényes megoldás lehet, ha a subject elküldi a változott adatot, paraméterben átadva az observernek (különösen, ha kisebb méretű az adat)
 - > Esetfüggő, melyik megközelítés a praktikusabb

LÉTREHOZÁSI MINTÁK

Dependency Injection (DI)

Abstract factory

Singleton

(Factory Method – nem tananyag)

Létrehozási minták

- Objektumok létrehozásával kapcsolatos
- **A new-val való közvetlen objektum létrehozás sokszor rugalmatlan**
 - > Emlékezzünk a korábbi alapelvünkre: zárjuk egységbe a kód azon részeit, melyeknél arra számítunk, változni/bővülni fognak
 - > Ezek a részek gyakran az objektumok **létrehozásával** kapcsolatosak
 - Pl. nem akarjuk beégetni, milyen típusú objektumot hozzon létre egy adott kódsor (a new operátor esetében ez be van égetve, ez maga a típus, mely a new után áll, pl. *new TextDocument()* esetén TextDocument típus)
 - > Rövidesen nézünk példákat ...

Létrehozási minták

- Megoldási lehetőségek

- > Dependency Injection -DI
- > Factory Method
- > Abstract Factory
- > Singleton – ez egy kicsit speciális célokat szolgál
- > ...



Dependency Injection, DI (függőséginjektálás)

Dependency Injection

- Ez nem egy klasszikus tervezési minta, nincs a GoF könyvben, de logikailag ide passzol
- Egy példán keresztül világítjuk meg
 - > A célunk egy olyan **SecurityService** osztály megírása, mely felhasználókhoz kapcsolódó biztonsági szolgáltatásokat biztosít, úgymint jelszó megváltoztatása, login, stb.
 - > A felhasználókat valamilyen adatbázisban tároljuk.

SecurityService példa – kezdeti verzió

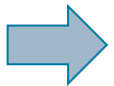
- Kód: lásd DesPattCode.DependencyInjection.Initial mappa

```
class SecurityService
{
    // A userId azonosítójú felhasználó jelszavát megváltoztatja
    // newPassword-re (ha teljesíti a validációs szabályokat)
    // Visszatérési értékben jelzi, hogy sikerült-e a jelszó megfelel-e
    // a validálási szabályoknak.
    public bool ChangePassword(int userId, string newPassword)
    {
        var userRepository = new UserRepository();
        // Adatbázisból régi jelszó lekérdezése userRepository segítségével
        var oldPassword = userRepository.GetUserPassword(userId);

        // Jelszó validálás
        bool isValid = isPasswordValid(newPassword, oldPassword);
        if (!isValid)
            return false;

        // Adatbázisból jelszó módosítása userRepository segítségével
        userRepository.UpdateUserPassword(userId, newPassword);

        return true; // ha minden stimmel
    }
    ...
}
```

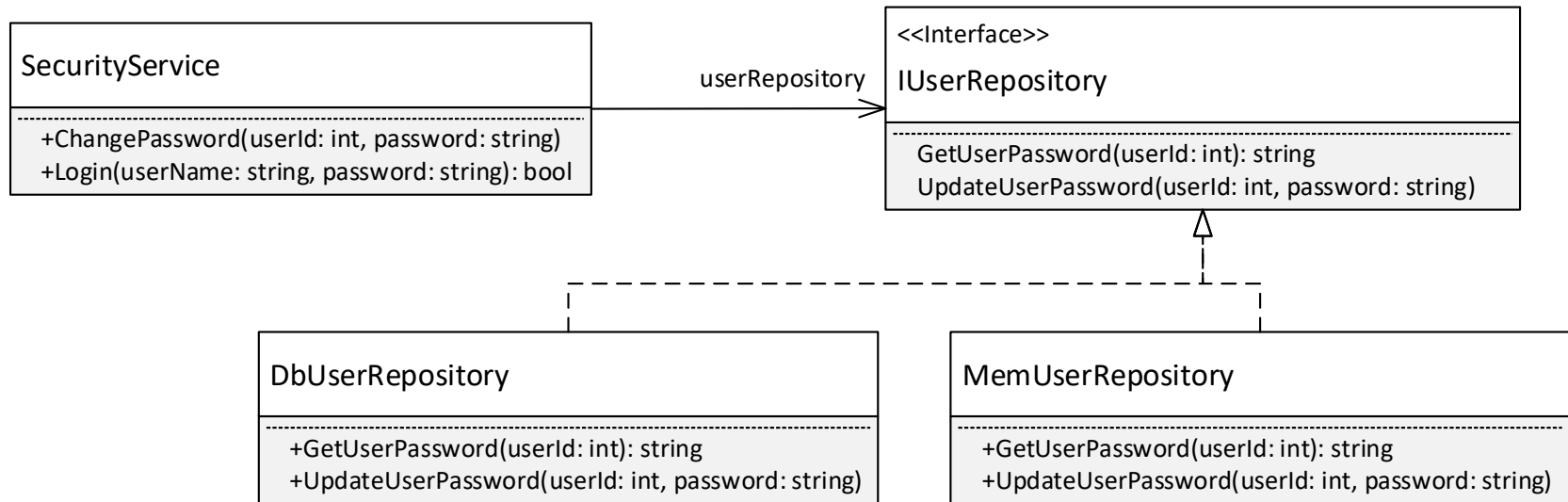


Magyarázat

SecurityService példa – kezdeti verzió

- Arra figyeltünk, hogy a felhasználók adatainak lekérdezése, módosítása (perzisztencia) le van választva egy **UserRepository** osztályba, hiszen az egy másik felelősségi kör (SRP elv).
- Probléma1: a **ChangePassword**-be a **UserRepository** osztály alkalmazása **be van égetve, csak ezzel tud dolgozni. Rugalmatlan,** csak adott adatbázis alapú felhasználó tárolással tud dolgozni. Ha más implementációt szeretnénk, a `var userRepository = new UserRepository()` sorokat mind át kell írni, ráadásul számos helyen.
- Probléma2: Az előző problémából adódóan a **ChangePassword** nem unit tesztelhető. A **ChangePassword** tesztelésénél NEM akarjuk tesztelni az adatbázis alapú perzisztenciát végző kódot, hanem csak a **SecurityService**-ben levő logikát. Ezen túl az adatbázis alapú perzisztencia nagyban feleslegesen lassítja a unit teszt futását.
- Megoldás: alkalmazzuk a Strategy pattern, vezessünk be egy **IUserRepository** interfészt több lehetséges implementációval (adatbázis alapú az éles működéshez, memória alapú a unit teszteléshez), a **SecurityService** pedig interfészként hivatkozzon rá. Lásd WithStrategy mappa...

SecurityService példa – Strategy-vel



- Csak alkalmaztuk/gyakoroltuk a Strategy mintát
 - > **IUserRepository** – interfész
 - > **DbUserRepository** – egy implementáció, mely adatbázisba dolgozik (éles környezethez)
 - > **MemUserRepository** – egy implementáció, mely a memóriába dolgozik (így gyors) unit teszteléshez

SecurityService példa – Strategy-vel

- Lásd DesPattCode.DependencyInjection.WithStrategy mappa

```
class SecurityService
{
    // Interfészként hivatkozunk a user repository implementációra
    IUserRepository userRepository;

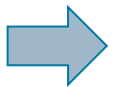
    // Egy központi helyen, a konstruktorban példányosít egy implementációt
    public SecurityService() { userRepository = new DbUserRepository(); }

    // A userId azonosítójú felhasználó jelszavát megváltoztatja
    // newPassword-re (ha teljesíti a validációs szabályokat)
    // Visszatérési értékben jelzi, hogy sikerült-e a jelszó megfelel-e
    // a validálási szabályoknak.
    public bool ChangePassword(int userId, string newPassword)
    {
        // Adatbázisból régi jelszó lekérdezése userRepository segítségével
        var oldPassword = userRepository.GetUserPassword(userId);

        // Jelszó validálás
        bool isValid = isPasswordValid(newPassword, oldPassword);
        if (!isValid)
            return false;

        // Adatbázisból jelszó módosítása userRepository segítségével
        userRepository.UpdateUserPassword(userId, newPassword);

        return true; // ha minden stimmel
    }
}
```



Magyarázat

SecurityService példa – Strategy-vel

- Az eredeti(Initial) **SecurityService** továbbfejlesztése.
- Interfészként hivatkozik a **UserRepository** implementációra: Ha más implementációt szeretnénk, akkor a műveleteket, pl. a **ChangePassword**-öt nem kell megváltoztatni:
- Még mindig maradt egy probléma, a konstruktorban a new-val a példányosítás! **A new után csak konkrét implementációt adhatunk meg. így ha át szeretnénk térni egy másik implementációra, akkor a SecurityService kódját még mindig módosítani kell** (igaz, már csak egy helyen, a konstruktorban), ez sérti az Open/Closed elvet (meg kell nyitni az osztályt módosításra).
- Megoldás: **Dependency Injection(DI)** alkalmazása, konstruktor, vagy esetleg metódus paraméterben adjuk át az implementációt a **SecurityService** osztálynak. Lásd WithStrategyAndDI mappa.

Dependency Injection (DI) alkalmazása a Strategy mellett)

- Lásd DesPattCode.DependencyInjection.WithStrategyAndDI mappa

```
class SecurityService
{
    // Interfészként hívkozunk a user repository implementációra
    IUserRepository userRepository;

    // Konstruktor paraméterként kap egy IUserRepository implementációt
    public SecurityService(IUserRepository userRepository) {
        this.userRepository = userRepository;
    }

    // A userId azonosítójú felhasználó jelszavát megváltoztatja
    // newPassword-re (ha teljesíti a validációs szabályokat)
    // Visszatérési értékben jelzi, hogy sikerült-e a jelszó megfelel-e
    // a validálási szabályoknak.
    public bool ChangePassword(int userId, string newPassword)
    {
        // Adatbázisból régi jelszó lekérdezése userRepository segítségével
        var oldPassword = userRepository.GetUserPassword(userId);

        // Jelszó validálás
        bool isValid = isPasswordValid(newPassword, oldPassword);
        if (!isValid) return false;

        // Adatbázisból jelszó módosítása userRepository segítségével
        userRepository.UpdateUserPassword(userId, newPassword);

        return true; // ha minden stimmel
    }
}
```

Dependency Injection (DI) alkalmazása a Strategy mellett)

- Folytatás, **DemoSecurityService** osztályban **SecurityService** használatra példák:

```
// Éles alkalmazásban a DbUserRepository-val használjuk a SecurityService-t
var securityService = new SecurityService( new DbUserRepository() );
securityService.ChangePassword(111, "abrakadabra");

// Unit teszt esetében a gyors MemUserRepository-val használjuk a
// SecurityService-t
var securityService2 = new SecurityService( new MemUserRepository() );
// ... teszt előkészítés ...
// tesztelendő kód futtatás
bool wasValid = securityService2.ChangePassword(111, "abrakadabra");
// ... teszt validálás ..
```

Magyarázat

Minden pont fontos!

- A **SecurityService** Interfészként hivatkozik a **UserRepository** implementációra, és az implementációt is kívülről kapja meg (dependency injection) jelen példában konstruktor paraméterként.
- Minden korábbi problémától megszabadultunk, sehol nem szerepel a **SecurityService** osztályban egyik **IuserRepository** implementáció sem (próbáljuk rákeresni a fájlban a DbUserRepository és MemUserRepository szövegekre – nem lesz találat).
- A **SecurityService-et** így bármelyik **IuserRepository** implementációval is akarjuk használni, a **SecurityService** kódját nem kell módosítani (lásd **DemoSecurityService** osztály)

További DI gondolatok

- A DI-t már előző előadáson is ösztönösen használtuk, amikor a **DataProcessor** osztálynak a **ICompressionStrategy** és **ICancellationStrategy** implementációkat adtunk át!
- Általában a konstruktor paraméterként való megadást preferáljuk, de ha futás közben szükség van az implementáció megváltoztatására, akkor egy setter metódust használunk, pl. a **SecurityService**-ben.:

```
public void SetUserRepository(IUserRepository
    userRepository)
{
    this.userRepository = userRepository;
}
```

- A gyakorlatban sokszor a keretrendszer extra szolgáltatásokat (ún. DI containert) nyújt ahhoz, hogy a DI-t igazán kényelmesen tudjuk használni, pl. .NET, Java Spring. Ez a tárgy keretében nem szerepel.

További DI gondolatok

- Miért hívják függőséginjektálásnak?
 - > Az osztály a függőségeit (osztályok, melyekre épít, a példánkban a UserRepository) nem maga példányosítja a new operátorral, hanem ezek példányait kívülről kapja meg.
- Bizonyos esetekben a DI nem használható, mert a feladat olyan jellegű, hogy az osztálynak kell adott pontban – még ha közvetve is – példányosítani az objektumokat.
 - > Ez esetben a Factory Method és az Abstract Factory minták használhatók,
 - > Illetve a Prototype és a Builder minták, de ezeket nem tanuljuk

private konstruktor + static instance

Pl: Logger - fölösleges mindegyik osztályba példányosítani!

Singleton (Egyke)

Singleton

- Célja
 - > Biztosítja, hogy egy osztályból csak egy példányt lehessen létrehozni, és ehhez az egy példányhoz globális hozzáférést biztosít
- Globális hozzáférés
 - > Az egy példány kódban bárholnán kényelmesen elérhető, anélkül, hogy függvényparaméterben kellene átpasszolni
- Példa
 - > Pl. egy központi ablakkezelő (pl. WindowManager) vagy Application objektum
- Megoldás
 - > Singleton
 - Legyen az osztály felelőssége, hogy csak egy példányt lehessen belőle létrehozni
 - Maga az osztály biztosítson globális hozzáférést ehhez az egy példányhoz
 - > A programozási nyelvek segítségét igényli

Singleton C# példa

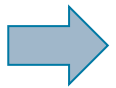
```
class WindowManager
{
    // Tárolja az egyetlen példányt, kezdetben null.
    private static WindowManager instance;

    // Globális hozzáférést biztosító statikus művelet
    public static WindowManager GetInstance()
    {
        // Az egy példányt az első hozzáférés alkalmával hozzuk létre
        if (instance == null)
            instance = new WindowManager();
        return instance;
    }

    // Védett konstruktor
    protected WindowManager() { }

    // Az osztály egyik művelete
    public void DoSomething() { /*...*/ }
}

...
// Valahol a kódban hozzáférünk hozzá a WindowManager singleton példányhoz
WindowManager.GetInstance().DoSomething();
```



Magyarázat

Singleton C# példa magyarázat

- Az egyetlen példányt maga az osztály tárolja egy statikus, pl. **instance** nevű védett (private) változóban.
- Az egyetlen példányt a többi osztály egy statikus, pl. **GetInstance** nevű metódussal éri el
 - > Ez az első hívás alkalmával hozza létre a példányt és el is tárolja a statikus instance változóban
 - > A későbbi hívások során ugyanezt a példányt adja vissza
- Az osztály konstruktora védett (private vagy protected), így más osztályok nem tudják a new operátorral példányosítani. Ez garantálja, hogy más osztály nem tudja megkerülni a **GetInstance** használatát, így további példányokat nem tud létrehozni.
- Mivel a **GetInstance** statikus, a kódban a **WindowManager.GetInstance()** hívással bárhol el tudjuk érni az egyetlen **WindowManager** objektumot.

Singleton

- C#-ban a hozzáférő statikus művelet helyett gyakori a statikus property használata (az alábbi példa csak a különbséget mutatja):

```
class WindowManager
{
    ...
    // Globális hozzáférést biztosító statikus művelet
    public static WindowManager Instance
    {
        get
        {
            // Az egy példányt az első hozzáférés alkalmával hozzuk létre
            if (instance == null)
                instance = new WindowManagerP();
            return instance;
        }
    }
    ...
}

...
// Valahol a kódban hozzáférünk hozzá a WindowManager singleton példányhoz
WindowManager.Instance.DoSomething();
```

Szálbiztos C# implementáció *

- **Nem kell tudni!**
- A korábbi megoldásunk nem volt szálbiztos, vagyis több szálú környezetben kis valószínűséggel, de előfordulhat, hogy több objektum jön létre
- A statikus tagváltozók inicializálása az osztály első használatakor történik
- A C# fordító olyan kódot generál az inicializáláshoz, ehhez, ami szálbiztos.

```
class WindowManager
{
    // Tárolja az egyetlen példányt
    // Itt inicializáljuk, ez szálbiztos
    private static WindowManager instance
        = new WindowManager();

    // Globális hozzáférést biztosító statikus property
    public static WindowManager Instance
    {
        get
        {
            return instance;
        }
    }
    // Védett konstruktor
    protected WindowManager() { }

    // Az osztály egyik művelete
    public void DoSomething() { /*...*/ }
}
```

Singleton

- Használatát nem szabad túlzásba vinni
- Sok esetben használata ellenjavallt (anti-pattern), mert a **GetInstance** mindig az osztály típusával tér vissza, nem tudunk egy alternatív „dummy”/fake/mock implementációval visszatérni, ami pedig a unit tesztelésnél sokszor alapelvárás.
 - > A Dependency Injection általában egy sokkal jobb tervezési minta!

Abstract Factory (Absztrakt gyár)

Abstract Factory

- Célja

- > Interfészt biztosít ahhoz, hogy egymással összefüggő objektumok családait hozzuk létre anélkül, hogy specifikálnánk a konkrét osztályukat
- > Így az objektumok létrehozása egy interfészen keresztül történik, a kódunk nem fog függeni a létrehozott objektumok konkrét osztályától/típusától

Próbáljuk megérteni egy példa segítségével 

Abstract Factory példa

- A célunk
 - > Egy modern ablakos/grafikus (GUI) felülettel rendelkező alkalmazás elkészítése
 - > Az alkalmazás felülete felületelemekből komponálódik (pl. Window, Button, Checkbox, Scrollbar, ...)
 - > Az alkalmazásnak több megjelenési témát kell támogatnia („look-and-feel”, „skin”). Pl. Win10, WinClassic, OSX, stb.
 - > A felületelemek megjelenése az aktuálisan kiválasztott megjelenési témát kell kövesse

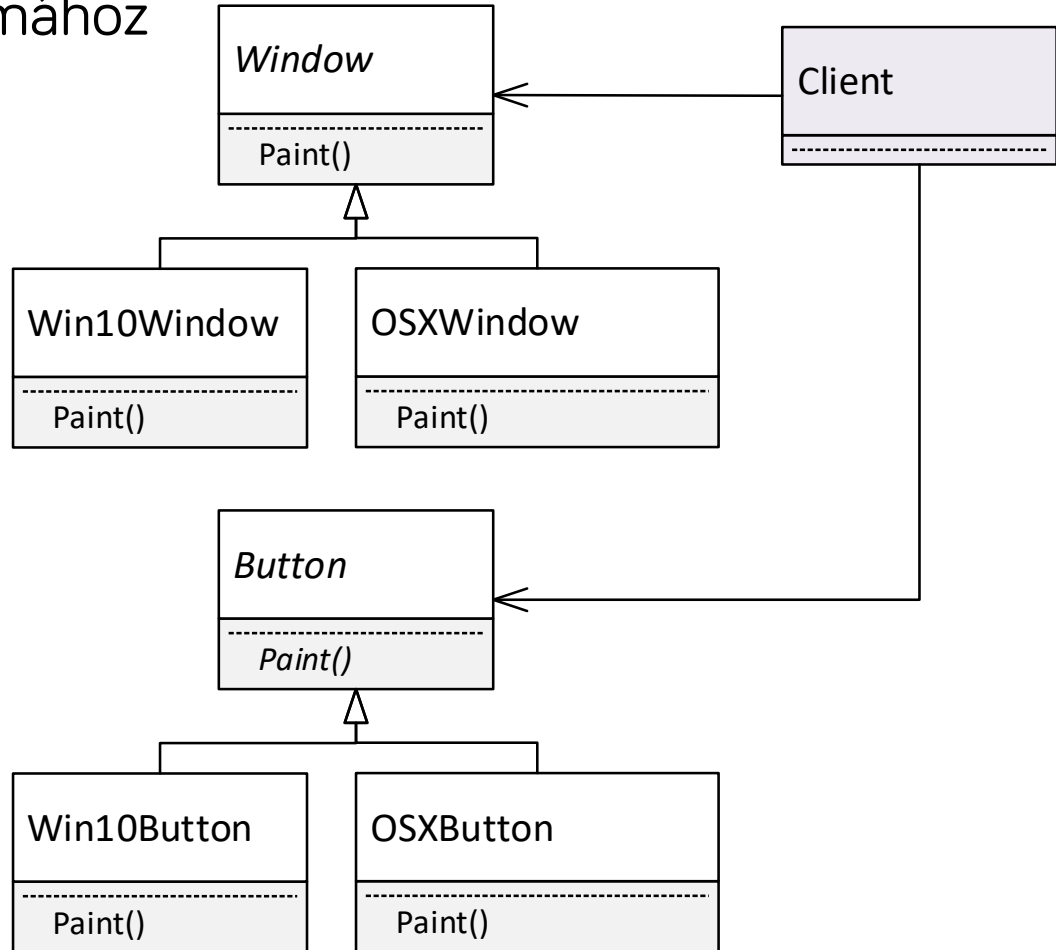
Abstract Factory példa

- Első lépésben minden felületelemhez külön verziót készítünk minden megjelenési témához

A kliens (Client)

reprezentálja az alkalmazás azon részét, mely a felületelemeket tárolja és példányosítja:

- Ez független kell legyen a konkrét megjelenítési téma osztályaitól (különben a téma nem lenne megváltoztatható).
- A Client emiatt interfészként vagy absztrakt ősként hivatkozik a felületelemekre



Abstract Factory példa

- Kód: lásd DesPattCode.AbstractFactory mappa. A kliens osztályt ilyen lépésekben fejlesztjük:
- **Client1_Initial** osztály: Ez a kiindulási verzió: ebbe még teljesen be vannak építve a téma specifikus osztályok. Új témára átálláskor a kódját jelentősen módosítani kell.
- **Client2_UsingInterfaces** osztály: A kiindulási verziót úgy alakítottuk át, hogy a GUI elem hivatkozásokat közös ősré/interfészekre cseréltük. De a GUI elemek példányosítása még a new operátor használata miatt mindig téma specifikus, így új témára átálláskor a kódját jelentősen módosítani kell.
- **Client3_UsingAbstractFactory** osztály: A GUI elemek példányosítása már az Abstract Factory mintával történik, a példányosítás is téma független lett. Új témára átálláskor a kliens kódját így már nem kell módosítani.

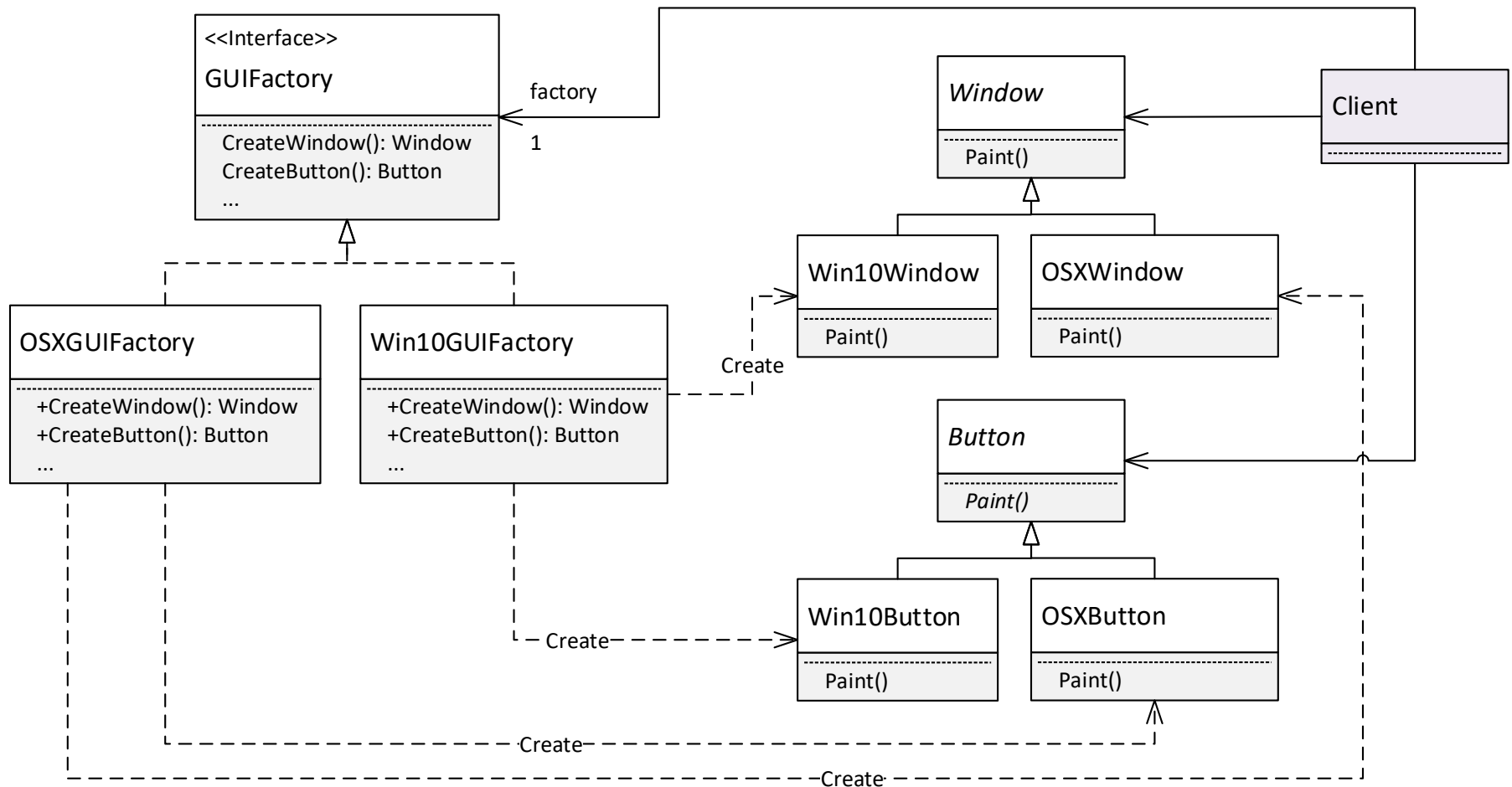
Abstract Factory osztálydiagram + magyarázat (+ mindenképpen érdemes Visual Studioban vagy GitHubon a teljes kódot megnézni!)



Abstract Factory - példa

- Ne drótozzuk bele a felhasználói felület elemeket az alkalmazásba. Sem az elemek **létrehozását**, sem pedig a **használatukat**.
 - > Zárjuk egységbe ezek létrehozását, bízzuk más (abstract factory) objektum(ok)ra
 - > Az alkalmazásban csak egy interfészen keresztül hivatkozunk rájuk (így az implementációjuk kicserélhető)

Abstract Factory struktúra



Abstract factory - Megoldás

- Vezessünk be egy **GUIFactory** interfészt felületelemek létrehozására.
- A **GUIFactory** mindegyik művelete egy a művelet nevének megfelelő felületelemet gyárt és azzal tér vissza (általános, témafüggetlen típusként tér vissza): pl. **CreateWindow()** → **Window** visszatérési típus, **CreateButton()** → **Button** visszatérési típus).
- Mindegyik megjelenítési témához bevezetünk egy **GUIFactory** implementációt: (Win10 → **Win10GUIFactory**, OSX → **OSXGUIFactory**), amelynek metódusai az adott témához tartozó felhasználói felületelem objektumokkal térnek vissza. Pl. a **Win10GUIFactory.CreateWindow()** **Win10Window**-t ablakot példányosít, míg az **OSXGUIFactory.CreateWindow()** **OSXWindow**-t (a példányosítás a szokványos new operátorral történik).
- A klienst felkonfiguráljuk valamelyik **GUIFactory** implementációval (amelyik témát aktuálisan be kívánjuk állítani). A kliens eltárol erre egy hivatkozást egy tagváltozóban, a hivatkozás/tag típusa az általános **GUIFactory** interfész.

Abstract factory - Megoldás

- A kliens a felületelemek létrehozására az eltárolt **GUIFactory** objektumot használja a new operátor helyett: pl. ablak létrehozására a **CreateWindow-t**, gomb létrehozására a **CreateButton-t** hívja. Ezek a műveletek a beállított/eltárolt **GUIFactory** implementációnak (amire előzetesen felkonfiguráltuk) megfelelő témájú konkrét felületelem objektumokat hoznak létre.
- A kliens a létrehozott felületelem objektumokra csak az közös interfészükön/osztályukon keresztül hivatkozik.
- Összegezve: a kliens mind a létrehozott témafüggő felületelem osztályokra, mind a felületelemek létrehozására használt témafüggő **GUIFactory** implementációra csak a témafüggetlen közös interfészükön/ősosztályukon keresztül hivatkozik, a kliens kódja teljesen témafüggetlen lesz: a kliens kódjában semmilyen témafüggő kód/osztály nem szerepel. Így a téma megváltoztatásakor a kliens kódján nem kell változtatni semmit.
- Megjegyzés: a példában a felületelemeknek közös őse van, de lehetne közös interfésze is, feladatfüggő, melyik megközelítést célszerű változtatni

Abstract Factory kód

- GUIFactory

```
// Abstract Factory interfész.  
// Mindegyik művelete egy a művelet nevének megfelelő vezérlőt gyárt.  
// Mindegyik művelet visszatérési típusa az adott vezérlő  
// interfész/közös ős típusa (vagyis független a témától!).  
interface GUIFactory  
{  
    Window CreateWindow();  
    Button CreateButton();  
    Scrollbar CreateScrollBar();  
}
```

Abstract Factory kód

- Win10GUIFactory és OSXGUIFactory

```
// A GUIFactory Win10 implementációja, Win10 témájú felületelemeket gyárt.
// Minden művelete egy a művelet nevének megfelelő típusú, az osztály
// nevének megfelelő (jelen esetben Win10) témájú felületelemet gyárt.
class Win10GUIFactory : GUIFactory
{
    public Window CreateWindow() { return new Win10Window(); }
    public Button CreateButton() { return new Win10Button(); }
    public Scrollbar CreateScrollbar() { return new Win10Scrollbar(); }
}

// A GUIFactory OSX implementációja, OSX témájú felületelemeket gyárt.
// Minden művelete egy a művelet nevének megfelelő típusú, az osztály
// nevének megfelelő (jelen esetben OSX) témájú felületelemet gyárt.
class OSXGUIFactory : GUIFactory
{
    public Window CreateWindow() { return new OSXWindow(); }
    public Button CreateButton() { return new OSXButton(); }
    public Scrollbar CreateScrollbar() { return new OSXScrollbar(); }
}
```

Abstract Factory kód

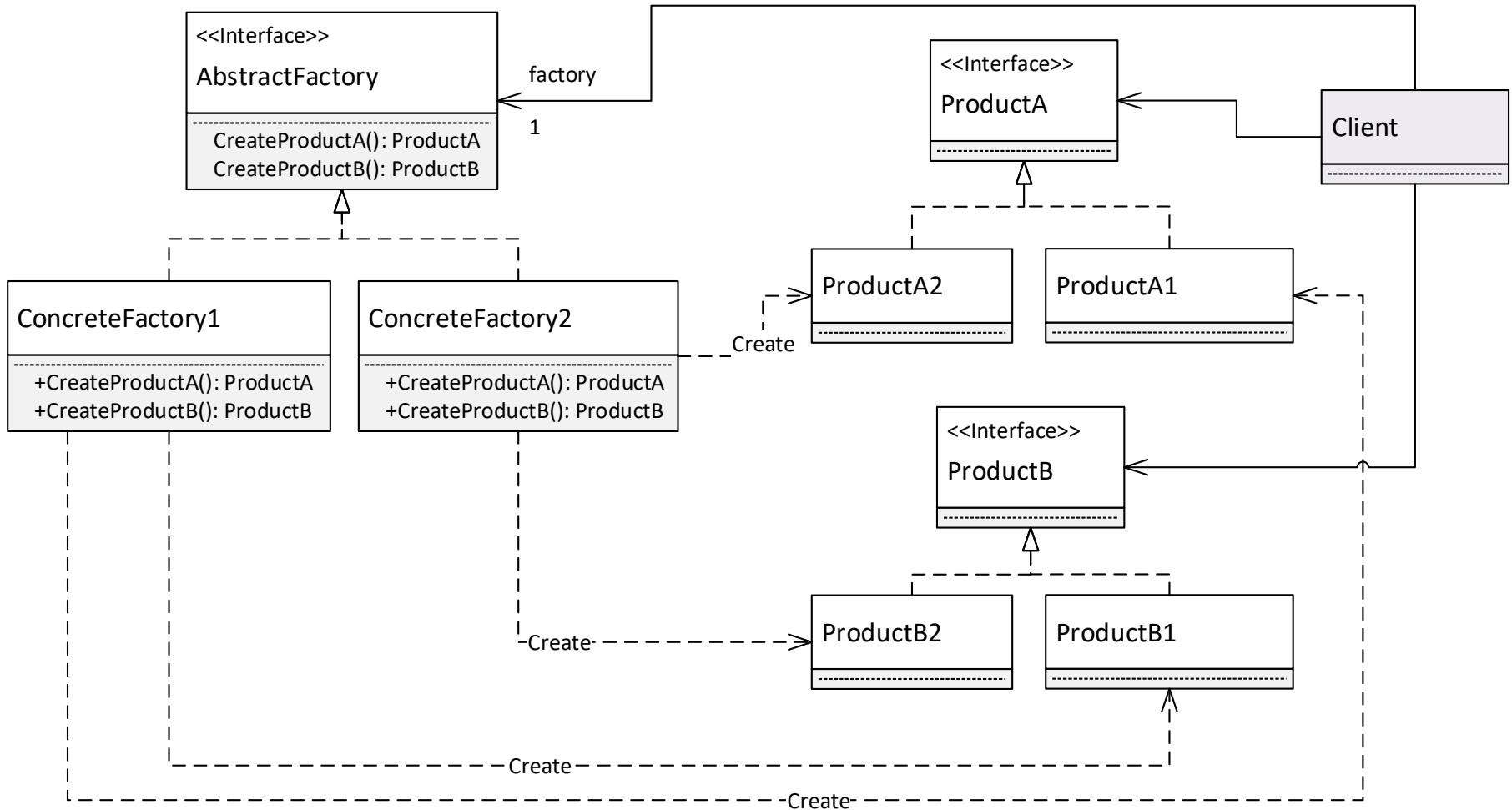
- Client3_UsingAbstractFactory

```
class Client3_UsingAbstractFactory
{
    // GUI elemek
    private Window wnd;
    private Button button;
    // ...
    // Factory interfész, ezt használja majd a kliens a GUI elemek létrehozásához.
    private GUIFactory factory;

    public void SetFactory(GUIFactory fy) { // Művelet a factory beállításához
        factory = fy;
    }

    // Felületelemek példányosítása new helyett a factory segítségével
    public void InitGUIElements() {
        wnd = factory.CreateWindow();
        button = factory.CreateButton();
        //...
    }

    public void DoSomethingComplex() {
        // Demonstráljuk a GUI elemek kirajzolását
        wnd.Show(); wnd.Paint(); button.Paint();
        //...
    }
}
```

Abstract factory

- Használjuk, amikor
 - > A rendszernek függetlennek kell lennie az általa létrehozott dolgoktól (“termék” objektumok, pl. felhasználói felület elemek)
 - > A rendszernek több termékcsaláddal kell együttműködnie
 - > A rendszernek szorosan összetartozó “termék” objektumok adott családjával kell dolgoznia, és ezt akarjuk kényszeríteni a rendszerben (pl. Win10 scrollbar ne lehessen OSX ablakkal együtt használni)

Abstract factory

- Előnyök
 - > Elszigeteli a konkrét osztályokat
 - > A termékcsaládokat könnyű kicserélni
 - > Elősegíti a termékek közötti konzisztenciát
- Hátrányok
 - > Nehéz új termék hozzáadása. Ekkor az Abstract Factory egész hierarchiáját módosítani kell, mert az interfész rögzíti a létrehozható termékeket
 - > Megjegyzés: ezt bizonyos esetekben ki lehet kerülni, pl. ha az előző példában minden termék egy közös GraphObject osztályból származik)

A létrehozási minták áttekintése

- A létrehozási minták célja: hozzuk létre az objektumokat úgy, hogy a rendszerünk rugalmasabb, könnyen bővíthető, a meglevő osztályok könnyebben újrafelhasználhatók legyenek
 - > Abstract factory: külön factory hierarchia létezik termékcsaládok létrehozására
 - > Singleton: csak egy példány létezen, ami bárholnan közvetlenül elérhető
 - > Factory method (nem tananyag): a leszármazottra bízta az objektum létrehozását, a Template Method minta alkalmazása objektum létrehozására.
 - > ...
- A modern nyelvekben a reflexiós technikák segítségével a legtöbb létrehozási minta kiváltható, ma már gyakran ezt is használjuk
 - > A létrehozandó típus reflexió esetén ugyanis sztring formájában is megadható, ami tetszőleges módon összeállítható
 - > A reflexió hátrányai:
 - A többi megoldáshoz képest jelentősen lassabb. Ez van, amikor számít (nagy számú objektum létrehozásánál), van, amikor nem.
 - Nem típusbiztos, ha elgépelünk egy típusnevet, akkor csak futás közben derül ki, fordításkor nem.

Szoftvertchnológia és -technikák

8. Előadás – Benedek Zoltán
Tervezési minták 3



Automatizálási és
Alkalmazott
Informatikai Tanszék

Ez az oktatási segédanyag a Budapesti Műszaki és Gazdaságtudományi Egyetem oktatója által kidolgozott szerzői mű. Kifejezett felhasználási engedély nélküli felhasználása szerzői jogi jogsértésnek minősül.

Tartalom

Tervezési minták

- > Command
- > Command Processor
- > Memento
- > Adapter
- > Composite

Command (Parancs)

(Action)

Command

- Cél
 - > Egy kérés **objektumként** való egységbezárása.
- Bővebben
 - > Egy kérés általában egy **függvényhívásként** jelenik meg a kódban. Ezzel szemben Command esetében a kérés **objektumként** jelenik meg.
 - > Ez lehetővé teszi a kliens különböző kérésekkel való felparaméterezését, a kérések sorba állítását, naplózását és visszavonását (undo).
- Alternatív név: Action
- Nagyon rendszerfüggő (C++, .NET UWP, Java, stb.) a koncepció értelmezése és az implementáció is

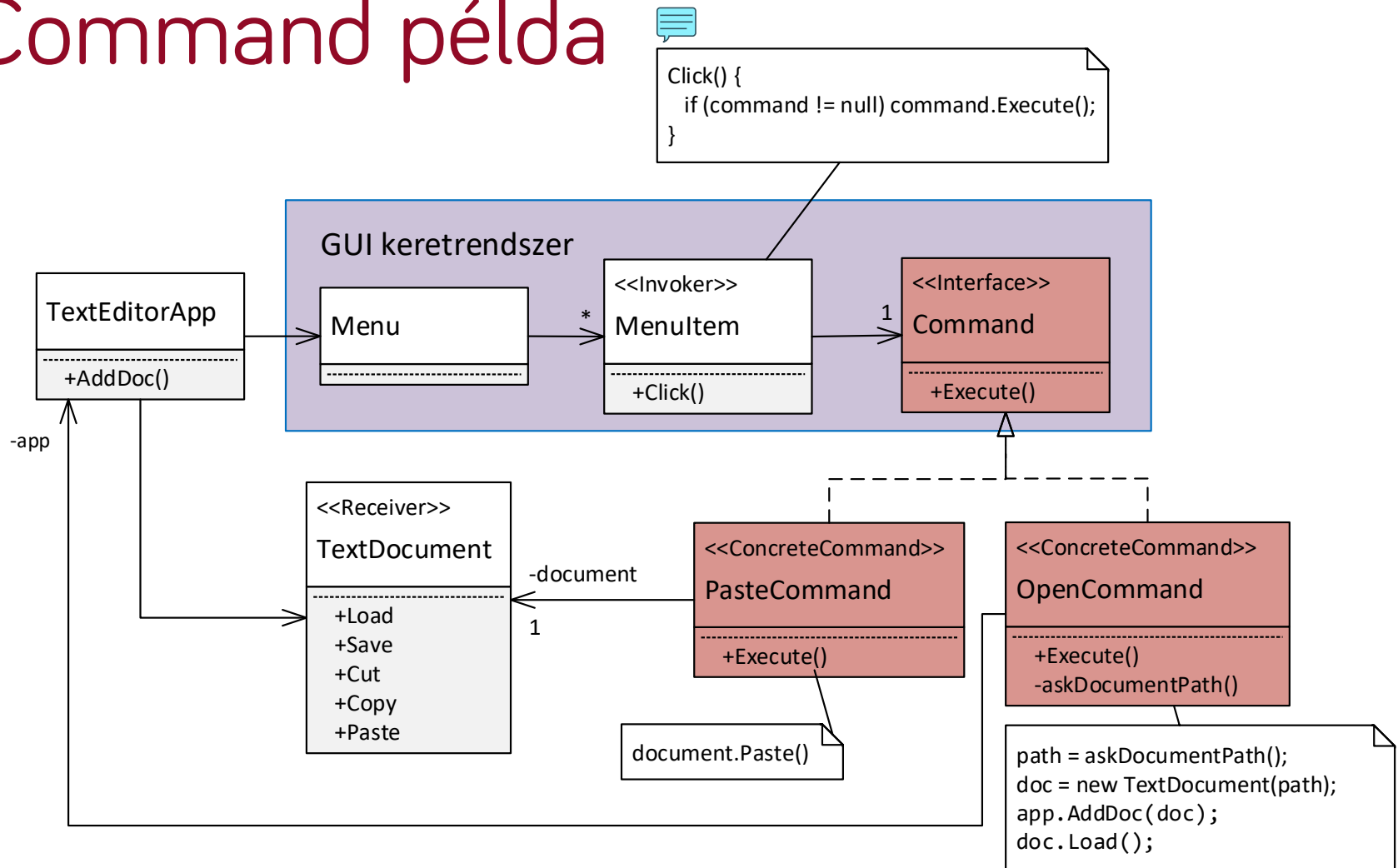
Példa

- Példa: felhasználói parancsok
 - > Menü, gomb, toolbar gomb
 - > Résztvevők pl.: Alkalmazás, Dokumentum, Menü, Almenü, Menüpont, stb.
 - > Probléma: a GUI keretrendszer írói nem építhették bele az alkalmazásfüggő menüelem kiválasztás kezelést →
 - > Hogyan reagáljunk a menüpont kiválasztása által generált eseményekre?
 - Command minta, ezt nézzük most
 - Callback függvény – nem objektum-orientált (strukturált) megoldás
 - Delegate alapú megoldás – .NET
 - Adapter alapú megoldások – Java

Command Példa

- Adott egy GUI keretrendszer
 - > Ebben beépített Menu (menü), MenuItem (menüelem) osztályok felhasználói parancsok futtatására
 - > A feladat: az alkalmazásfejlesztő ugyanazon beépített MenuItem osztály objektumait teljesen eltérő , ráadásul alkalmazásfüggő kódok futtatására akarja használni. Pl.:
 - File/Open menüelem MenuItem objektuma a fájlmegnyitás kódját kell futtassa
 - Edit/Paste menüelem MenuItem objektuma a paste (beillesztés) logikához tartozó kódot kell futtassa
 - > Ha ugyanaz az osztály (MenuItem)-> ugyanazok a tagfüggvények. **Hogyan lehet mégis eltérő kódot futtatni eltérő MenuItem objektumok esetén?**

Command példa



Zárjuk külön Command interfészt implementáló osztálybeli objektumokba a kéréseket, és a menüelemeket ezekkel paraméterezzük fel

Magyarázat



Command példa magyarázat

- A példa azt illusztrálja, hogy a Command minta használatával hogyan lehet a File/Open és az Edit/Paste menüelemekhez a megfelelő kód futtatását elérni egy szövegszerkesztő alkalmazásban.
- `TextEditorApp` osztály: magát az alkalmazást reprezentálja. Tartalmaz egy `Menu` objektumot, ez az alkalmazás menüje.
- A `Menu` több menüelem (`MenuItem`) objektumból áll (pl. File/Open, Edit/Paste).
- A GUI keretrendszerben bevezetünk egy `Command` interfészt egyetlen, `Execute` metódussal
- Az általános `MenuItem` osztálynak van egy hivatkozása egy `Command` objektumra. Ez a kezdetben null, de bármilyen `Command` interfészt implementáló osztálybeli objektumra ráállítható.

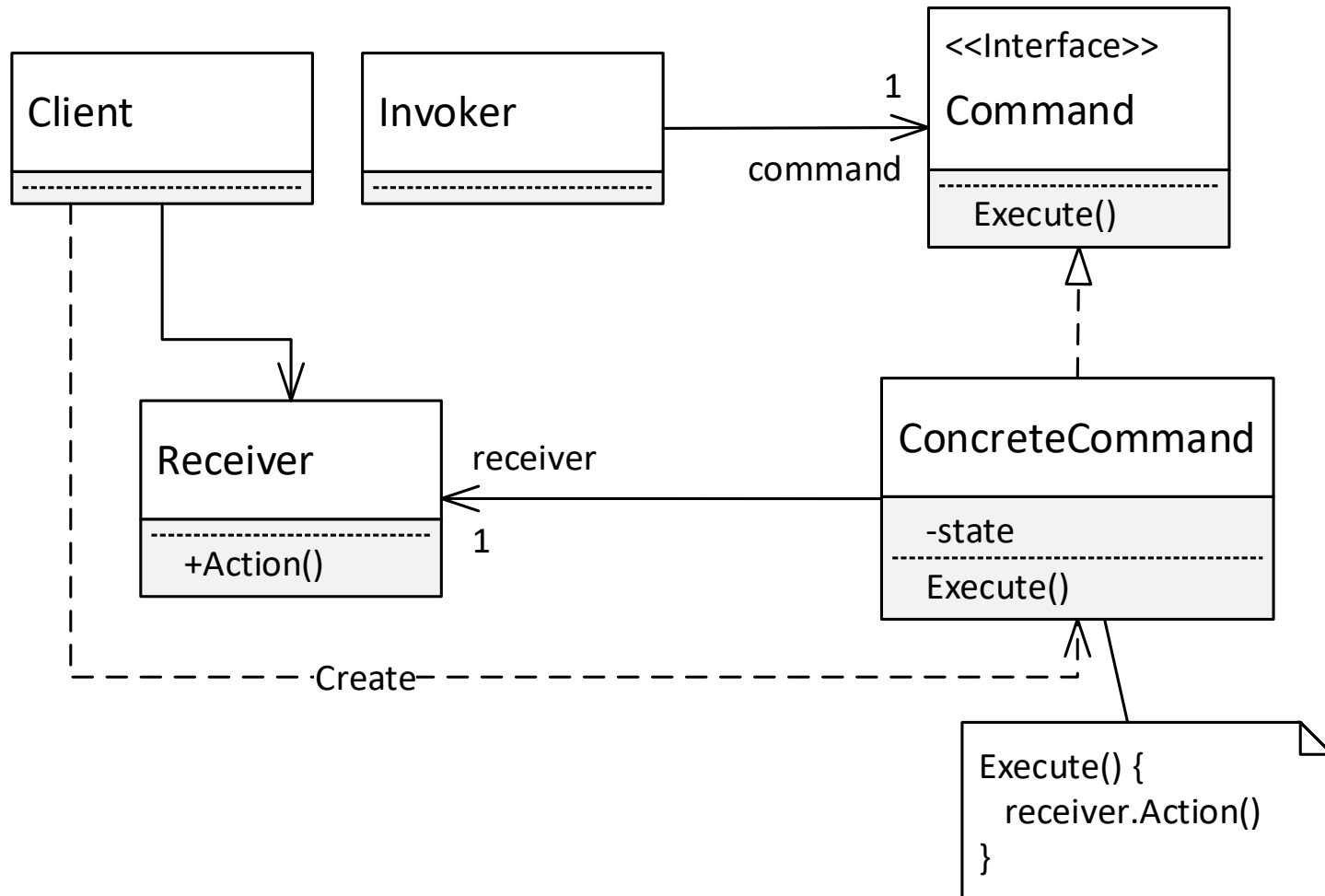
Command példa magyarázat

- Minden „parancshoz” bevezetünk egy Command implementációt: a Paste-hez egy PasteCommand osztályt, az Open-hez egy OpenCommand osztályt. Ezek Execute műveletébe a parancsspecifikus kódot tesszük (PasteCommand.Execute -> beillesztés megvalósítása, OpenCommand.Execute -> fájlmegnyitás megvalósítása).
- A Paste MenuItem command hivatkozását egy PasteCommand objektumra állítjuk, az Open MenuItem-ét egy OpenCommand-ra
- Amikor a felhasználó kattint egy adott menüelemen, meghívódik a MenuItem Click művelete, mely meghívja a MenuItem objektumhoz beregisztrált command objektum Execute műveletét.
- Ezzel pont elértük a célunkat: futás közben a Paste MenuItem kattintás a hozzá beregisztrált PasteCommand.Execute meghívását eredményezi -> ez beilleszti a szöveget. Ezzel analóg módon az Open MenuItem kattintás a hozzá beregisztrált OpenCommand.Execute meghívását eredményezi -> ez megnyit egy új dokumentumot.
- Általánosságban: **Azáltal, hogy ugyanolyan MenuItem osztálybeli objektumokhoz eltérő Command implementációkat regisztráltunk be, eltérő Execute művelet fut le, vagyis el tudtuk érni, hogy a kattintás eseményre más-más kód fusson le.**
- Megjegyzés: a kapcsolódó C# mintakódban a Command interfész neve ICommand követve a .NET konvenciókat.

Command megjegyzések

- **Példakód:** lásd DesPattCode/Command mappa (a futtatáshoz nevezzük át a Program osztály Main2 függvényét Main-re).
- Ízlés kérdése , mennyi logikát teszünk a `Command.Execute`-ba. Két megközelítés lehetséges
 - > Beletesszük a logika részletes implementációját. Erre példa az `OpenCommand.Execute`.
 - > A részletes implementációt más osztályba (ún. „Receiver”) tesszük, a `Command.Execute` ennek delegálja a kérést. Erre példa a `PasteCommand.Execute`, mely a kéréseket a `TextDocument` osztálynak továbbítja.
- A Menu és MenuItem nem szükséges része a Command mintának, a minta szempontjából lényegtelen, mi futtatja a parancsot.

Command minta általánosságában



Command – mikor használjuk

- Használjuk, ha
 - > Ha strukturált programban callback függvényt használnánk, objektumorientált programban használjunk commandot helyette. Más megközelítésben: ezzel tudunk objektumspecifikus – és nem osztályspecifikus – kódot futtatni (ilyen volt a Menultem példa is).
 - > **Visszavonás támogatására** – eltároljuk az előző állapotot a command-ban. Lásd Command Processor minta rövidesen!
 - > **Szeretnénk a kéréseket különböző időben kiszolgálni (a parancs kiadásától, megszületésétől „leválasztva”)**. Ilyenkor várakozási sort használunk, ebbe tesszük a command objektumokat. Az egyes command objektumokban tároljuk a parancs paramétereit, majd akár különböző folyamatokból/szálakból is futtathatjuk őket.

Command további gondolatok

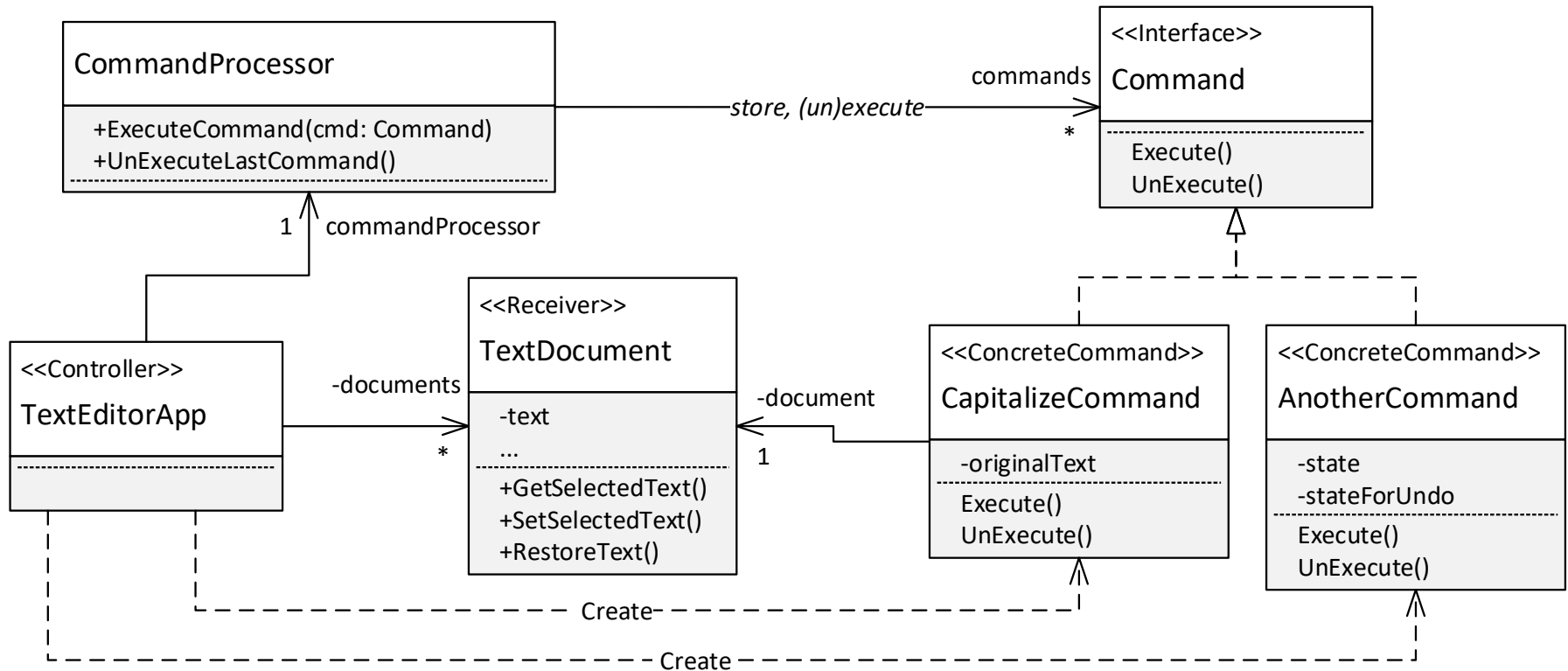
- Általánosítva az alapgondolata a következő: Elválasztja a parancsot **kiadó** objektumot (pl. MenuItem) attól az objektumtól, amelyik tudja, hogyan kell **lekezelni** (adott Command implementáció).
- Könnyű hozzáadni új parancsokat, mert ehhez egyetlen létező osztályt sem kell változtatni. Hogyan tegyük ezt meg?
- Összetett parancsok támogatása (lásd Composite minta később)
- Ismétlésképpen: nagyon rendszerfüggő (.NET UWP, Java, stb.) a koncepció értelmezése, sokszor némiképpen mást értenek alatta!
 - > Pl. UWP-ben is mást jelent, nincs minden parancsfuttatáshoz külön Command objektum létrehozva, viszont a parancs futtatásán túl kezelni, hogy az adott parancs az adott pillanatban engedélyezve vagy tiltva van-e.

Command Processor (Parancsfuttató)

Command Processor

- A Command minta egy változata
- „Beépítve” támogatja parancsok visszavonását (Undo)
- Alapelvek
 - > **UnExecute** művelet bevezetése a Command interfészbe
 - Minden command objektumnak támogatnia kell a változtatásának visszavonását is az UnExecute művelethez
 - Minden command objektum az Execute során eltárolja tagváltozóiban azt az állapotot, mely a visszavonáshoz szükséges
 - > **CommandProcessor** osztály bevezetése
 - Eltárolja a már futtatott command objektumokat, hogy ha később a parancs visszavonására kerül sor, rendelkezésre álljon a command objektum
 - A parancsok futtatását és visszavonását rajta keresztül végezzük (ExecuteCommand és UnExecuteLastCommand műveletek)

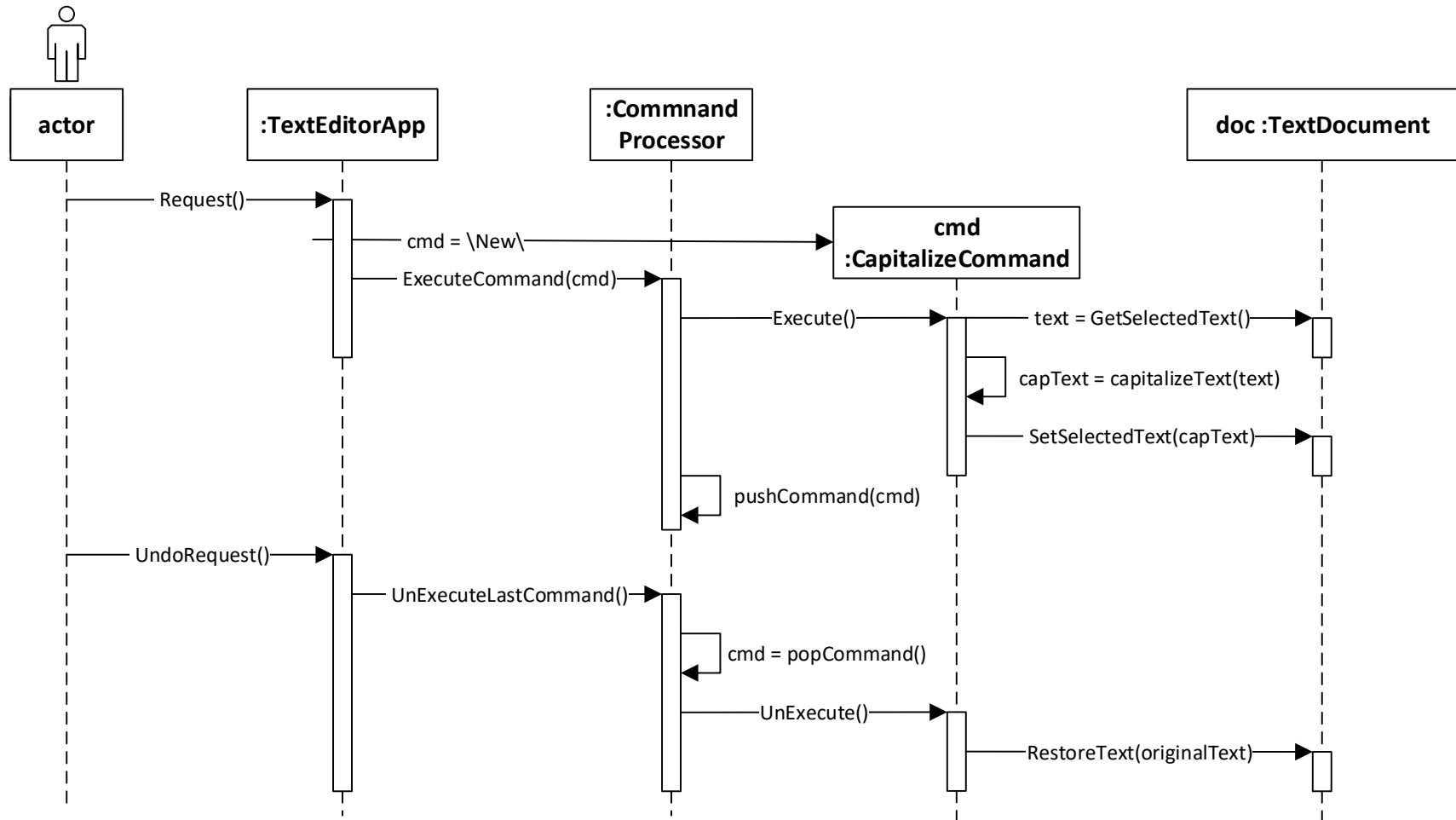
Command Processor példa



Capitalize parancs: a kijelölt szöveget nagybetűssé alakítja

Command Processor példa

- Kijelölt szöveg nagybetűssé alakításának lépései majd visszavonása



Példa magyarázat

- Kijelölt szöveg nagybetűssé alakításának lépései
 1. A felhasználó kijelöl egy szöveget és kéri ennek nagybetűssé alakítását. Lényegtelen, milyen módon, a példánkban a `TextEditorApp` fogadja a kérést.
 2. A `TextEditorApp` létrehoz egy `CapitalizeCommand` objektumot, és meghívja a **CommandProcessor.ExecuteCommand** műveletét, paraméterként átadva neki a parancs objektumot. Ez a művelet:
 1. Meghívja a `command` objektum `Execute` műveletét (így lefut a parancs kódja – a példában nagybetűssé alakítja a kijelölt szöveget)
 2. Eltárolja a parancs objektumot egy `command` gyűjteményben a `pushCommand` művelettel (hogyan az esetleges későbbi Undo során meglegyen)

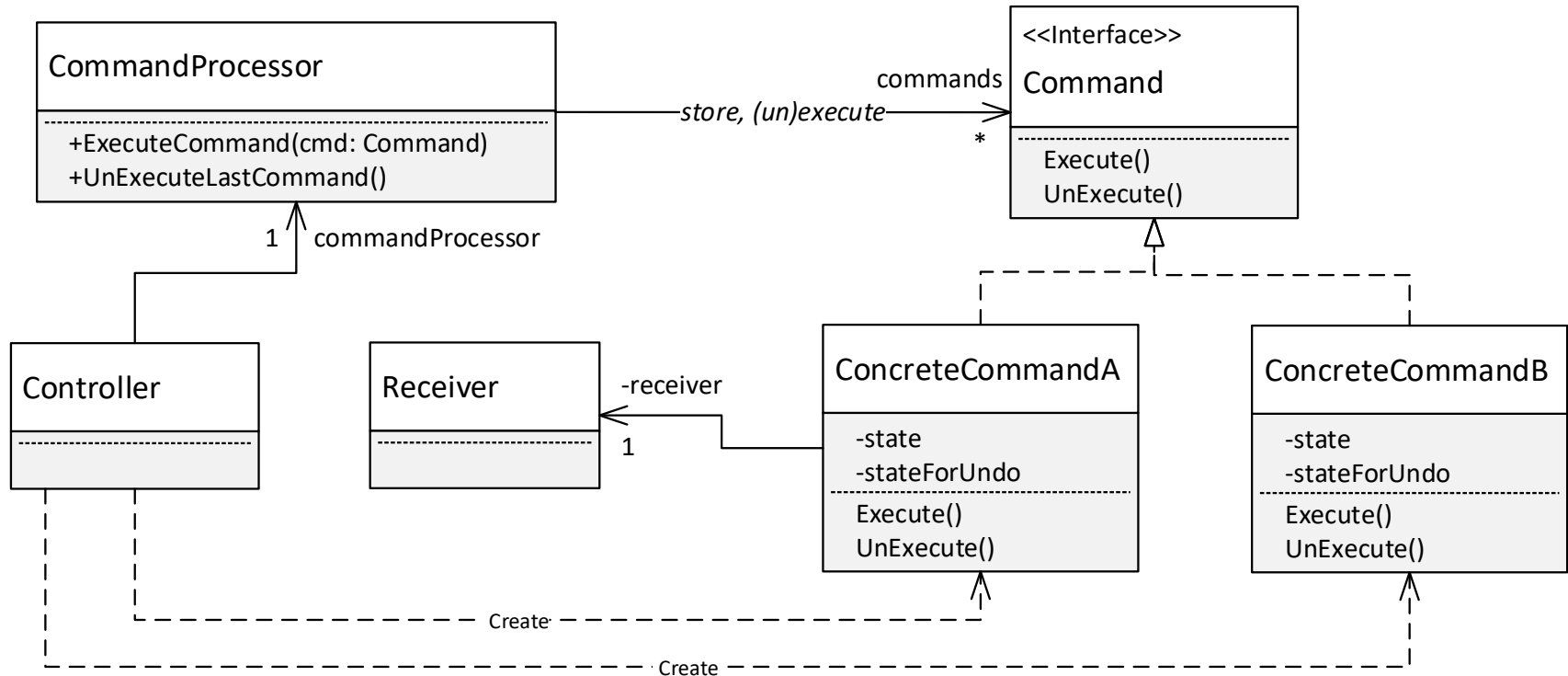
Példa magyarázat

- A felhasználói visszavonás (Undo) lépései
 1. A felhasználó kéri az utolsó parancs visszavonását. Lényegtelen, milyen módon, a példánkban a `TextEditorApp` fogadja a kérést.
 2. A `TextEditorApp` meghívja a **`CommandProcessor.UnExecuteLastCommand`** műveletét. Ez a művelet:
 1. Kiveszi a legutoljára eltárolt parancs objektumot a parancs gyűjteményéből (`popCommand` művelet)
 2. Erre a parancs objektumra meghívja az `UnExecute` műveletet (így lefut a parancs azon kódja, mely visszacsinálja a parancs által korábban végrehajtott változtatásokat)

Példa

- Kód: lásd DesPattCode/CommandProcessor mappa
 - > A kód futtatható és debugolható, a Program osztályban a Main2 függvényt nevezzük át Main-re

Command Processor általánosságában



Command Processor megjegyzések

- A Command két műveletét Execute-nak és UnExecute-nak neveztük. Szokásos a Do és Undo nevek használata is, illetve az UnExecute-ra a Revert alternatíva.
- A parancs végrehajtás során törekedjünk arra, hogy a visszaállításhoz csak azon állapotot mentsük el, mely a visszaállításhoz mindenképpen szükséges
 - > Ha pl. egy szövegszerkesztő esetén minden parancs során a teljes dokumentum tartalmát elmentjük, akkor nagyméretű dokumentum esetén hamar kifuthatunk a memóriából (vagy csak nagyon kevés lépés visszavonását tudjuk támogatni).

Memento

Memento

- Cél

- > Az egységbezárás megsértése nélkül a külvilág számára elérhetővé tenni az objektum belső állapotát:
 - Vagyis pl. anélkül, hogy a védett változókat publikussá tennénk
 - Így az objektum állapota később visszaállítható

- Példa: Visszavonás (Undo) funkció megvalósítása egy dokumentum esetén

Célok kifejtése



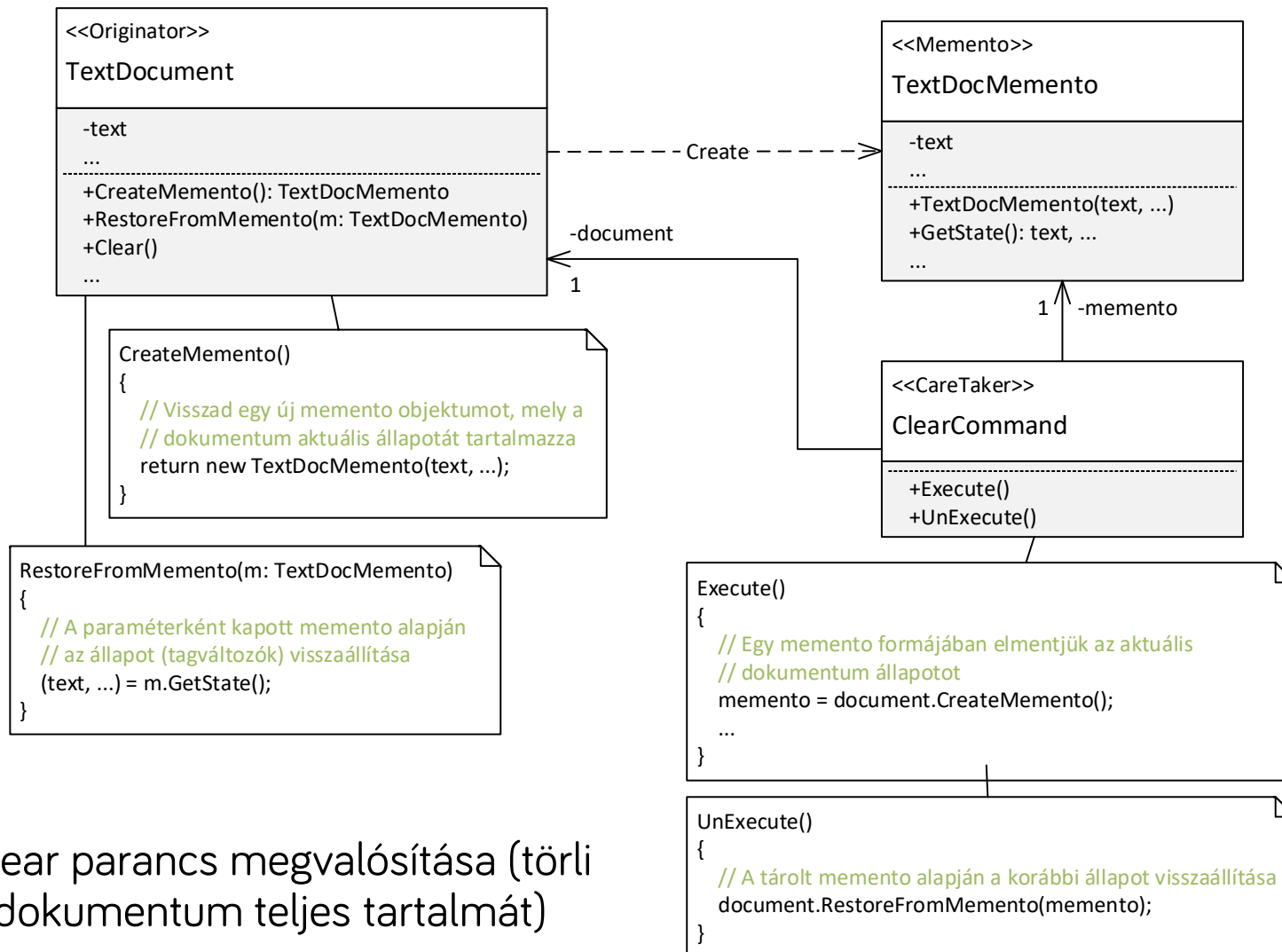
Memento

- Visszavonható műveletek
- Bár a Memento a Command/Command Processor minta nélkül is használható, jellemzően ezekkel célszerű kombinálni, mi is ezt tesszük
 - > A visszavonás sokszor nehéz vagy lehetetlen anélkül, hogy az objektum (pl. dokumentum) **teljes állapotát** elmentenék, majd visszaállítanánk a visszavonás során (pl. dokumentum teljes tartalmát törölő „Clear” parancs visszavonása).
 - > Az objektum teljes állapota viszont általában nem elérhető más osztályok számára, mert az egységbezárás miatt a tagváltozók védettek (private).
 - > Csak az visszavonáshoz való állapotmentés lehetősége miatt kellene ezeket a változókat publikussá tenni. **Nem tesszük (teljesen szembe menne az egységbezárás elvével)!** Inkább alkalmazzuk a Memento mintát.
- A Memento minta lényege, hogy egy **objektum (pl. dokumentum) adott állapotát egy Memento objektumba csomagoljuk be, és ilyen „becsomagolt” formában tesszük elérhetővé (a visszavonás megvalósításához)**

Memento példa

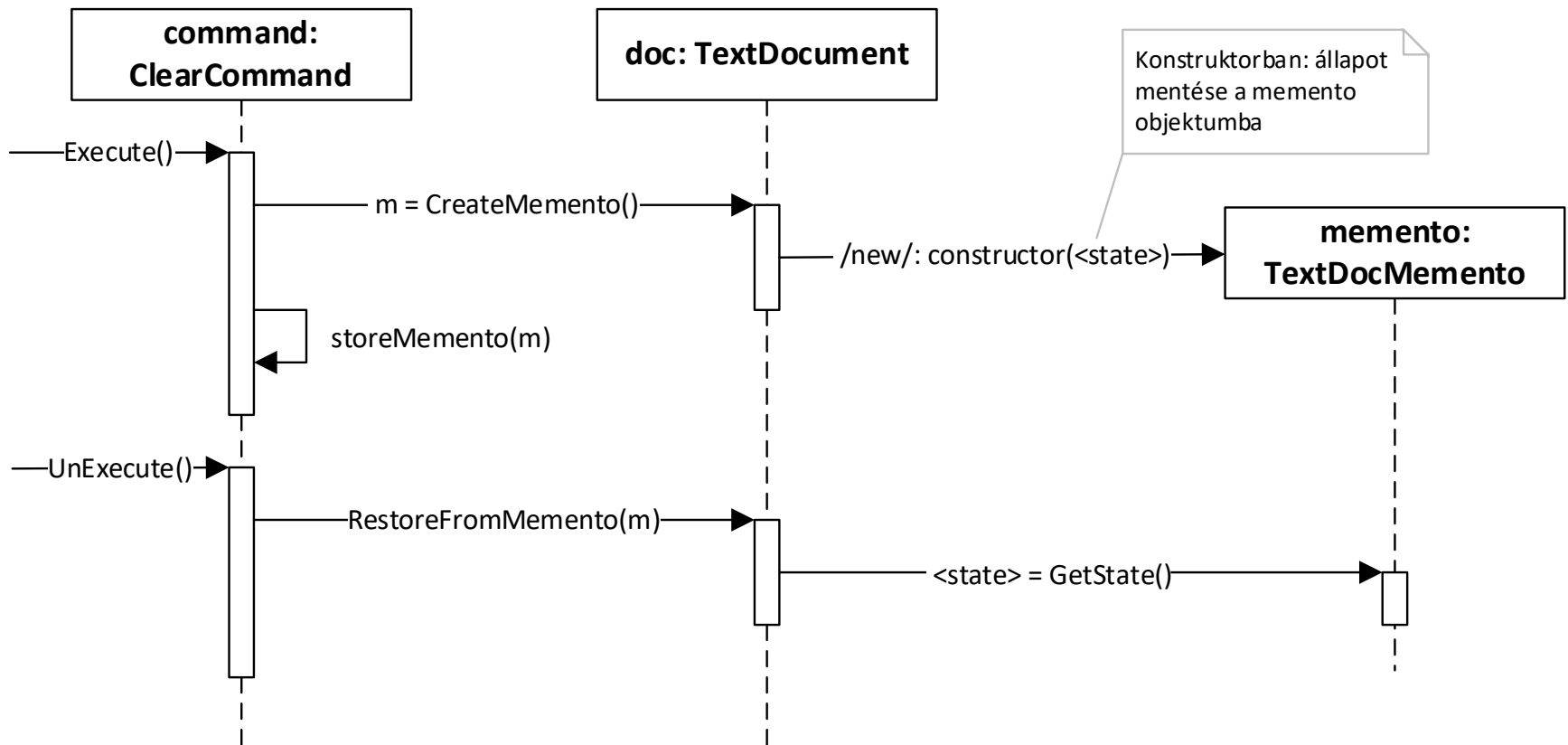
- Feladat : Clear parancs megvalósítása (törli a dokumentum teljes tartalmát)
- Ehhez bevezetjük a ClearCommand parancs osztályt (Command Processor minta, Command implementáció)
- A ClearCommand az Execute műveletében el kell mentse a TextDocument teljes állapotát, és UnExecute során vissza is kell állítsa
 - > De a TextDocument nem fér hozzá a TextEditor állapotához (text és egyéb tagváltozók private-ok és nincs mindenhez lekérdező/beállító függvény sem)
 - > Nem is akarjuk ezeket publikussá/közvetlen hozzáférhetővé tenni (egységbezárás megőrzése)
 - > Helyette: bevezetünk egy memento (TextDocMemento) osztályt. Ennek minden objektuma a dokumentum adott időpontbeli állapotát tárolja a tagváltozóiban (a tagváltozói „tükröképei” a dokumentum tagváltozóinak)

Memento példa



Clear parancs megvalósítása (törli a dokumentum teljes tartalmát)

Memento szekvenciadiagram



Megjegyzés: a `<state>` az ábrán a mentendő dokumentum állapotot jelenti (text és egyéb tagváltozók, melyeket a clear parancs módosít)

Memento példa

- Parancs (Clear példa) futtatása lépések (a `ClearCommand.Execute` a „belépési pont”)
 1. Egy memento formájában elmentjük az aktuális dokumentum állapotot
 - `document.CreateMemento()` hívása, mely visszaad egy új memento objektumot, mely a dokumentum aktuális állapotát tartalmazza a tagváltozóiban
 - A parancs objektum elmenti egy tagváltozóba ezt a memento objektumot a későbbi visszaállításhoz
 2. Lefut a parancs lényegi kódja, a clear „kipucolja” a dokumentum belső állapotát (text és egyéb tagváltozók kezdőértékre állítása)

Memento példa

- Parancs (Clear példa) visszavonása lépések (a `ClearCommand.UnExecute` a „belépési pont”)

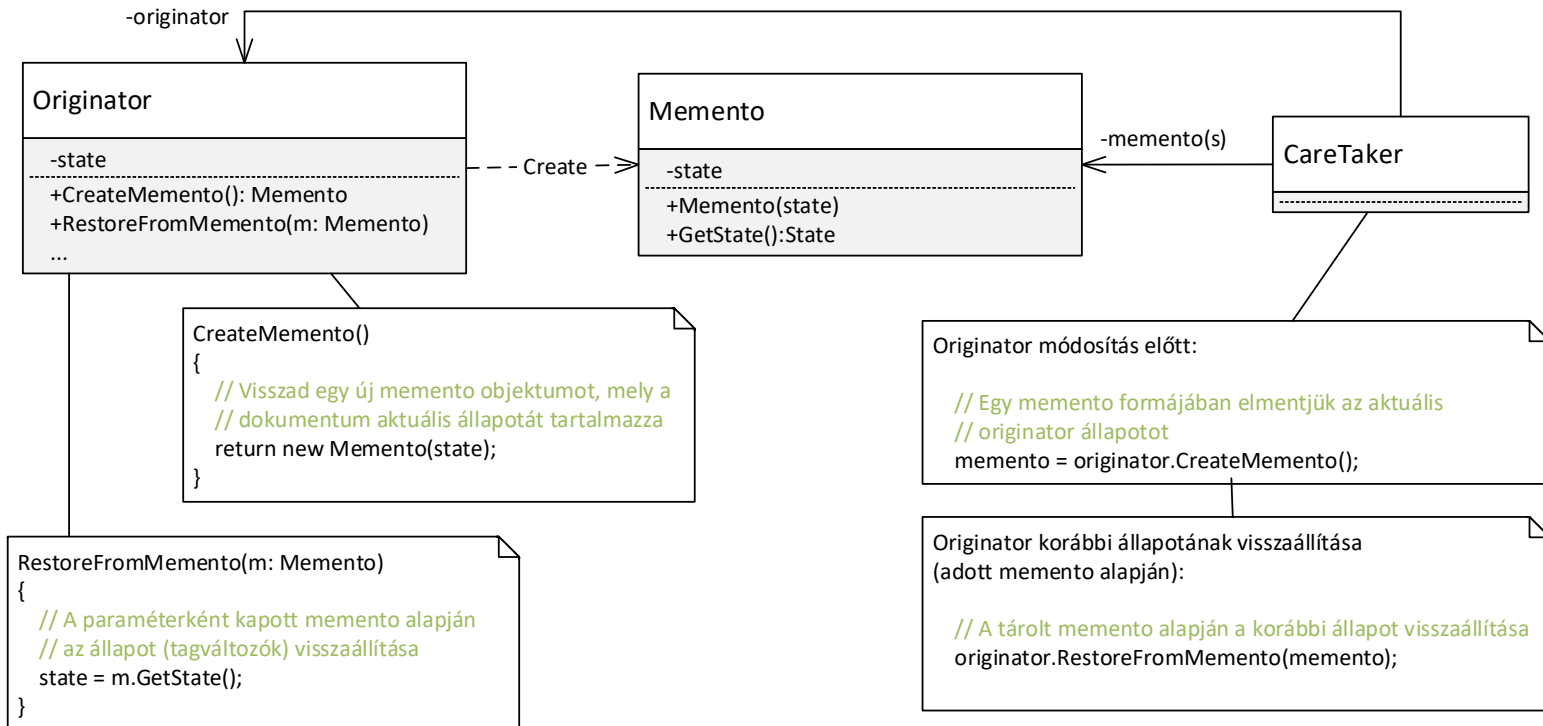
Visszaállítjuk a korábbi dokumentumállapotot

- A tagváltozóban tárolt memento objektumot paraméterként átadva
`document.RestoreFromMemento()` hívása
- A `document.RestoreFromMemento()` a paraméterként kapott memento alapján visszaállítja a dokumentum korábbi állapotát (tagváltozók értékének visszaállítása)

Memento példa

- Kód: lásd DesPattCode/ MementoWithCommandProcessor mappa (a futtatáshoz nevezzük át a Program osztály Main2 függvényét Main-re).
- Ez kicsit összetettebb implementáció, mint amit az előző diákon néztünk
 - > A `TextDocument` osztálynak nem csak a `text` tagváltozó adja az állapotát, hanem a `selectionStartIndex` és `selectionLenght` tagok is (ezek határozzák meg az aktuális kijelölést a dokumentumban)
 - > Ezeket is elmentjük a memento osztályunkba és visszaállítjuk az `UnExecute` során
 - > Így kicsit életszerűbb a példa
 - > A példa futtatható, az `App/Program` osztály `Main2` függvényét kell `Main`-re nevezni a futtatáshoz és debuggoláshoz

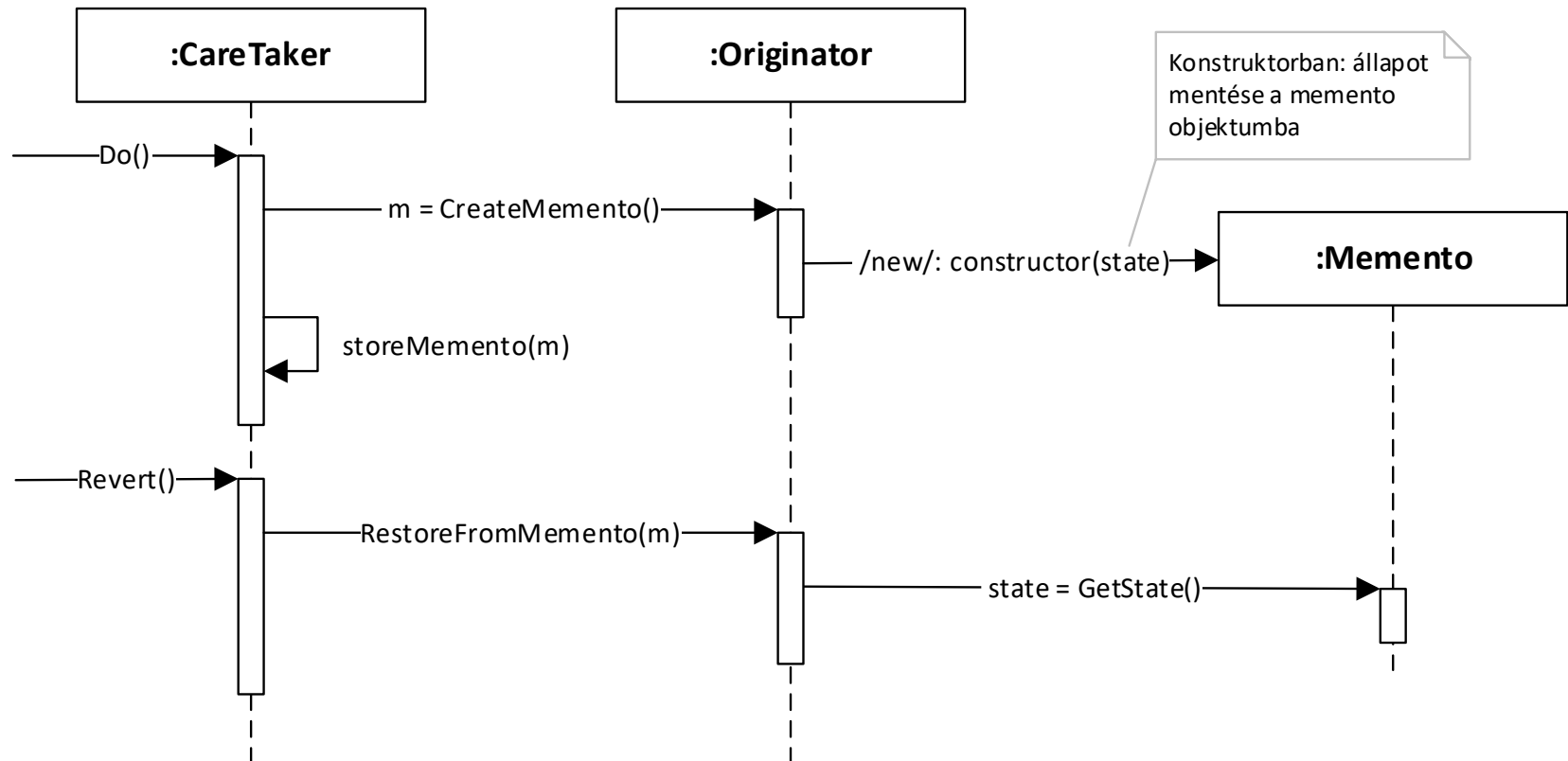
Memento minta általánosságában



- **Originator:** az ő állapotát kell tudni visszaállítani.
 - > A **CreateMemento()** elment (pontosabban visszaadja a state állapotot egy Memento objektum formájában)
 - > A **SetMemento()** visszaállít (pontosabban beállítja a state állapotot a paraméterben megkapott Memento objektum alapján)
- **Memento:** az Originator állapotát tárolja és elméletileg csak az Originator számára biztosít hozzáférést az állapothoz (state), de e nem minden programozási nyelven megvalósítható.
- **CareTaker:** nyilvántartja a mementot/mementokat (nem kell feltétlen command legyen)

Memento minta általánosságában

- Szekvenciadiagram



Memento megjegyzés

- Eddig azt mondtuk, hogy a Memento abban segít, hogy ne kelljen publikussá tenni az Originator (pl. TextDocument) tagváltozóit
 - > Nem tesszük publikussá
 - > És nem is vezetünk be olyan publikus műveleteket, melyekkel direktben le lehetne kérdezni és be lehetne állítani őket (ez is sértené az egységbezárást)
 - Vagyis marad a Memento, mint célszerű megoldás

Memento

- Használjuk, ha
 - > Egy objektum (rész)állapotát később vissza kell állítani és ennek támogatásához meg kellene sérteni az objektum egységbezárását
- Előnyök:
 - > Megőrzi az egységbezárás határait
- Hátrányok:
 - > Memento használata sokszor erőforrásigényes (pl. teljes dokumentum állapot mentése sok példányban)

Adapter (Illesztő)

(Wrapper)

Adapter

- Cél

➤ Egy osztály interfészét olyan interfésszé konvertálja, amilyent a kliens vár. Lehetővé teszi olyan osztályok együttműködését, melyek egyébként az inkompatibilis interfészeik miatt nem tudnának együttműködni.

- Alternatív név: Wrapper

- Elve



?



Nem megfelelő interfész



Megoldás: adapter

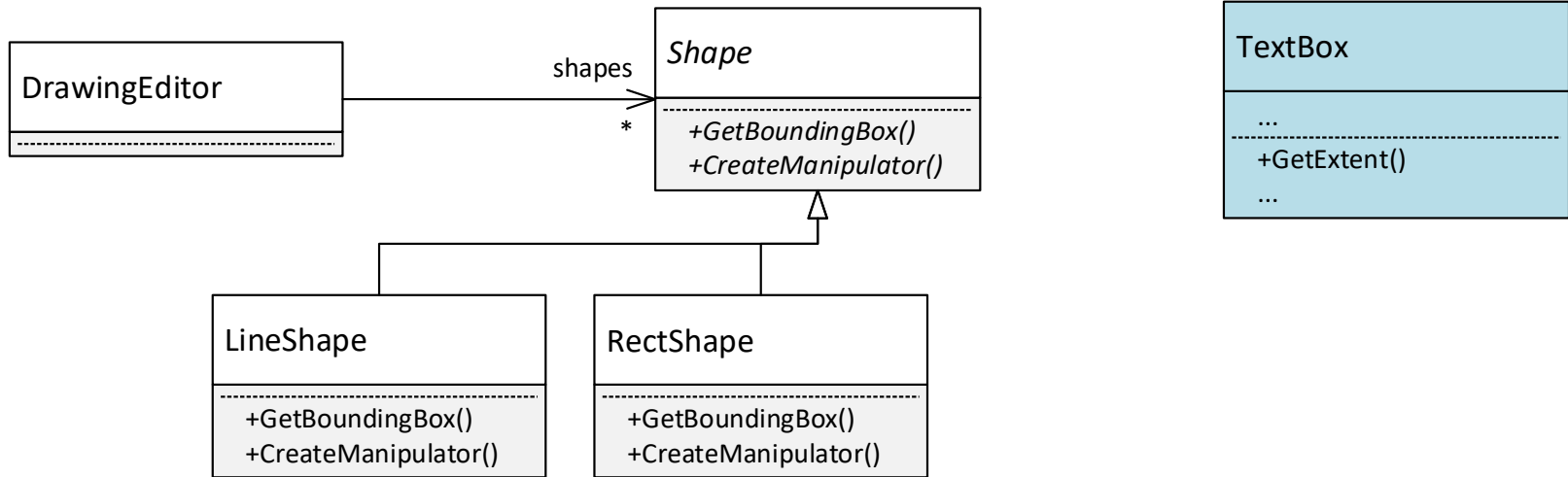
Adapter - példa

- A feladatunk egy vektorgrafikus alkalmazás elkészítése
 - > A felhasználó különböző grafikus alakzatokat tud elhelyezni a felületen. Támogatni kell a vonal, téglalap, stb. alakzatot, valamint a **szerkeszthető szövegdobozt** is (amibe lehet gépelni futás közben)
 - > Bevezetünk egy **Shape** **őszosztályt**, hogy az **alakzatokat egységesen szeretnék kezelni** (esetünkben **egy heterogén kollekcióban tudjuk tárolni**). A grafikus alakzatokat ebből a Shape osztályból származtatjuk. Pl. LineShape, RectShape, EditableTextShape.
 - > **Probléma:** az EditableTextShape (egy szerkeszthető szövegdoboz teljes logikájának) megvalósítása nagyon-nagyon nehéz, beláthatatlan mennyiségű munka.

Adapter – példa folytatás

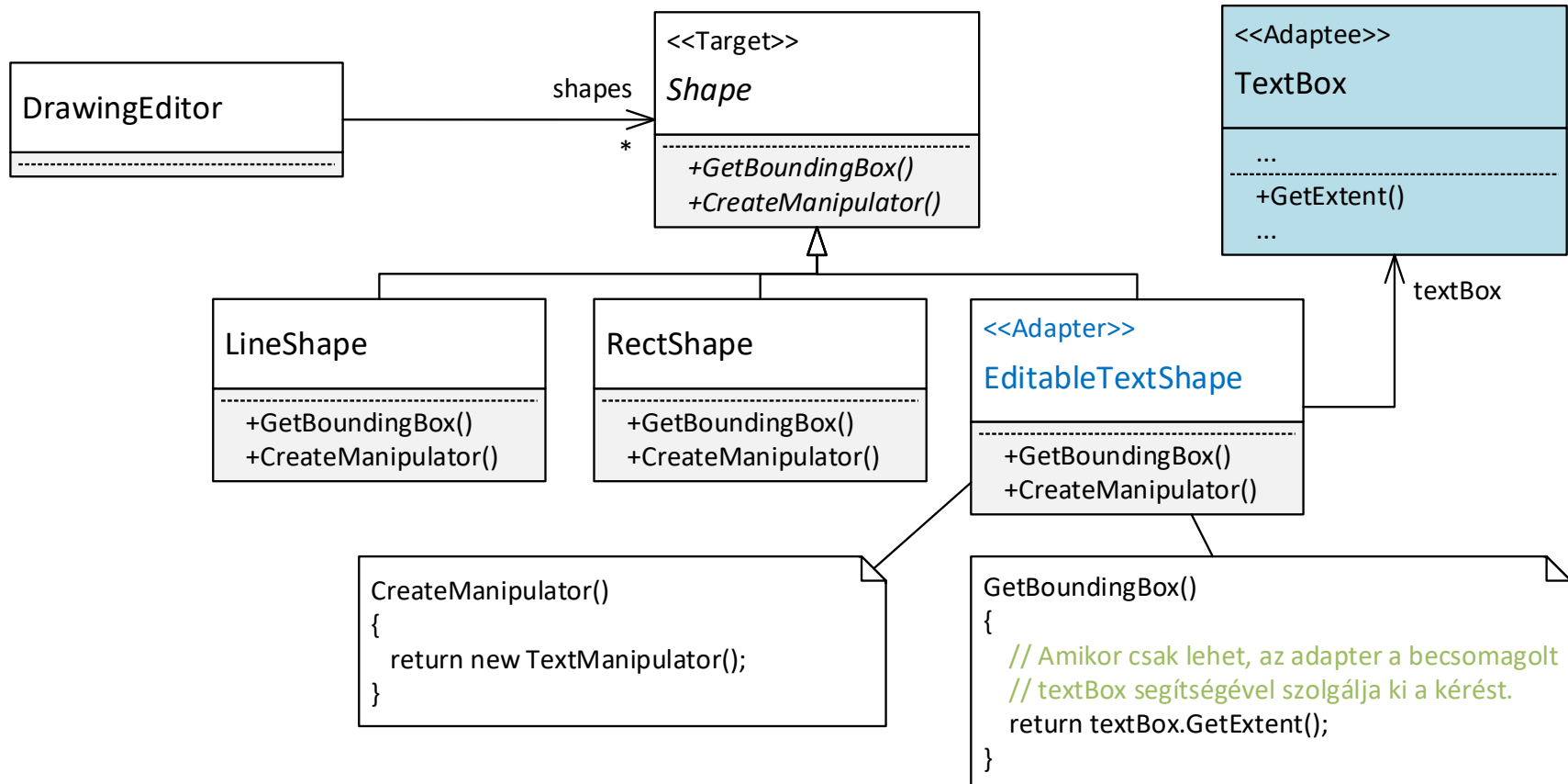
- > T.f.h. az alkalmazást egy adott keretrendszerre (pl. UWP) építve írjuk, amiben **már van egy beépített szerkeszthető szövegdoboz** (TextBox osztály), ami tudja mindazt, amit az EditableTextShape-től elvárunk. Jó lenne ezt felhasználni.
- > Probléma: a **beépített TextBox osztályt nem tudjuk közvetlenül felhasználni, mert nem megfelelő az interfésze**, ugyanis nem a Shape osztályból származik (emiattnem tudjuk a többi Shape-el együtt egységesen kezelni). Mivel egy „beépített” osztály, a forráskódját nem is tudjuk módosítani (hogya Shape osztályból származzon)
- > Probléma: nem tudjuk újrafelhasználni a meglévő osztályt!
- > Megoldás: Adapter minta használata (Object Adapter vagy Class Adapter)

Kiindulás

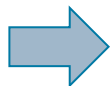


- A **DrawingEditor** egy **Shape** listát tárol (heterogén kollekció), a **TextBox** nem tehető bele
- A **GetBoundingBox** művelet egy befoglaló téglalapot ad vissza
- A **CreateManipulator** egy „manipulátor” objektumot, amivel az adott típusú alakzat szerkeszthető (pl. átméreteztető, stb., nincs jelentősége a példában)

Példa megoldás Object Adapterrel



Magyarázat



Példa megoldás Object Adapterrel

- Az eredeti tervünknek megfelelően leszarmaztatunk az `Shape` osztályból (vagy, ha a `Shape` interfész, implementáljuk azt) → Ez lesz az `EditableTextShape` osztály. Így az `EditableTextShape`-nek jó lesz az interfésze, a `DrawingEditor` tudja ezt is kezelni.
- Az `EditableTextShape` nem maga valósítja meg a műveletek többségét. Helyette:
 - > Példányosít és becsomagol egy `TextBox` osztályt
 - > A műveletek többségénél továbbhív a `TextBox` osztályba, delegálja a kéréseket. A példában a `GetBoundingBox` művelet a `TextBox` `GetExtent`-et hívja.
- Így a `EditableTextShape` fel tudja használni a már meglévő komplex logikát (`TextBox` osztály)
- Megjegyzés: a példában a `Shape` egy absztrakt osztály, lehetne interfész is, nincs jelentősége.

Példa megoldás Object Adapterrel

- Kód: lásd DesPattCode/Adapter mappa
- A legfontosabb rész maga az adapter

```
class EditableTextShape : Shape
{
    // A becsomagolt/adaptálandó osztály
    TextBox textBox;

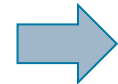
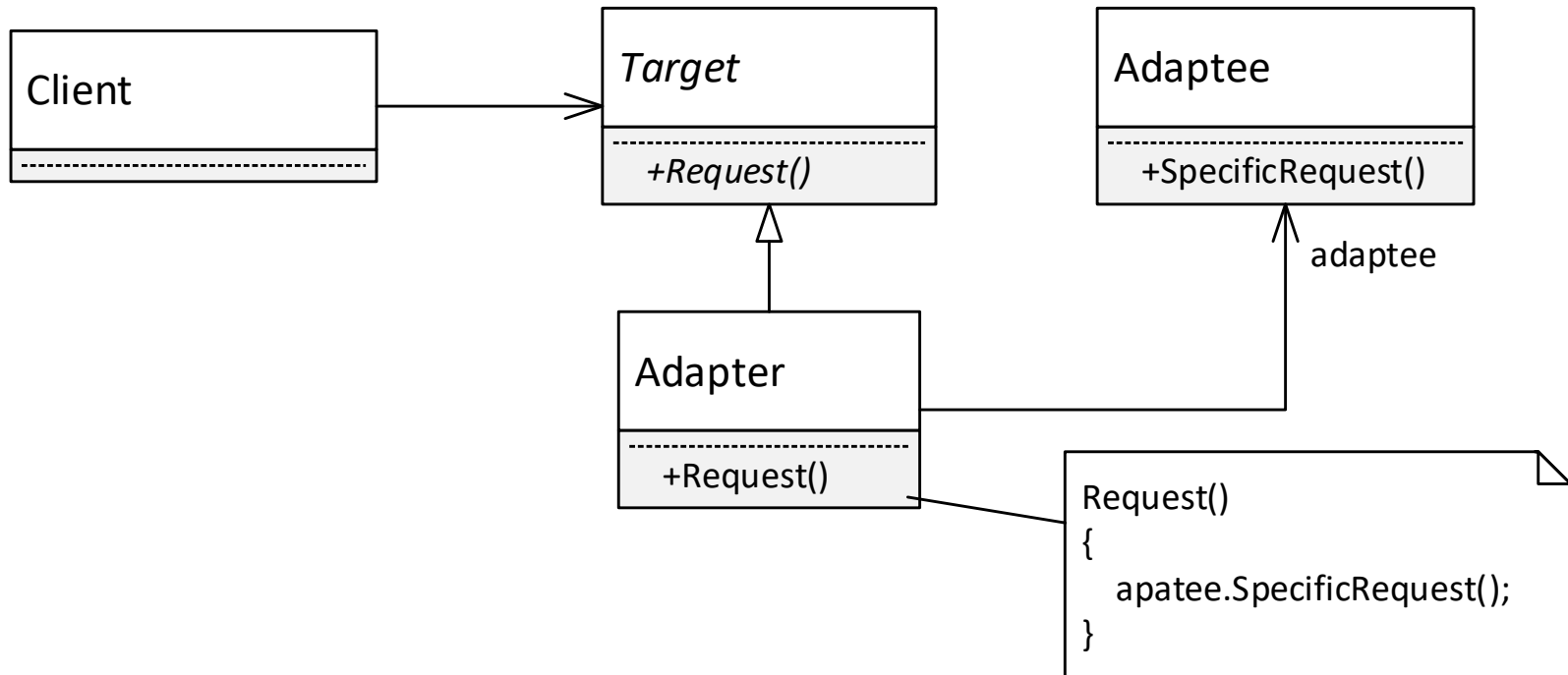
    // ...

    protected override Rect GetBoundingBox()
    {
        // Ez a lelke, ahol csak lehet, továbbítjuk a kérést
        // a becsomagolt/adaptálandó osztálynak, felhasználjuk
        // a kódját
        return textBox.GetExtent();
    }
}
```

Adapter

- Adapter minta általánosságában
 - Két változat
 - > Object Adapter
 - > Class Adapter
- Object Adapter → kompozícióval (has-a)
Class Adapter → örökléssel (is-a)

Object Adapter struktúra

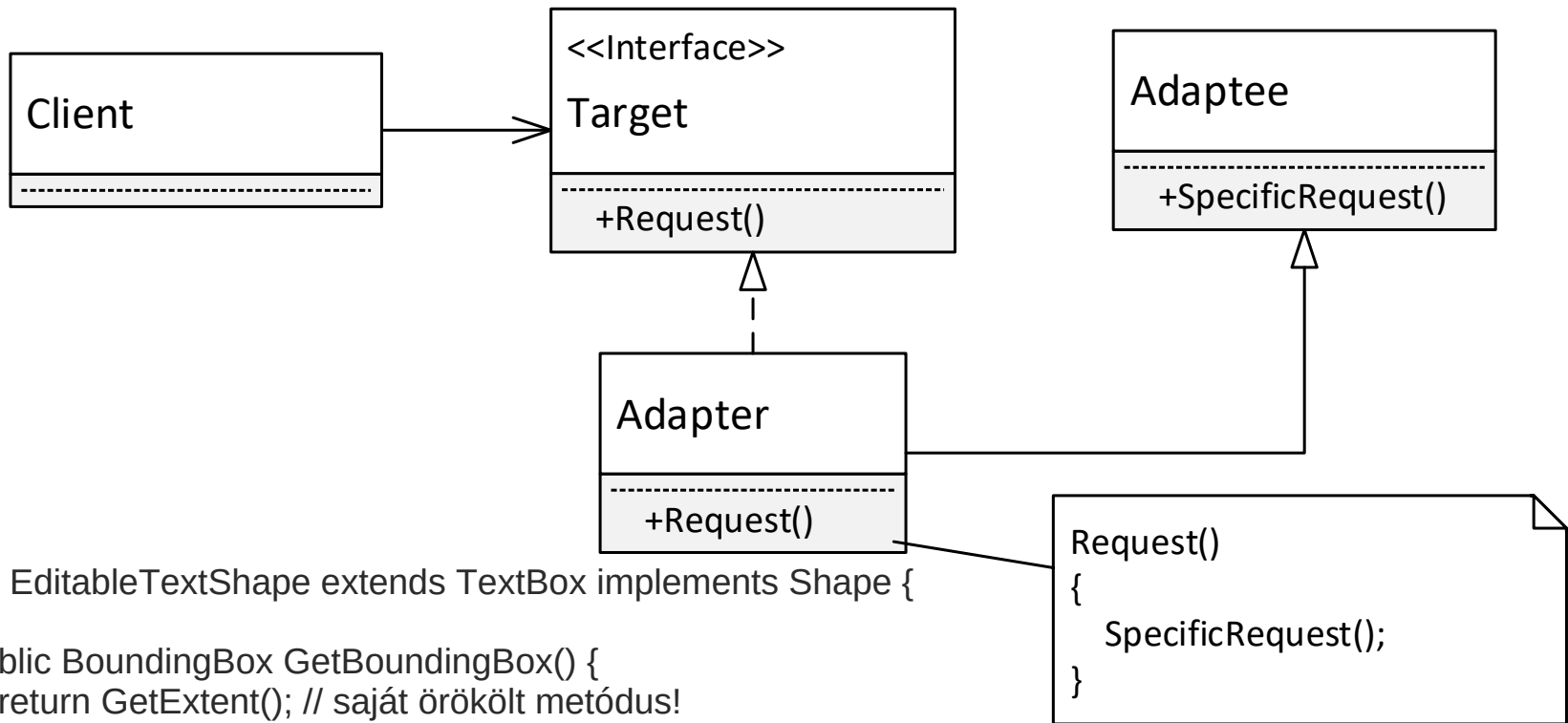


Magyarázat

Object Adapter magyarázat

- **Adaptee**: Az adaptálandó osztály, melynek nem megfelelő az interfésze (emiat nem tudjuk az adott környezetben triviális egyszerűséggel felhasználni).
- **Adapter** (*EditableTextShape*) : Illesztő, az Adaptee interfészt a Target interfésszé „konvertálja”. Tartalmaz egy hivatkozást egy adaptee objektumra (becsomagol egyet). A műveletei, ahol csak tehetik, a becsomagolt adaptee műveleteit hívják.
- **Target** (*Shape*): Interfész, amit a kliens használ. A gyakorlatban lehet interfész vagy absztrakt osztály is.

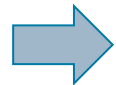
Class adapter struktúra



class EditableTextShape extends TextBox implements Shape {

```
public BoundingBox GetBoundingBox() {
    return GetExtent(); // saját örökölt metódus!
}
```

Alapja: tartalmazás (becsomagolás)
helyett leszármaztatás.



Magyarázat

Class Adapter magyarázat

- A szerepek alapvetően megegyeznek az Object Adapterével, ami különbség:
 - > Az Adapter nem becsomagolással használja fel az Adaptee kódját, hanem leszármaztatással. Az Adapter műveletei, ahol csak tehetik, az ősz Adaptee műveleteit hívják.
 - > Mivel a legtöbb nyelv (pl. Java, C#) nem támogatja a többszörös öröklést, a Target itt csak interfész lehet, őssztály nem!
 - > Kevésbé rugalmas, mint az Object Adapter, általában az Object Adapter változatot preferáljuk!

Adapter összefoglaló

- Használjuk: ha nem tudunk valamilyen osztályt újrafelhasználni egy környezetben, mert nem jó az interfésze
- Megjegyzések
 - > (Az Object Adapter esetében nem minden esetben szükséges tagváltozóként tárolni az Adaptee-t, ritkább esetben az Adapter művelete maga példányosítja majd el is dobja az Adaptee objektumot. Ekkor nem asszociáció, hanem csak függőség van az Adapter és az Adaptee között.)

Composite (Összetett)

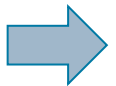
Composite

- Célja

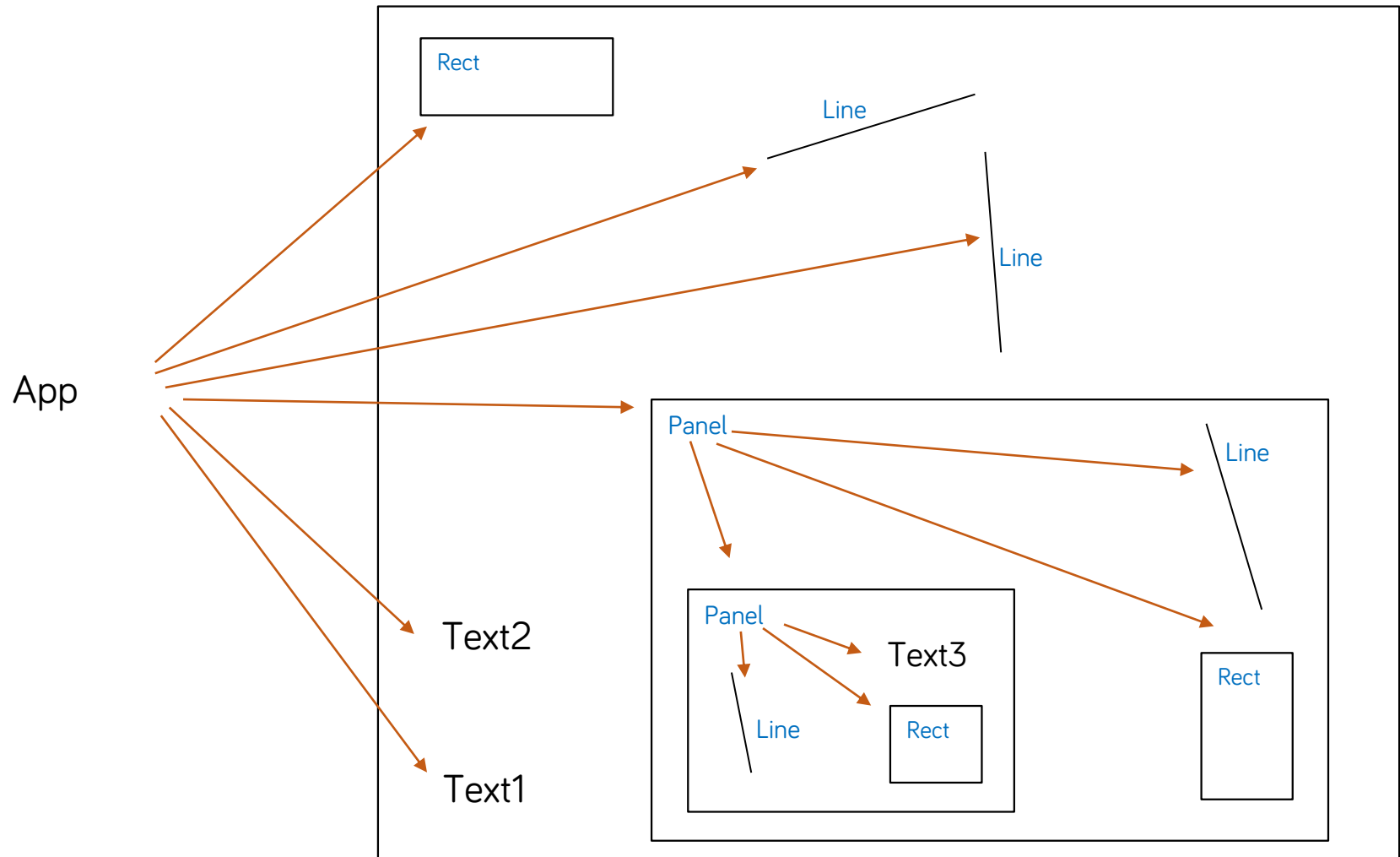
- > Rész-egész viszonyban álló objektumokat fastruktúrába rendezi
- > A kliensek számára lehetővé teszi, hogy az egyszerű és összetett (kompozit) objektumokat egységesen kezelje

- Példa

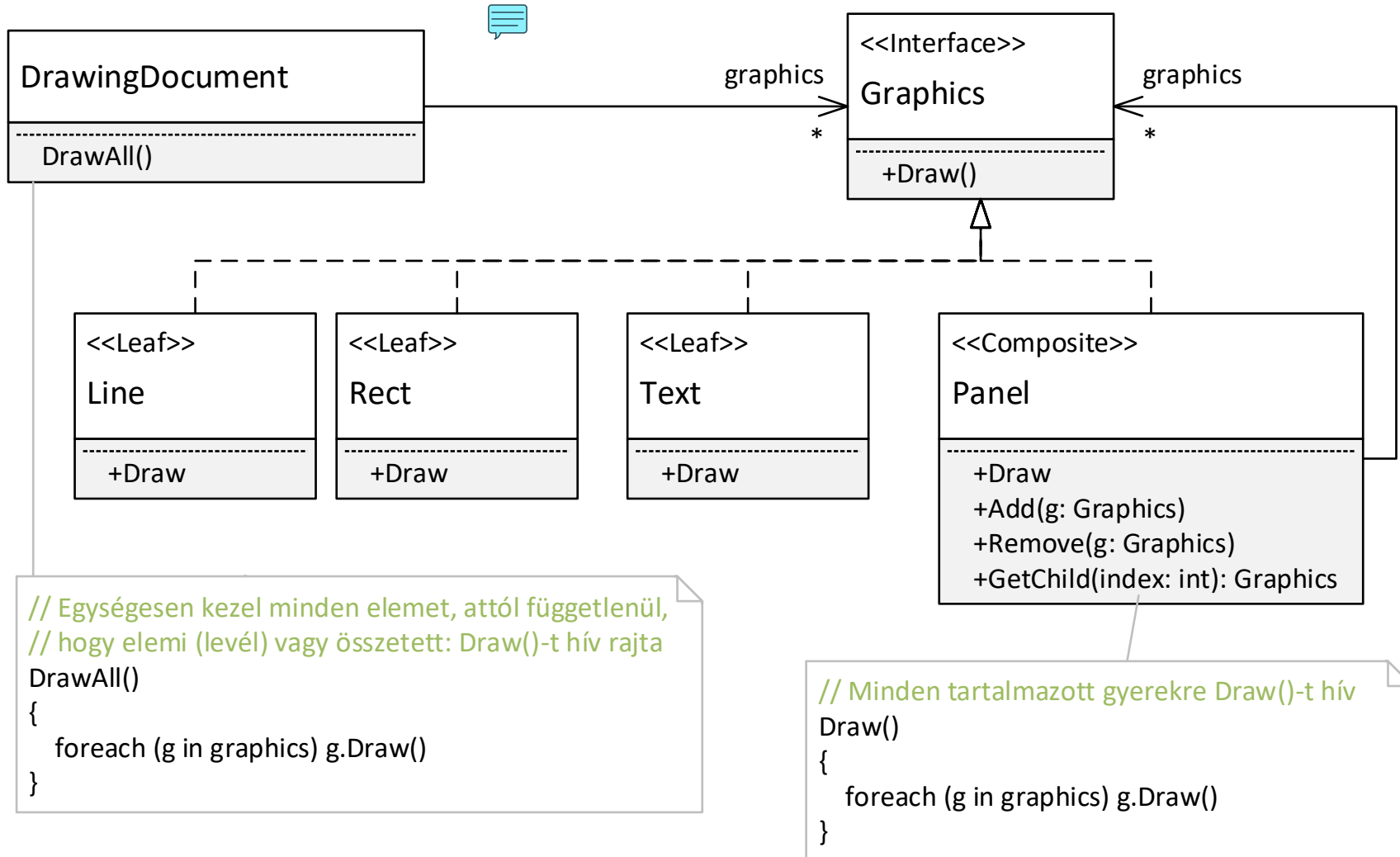
- > Olyan grafikus alkalmazás, amely lehetővé teszi elemi és összetett grafikus objektumokat tartalmazó rajzok létrehozását
- > A Panel egy olyan összetett grafikus elem, mely grafikus alakzatokat tartalmaz (tetszőlegesenek, akár más Panel objektumok is lehetnek rajta!)



Composite példa



Composite példa



Magyarázat



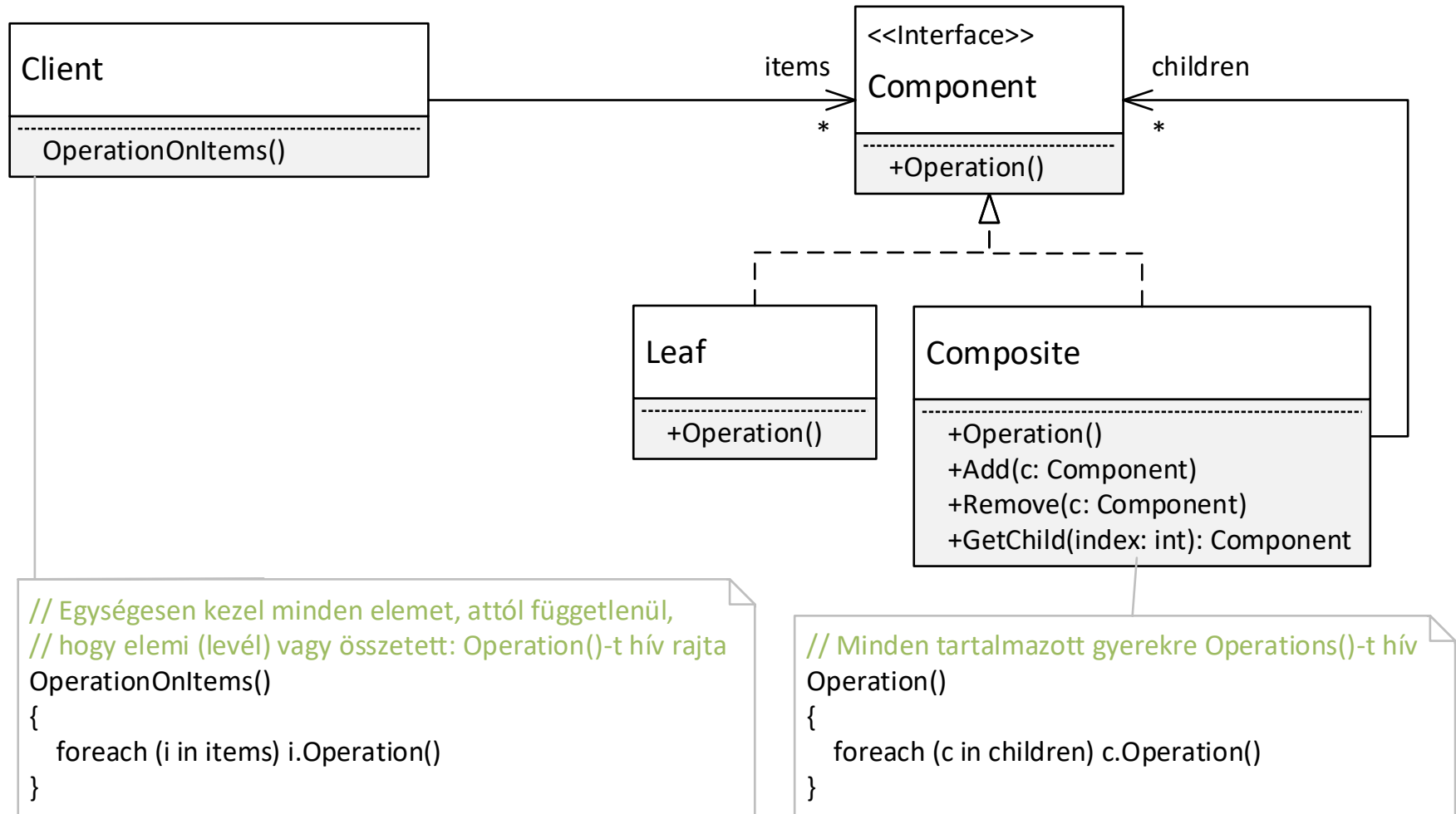
Composite példa magyarázat

- **DrawingDocument:** egy olyan dokumentumot reprezentál, mely különböző grafikus alakzatokat tárol és kezel
- Ezen grafikus alakzatok lehetnek
 - > **Elemiek** (levél, leaf), mint pl. Line, Rect, Text.
 - > **Összetettek**, mint pl. a Panel (mely maga is tartalmazhat elemi és összetett alakzatokat).
- **Graphics:** a grafikus alakzatok közös interfésze/őse, attól függetlenül, hogy elemi vagy összetett az alakzat

Composite példa magyarázat

- A Composite minta egyik alapelve, hogy **egységesen kezeli az elemi (Line, Rect, Text) és az összetett (Panel) objektumokat**. Ez a példában két pontban is megjelenik:
 - > A DrawingDocument **közös gyűjteményben tárolja az elemi és összetett objektumokat** (ez a graphics tag). Megteheti mert az összetett Panel is implementálja a Graphic interfészt.
 - > A DrawAll műveletben a kirajzoló kód **nem különböztetni meg – vagyis egységesen kezeli - az elemi és összetett objektumokat** (nincsenek típus szerint leválogatva): mindre egységesen a Draw() műveletet hívja.
- A összetett Panel Draw művelete az általa tartalmazott alakzatokat rajzolja ki, pont ez volt a célunk!
- Kód: lásd DesPattCode/Composite mappa

Composite általánosságában



Composite

- Használjuk, ha
 - > Objektumok rész-egész viszonyát szeretnénk kezelni (jellemzően fastruktúrában)
 - > A kliensek számára el akarjuk rejtetni, hogy egy objektum elemi objektum vagy kompozit objektum: bizonyos műveletek szempontjából egységesen szeretnénk kezelni őket

Composite megjegyzések

- A gyerekek kezelése nem tud egységes lenni!
 - > Hiszen az csak az összetett (példánkban a Panel) osztály objektumaira értelmezett (Add, Remove, GetChild)
 - > A kliens pl. típusellenőrzéssel tudja megnézni, hogy az adott elem összetett-e (pl. C#-ban az „is”, Java-ban az „instanceof” operátorral), pl.:
 - if (item is Composite) ...
 - > Az Add, Remove, GetChild műveletek hívásához a kliensnek a kompozit osztályra/interfészre kell castolnia a hivatkozását

Tervezési minták összefoglaló

További tervezési minták

- A klasszikus „GoF” minták közül is kimaradt pár:
 - > Prototype, Builder, Bridge, Mediator, Chain of Responsibility, Visitor, Decorator, Iterator, State, stb.
- Számos nem általános tervezési minta létezik, pl.
 - > Elosztott, konkurens rendszerekre jellemző minták
 - Szolgáltatás hozzáférés
 - Konfiguráció
 - Esemény kezelés
 - Szinkronizáció
 - Konkurencia
 - > Valós idejű rendszerekre jellemző minták
 - > Vállalati rendszerekre jellemző minták

Összefoglalás

- Tapasztalati tudást hordoznak
 - > Mi is rá tudunk jönni, de
 - Jó sokáig tart
 - Nem jövünk rá
 - Miért ne tanuljunk mások tapasztalataiból
 - > Értékes tudás!
- Célunk
 - > Ismerjünk meg minél több mintát
 - > Hosszútávon legalább arra emlékezzünk, hogy egy adott problémakörben mely mintákat lehet jól használni

Szoftvertechnológia és -technikák

9. Előadás

Architekturális tervezés



Automatizálási és
Alkalmazott
Informatikai Tanszék

Ez az oktatási segédanyag a Budapesti Műszaki és Gazdaságtudományi Egyetem oktatója által kidolgozott szerzői mű. Kifejezett felhasználási engedély nélküli felhasználása szerzői jogi jogsértésnek minősül.

Tartalom

Tervezési szemlélet

Alapvető architekturális
minták

- > Rétegek
- > MVVM
- > MVC
- > Document View

Mi az architektúra?

Architektúra: Megmondja mit és hogyan építs fel.
Keretrendszer: Architektúra konkrét megvalósítása.

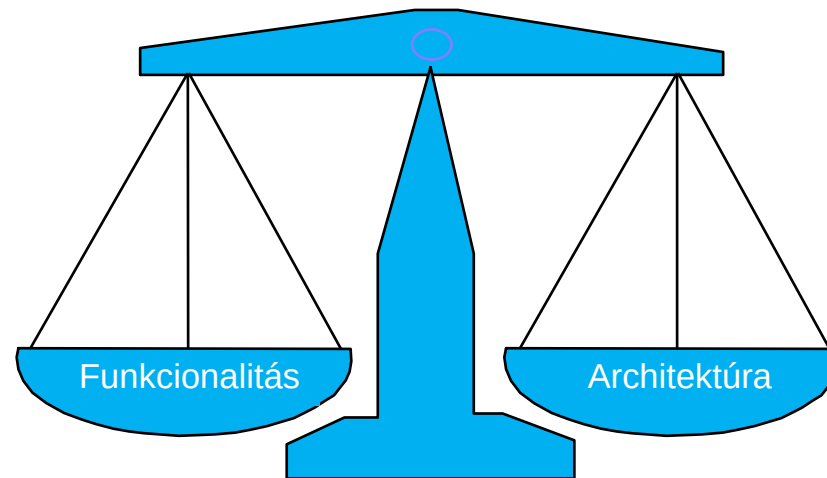
- A szoftverrendszer átfogó/magasszintű szervezése, strukturálása
 - > A rendszert alapvető strukturális elemekre dekomponáljuk
 - > Meghatározzuk ezen elemek kapcsolatát és együttműködését
- „From mud to structure.”
 - > Az architekturális elem meghatározó a rendszer felépítése, teljesítménye, stb. szempontjából.
 - Amiből már nem lehet elvenni ahhoz, hogy megértsük és elmagyarázzuk a rendszer (alapvető) működését.
 - Nem foglalkozik az alacsonyszintű, belső kérdésekkel (modul belső felépítése, algoritmus, implementáció, stb.)

Architektúra tervezés

- Mikor?
 - > A projekt kezdeti fázisában Layered? Microservices?
 - > A magasszintű követelmények meghatározásával párhuzamosan
- Nagy tapasztalati tudást igényel
 - > Ha rosszul találjuk ki, később nagyon-(nagyon) nehéz és költséges architektúrát változtatni.

Az architektúra és a funkcionalitás

- Az architektúrának és a funkcionalitásnak egyensúlyban kell lenni!
 - > Funkció: működjön jól! $\longleftrightarrow$ Architektúra: legyen jól strukturált!

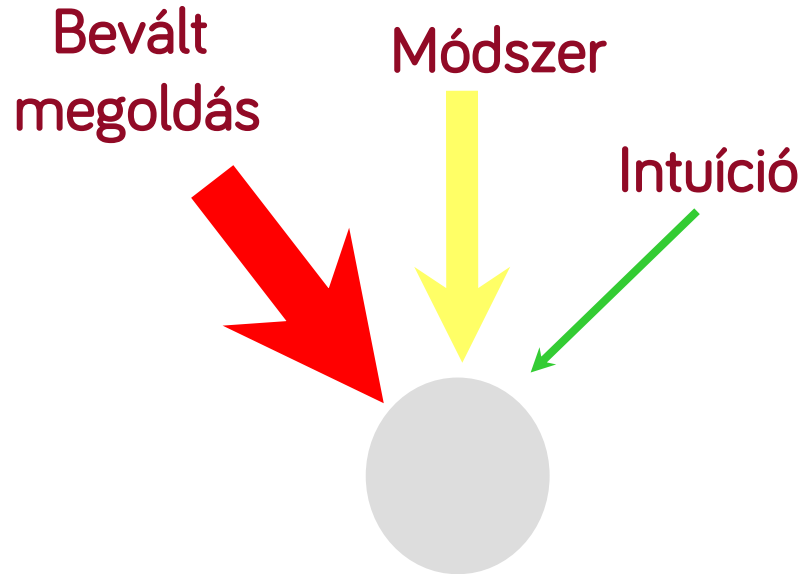


- > Az architektúra „léte” és jó megválasztása a rendszerek sikerének egyik alapkulcsa

Architektúra tervezés jelentősége

- Ha ad-hoc épül a rendszer (nincs architektúra):
 - > A rendszer nagyon nehezen lesz:
 - Megérthető
 - Bővíthető
 - Karbantartható
 - Dokumentálható
 - > Nem lesznek újrafelhasználható modulok/minták

Az architektúra forrásai



- Az esetek többségében választunk egy a projekt követelményekhez illeszkedő már bevált architekturális mintát
 - > Vagyis nem „intuitíven” vagy „módszeresen” határozzuk meg az architektúrát.

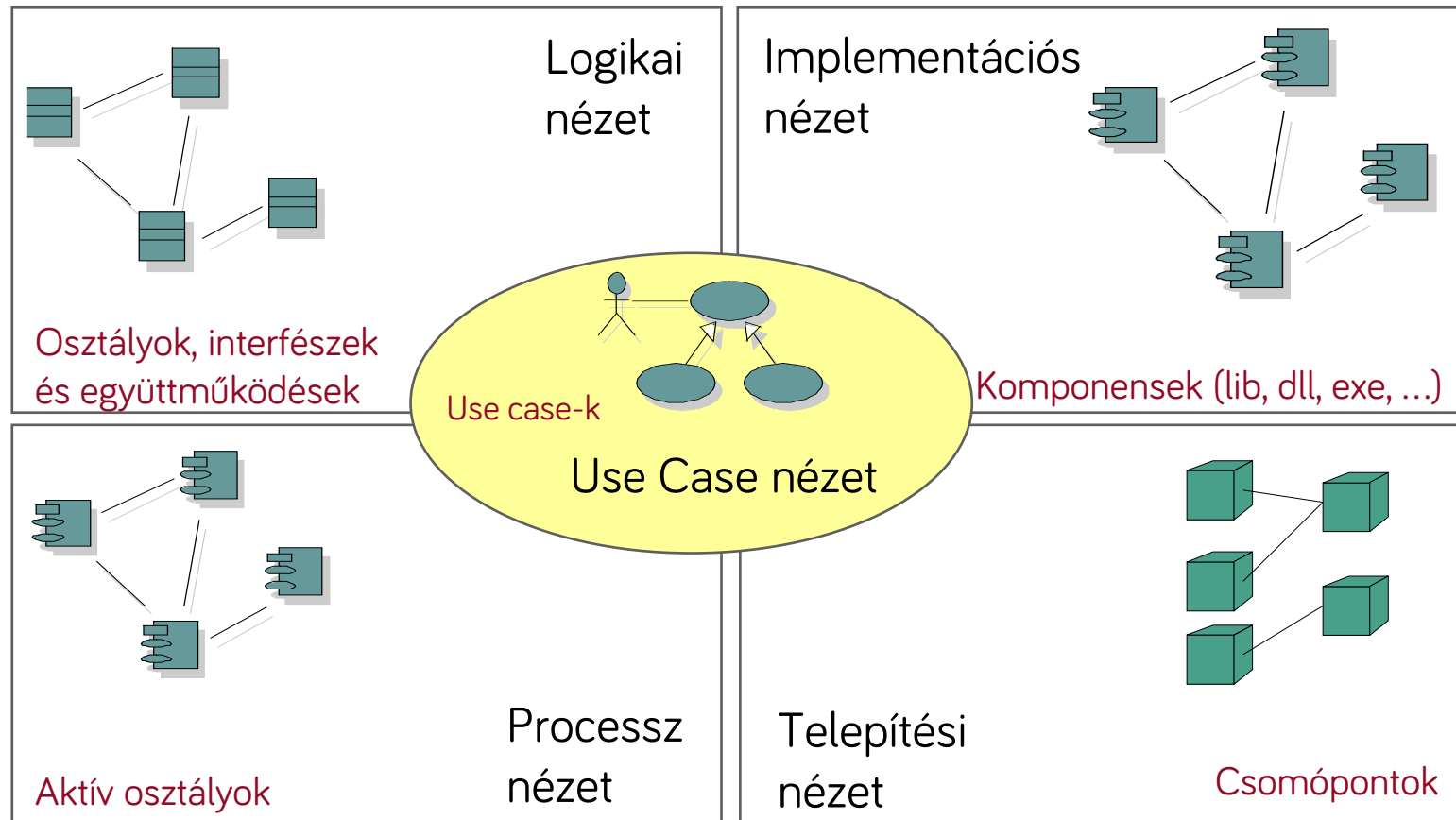
Pár architektúra minta példa

- Felhasználói felülethez kapcsolódók
 - > Model-View-Controller (MVC)
 - > Document-View
 - > Model-View-ViewModel (MVVM)
- Általános
 - > Pipes and filters
 - > Layers
- Elosztott alkalmazások
 - > Client-server
 - > Three-tier architecture
 - > Service-oriented architecture
 - > Microservices architecture
- ...

Jelölésrendszer

- Sokszor csak „dobozkák” és a köztük feltüntetett kapcsolatokkal
- Különböző UML diagramok

Az architektúra 4+1 nézete (haladó)



Magyarázat: következő dia...

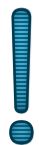
Az architektúra 4+1 nézete (haladó)

- **Használati eset**
 - > Funkcionális követelmények.
- **Tervezési vagy logikai**
 - > Főbb csomagok (névterek), alrendszerek, osztályok.
- **Implementációs nézet**
 - > Komponensek, dll-ek, exe, JAR csomagok, forráskódok szervezése.
- **Processz nézet**
 - > Konkurens aspektus, folyamatok, szálak, holtpont, startup, shutdown, teljesítmény, skálázhatóság.
- **Telepítési nézet**
 - > A futtatható és egyéb komponensek mely számítási csomópontokra (pl. számítógépekre, virtuális gépekre) kerülnek.

Alapelvek

UI és logika különválasztása

Separation of Concerns tervezési alapelv



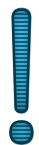
Alapelv – UI és logika különválasztása

- A felhasználó felület kódját (UI) és az alkalmazás/üzleti logika kódját válasszuk külön!
 - > Ne kódoljunk mindent bele az eseménykezelőkbe
 - > alkalmazás/üzleti logika alatt legtöbbször az alkalmazás adatainak kezelését értjük
- A lényeg a függőség iránya: a logika legyen független a felhasználói felülettől:



- <https://github.com/bzolka/AUT-SZTT> repóban a ArchCode/SeparateUIAndLogic mappa

Demo

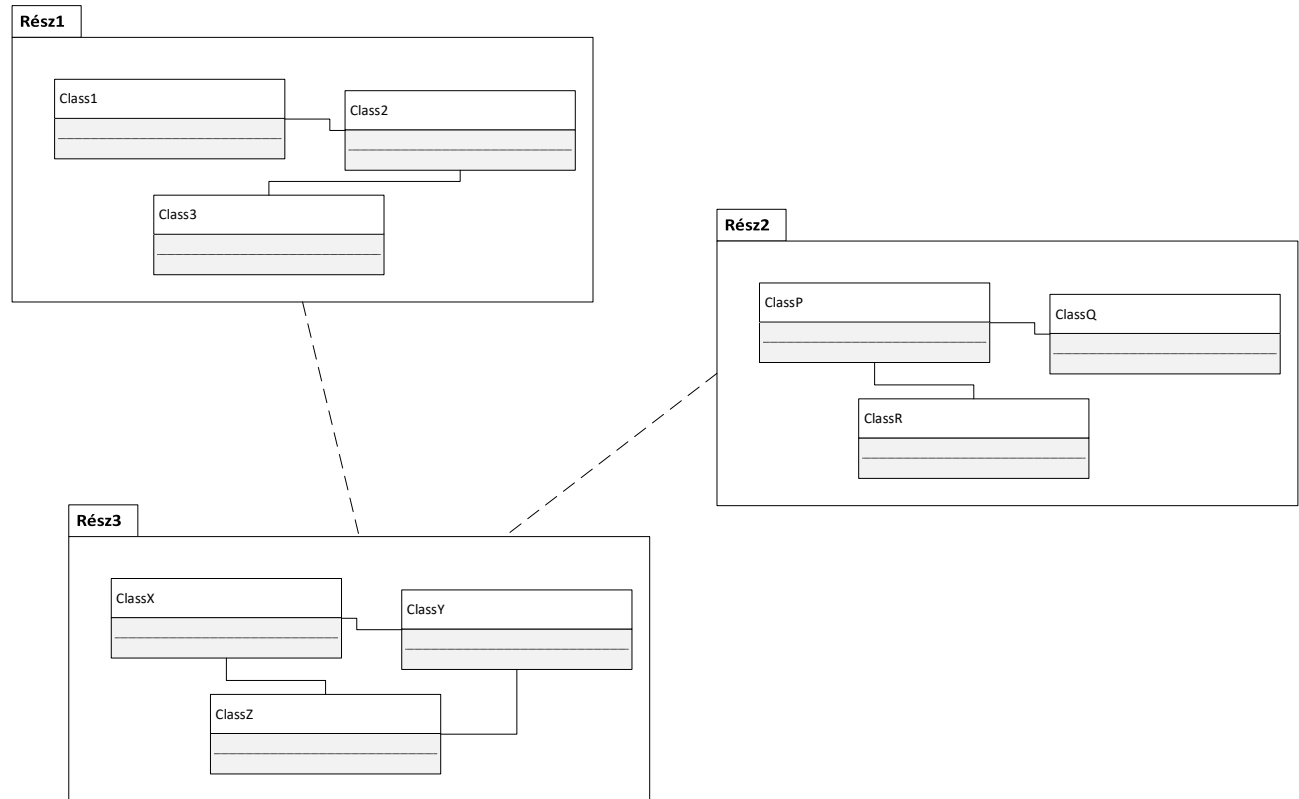


Alapelv - Separation of Concerns, SoC

- Vonatkozások/felelősségi körök különválasztása
- Elve: a kódrészeket (osztályokat) aszerint különítsük el, hogy milyen vonatkozású/felelősségi körbe tartozó feladatot végeznek
 - > Az ugyanazzal foglalkozó kód egy körbe
 - > A körök minél kevésbé lapolódjanak át és függjenek egymástól
- A UI és logika (mint felelősségi körök) különválasztása egy példa erre
- Példák felelősségi körökre
 - > Technikai felelősségi körök: megjelenítés, felhasználói bemenet kezelése, üzleti logika, adatok kezelése, adatok tárolása (perzisztencia), külső rendszerekkel való kommunikáció, naplózás, stb.
 - > Funkcionális: pl. egy webshop esetében vevőadatok kezelése, megrendelés kezelése, fizetés, számlázás, stb.

Separation of Concerns, SoC

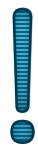
- A SoC **laza csatoláshoz** (loose coupling) és **nagy kohézióhoz** (high cohesion) vezet.



- Egy felelősségi körön belül „**high cohesion**” – a kód/osztályok ugyanarra fókuszálnak
- A felelősségi körök/részek között „**loose coupling**” – a részeknek keveset kell tudni egymásról, kicsi a függőség

SoC tipikus példa

- SoC tipikus példa:
 - > UI és üzleti/alkalmazás logika felelősségi kör különválasztása
 - > A különválasztás után nagyobb a kohézió. Miért is?
 - Az egyes részek egy dologra fókuszálnak
 - UI kód NEM tartalmaz (üzleti/alkalmazás) logikát
 - A logika kód NEM tartalmaz felhasználó felülethez tartozó „logikát”
- Vagyis nem keverednek a különböző felelősségű részek.



SoC előnyök (UI és logika különválasztás példával illusztrálva)

- #1. Az egyes részek önmagukban **könnyebben megérthetők**, átláthatók (mivel egy dologra fókuszálnak)
- #2. Az egyes részek **külön/párhuzamosan fejleszthetők** (nagy projektben külön UI-hoz értő és „logikához” értő szakértő fejlesztők)
- #3. **Könnyebb bővíteni**, karbantartani (pl. a UI és a logika egymástól „függetlenül” bővíthető, lecserélhető)
 - Pl. pár év után lecserélődik az UI technológia, a logika nagy része változatlan maradhat (nem kell „hozzányúlni”, tesztelni)
- #4. **Újrafelhasználhatóság** (a UI és a logika egymástól „függetlenül” újrafelhasználható)
- #5. Az egyes részek önmagukban **unit tesztelhetők** (pl. az üzleti/alkalmazáslogika a UI-tól függetlenül)

Különválasztás módja

- Esetfüggő, hogyan célszerű
- Lehetőségek (akár kombináltan)
 - > Java
 - Package-ek
 - JAR csomagok*
 - > .NET
 - Névterek
 - Mappák
 - Szerelvények (dll, exe)*

* - nem kell tudni

SoC összefoglaló

- Ne feledjük: a SoC elv teljesen általános, a UI és logika szétválasztása csak egy példa!
 - > A SoC az egyik **legfontosabb általános** tervezői elv
- Osztályok szintjén is, de architektúra szinten kiemelten fontos
- A „részeket” szokás moduloknak, komponenseknek is nevezni
- SoC és SRP hasonló: kéz a kézben együtt ...

Rétegek (layers)

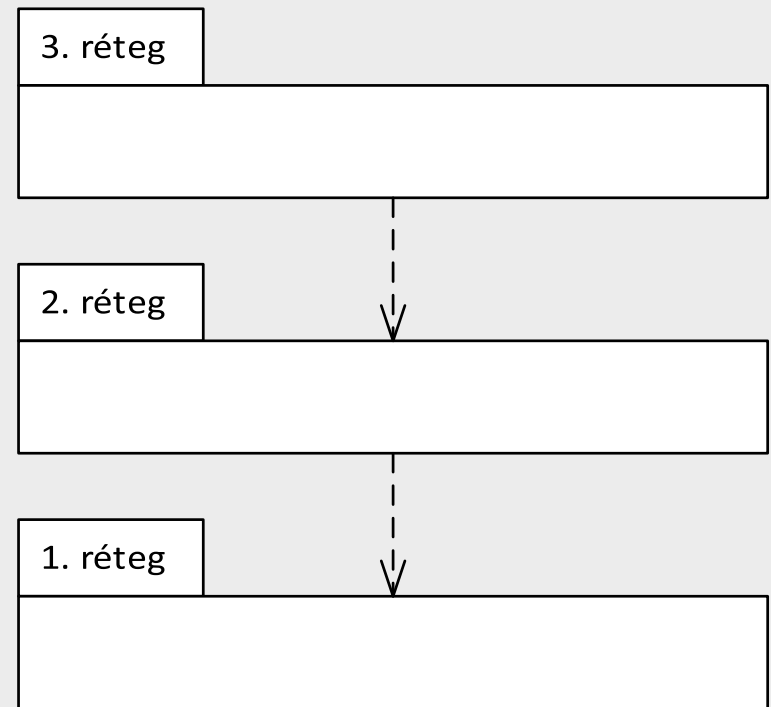
Rétegelés

- Legalapvetőbb szervezési elv
 - > A kódot rétegekbe szervezzük
- Hol fordul elő
 - > Hálózati protokollok
 - > Példa: eszközmeghajtók -> operációs rendszer -> .NET/Java -> alkalmazás
 - > Vállalati információs rendszerek (adatbázisban tárol adatokat, pl. a Neptun rendszer)
 - > Stb.
- Ha a rendszer alacsony és magasabb szintű funkciók keveréke

Alapelvek

- Akárhány réteg lehet
- Egy adott réteg
 - > szolgáltatásokat nyújt az a felette levő réteg számára
 - > A saját szolgáltatásait az alatta levő réteg szolgáltatásaira építve valósítja meg
- Fontos a függőségek iránya: egy réteg függ az alatta levőtől, fordítva nincs függőség
- Jelölése: egyszerű „dobozkák”, vagy UML-ben csomagokkal

Példa három rétegre:



Előnyök – a SoC rétegekre vetítve

- Ha túl komplex a feladat: vezessünk be egy új absztrakciós szintet (új rétegbe) és oldjuk meg erre építve
- Egy réteg működése önmagában is megérthető, nem kell a többit is megértsük
- Külön/párhuzamos fejlesztés lehetősége
 - > Az interfészek kialakítása után a rétegek egymástól függetlenül párhuzamosan fejleszthetők.
 - > Az egyes rétegek eltérő technológiai ismeretei igényelnek
 - (pl. UI fejlesztő, adatszakértő, stb.), nem kell mindenkinek mindenhez értenie a fejlesztőcsapatban.
- Könnyeb bővíthetőség, valamint az egyes rétegek újrafelhasználhatók. Pl. ugyanahhoz az üzleti logikai réteghez készíthetünk desktop és webes felületet (frontendet) is.
- Az egyes rétegek automata „unit” tesztelhetősége általában könnyebben megoldható.

Hátrányok

- Egyszerűbb feladatnál felesleges komplexitás
- A változások sok esetben kaszkádoltan végigvonulnak az összes rétegen, több helyen kell módosítani
 - > Pl. felveszünk egy új adatbázis mezőt, akkor a UI és az adatbázis réteg között minden réteget módosítani kell
- Teljesítmény

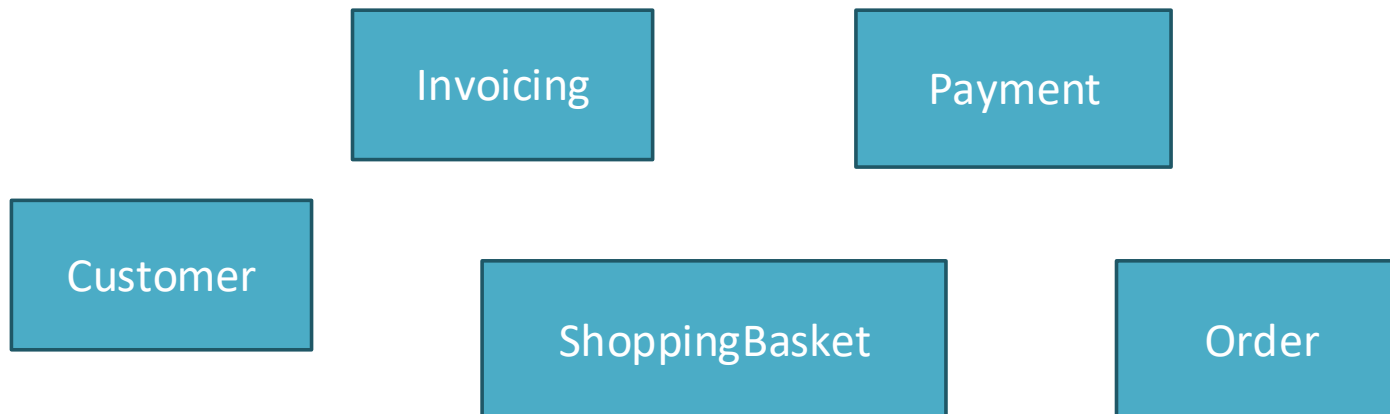
Többrétegű architektúra példák

- Kétrétegű
- Háromrétegű
- Kliens-szerver

Kétrétegű architektúra

Eggrétegű - WebShop példa,

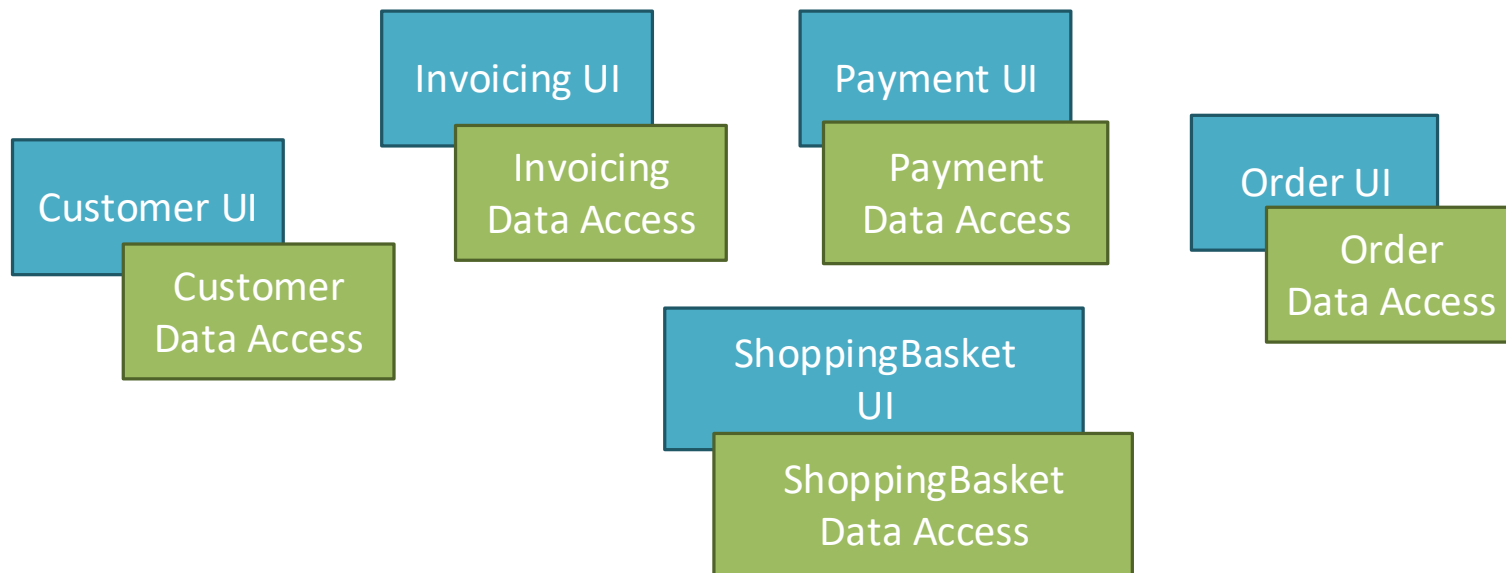
- Modulok
 - > SoC, egyelőre csak funkcionális szétválasztás
 - > Mindegyik valamilyen adatot tárol/kezel/jelenít meg



További
szétválasztás →

WebShop példa továbbfejlesztve

- SoC funkcionálisan
- Felhasználói felület (UI) és adathozzáférés (data access) vonatkozásában is különválasztva



Adathozzáférés: adatok tárolása,
mentése, betöltése, vagyis perzisztencia

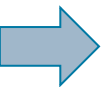
Adatthozzáférés

- Szerepe: adatok lekérdezése, mentése
- Data Access Layer, röviden DAL (sokan csak DAL-ként emlegetik).
- Perzisztencia réteggként is szokás rá hivatkozni
- .NET világában inkább „DAL”, Java világában inkább „Persistence” –ként hivatkoznak rá

Felhasználói felület szerepe

- Lehetővé teszi a felhasználó számára, hogy interakcióba lépjen az alkalmazással (pl. ablakok, vezérlők, weboldalak, konzol alkalmazásnál parancssor segítségével)
- Adatok megjelenítése a felhasználó számára
- A felhasználói kérések fogadása (lekérdezések és módosító parancsok).
- A felhasználó által megadott adatok (első körös) validálása

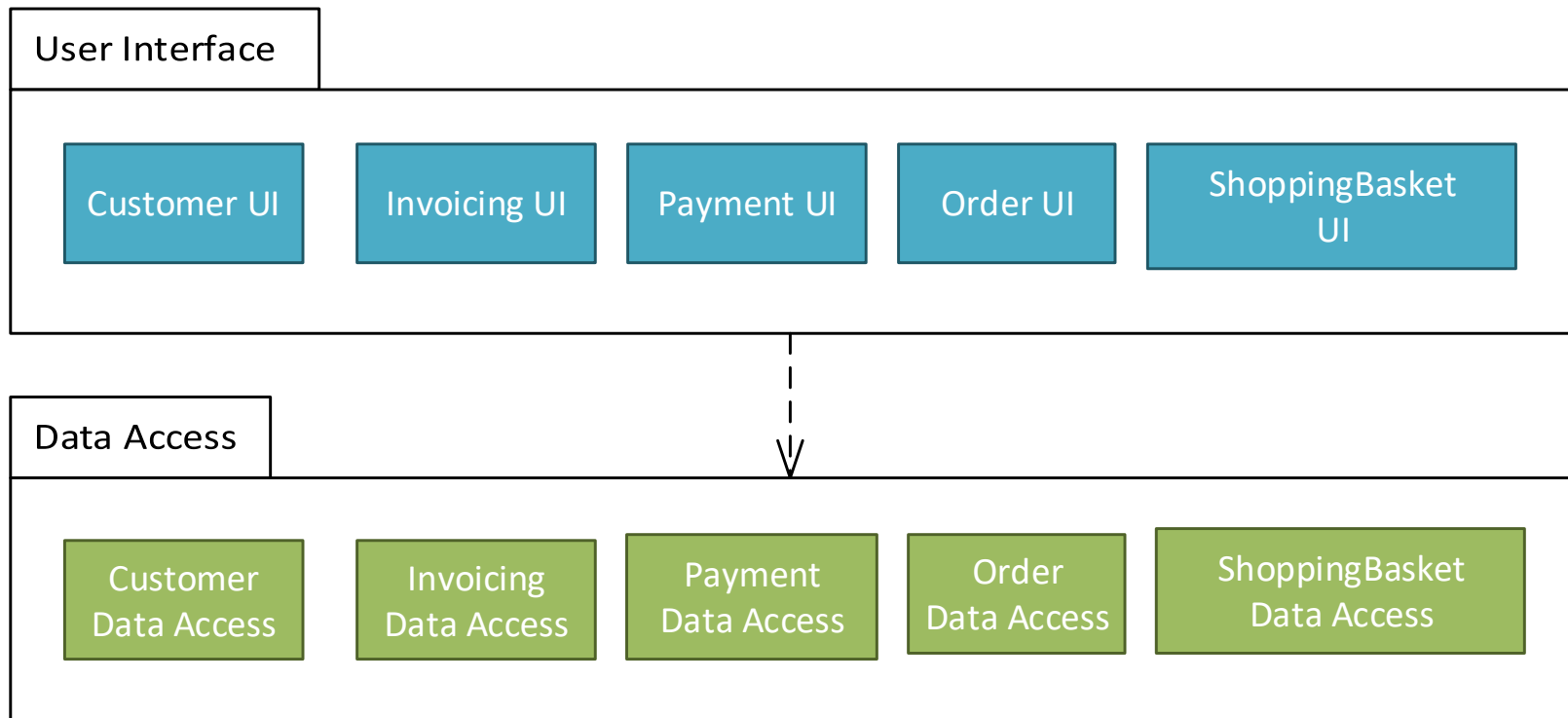
Szervezzük rétegekbe a
korábbi alkalmazást



Kétrétegű WebShop

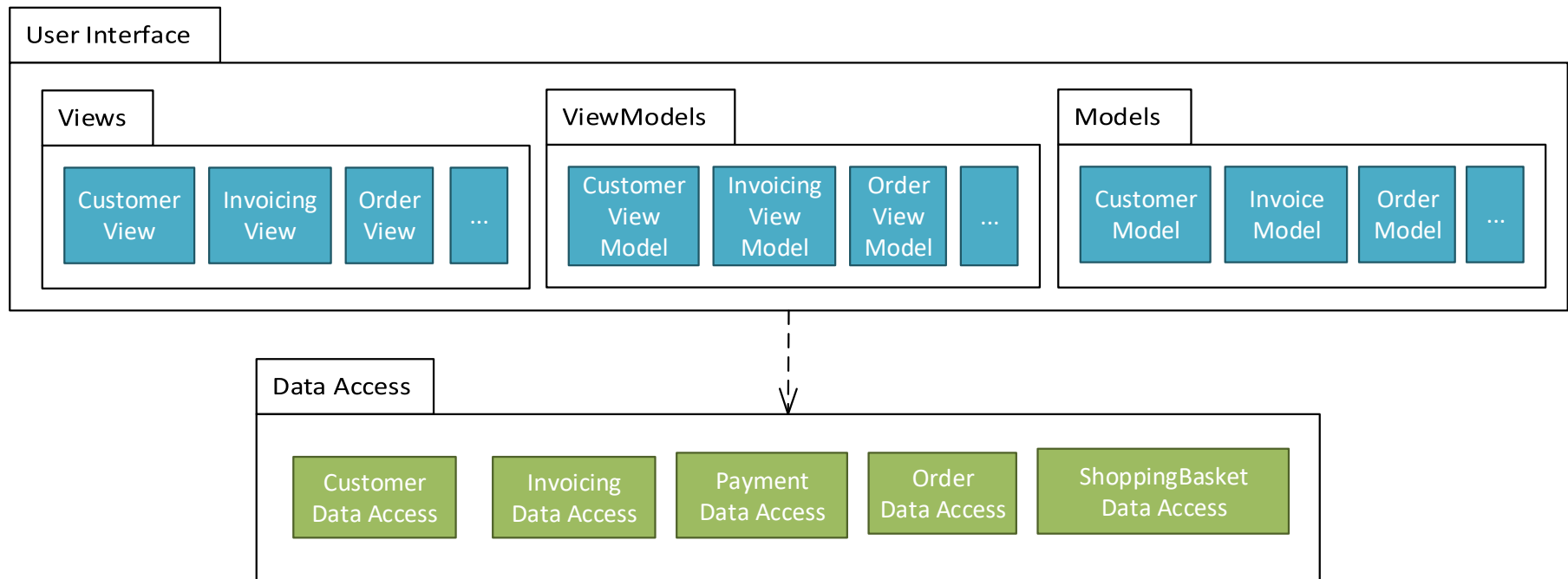
- Csoportosítsuk a **UI** és az **adathozzáférési** részeket két rétegbe

UI dolgozza fel a logikát!!! NINCS középső üzleti logikai réteg!!
DAL: csak adatbázissal történő kommunikációért felel!



A rétegek belül tovább partícionálhatók

- Például UI rétegben MVVM minta (lásd később)



Kétrétegű előnyök

- Szokásos SoC előnyök, pár kiemelve
 - > Ha le akarjuk cserélni a perzisztencia módját (pl. sima fájl, XML, SQL adatbázis, stb.), akkor csak a perzisztencia réteget kell lecserélni
 - > Ugyanazokhoz az adatokhoz több különböző felhasználói felület készíthető

Milyen esetben használjuk a kétrétegű architektúrát?

- Olyan alkalmazásoknál, melyek adatokat kezelnek (jellemzően adatbázisban)
- Mikor elég a két réteg
 - > Ha tipikusan egyszerű „minimál” adatszervezők vannak (Create/Read/Update/Delete – röviden CRUD)
 - > Nincs komolyabb üzleti/feldolgozási/validációs logika az alkalmazásban !!!!!
 - > Gyorsan össze kell dobni egy egyszerű/prototípus alkalmazást
 - > Egyszerűbb alkalmazások esetében használjuk bátran

Milyen esetben használjuk?

- Mikor nem elég a két réteg
 - > Ha az alkalmazásban **jelentős az üzleti logika**, akkor azt a SoC elvek alapján sem a UI, sem az adathozzáférési rétegbe nem praktikus tenni az üzleti logikát
 - Más a felelősségi kör
 - Pl. üzleti logika nem lesz megérthető, tesztelhető, bővíthető, újrafelhasználható a UI vagy DAL nélkül (attól függően, melyikbe tesszük), ha a kódban össze van nőve ezekkel.
 - A kétrétegű esetben gyakorlatban sokszor a UI-ba kerül, így viszont nem lehet ugyanahhoz a logikához különböző frontendeket (pl. web, desktop, mobil) írni.

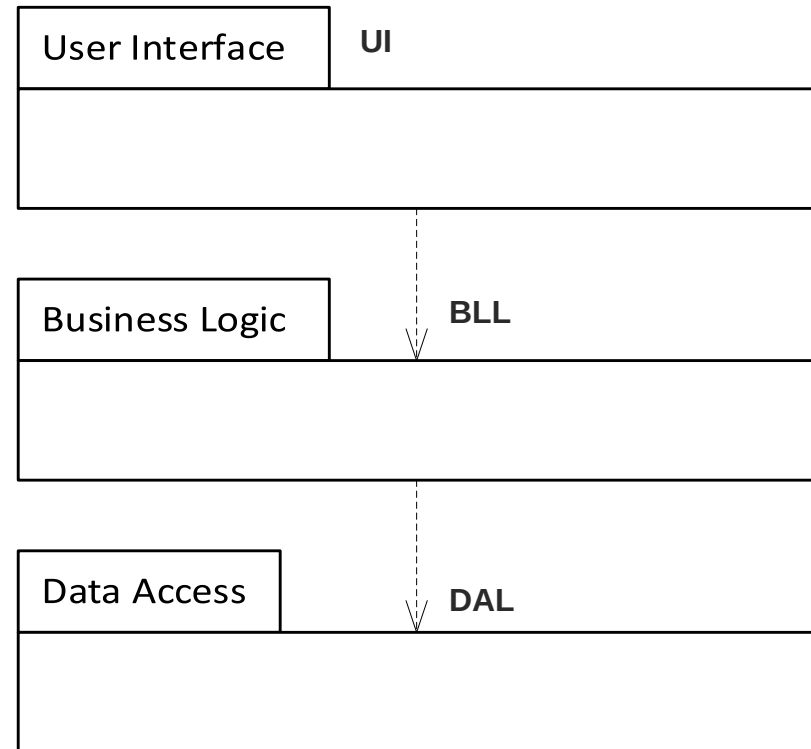
Háromrétegű architektúra

Háromrétegű architektúra alapok

- Kibővíti a kétrétegű architektúrát egy üzleti logikai réteggel
- Business Logic Layer, rövidítve BLL

BLL feladatai:

- validáció
- feldolgozás, üzleti szabályok (if ...)
- tranzakciókezelés (@Transactional)
- Logolás / audit

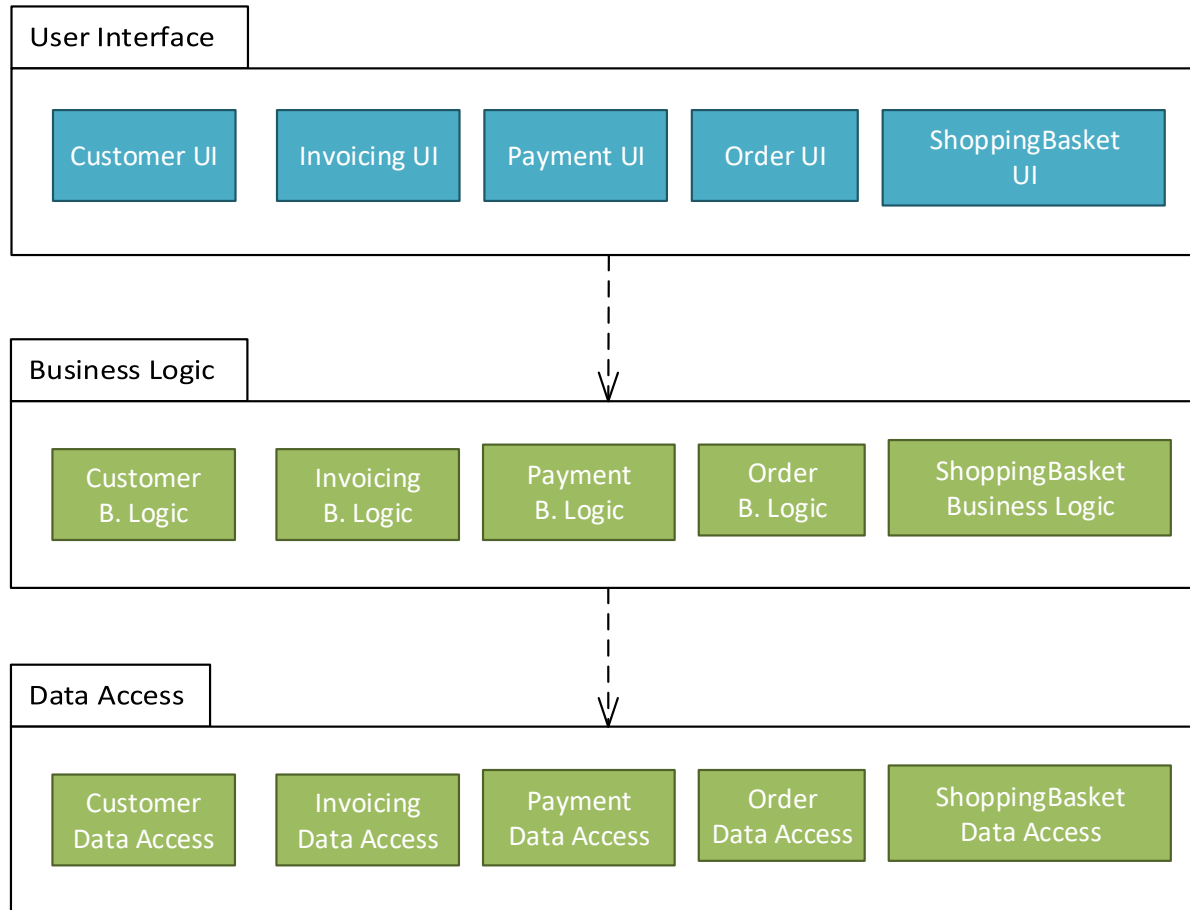


Üzleti logikai réteg

- Üzleti logikai szabályokat, validációs szabályokat, stb.-t tartalmaz valamint folyamatokat ír le
- Pl.:
 - > **Egyszerű validációs szabály:** árucikk neve nem maradhat üresen; a születési dátum nem lehet a jövőben
 - > **Logikai szabály:** pl. számla esetén kell legalább egy tétel; a végösszeg meg kell egyezzen a tételek összegével; bruttó ár számítása nettó árból; kedvezmény az összeg/vevő függvényében; két hallgató Neptun kódja nem lehet azonos
 - > **Folyamatok:** egy online megrendelést előbb-utóbb ki kell szállítani és ki kell számlázni (vagy „cancel”-elni kell, ha nincs készleten áru).
- Független a megjelenítéstől

Háromrétegű WebShop

Egyszerűsítéssel



Az ábra egyszerűsít: a rétegeken belül gyakran nem ilyen egyszerű egymás mellé rendeltség van. Pl. a bruttó-nettó számítási szabályok közősek az Order és az Invoicing BLL modulokra, így ezt érdemes külön kódba kiszervezni, melyet az Order és az Invoicing is felhasznál.

Előnyök és hátrányok

- Előnyök

- > Szokásos SoC előnyök az üzleti logika vonatkozásában:
- > Az üzleti logika önmagában megérthető, fejleszthető, bővíthető, újrafelhasználható, tesztelhető
- > Újrafelhasználhatóság: sokszor egy alkalmazás/rendszer több modulja is felhasználja ugyanazt a logikát:
 - pl. nettó-bruttó számítás szabályait az Order és az Invoicing modulok is

- Hátrányok

- > Megnövekedett komplexitás, több munka (extra réteg)

Kódszervezés

- Esetfüggő, hogyan célszerű
- Lehetőségek (akár kombináltan)
 - > Java
 - Package-ek
 - JAR csomagok*
 - > .NET
 - Névterek
 - Mappák
 - Szerelvények (dll, exe)*

* - nem kell tudni

Felhasználói felület kialakításához kapcsolódó architektúrák

Felhasználói felület kialakításához kapcsolódó architektúrák

- Kétrétegű (felület és logika két külön rétegben)
 - > Már láttuk az előző „fejezetben”
- MVC – Model-View-Controller
- MVVM – Model-View-ViewModel
- Document-View
- ...

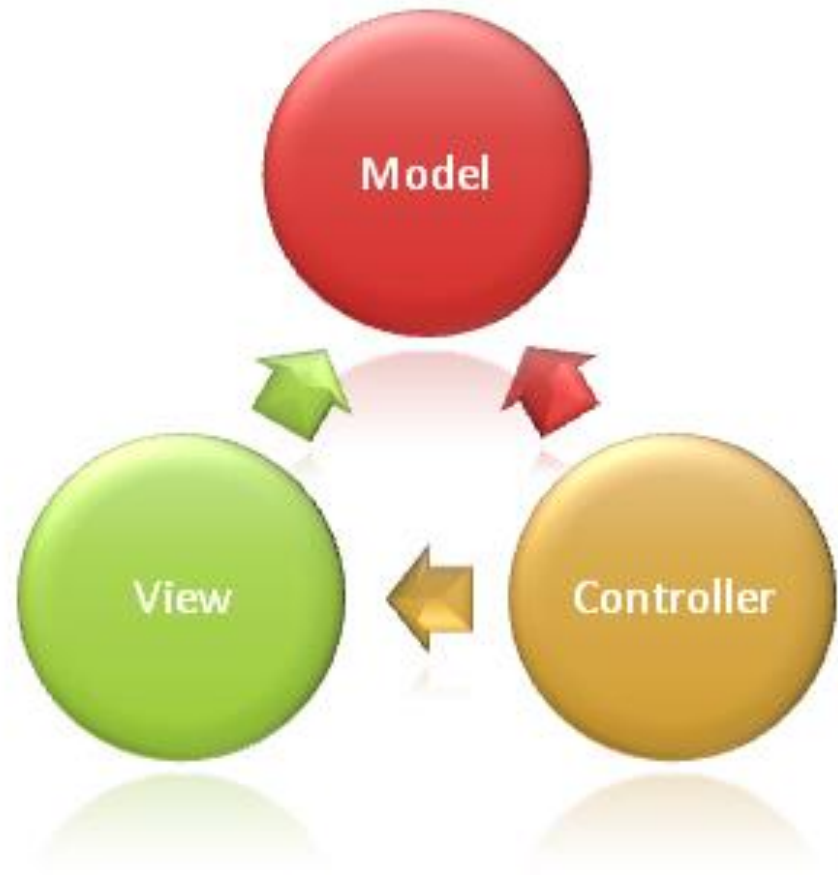
A Model-View-Controller (MVC) architektúra

Szereplők és függőségek

A kódot
(osztályokat) **három**
felelősségi körbe
csoportosítjuk:

- Controller
- Model
- View

Függőségek:



Szereplők és függőségek

- **Controller:** fogadja a felhasználói bemenetet és interakciókat
 - > Pl. egy webalkalmazásnál ez kapja meg a felhasználói interakciók hatására bekövetkező hálózati kéréseket a felhasználói által megadott adatokkal.
 - > Fogadja a felhasználói adatokat és továbbítja a model számára.
 - > A kiválasztja ki a megfelelő View-t (vagy View-kat) a megjelenítéshez
 - > Átadja a Model megfelelő adatait a View-nak a megjelenítéshez

Szereplők és függőségek

- **Model:** az adatok reprezentálásáért, kezelésért felel (+ üzleti logika, validációs szabályok, stb.)
 - > A felhasználói bemenetet a Controllertől kapja meg.
 - > *Független a felhasználói felülettől!*
- **View:** az adatok megjelenítésért felel



Előnyök

- UI – logika (SoC) különválasztás előnyök mindegyike itt is él: lásd korábban! Többek között:
 - > Nem keverednek a különböző felelősségű részek a kódban, könnyebb megérteni, karban tartani, tesztelni, ...
 - > Lásd függőségek ábra: a Model (vagyis a logika) nem függ sem a Controller-től sem a View-tól!
A UI kód jellemzően gyakrabban változik, mint az alkalmazáslogika: ha nem lenne különválasztás, akkor minden UI változáskor a logika osztályait is újra kellene tesztelni.

Hátrányok

- Extra komplexitást jelent
 - > Új absztrakciók megértése és az ehhez való illeszkedés
 - > Kódban navigáció körülményesebb („ugrálás” az adott funkció model/view/controller részei között)
- Csak indokolt esetben használjuk
 - > Ha jól illeszkedik az adott feladathoz !!!
 - > Ha az általunk választott keretrendszer is támogatja.

Jellemzők

- A konkrét megvalósítása nagyon függ attól, hogy desktop/mobil/web környezetben alkalmazzák
- Ma a (tradicionális) webalkalmazások esetében gyakori:
 - > Pl:
 - Java → Spring MVC
 - .NET → ASP.NET Core MVC
 - > Desktop alkalmazásoknál már kevésbé elterjedt.
- *A gyakorlatban akkor használjuk, ha a keretrendszer, amit használunk, beépítve támogatja*

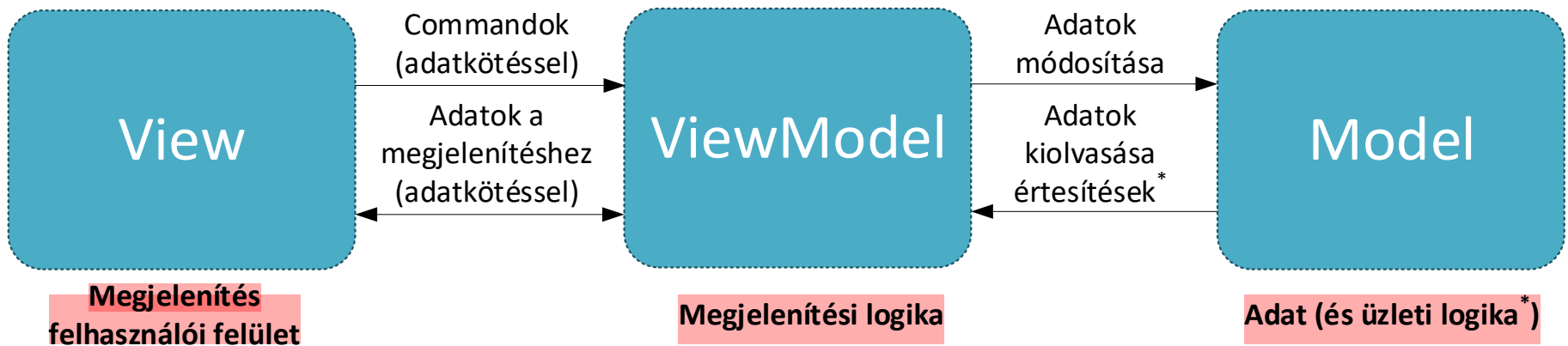
Model-View-ViewModel (MVVM) architektúra

Bevezetés

- Eseményvezérelt és vizuális programozás tárgyból
 - > Részletesen
 - > Gyakorlati példák
- Komolyabb platformszolgáltatásokat igényel
 - > Pl. adatkötés (data binding)

Felelősségi körök

View fogadja a kérést -> ViewModel-t hívja -> Model-t hívja -> View automatikusan frissül



laza csatolás (Observer mint használata!!
kliens oldalon erős!

Felelősségi körök

- **Model:** domain adatosztályok, adatokat reprezentálják
 - > pl. Person, Employee, Student, Order, OrderItem, Tweet, Contact
 - Ezek tagváltozóknak/propertyknak a szükséges információt hordozzák. Pl. Contact esetén Name, Address, PhoneNumber.
 - > **Bizonyos esetekben az üzleti logika is ebben van** (pl. egy számológép app), más esetekben külön „service” osztályokban, melyeket a ViewModel használ, és a „service” osztályok kérdezik le és módosítják a Model adatosztályokat.
 - > **Független a megjelenítéstől!**

Felelősségi körök

- **View:** csak megjelenítés (+ felhasználói események innen erednek)
 - Felületelemekből épül fel (pl. szövegdoz, jelölőnégyzet, stb.),
 - Jellemzően valamilyen deklaratív leíró nyelvvel definiálható (pl. XAML).
- **ViewModel:** *megjelenítési* logika, összeköti a modellt és a nézetet
 - Adatkötés segítségével biztosítja a nézet számára a megjelenítendő adatokat (ehhez felhasználja a modell osztályait).
 - Command objektumok segítségével fogadja az interakciókat a nézet felől (ezek hatására transzformációkat végezhet és továbbítja a változásokat a modell felé). Pl. adott gomb lenyomása futtatja a View-ban a gombhoz kötött ViewModelben definiált parancsot.

Előnyök

- Lásd SoC előnyök
 - > Kiemelve: a model osztályok függetlenek a megjelenítéstől!
- Párhuzamos fejlesztés lehetősége, szakértői tudás különválasztása
 - > A „designer” megtervezi, kialakítja a View-t
 - > A fejlesztők lekódolják a Modelt és ViewModelet
- A UI logika a ViewModelben van, leválasztva a megjelenítéstől (View), a ViewModel kódja unit tesztelhető

DOCUMENT-VIEW ARCHITEKTÚRA

Bevezető

- Az alkalmazások többsége adatokat jelenít meg, melyet a felhasználó módosíthat.
- Alapigazság: ne keverjük bele a UI-ba az alkalmazáslogikát (lásd korábban)
 - > Válasszuk külön az adatok **kezeléséért** felelős kódot az adatok **megjelenítéséért** felelős kódtól.
- Egy lehetséges megoldás a Document-view architektúra
 - > Alternatíva pl. a Model-View-Controller (MVC)

Bevezető

- Az MVC napjainkban a **webalkalmazások** esetében gyakori
- A Document-View a **desktop** alkalmazások esetében használatos (dokumentum alapú alkalmazások esetén, pl. Word-höz hasonló)

Kapcsolat az MVC-vel (némi egyszerűsítéssel élve):

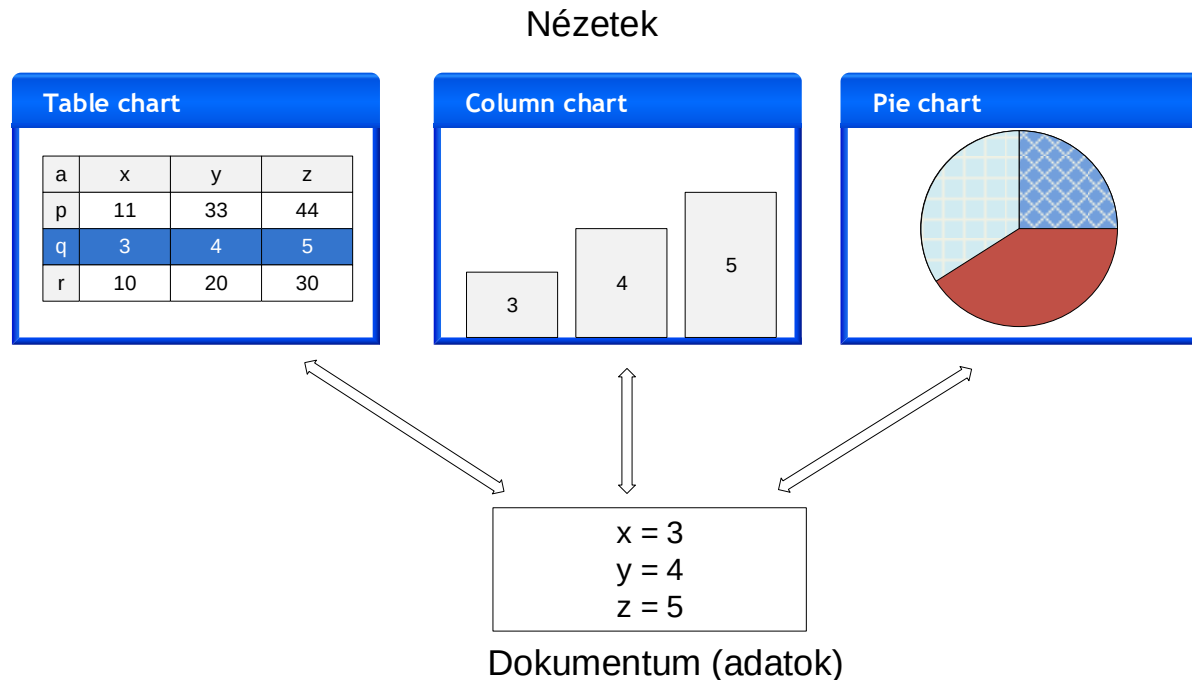
- Model → Document
- View+Controller összevonva → View

A Document-view architektúra szereplői

- **Document (dokumentum)** adat és üzleti logika = **Model**
 - > Feladata az **adatok tárolása, menedzselése**.
 - > Olyan osztály(ok), melyek az adatokat tagváltozóikban tárolják, és olyan tagfüggvényekkel rendelkeznek, melyek kezelik ezeket az adatokat (pl. Load, Save), és elérhetővé teszik más osztályok számára (pl. a View részére)
- **View (nézet)** Observer minta!!!
 - > Feladata az adatok **megjelenítése** a dokumentum adatai alapján és a **felhasználói interakciók kezelése** (pl. menük, egér, billentyűzet).
 - > A felhasználói interakciók során általában a nézet a dokumentum tartalmát módosítja
 - > A view általában egy ablakként, tabfülként vagy egy „panelként” jelenik meg a kliensalkalmazásokban

Támogatja a következőket

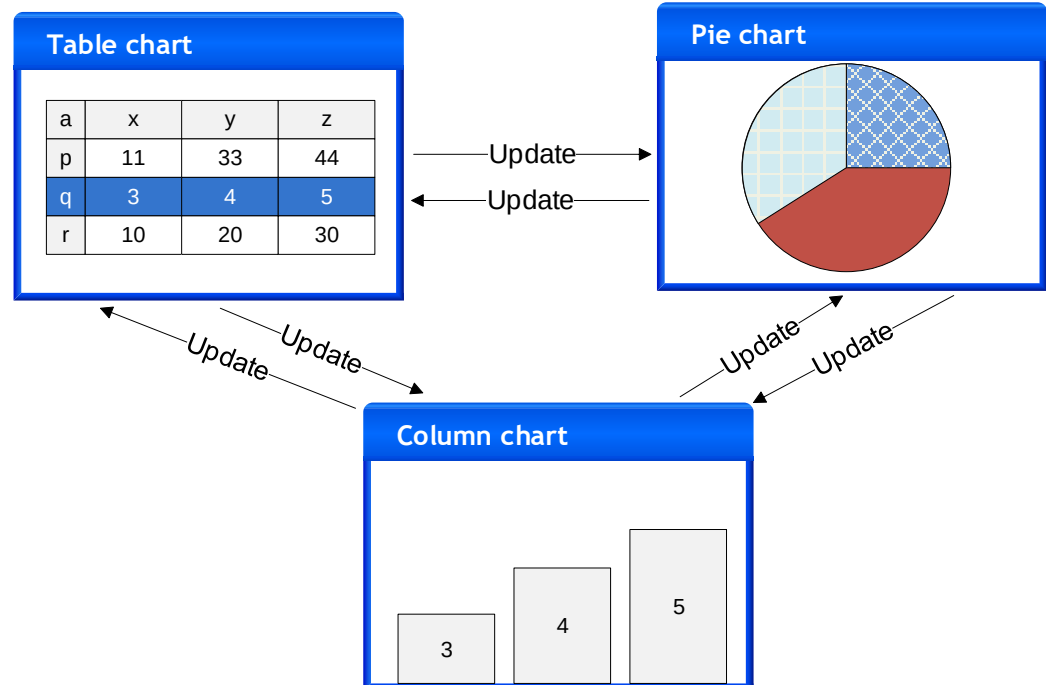
- Több dokumentum egyidejű megnyitása
 - > pl. Firefox tabok, MS Word dokumentumok
- Egy dokumentumhoz több és többféle nézet kapcsolódhat
 - > pl. Excel, de több alkalmazásban a View/New Window menüvel



Egy dokumentumnak több nézete lehet!

- Meg kell oldani, hogy az egyes view-k konzisztens nézetét jelenítsék meg az adatoknak.
- Például ha a felhasználó megváltoztatja az egyik nézeten az adatokat, frissíteni kell a többit. Hogyan? Közvetlen egymásra hivatkozás+függvényhívással (minden nézet tartalmazzon egy referenciát az összes többire, hogy változás esetén tudjon mindenki mást értesíteni)?

Demo
(Word, VS)

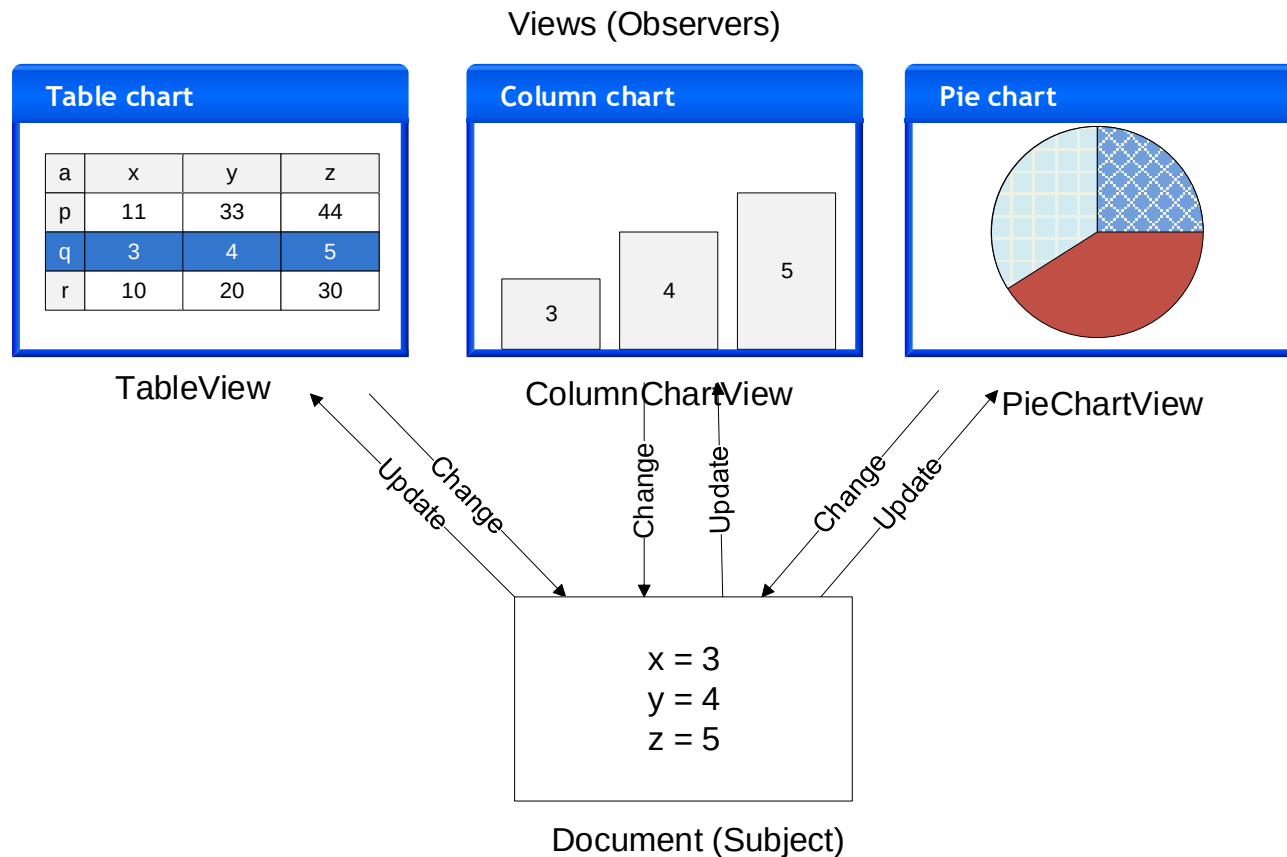


Nézetek frissítése

- Közvetlen hivatkozás+függvényhívás hátrányok
 - > Függőség a konkrét osztálytól.
 - Pl. a `TableView` függ a `ColumnChartView` és a `PieChartView` osztályoktól
 - > Ha új nézetet szeretnék bevezetni, minden MEGLÉVŐ nézet osztályt módosítani kell ☹ ☹ ☹ ☹ ☹
 - > Az alkalmazáslogika nem újrafelhasználható, mert össze van vonva a megjelenítéssel.
 - Cél lenne, hogy úgy jelenjen meg, hogy ne legyen benne hivatkozás egy (konkrét) megjelenítési osztályra sem, mert akkor fel tudnánk több helyen használni
 - > Nehéz karbantartani, továbbfejleszteni, újrafelhasználni, mert túl szoros a csatolás az osztályok között

A nézetek frissítése

- A jó megoldás: Observer minta alkalmazása



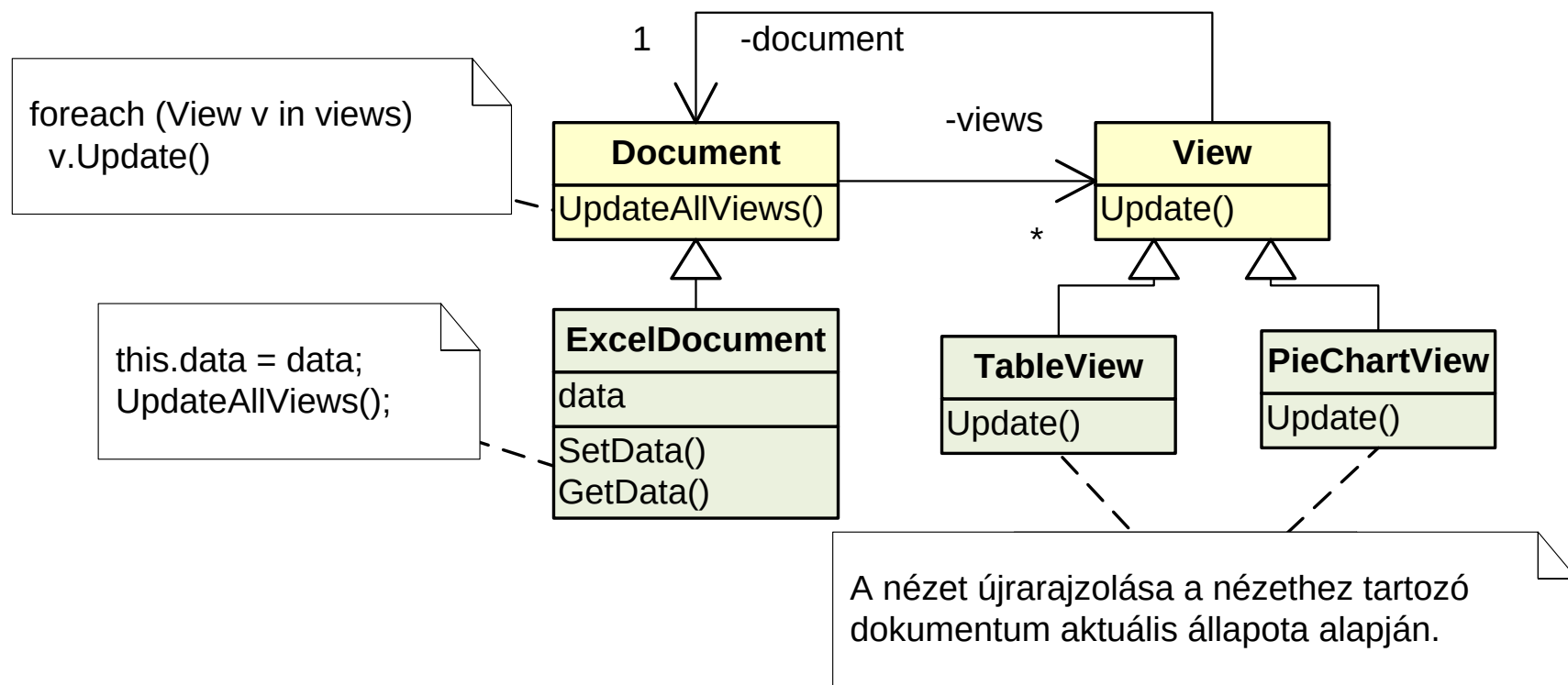
Magyarázat

A nézetek frissítése

- Magyarázat
 - > Emeljük ki az adatokat és az azon értelmezett műveleteket egy osztályba, ez lesz a dokumentum (subject)
 - > A dokumentumhoz különböző view-kat lehet beregisztrálni (observer)
 - > Ha valamelyik view megváltoztatja a dokumentum adatait, a dokumentum értesíti az összes beregisztrált view-t a változásról.
 - > Az értesítés hatására a view lekérdezi a dokumentum állapotát és frissíti magát
 - > A dokumentum csak egy közös View interfészen/ősoosztályon keresztül tárolja a beregisztrált view-kat (nem függ az egyes típusoktól).

Statikus nézet (osztálydiagram)

- Példa (Excel)



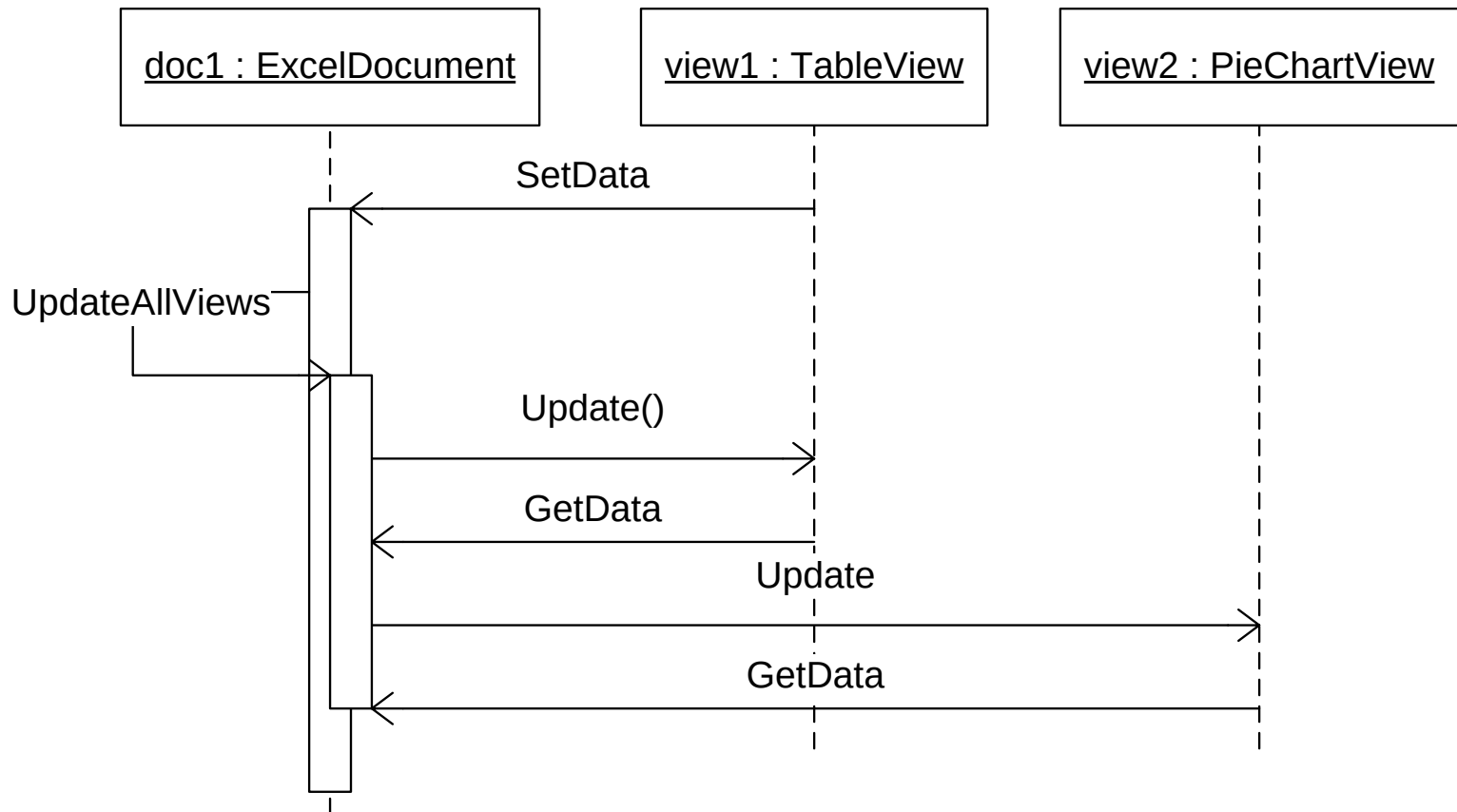
Osztálydiagram magyarázat

- Document
 - > Az osztály objektumai reprezentálnak egy-egy dokumentumot, és a views nevű listájában tárolja a beregisztrált nézeteket.
 - > Tartalmazza a dokumentumokra közös kódot, ha több különböző típusú dokumentum is van (a példánkban nincs)
- ExcelDocument
 - > Egy példa Document leszármazottra.
 - > Tárolja a data, stb. tagváltozójában a dokumentum adatait (pl. cella értékek).
 - > Publikus lekérdező függvényeket biztosít a többi osztály számára az adatokhoz (elsősorban a nézet számára), pl. GetData.
 - > Publikus adatmódosító műveleteket biztosít a többi osztály számára, pl. SetData. Ezek módosítják a tagváltozókat, majd az UpdateAllViews hívásával értesítik a többi nézetet a változásról.
 - > Az UpdateAllViews frissíti a beregisztrált nézeteket (Update-et hív mindre).

Osztálydiagram magyarázat

- View
 - > Az egyes nézetek közös őse vagy interfésze, lehetővé teszi egységes kezelésüket.
 - > Tartalmaz egy referenciát a dokumentumra, amin keresztül a leszármazott nézetek elérhetik a dokumentumot, melynek adatait megjelenítik.
- TableView, PieChartView, stb.
 - > A dokumentum egyes nézeteit reprezentálják.
 - > A View-ból származnak.
 - > Felüldefiniálják vagy implementálják a View Update műveletét. Az Update műveletben a nézet a dokumentum aktuális állapota alapján frissíti magát.

Dinamikus nézet (szekvenciadiagram)



Konklúzió

- Document-view előnyök
 - > Korábban megadot **SoC előnyök**
 - Pl. a document kódjában csak egy View lista van, így a document független az egyes View-t implementáló osztályoktól (laza csatolás!)
 - > Egy egyszerű mechanizmust kaptunk arra, hogy az összes view **konzisztens nézetét** mutassa az adatoknak.
 - > A rendszer **könnyen kiterjeszthető új view osztályokkal**. Sem a document, sem a többi view osztályt nem kell ehhez módosítani.
- Document-view hátrányok
 - > Megnövekedett komplexitás
 - > Csak indokolt esetben, ha illeszkedik a konkrét feladathoz

Szoftvertechnológia és -technikák

10. Előadás

*A szoftverfejlesztés fázisai, dokumentáció,
tesztelés alapjai*

A szoftverfejlesztés fázisai

It's alive!

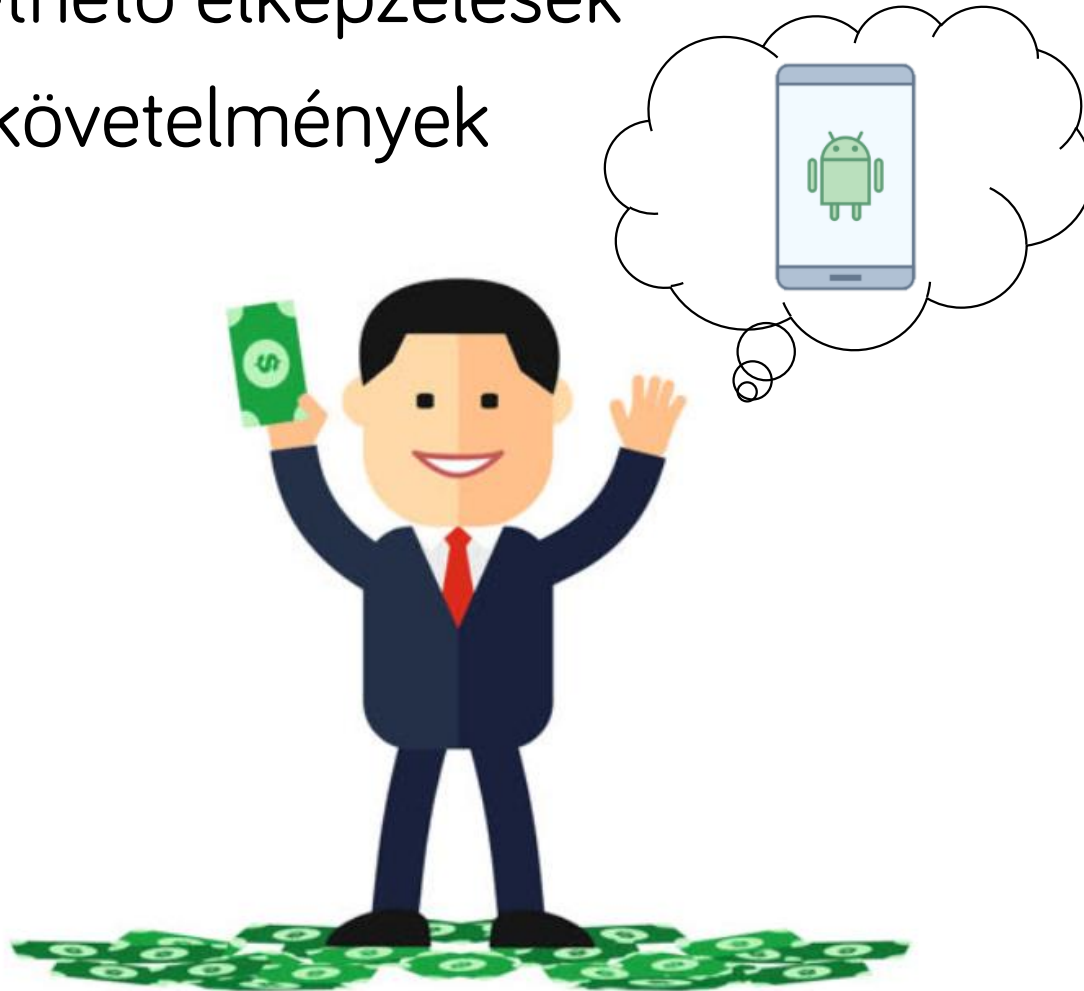


Ideális világ



Ideális világ - Megrendelő

- Jól követhető elképzelések
- Pontos követelmények



Ideális világ - Fejlesztés

- Könnyen, tisztán megértjük a feladatot
- Mehet a fejlesztés!



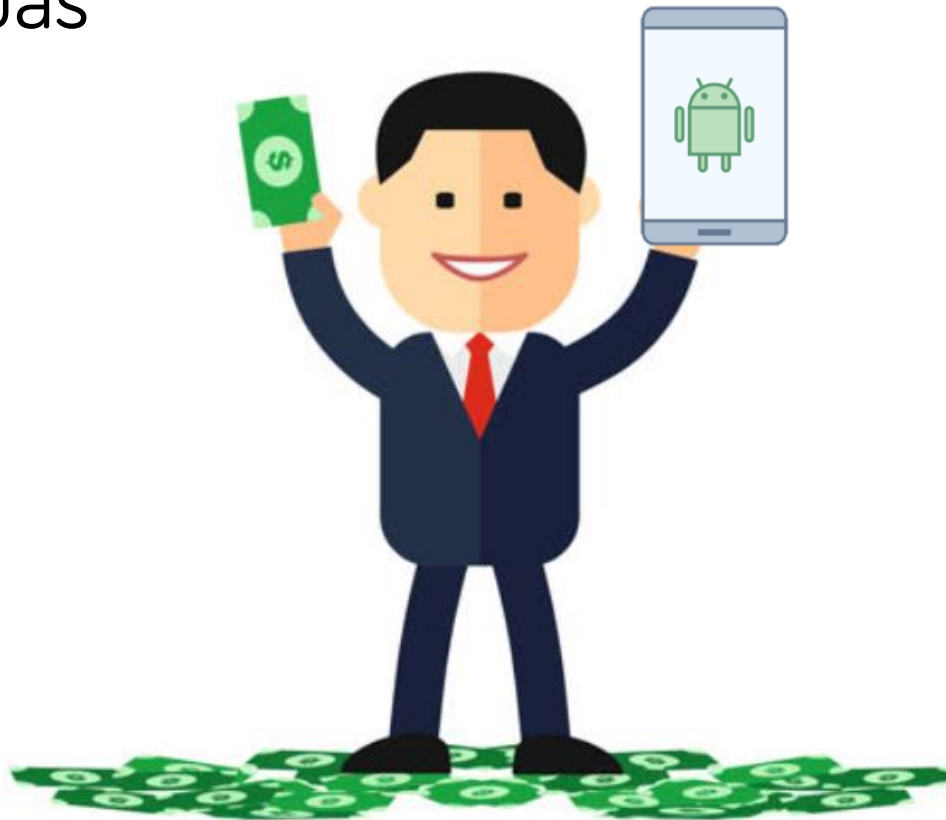
Ideális világ – Kész termék

- Elkészül az alkalmazás határidőre



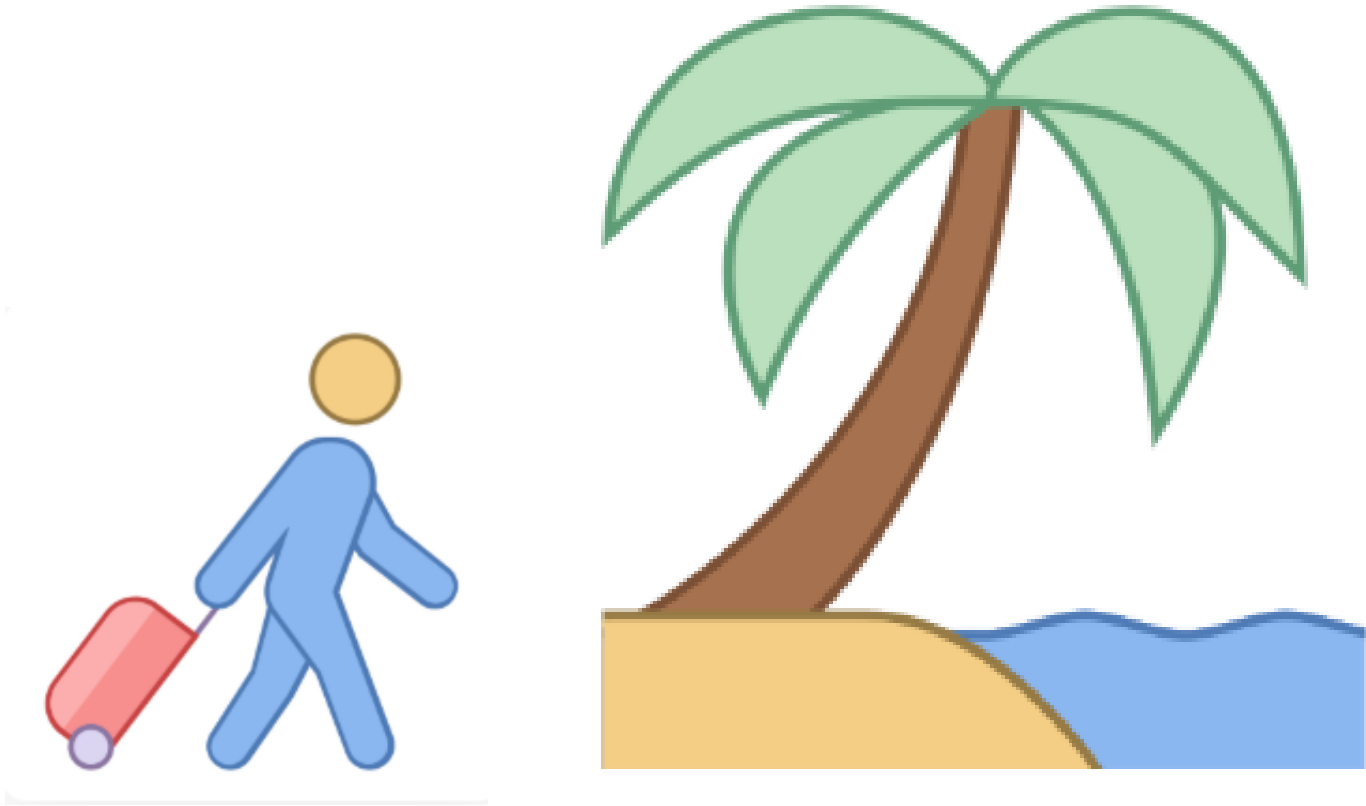
Ideális világ - Átadás

- Elégedett megrendelő
- Könnyű átadás

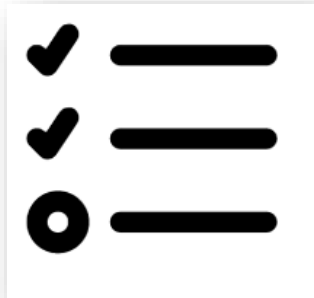


Ideális világ

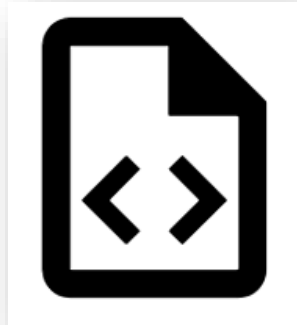
- Mehetünk nyaralni...



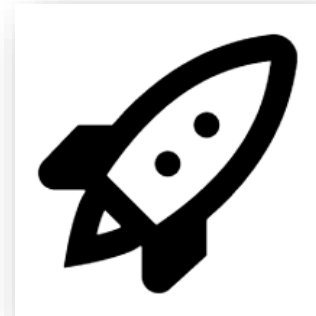
Ideális világban a szoftverfejlesztés fázisai



TERVEZÉS



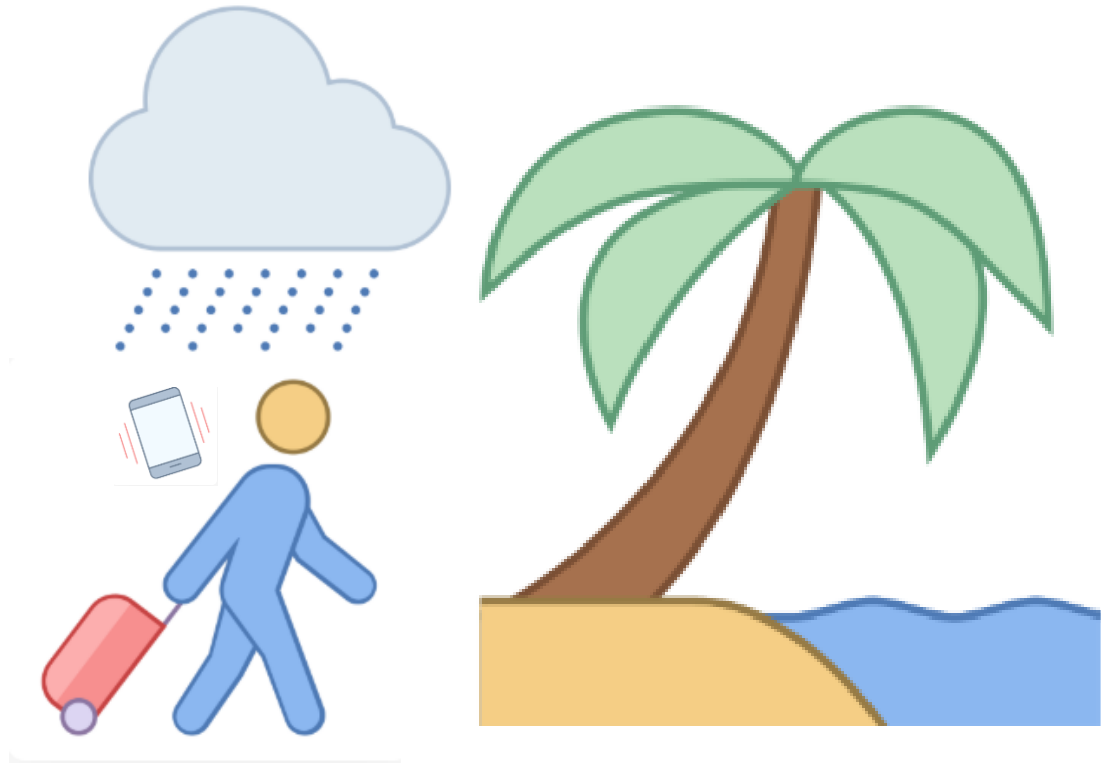
FEJLESZTÉS



TELEPÍTÉS

A valóság ennél árnyaltabb...

- Áramszünet után fura hiba



A szoftverfejlesztés valós fázisai



SPECIFIKÁCIÓ KÉSZÍTÉS
KÖVETELMÉNYEK DEFINIÁLÁSA



TERVEZÉS



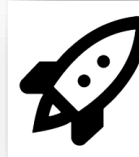
IMPLEMENTÁLÁS



HIBAKERESÉS



TESZTELÉS



ÜZEMBE HELYEZÉS, TELEPÍTÉS

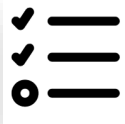


ÜZEMELTETÉS, KARBANTARTÁS

A szoftverfejlesztés valós fázisai



SPECIFIKÁCIÓ KÉSZÍTÉS
KÖVETELMÉNYEK DEFINIÁLÁSA



TERVEZÉS



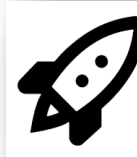
IMPLEMENTÁLÁS



HIBAKERESÉS



TESZTELÉS



ÜZEMBE HELYEZÉS, TELEPÍTÉS



ÜZEMELTETÉS, KARBANTARTÁS

Követelmények definiálása

- A Projekt fogalma:
 - > egyedi jellemzőkkel rendelkező rendszer kidolgozására vállalkozik
 - > különféle **koordinált tevékenységek** alkotják
 - > meghatározott a **célja**
 - > jól definiált kezdő és befejező **dátummal** rendelkezik
 - > **átmeneti** vállalkozás (a projekt végeztével a kialakított szervezeti és strukturális felépítés megszűnik vagy átalakul)
 - > egyéb keretek (például költség, információ biztonság stb.)

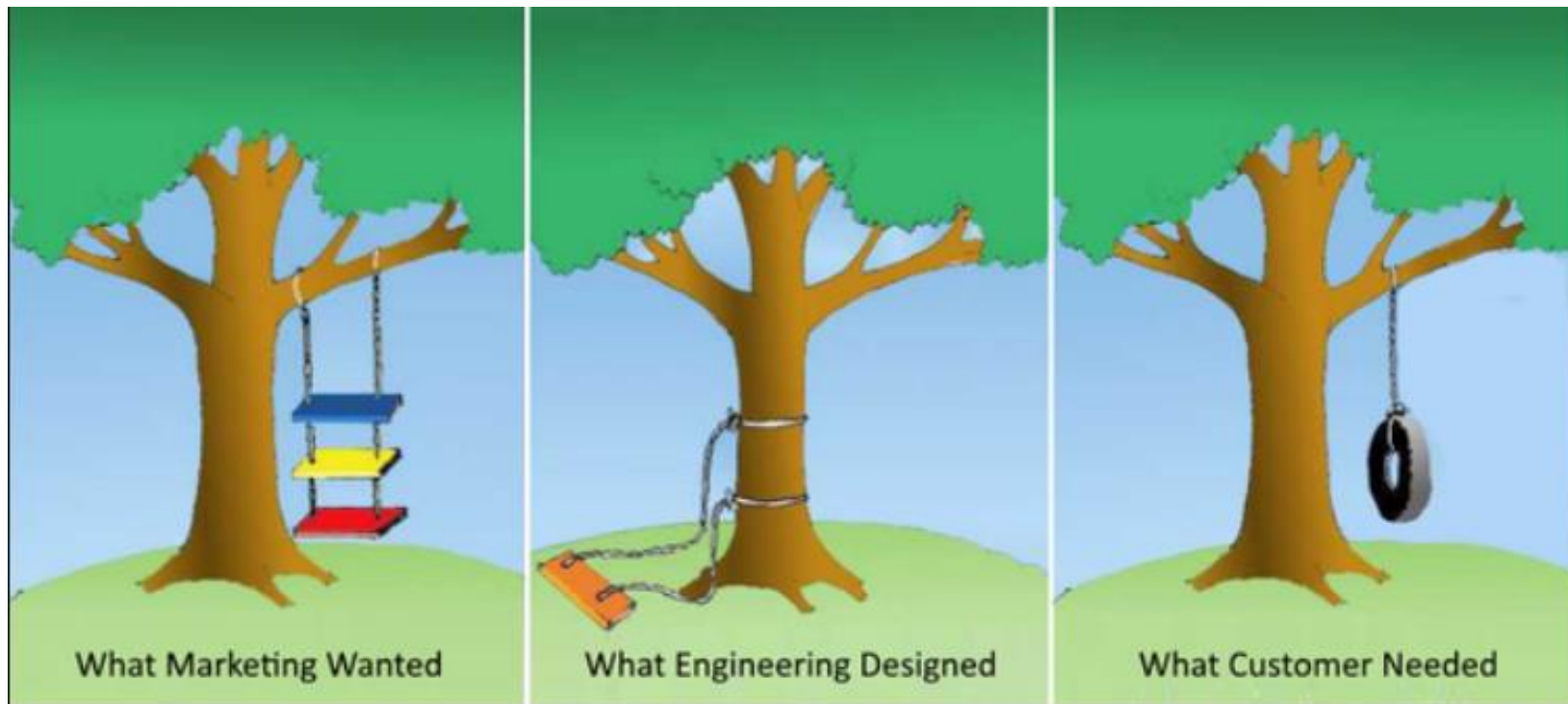
Követelmények definiálása

- Meghatározó tényezők: **idő, költség, teljesítmény és minőség**
- Példa: A megrendelő nem bíz a műholdakban, és szeretne egy saját navigációs, útvonal optimalizációs rendszert, ezzel bíz meg minket (gondoljunk a Waze-re)

Követelmények definiálása

- Mikorra szeretné ezt a megrendelő? Milyen határidővel?
 - > 2 év múlva
- Milyen platformra szeretné a megrendelő?
 - > Android, iOS? Mindkettő?
- Mit kell pontosan tudni az appnak?
 - > Milyen pontossággal működjön?
 - > Milyen gyorsan?
 - > Milyen hibahatárral?
 - > Milyen funkció vannak pontosan?

Követelmények definiálása



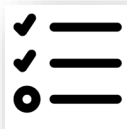
Követelmények rögzítése

- A követelmények részletei meghatározzák a költségeket (és átfutást)
- A részletek adják a projekt funkcionális kereteit
 - a szerződés kereteit
 - a change requestek alapját

A szoftverfejlesztés valós fázisai



SPECIFIKÁCIÓ KÉSZÍTÉS
KÖVETELMÉNYEK DEFINIÁLÁSA



TERVEZÉS



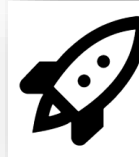
IMPLEMENTÁLÁS



HIBAKERESÉS



TESZTELÉS



ÜZEMBE HELYEZÉS, TELEPÍTÉS



ÜZEMELTETÉS, KARBANTARTÁS

Szoftvertervezés

- A szoftvertervezési folyamatnak kell biztosítania, hogy a fejlesztés végén egy „jó szoftver” szülessen
- A „jó szoftver” egy megfelelő kompromisszum a különféle elvárások célkeresztjében
 - A „jó szoftver” ritkán az elképzelhető legjobb...



Milyen a jó szoftver?

- Funkcionálisan teljes
 - > Ellátja a kitűzött funkcióit
- Robosztus
 - > Hibatűrő (felismeri és orvosolja a hibát)
 - > Jól viseli a változtatásokat
- Mérhető
 - > Futásidő (pl. UI betöltése)
- Debugolható:
 - > Pl. logok
 - > Monitorozható

A jó szoftver

- Karbantartható
 - > Architektúra, kommentek, dokumentáció
- Megbízható
- Újra felhasználható
- Tovább fejleszthető

Szoftvertervezés

- A követelmények alapján elkészül a szoftverterv
- Kialakul a rendszer architektúrája:
 - > Milyen egységek vannak?
 - > Ezek hogyan kapcsolódnak
- A szoftvertervezés eredményei modellek és dokumentáció (terméktípustól függően más és más jellegű és mennyiségű)

Minőség és költségek

- A felhasználó néha nem fizeti meg a lehető legjobb minőséget → ???
- Valóság: egyszerűsítünk, butítunk, ...
- Példa:
 - > Jobbgombos menüt sok idő lenne ide betenni
 - > Jó lesz, ha lenyomja az „n” betűt, létrejön az új elem
 - Beleírjuk egy tooltipbe és a kézikönyvbe

A szoftverfejlesztés valós fázisai



SPECIFIKÁCIÓ KÉSZÍTÉS
KÖVETELMÉNYEK DEFINIÁLÁSA



TERVEZÉS



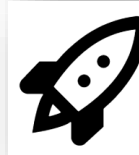
IMPLEMENTÁLÁS



HIBAKERESÉS



TESZTELÉS



ÜZEMBE HELYEZÉS, TELEPÍTÉS



ÜZEMELTETÉS, KARBANTARTÁS

Implementálás

- A szoftvertervezési fázisban készített szoftverterv ebben a szakaszban valósul meg
- A funkciók itt valósulnak meg
- Általában az aktív fejlesztés leghosszabb szakasza

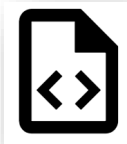
A szoftverfejlesztés valós fázisai



SPECIFIKÁCIÓ KÉSZÍTÉS
KÖVETELMÉNYEK DEFINIÁLÁSA



TERVEZÉS



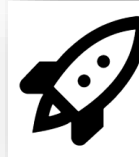
IMPLEMENTÁLÁS



HIBAKERESÉS



TESZTELÉS



ÜZEMBE HELYEZÉS, TELEPÍTÉS



ÜZEMELTETÉS, KARBANTARTÁS

Hibakeresés

- A “tökéletes” és “hibátlan” szoftver legendája
- Tévedni emberi
 - > Kapkodás
 - > Monotonitás
 - > Fáradtság
- Hibatípusok
 - > Implementációs hiba (“if (a=5) ...”)
 - > Kommunikációs hiba
 - > Rossz kódolás, verziókezelés, ...

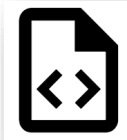
A szoftverfejlesztés valós fázisai



SPECIFIKÁCIÓ KÉSZÍTÉS
KÖVETELMÉNYEK DEFINIÁLÁSA



TERVEZÉS



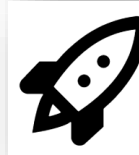
IMPLEMENTÁLÁS



HIBAKERESÉS



TESZTELÉS



ÜZEMBE HELYEZÉS, TELEPÍTÉS



ÜZEMELTETÉS, KARBANTARTÁS

Tesztelés

- Tény: minden szoftverben vannak hibák
- A minőség ellenőrzők megvizsgálják a szoftvert, hogy a specifikáció szerint működik-e?
- A hibák visszajutnak a fejlesztőkhöz és a tesztelés előlről indul
- Gyakran használunk átmeneti állapotot: béta verzió, meghívott felhasználók stb.

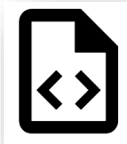
A szoftverfejlesztés valós fázisai



SPECIFIKÁCIÓ KÉSZÍTÉS
KÖVETELMÉNYEK DEFINIÁLÁSA



TERVEZÉS



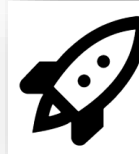
IMPLEMENTÁLÁS



HIBAKERESÉS



TESZTELÉS



ÜZEMBE HELYEZÉS, TELEPÍTÉS



ÜZEMELTETÉS, KARBANTARTÁS

Telepítés (deployment)

- Telepítés, terjesztés
- Más környezet – más működés
- Pontosán ugyanaz a verzió menjen ki, mint amit a tesztelők jóváhagytak!

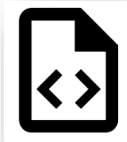
A szoftverfejlesztés valós fázisai



SPECIFIKÁCIÓ KÉSZÍTÉS
KÖVETELMÉNYEK DEFINIÁLÁSA



TERVEZÉS



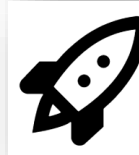
IMPLEMENTÁLÁS



HIBAKERESÉS



TESZTELÉS



ÜZEMBE HELYEZÉS, TELEPÍTÉS



ÜZEMELTETÉS,
KARBANTARTÁS

Üzemeltetés és Karbantartás

- A szoftverélelciklusának általában a leghosszabb szakasza
 - > Gondoljunk az újonnan vett és a használt autókra! Mennyi költség megy el az autók szervizelésére, karbantartására!

Üzemeltetés és Karbantartás

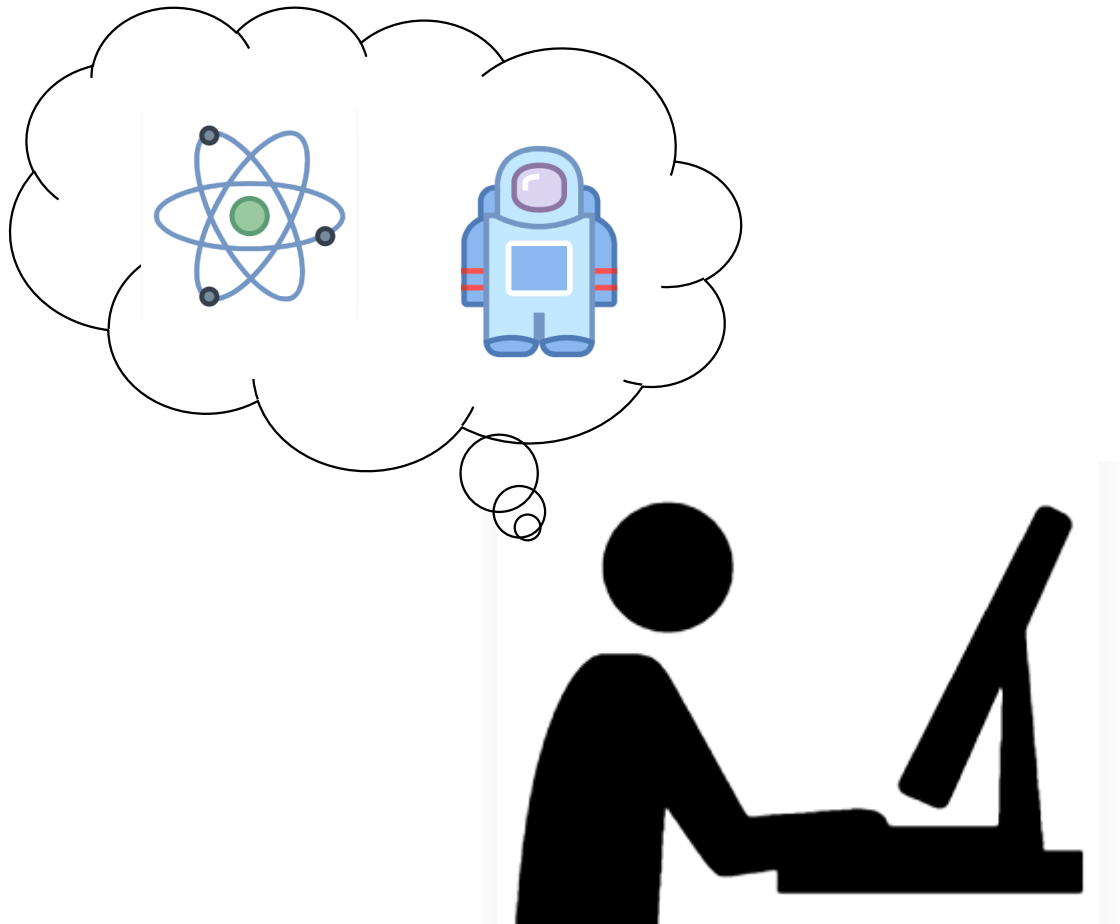
- Továbbfejlesztési lehetőségek
 - > Implementáció/Architektúra fejlesztése
 - > Új követelmények fellépése
- A hibák javítása
- Megváltozott környezethez illesztés

Eredmény a fázisok végén

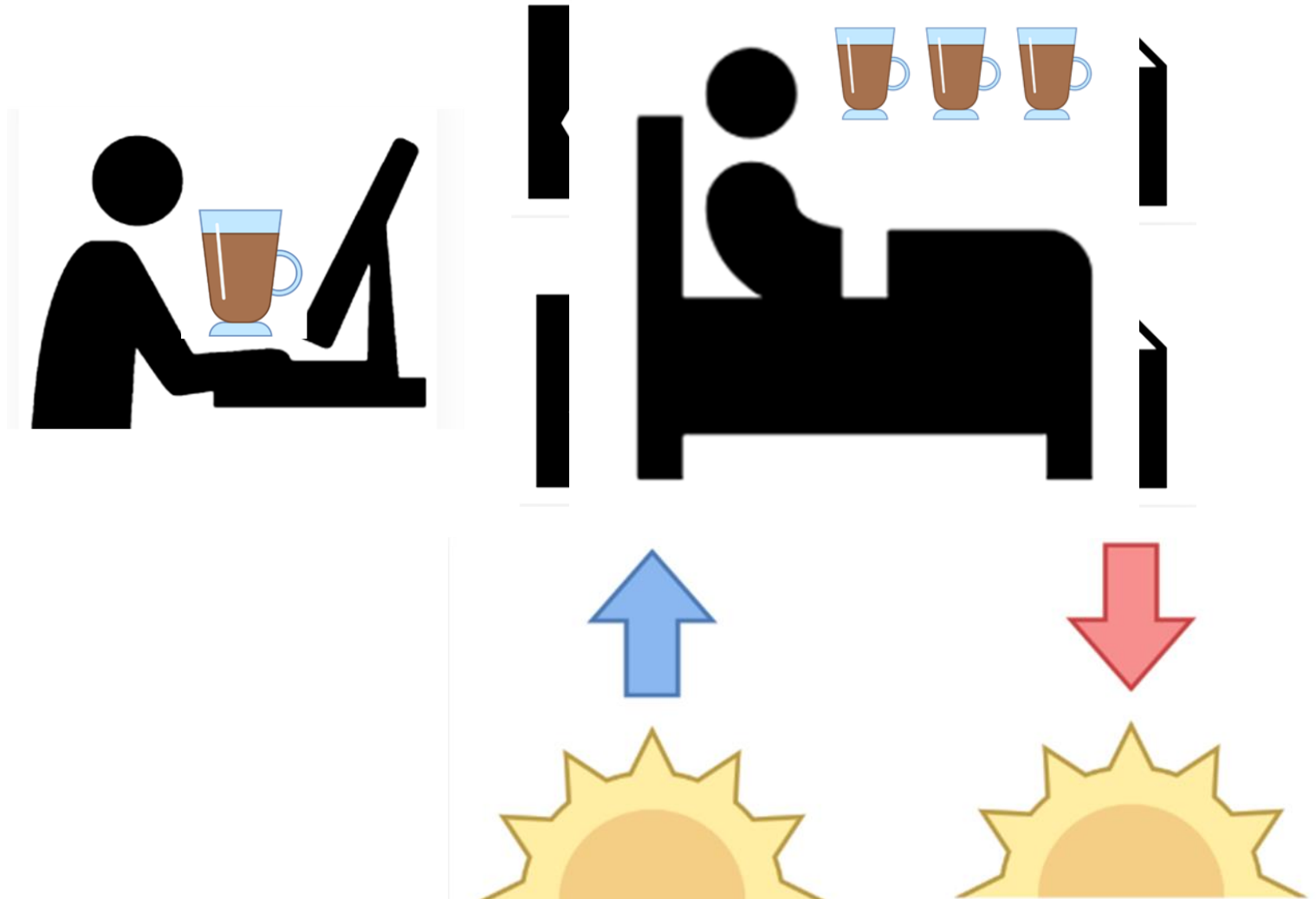
- Specifikáció, követelmények definiálása
 - > Modellek (Use Case), követelmény specifikáció
- Tervezés
 - > Modellek (többi UML modell) és dokumentáció
- Implementálás
 - > Forráskód
- Hibakeresés
 - > Javított forráskód
- Tesztelés
 - > Tesztelt, funkcionálisan működő forráskód
- Üzembehelyezés, telepítés
 - > Telepített, működő alkalmazás
- Üzemeltetés, karbantartás
 - > Alkalmazás (új verziók)

Szoftverfejlesztési dokumentáció

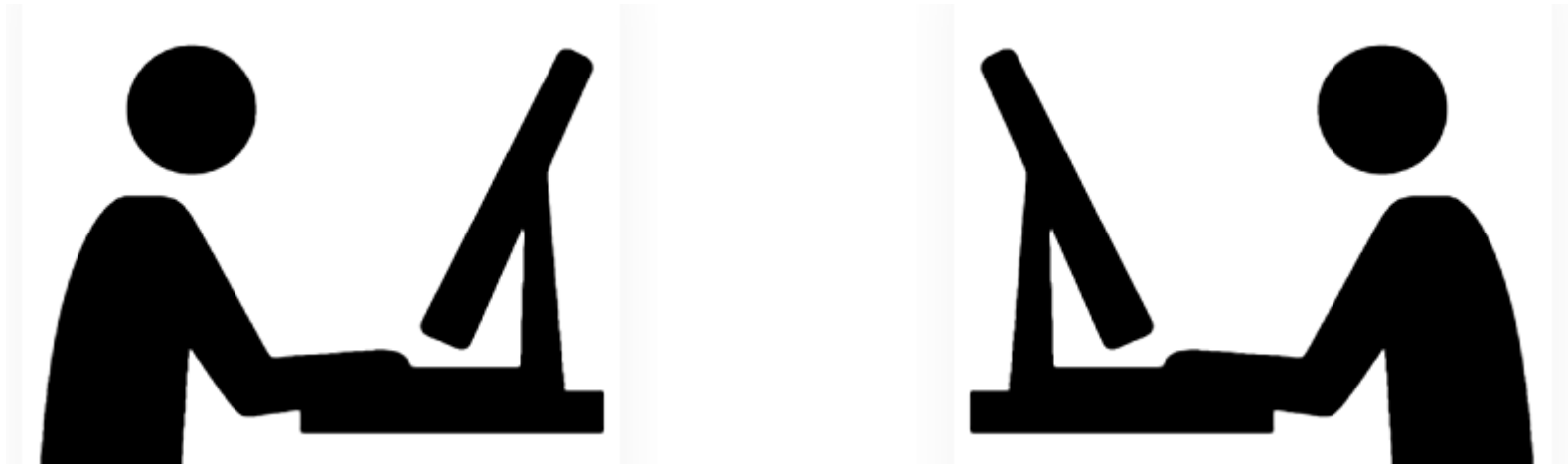
MoonLanding – az ötlet



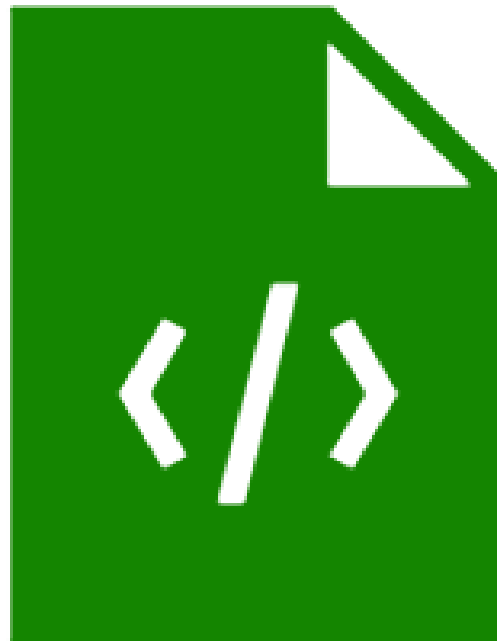
MoonLanding – az implementáció

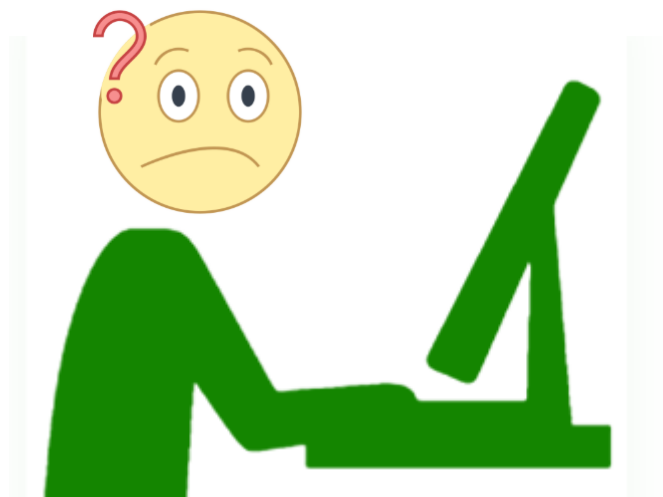


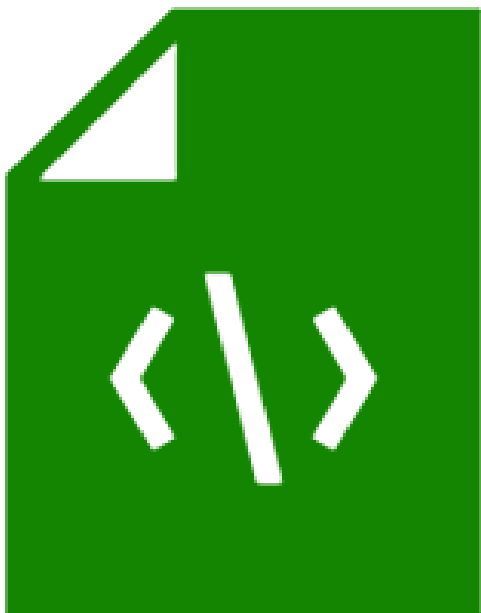
MoonLanding – a társ

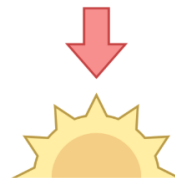






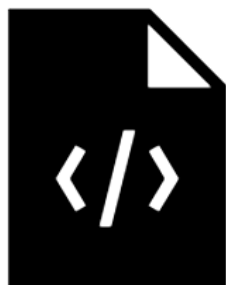
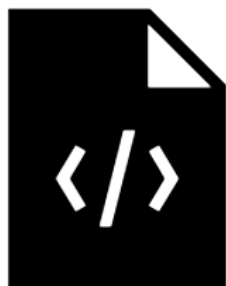


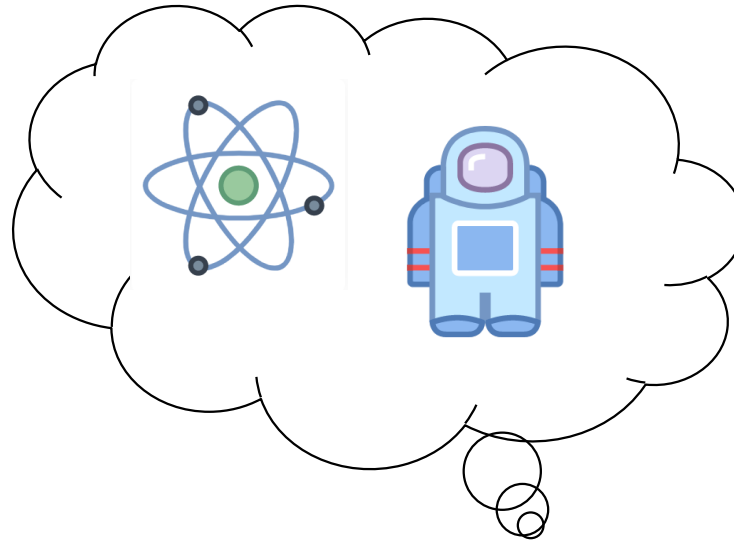


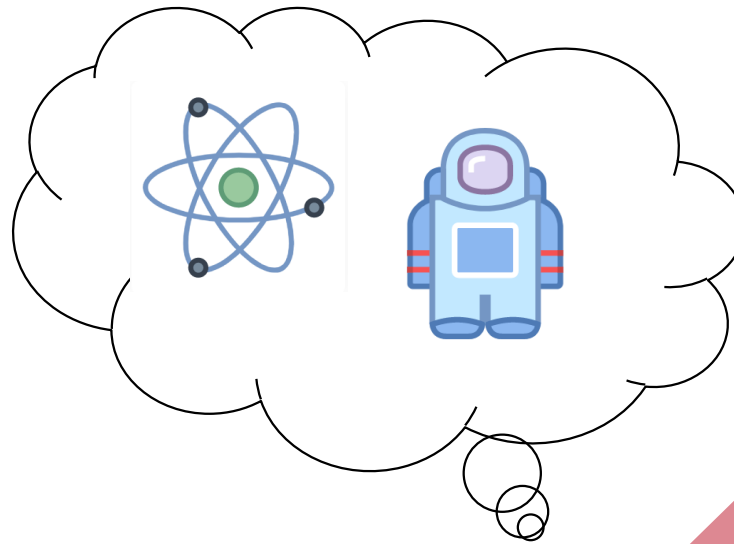


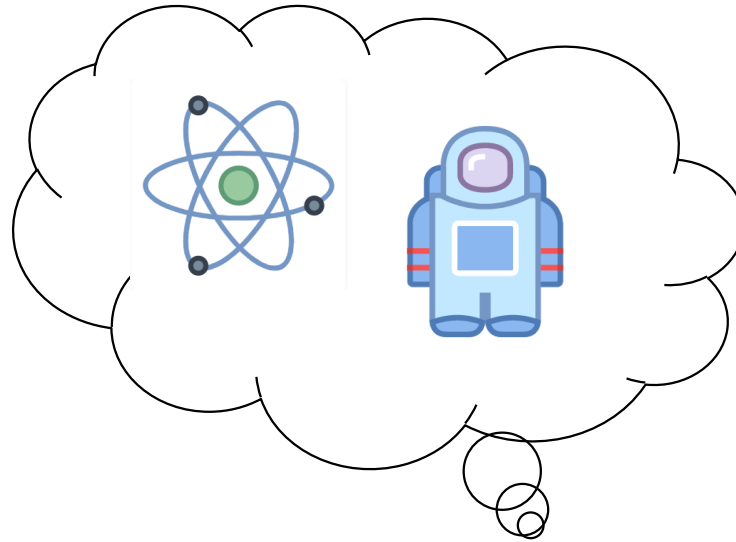


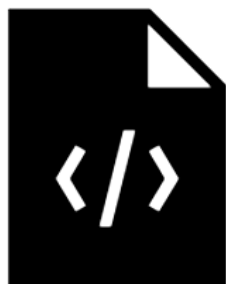
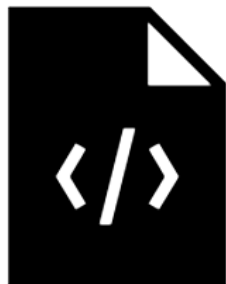






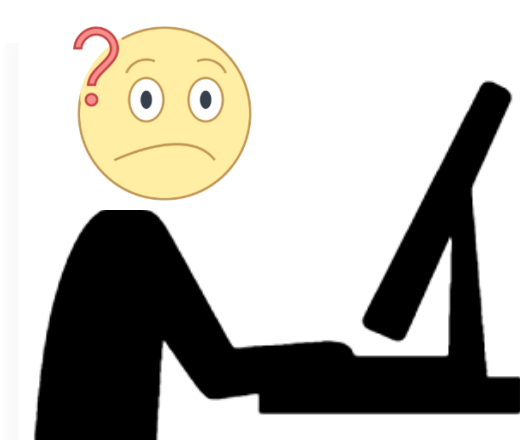
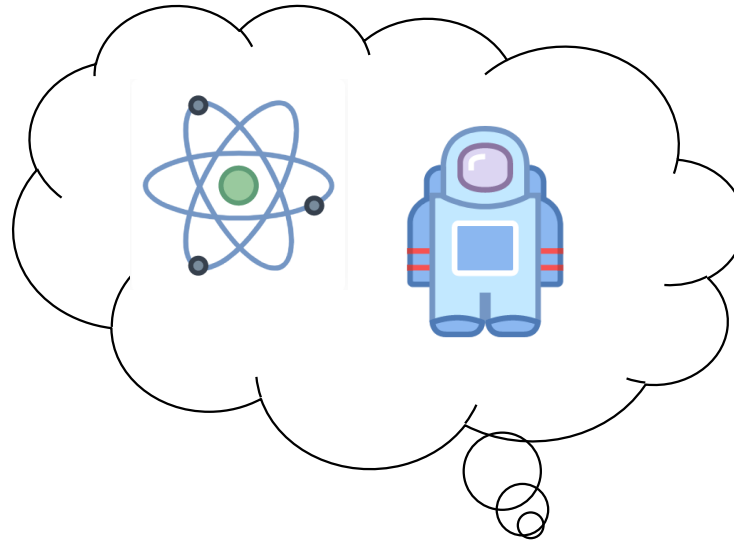












Hogyan lehetett volna ezt elkerülni?

- „Dokumentálással”

Dokumentáció

- **Kód dokumentáció célja: a következő fejlesztő generáció tudjon dolgozni, hibákat javítani, új funkciókat fejleszteni!**
 - > Lehetnek egyéb célok: pl a megrendelő fizet érte, törvényi előírás stb.

Dokumentáció formái

- A „dokumentáció” nem feltétlenül kiegészítő szövegesen írott dokumentum
- Alternatívák
 - > „Öndokumentáló” kód: kódolási stílus, beszédes nevek, általános és projekt specifikus tervezési minták stb.
 - > Unit tesztek: egyértelműsítik a komponens határokat, felelősségeket!
 - > UML diagramok – a közös nyelv! Részben generálható is

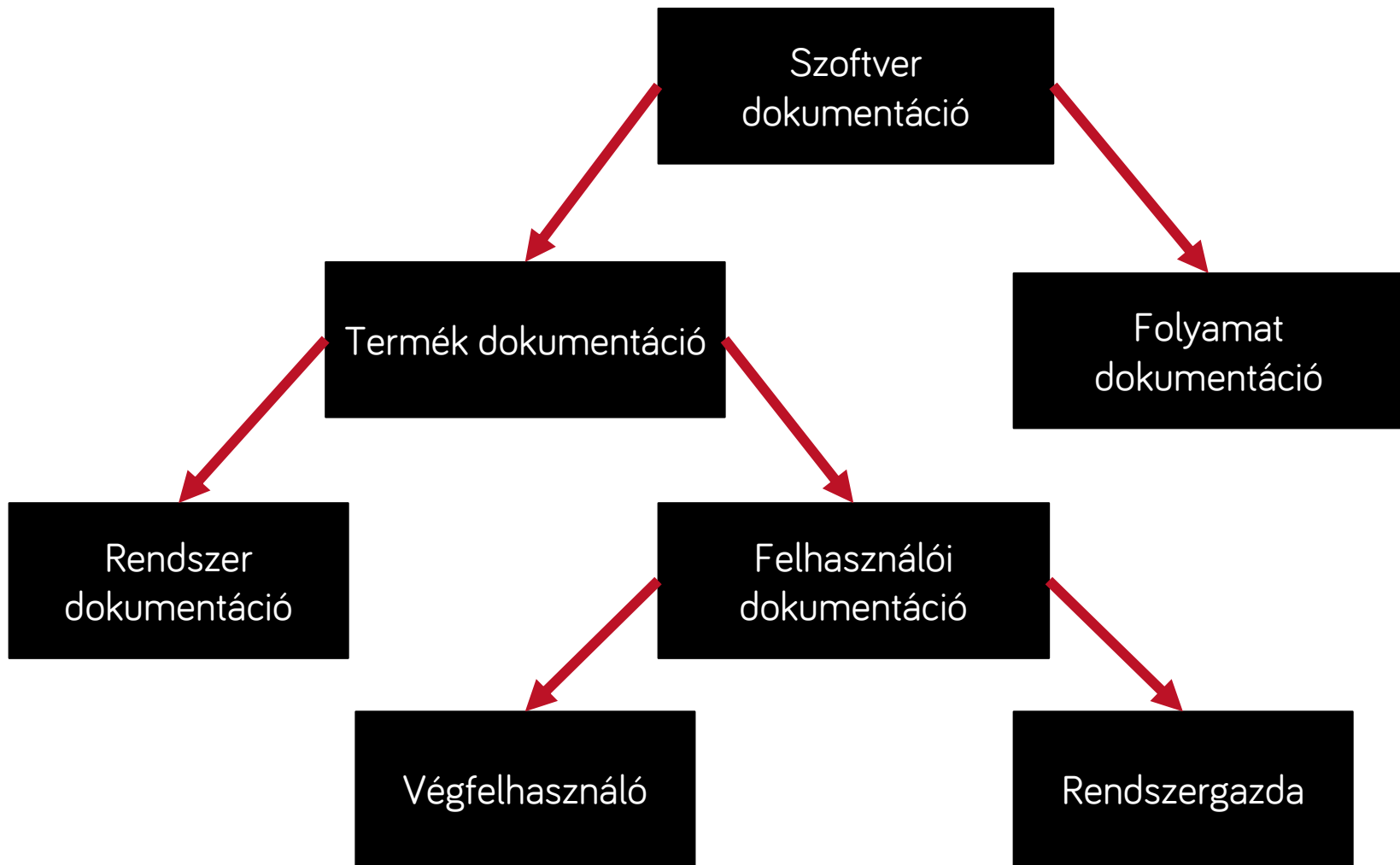
Dokumentáció

- A dokumentálás a szoftverkészítés folyamatát végigkísérő tevékenység.
- Lényegében az összes munkafázishoz kapcsolódik dokumentáció.

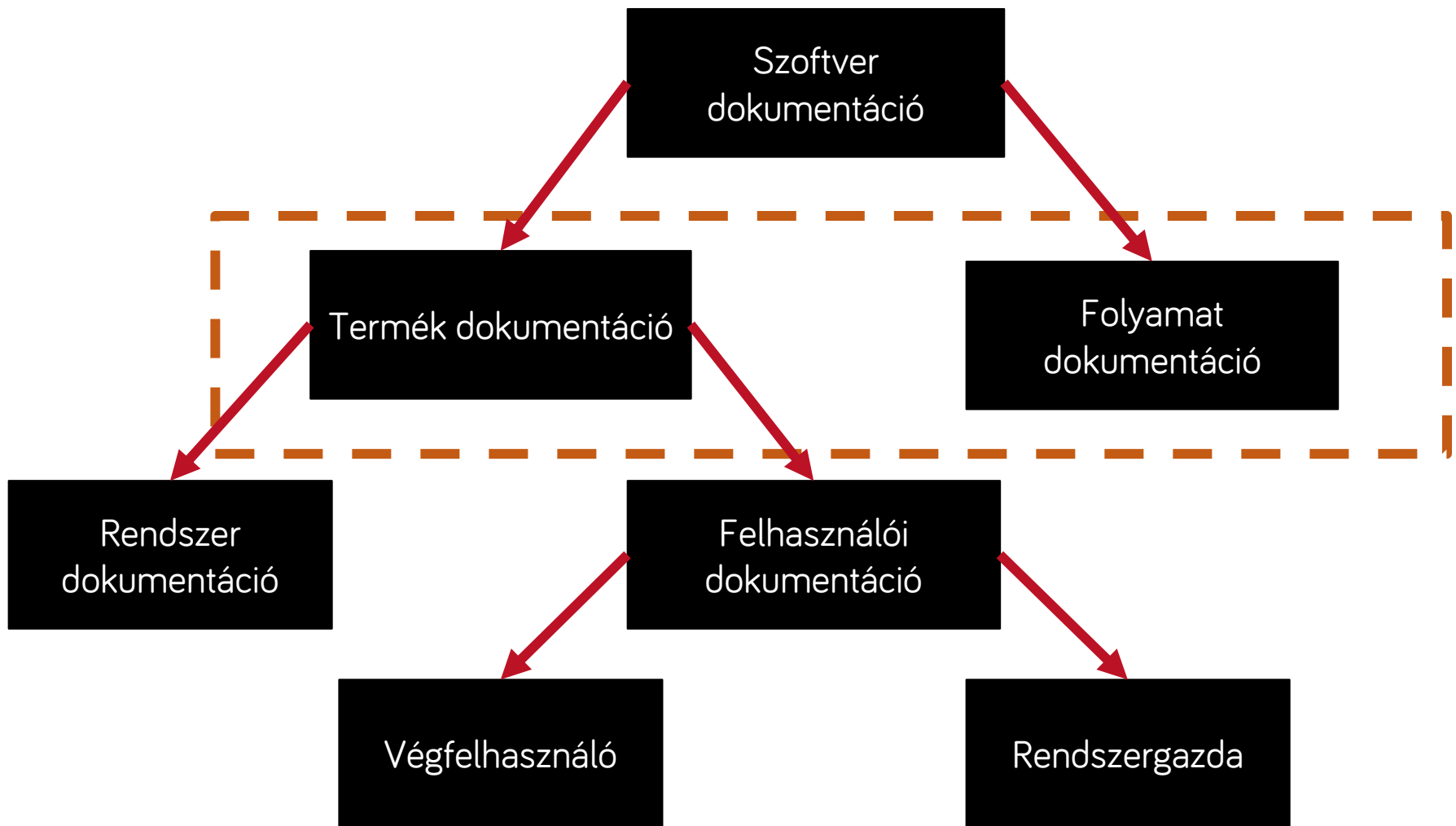
Szoftver

- A szoftver nem csupán programkód!
- Szoftver = programok, dokumentációk, konfigurációk, adatok együttese

Dokumentáció csoportosítása



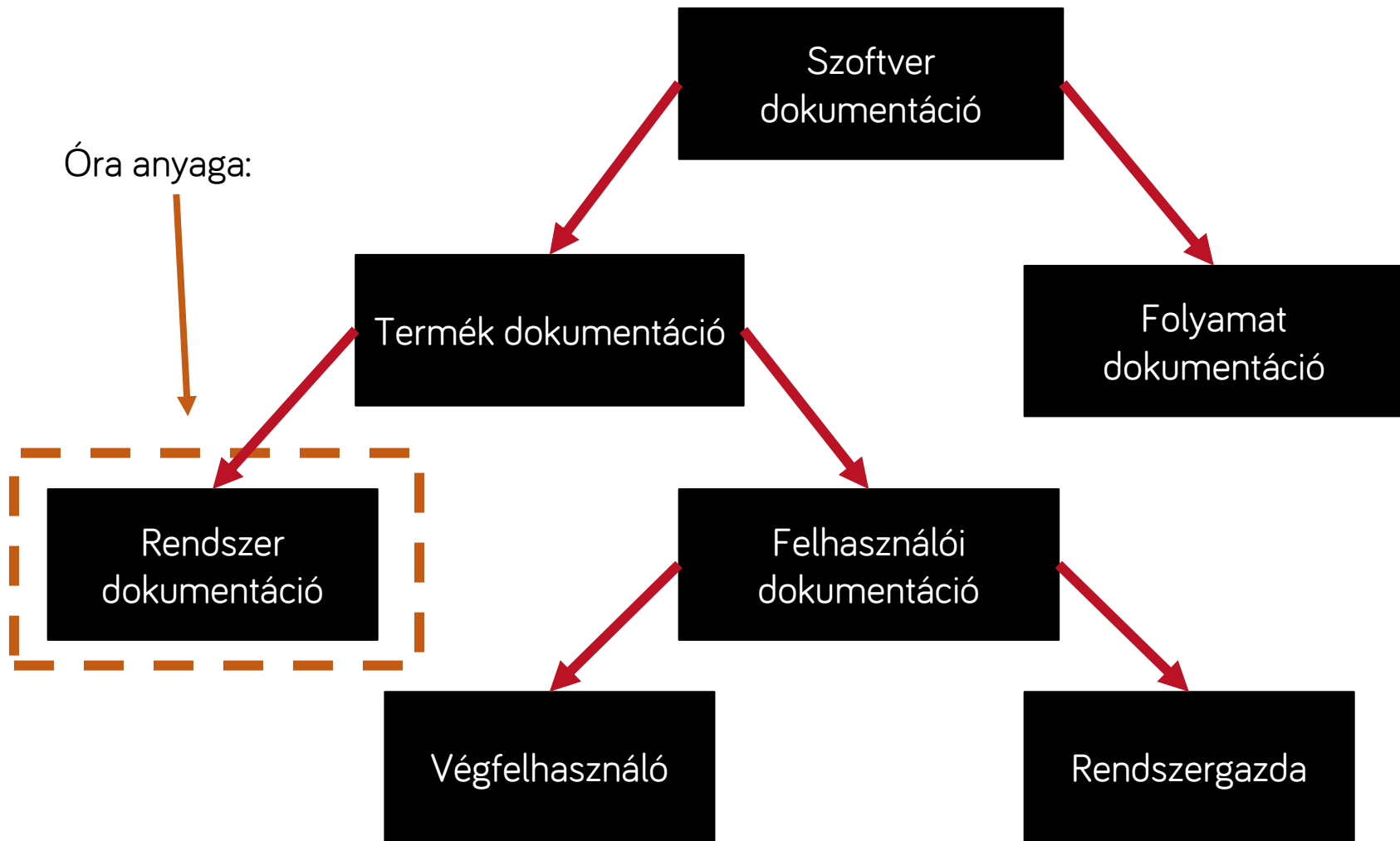
Dokumentáció csoportosítása



Termék vs Folyamat

- Termék dokumentáció:
 - > Leírja a szoftverterméket, ami éppen fejlesztésre kerül
- Folyamat dokumentáció:
 - > A folyamat dokumentáció leírja a folyamatot (pl. mi mennyi ideig fog tartani, projekt terv, meeting jegyzetek)

Dokumentáció csoportosítása



Rendszer dokumentáció

- A termék dokumentáció része
- Egy áttekintést ad a szoftverre
- Ez tartalmazza: FSD, SDD, API doc!
 - > Követelményspecifikációt
 - > Rendszertervet
 - > Forráskód dokumentációt
 - > Tesztelési dokumentációt
 - > Karbantartási dokumentációt

Követelmény specifikáció

- A követelmények definiálása fázishoz tartozik
- A követelmények alapján készítik el
- A gyakorlatban azt foglalja össze, hogy mit kell a rendszernek/modulnak/feature-nek tudnia
- A formátumának tisztának és jól követhetőnek kell lennie
- Elméletben a formája lehet: beszélt nyelven, formalizált leírással, formális matematikai leírással, formális leírónyelvekkel

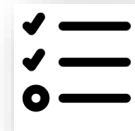


SPECIFIKÁCIÓ KÉSZÍTÉS
KÖVETELMÉNYEK DEFINIÁLÁSA

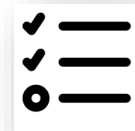
Követelmény specifikáció

Requirements

#	User story title	User story description	Priority	Notes
1	Facebook Integration 	A user wants to sign up via Facebook	Must Have	• We will need to talk to Cassie Owens. • There has also been some research done on this (see Facebook integration prototype)
2	Activity Stream 	A user wants to view the latest updates via the mobile dashboard so that they can get a better understanding of what is in place	Must Have	
3	Post Updates 	A user wants to be able to post status updates on the go	Must Have	The key things we will need to support: • Text status updates • Mentions • Support for images • Smart embedding for YouTube vids
4	API 	A developer wants to integrate with the mobile app so that they can embed the activity stream on their website	Should Have	• We should chat to Team Dyno as they did something similar.



- A tervezés fázisához tartozik
- A fő döntések dokumentálása a szoftver felépítésével kapcsolatban
- Része az architektúra kifejtése
 - > Pl. ha microservice-eket használunk a fejlesztés során, ez mindenképpen említésre kerül itt



- A felhasználói történetek (user story) kifejtése
 - > Az egyes user storyk milyen üzleti folyamatoknak felelnek meg
- A megoldás részletei
 - > Service-ek, modulok, komponensek részletezése és fontosságuknak a tárgyalása
- Diagrammok
 - > Pl. Use case, UML osztálydiagramok, szekvenciadiagramok

Rendszerterv felépítése

- Előszó (célközönség, dokumentum-történet)
- Bevezetés (szoftver célja, helye, szükségessége, előnyei, fejlesztési módszertan)
- Fogalomtár (technikai áttekintés)
- Rendszer architektúra (magas szintű áttekintés, UML csomag-, komponens-, állapotdiagram)
 - > architektúrális minták
 - > funkcionális megfeleltetés
- Adattervezés (adattárolás, formátumok leírása)

Rendszerterv felépítése

- Rendszer tervezés (alacsony szintű áttekintés)
 - > statikus terv (UML osztály-, objektumdiagram)
 - > dinamikus terv (UML állapot-, szekvencia- és aktivitásdiagram)
 - > interfész leírás
 - > felhasznált algoritmusok és minták
- Felhasználói felület (áttekintés, felületi terv)
- Implementációs ajánlások
- Függelék (pl. adatbázis terv, becsült hardver szükségletek)
- Tárgymutató

Forráskód dokumentáció

pl: Javadoc használata



IMPLEMENTÁLÁS

- Az implementációs fázishoz tartozik
- Elmagyarázza, hogy működik a kód
- Szoftverfejlesztőknek szól
- Többek között tartalmazhatja:
 - > A felhasznált keretrendszereket
 - > Adatkötés típusát
 - > Tervezési minták példákkal
 - > Formázási követelmények
 - > API-k leírása

Tesztelési dokumentáció



TESZTELÉS

- Tesztterv
 - > Stratégiák leírása (milyen tesztek lesznek)
 - > Milyen funkciókat kell tesztelni
 - > Milyen módon kell tesztelni őket
 - > Milyen prioritással
 - > Szerepek/felelősségek
- Teszteset dokumentáció
 - > Részletes leírása a tesztesetnek
 - > Mikor verifikálható a teszteset, milyen feltételeknek kell megfelelni
- Teszt ellenőrzőlista

Karbantartási dokumentáció

- Tartalmaznia kell a működés közben felmerült hibákat
- Fejlesztéseket, változtatásokat



ÜZEMELTETÉS, KARBANTARTÁS

Tesztelés

Miért fontos a tesztelés?

- A tesztelés minőségi mutató
 - > Ha nincs tesztelés, nem tudjuk, hogy mennyire jó az alkalmazásunk
 - > Pl. Hogy ne „haljon meg” váratlan felhasználás esetén az alkalmazásunk stb.



Error!

Automatikus vs Manuális

- A tesztelés lehet automatikus vagy manuális
- Manuális tesztelés:
 - Pl. Egy ember végig kattint az alkalmazáson
 - Lehet ad-hoc, forgatókönyv mentén stb.
- Automatikus tesztelés:
 - A gép végzi
 - Nagy rendszereknél szükség van automatikus tesztelésre
 - Mi most ezzel foglalkozunk

Blackbox vs Whitebox

- A programra tekinthetünk úgy, mint egy feketedoboz vagy egy fehérdoz
- Blackbox:
 - Nem látunk bele a programba a tesztelés folyamán, azt tudjuk csak vizsgálni, hogy milyen inputra milyen outputot kapunk
- Whitebox:
 - Belelátunk a programba és speciálisan úgy írjuk meg a tesztet

Teszt típusok

- Komponens tesztek
 - > Unit teszt
 - > Modul teszt
- Integrációs tesztek
- Rendszerteszt
- Átvételi teszt
- Regressziós teszt
- Biztonsági tesztek
- Terhelési tesztek
- Üzemeltetési funkciók és folyamatok tesztje

Unit teszt

- Ez egy komponensteszt, vagyis a rendszer önálló részeit vizsgálja
- A legkisebb szoftveres egységre fókuszál
- A metódusokat kezeli: kimenet-bemenetre fókuszál
 - > Jó-e a visszatérési érték

Unit teszt (példa)

```
public class PhoneValidator
{
    public bool IsValid(string phone)
    {
        return UseSomeRegExToTestPhone(phone);
    }
}
```

```
public void TestPhoneValidator()
{
    string goodPhone = "(123) 555-1212";
    string badPhone = "555 12"

    PhoneValidator validator = new PhoneValidator();

    Assert.IsTrue(validator.IsValid(goodPhone));
    Assert.IsFalse(validator.IsValid(badPhone));
}
```


Modul teszt

- Általában a nem funkcionális részeket teszteli:
 - > Memóriaszivárgás
 - > Sebesség
 - > Szinkronizáció
 - > Bottleneck

Integrációs teszt

- A komponensek közötti interfészeket teszteli
 - > A komponensek közötti kapcsolatot
 - > Az összeillesztéskor keletkező hibát
- Unit tesztek után szokott lenni
- Interfészek definiálásánál kezdődik el a tesztelés
- Hibakeresés nehéz

Rendszerteszt

- Az integrációs teszt után szokott következni
- A rendszerteszt a már kész szoftverterméket teszteli
 - > Hogy megfelel-e a követelményeknek
 - > Megfelel-e a specifikációnak
 - > Minden funkciót ellát-e
 - > Megfelel-e a rendszertervnek
- Feketedobozos teszt (blackbox)
- Van, hogy független cég végzi

Átvételi teszt

- Az átvételi folyamathoz tartozik
 - > Alpha teszt: kész termék tesztje a cégnél, de nem a fejlesztőcsapat által
 - > Béta teszt: végfelhasználók szűk csoportja végzi
 - > Felhasználói átvételi teszt: Ugyanaz, mint a béta teszt, csak sokkal több felhasználóval
 - > Üzemeltetői teszt: Rendszergazdák tesztelik a rendszert

Regressziós teszt

- Korábban megjelent hibát vagy forgatókönyvet tesztel újra
- Tipikusan valamilyen változtatás után végzik: jó-e még a rendszer?

Regressziós teszt

- Amikor fejlesztésnél írunk egy új funkciót, akkor azt szeretnénk, hogy a már meglévő funkciók ne sérüljenek
- Ezt úgy lehet megvalósítani, hogy egy új funkciót akkor fogadunk csak el, ha minden már megírt unit teszt sikeresen lefut
- Ez a regressziós teszt: minden változtatás eredménye az összes unit teszt lefutása

Funkcionális tesztek

- A szoftver működése determinisztikus
- Minél kevesebb teszttel, minél több funkciót fedjük le

További részletek

- <https://blog.prototypr.io/software-documentation-types-and-best-practices-1726ca595c7f>
- <https://www.altexsoft.com/blog/business/technical-documentation-in-software-development-types-best-practices-and-tools/>



Köszönöm a figyelmet!

Szoftvertchnológia és - technikák

11. Előadás - A vízesés és a RUP módszertan



Szoftverfejlesztési módszerek

- Egy mérnöki diszciplína, amely fókuszában a szoftver-alapú rendszerek fejlesztésének aspektusai szerepelnek.



Szoftverfejlesztési módszerek

- Vízesés
- Spirális
- Iteratív és inkrementális
- Agilis



A szerződés: megállapodás

- Mit adok – mit kapok
 - > Keretek: idő, minőség, ...

Két alap modell

1. Projekt alapú
2. Költség alapú

1. Projekt alapú

- A szerződésben rögzített témák
 - > „specifikáció”, feature set
 - > Ellenérték
 - > Egyéb paraméterek...
- Kinél van a kockázat?
 - > A fejlesztőnél:
 - > Ha a projekt mérföldköveinél szállítja ami benne van a szerződésben, kifizetik, de addig ...
- Van-e mozgástere a megrendelőnek?
 - > Nincs: ha a végén elkészül amit kért, ki kell fizetni



2. Költség alapú

- A szerződésben rögzített témák
 - > Fejlesztői/... erőforrások időarányos ára
 - > Minőségi és egyéb paraméterek
- Kinél van a kockázat?
 - > A megrendelőnél
 - > Bármilyen is lesz a projektből, a ráfordított időt ki kell fizetnie
- Van-e mozgásteret a megrendelőnek?
 - > Igen: bármit kérhet bármikor, úgyis kifizeti



Mi az alapvető probléma?

- Minden szervezet stratégiai célokban és költség keretekben gondolkodik
- Stratégia: jól definiált jövőbeni célok
- A pontos részletekre bontás nehéz
- Az elkészült projekt nem mindig juttatja el a céget a kitűzött stratégiai célhoz
 - > Az igazi probléma gyakran a megrendelő oldali folyamatokban, struktúrában stb. rejlik
- **Az informatika kiszolgáló terület!**
Alkalmazkodnunk kell!



Egy projekt „megnyerése”

- Szabályozott környezet
 - > Például előre rögzített pontszámítási módszer, ...
- Szabad környezet
 - > Gyakran kevésbé publikált elbírálási módszer
- Mindkét esetben nagy tere van a lobbinak



Leggyakoribb kiválasztási módszer

- Projekt alapú
 - Fix költségvetés
 - Fix feature set
 - Ár alapú döntés
- Kockázat a fejlesztőnél



A szabályrendszerekről

- Minden szabályrendszer motiválja a résztvevőket valamilyen irányba
 - > Lehet a szabályalkotó tudatos választása
 - > Váratlan „mellékhatások”
- Ezekkel érdemes tisztában lenni
 - > Kihasztnálni
 - > A veszélyeket időben felismerni

Mire motivál a projekt alapú szerződés?

- Fejlesztők

- Feature freeze: semmi változtatás az eredeti specifikáción, merev ellenállás, „bármilyen áron”
 - Vagy olyan változás, amit a megrendelő kifizet...

- Megrendelők

- Minél több/gazdagabb funkció fejlesztése
- Előzetes felmérés: minden belekerüljön amire szükség van/lehet
 - Költség tervezés, megtérülés vizsgálat, ...



A projekt alapú megközelítés veszélyei

- Az előzetes specifikáció (ajánlati kiírás) sosem lehet teljes
 - > Viszont ennek alapján készül az ajánlat
- A későbbi változtatások nehézkesek
- Okai:
 - > A megrendelő nem tudja, nem veszi a fáradságot
 - > Nem az írja a specet mint aki használni fogja
 - > Változó gazdasági, technológiai körülmények
 - > Az ördög mindig a részletekben rejlik

A kockázatok mérséklése

- Ajánlatba beépített puffer a fejlesztői oldalon
 - > Verseny környezetben kockázatos
- Rugalmasság, mindkét oldalon
 - > Barter megállapodások: nem módosul a teljes projekt összeg
- Lépcsőzetes kidolgozás és ajánlatok
 - > Részletes, kifizetett tervek
 - > Hátránya: lassú átfutás, sok döntési pont

Mire motiválhat a költség alapú szerződés?

- Fejlesztők

- > Hosszútávú projekt

- > A megrendelő folyamatosan kapjon értéket

- > Minél több munka, változtatások

- Megrendelők

- > Alul specifikálás: nem kell előre specifikálni, hiszen közben bármikor módosítható a funkció halmaz

A költség alapú megközelítés veszélyei

- Előzetes specifikáció/végig gondolás hiányában rengeteg, nagy költségű változtatás
 - > A változtatások gyakran architekturálisak
- Nem készül el a kívánt alap funkcionalitás sem
 - > Folyamatos változtatások
- A költségek irreálisan nagyokká válnak
 - > Különösen összevetve az elkészült terméket a rá költött összeggel

A kockázatok mérséklése

- Minden agilis módszertan nagy hangsúlyt helyez a tervezésre, közös vízióra!
 - > A termék elhelyezése a megrendelő stratégiai koncepciójában, életében
- Előzetes (nem kötelező érvényű) költség becslések

A rugalmasságról - kérdések

- „Miért nem úgy van elkészítve az alkalmazás, hogy ezt egy nap legyen beletenni?”
- „Ez miért nem paraméterezhető, miért van beledrótozva?”
- „Igaz, hogy nem kértük az ajánlatban kifejezetten, de minden minőségi informatikai projektben ezt paraméterezhetően csinálnák meg...”
- „Ez miért nem egy adatbázis mező amit mi tudunk módosítani, miért kell ehhez fejlesztés?”

A rugalmasságról - válaszok

- A rugalmasság sosincs ingyen
 - Ami igen, azt tessék belefejleszteni! 😊
- Előre nem tudni, hol lesz rá szükség!
- Az alkalmazást minden irányú bővítésre felkészíteni tipikus hiba
 - Költségek: fejlesztés és karbantartás
 - Overarchitecting, KISS/Yagni elveknek ellentmond stb.
 - Könnyen inkonzisztenciát okozhat
- Ezért nem feltétlenül hiba, ha valamit nem lehet paraméterezni pedig meg lehetett volna úgy is írni

Projekt alapú megközelítés elvárásai

- Fel lehessen használni a készen lévő „specifikációt”
- Ellenőrizhető, követhető dokumentáció
- Lehessen tervezni
 - > Tervező eszközökkel megközelíthető architektúra
 - > Esetleg kódgenerálás
 - > A terveket az implementációs fázisban ideálisan tudjuk felhasználni, ne legyen kétszeri munka
- Stabil technológiák, eszközök használata
 - > Tartani kell a határidőket

Költség alapú megközelítés elvárásai

- „Változtatás tűrés”
- Refaktorálás regresszió nélkül
- A kódban történt változtatások gyors ellenőrzése
- Jól unit tesztelhető alkalmazás
 - > A jó unit tesztek jellemzői
- Rugalmas, bővíthető architektúra

Az üzleti modell és a módszertanok

- Az üzleti modell / szerződés típus erősen befolyásolja az alkalmazott módszertant
 - > Fix költség – vízesés
 - > Költség alapú – evolúciós
- De vannak egyéb – üzleti – tényezők
 - > Például: a szállító hosszú távon tartja karban az alkalmazást, jó minőségű, olcsó javításokat, fejlesztéseket szeretne adni hosszú távon -> evolúciós módszertant választ az esetlegesen nagyobb kezdeti költségek ellenére

Összefoglalás

- Minden fejlesztési modellnek megvannak a maga erősségei és gyengeségei
 - > A kiválasztás soktényezős
- Néhány évente újabb és újabb fejlesztési módszerek jelennek meg
 - > Érdeemes követni az új megoldások fejlődését
- Mindig legyünk tisztában a környezettel amiben dolgozunk!

Vízesés modell



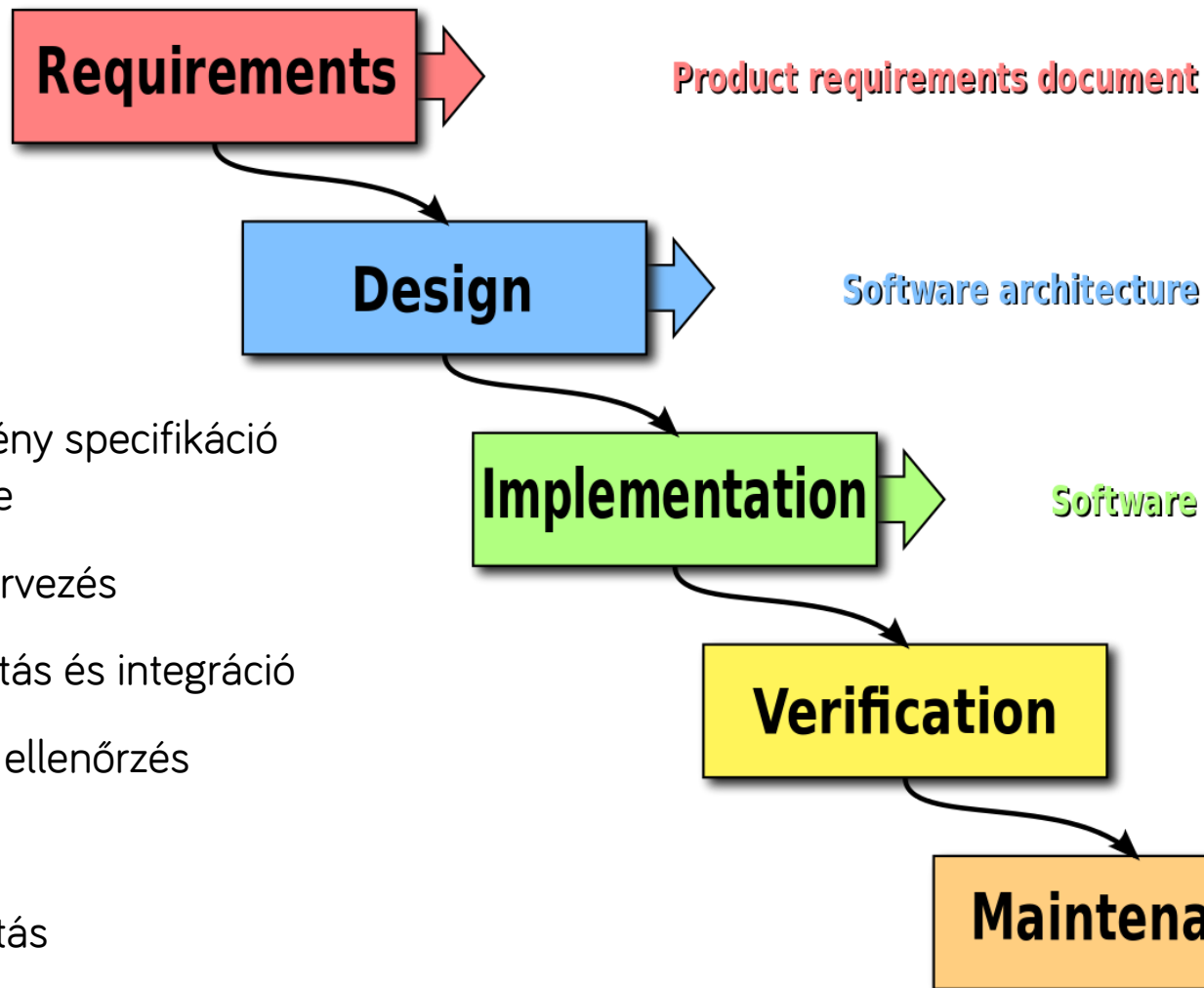
Vízesés modell

- **Példa:** A BKV megbízza a szoftverfejlesztő csapatunkat egy elektronikus jegyrendszer fejlesztésével.
- Jól rögzített határideje van:
 - az átadást 2022-re tűzik ki.
- Alphától omegáig minden ránk van bízva:
 - Mindent nekünk kell kitalálni, megvalósítani, telepíteni, átadni, karbantartani.
- A megrendelővel a projekt elején és végén találkozunk.

Vízesés modell

- Az elmúlt 50 év leggyakrabban használt modellje
- Ma is nagyon gyakori, főleg hazánkban
- Lineáris modell
 - > Egymást követik a lépések
 - > Akkor kezdünk el egy új fázist, ha az előző befejeződött

Vízesés modell lépései



1. Követelmény specifikáció elkészítése
2. Szoftvertervezés
3. Megvalósítás és integráció
4. Tesztelés, ellenőrzés
5. Telepítés
6. Karbantartás

1. lépés: Követelmények

- Részletes, közös képet alakít ki a Megrendelő és a Szállító között az elkészítendő termékről
- Akkor lépünk tovább, amikor elfogadásra került a specifikáció



1. lépés: Követelmények

- Egy specifikáció elemei:
 - > A projekt célja, helye a vállalatnál
 - > Terminológia, definíciók
 - > Felelősök
 - > Felhasználói csoportok
 - > Feltételezések, kényszerek, elő-követelmények, függőségek
 - > Funkcionális követelmények – Folyamatok, interfészek stb. tételes felsorolása és részletezése
 - > Nem funkcionális követelmények – Teljesítmény, biztonság, bővíthetőség, formátum, ...
 - > Becsült határidők
 - > Átadás / átvételi tesztek

2. lépés: Tervezés

- Konceptcionális tervek:
 - > Architektúra, felhasznált keretrendszer
 - > Interfészek, modul határok
 - > Részletes adatbázisterv
 - > Folyamatok leírása
 - > Képernyőtervek, navigációs térkép, stb.
 - > Használati minták
 - > Biztonsági, tesztelési tervek
- Prototipizálás:
 - > Iterációk a felhasználói visszajelzések alapján

2. lépés: Tervezés

- A tervezés lépése után a további módosítások már költségesek!
- A vízesés modell alapja a jó terv.
 - > Megrendelő által validálható tervek – például képernyő képek, folyamat ábrák!
 - > Megrendelői oldalon valódi ellenőrzés a munkát végzők által (kulcsfelhasználók)!
 - > Nehézséget és kockázatot jelent, ha a megrendelő közben egy transzformációban van benne.

3. lépés: Megvalósítás és Integráció

- Ebben a lépésben indul el a fejlesztés.
- Munka dekomponálása, fejlesztési tervek
- Többnyire a leghosszabb idő a teljes folyamatban

4. lépés: Tesztelés

- A minőség ellenőrzők megvizsgálják a szoftvert, hogy a specifikáció szerint működik-e?
- A hibák visszajutnak a fejlesztőkhöz és a tesztelés előlről indul
- Gyakran használunk átmeneti állapotot: béta verzió, meghívott felhasználók stb.

5. lépés: Telepítés

- Telepítés, terjesztés
- Pontosán ugyanaz a verzió menjen ki, mint amit a tesztelők jóváhagytak!

6. lépés: Karbantartás

- Karbantartási / továbbfejlesztési fázis
 - > Hibajavítás, tesztelés, ...
- A karbantartás a legdrágább fázis
 - > A hibajavítás kiemelten költséges lehet

A vízesés modell előnyei

- A megrendelők és a fejlesztők hamar (a projekt elején) megállapodnak, tisztázzák a fejlesztés fázisait.
- A projekt fejlődése könnyen mérhető és nyomonkövethető
 - > Melyik fázisban vagyunk éppen
 - > Tudjuk (jó becslésünk van arról, hogy mennyi van hátra

A vízesés modell előnyei

- Nem kell mindenkinek folyamatosan a projekten dolgoznia
 - > Pl. A tesztelőknek a tesztelés fázisában kell dolgozniuk, előbb nem
- A megrendelő (folyamatos) jelenléte nem szükséges
- Ha a tervezés jó, akkor előre becsülhető, mikorra lesz kész a fejlesztés és melyik fázis mennyi időt vesz igénybe

A vízesés modell kritikája

- Nem az készül el, amire szükség van végül
 - > Gyakran nem kapunk visszajelzéseket a termékről addig, amíg az el nem készült
- A végül megjelenő új követelmények koncepcionális változásokat indukálhatnak, ami újratervezést igényelhet
 - > Jelentőssé válhat a kidobott idő, energia
- A tervezők gyakran nincsenek tisztában az implementációs részletekkel
 - > Néha egyszerű tervváltoztatás helyett bonyolult implementáció készül el

A vízesés modell kritikája

- A tervező eszközök nem integrálódnak jól az implementációt segítő eszközökkel
- Túl hosszú lehet a teljes fejlesztési fázis, nincs részfelhasználás

Mikor használjuk?

- Fix projekt alapú a megállapodásunk
- A megrendelő csak a mérföldköveknél van jelen
- Pontos határidővel dolgozunk
 - > Például törvényi előírások
- Nem változnak a követelmények útközben

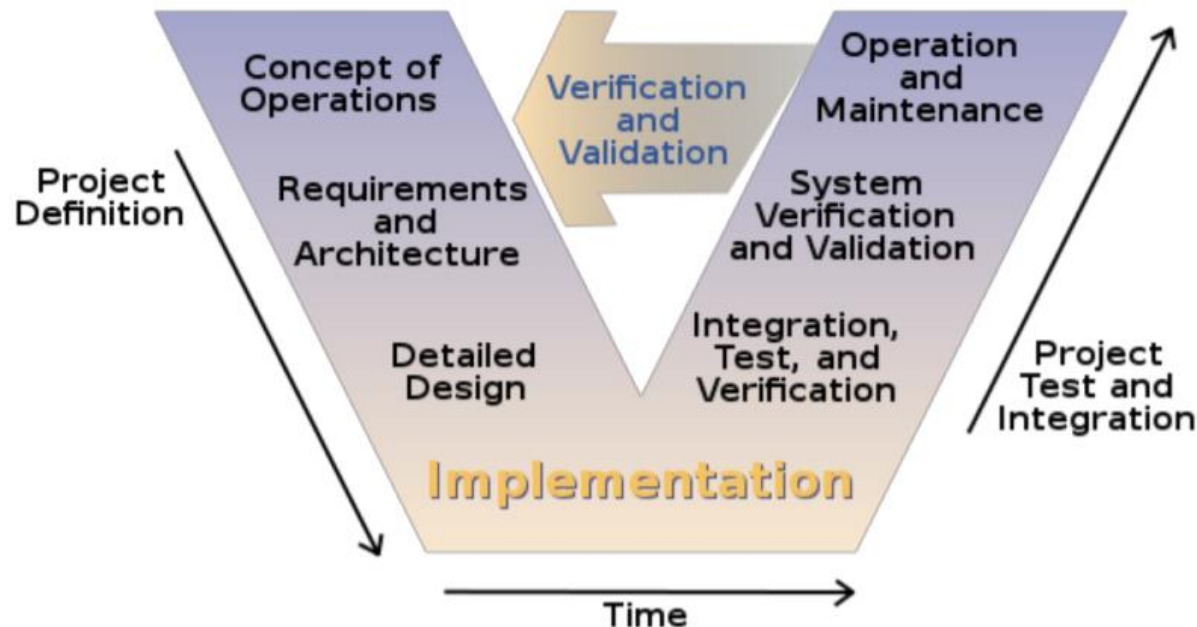
Esettanulmány (olvasmány)

- <https://medium.com/@kawomersley/would-you-rather-be-late-or-wrong-a-strategic-case-for-waterfall-development-35d81911022c>



V-modell

- Fejlesztési módszertan, a vízesés modell kiterjesztése
- Megvan a kapcsolat a fejlesztési és a tesztelési lépések között

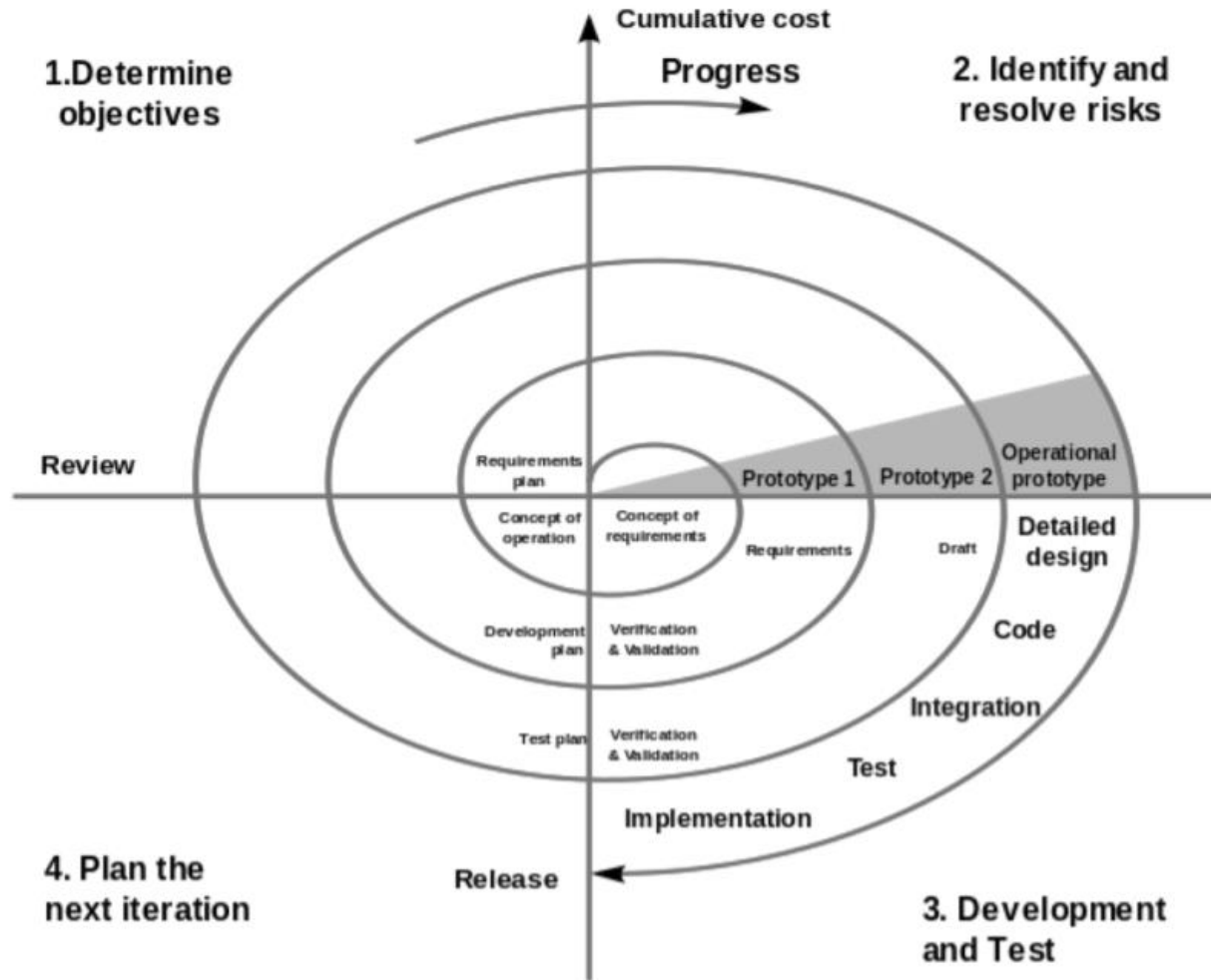


Evolúciós modellek

- A vízesés modell problémáinak elkerülése érdekében született meg egy másfajta megközelítés
- Evolúciós megközelítés
 - > Több ciklus, mindegyik a termék egy részéért felel
 - > Ez lehetővé teszi a követelmények módosítását

Jelenleg az agilis megoldások a legnépszerűbbek az evolúciós megközelítések közül

Spiral



Spiral

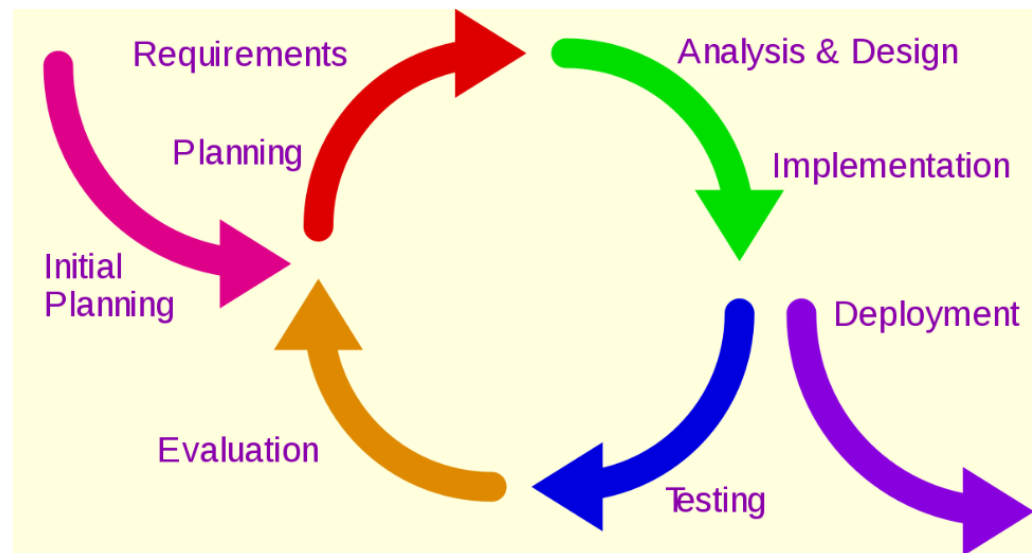
- Kulcs karakterisztika: kockázat menedzsment a fejlesztés megfelelő fázisában
- A projekt egyedi kockázatai alapján veszi át más folyamatmodellek elemeit (inkrementális, vízesés, evolúciós modellek)

Spiral

- Négy fő aktivitást ismételt ciklikusan (spirál formájában):
 1. Tervek kialakítása: célok, elvárások, megkötések
 2. Kockázatelemzés: kockázatot jelentő tényezők azonosítása és eltávolítása
 3. Implementáció: (alkalmazás)fejlesztés és verifikáció
 4. Eredmények kiértékelése és a következő iteráció tervezése

Iteratív és inkrementális

- Részegységekre osztjuk a feladatokat
- A rendszer ismétlődő ciklusok (iteratív) és egy időben kisebb részek kidolgozásával (inkrementálisan) fejlődik
 - > Minden iterációban frissül a terv és új funkciók kerülnek kidolgozásra
 - > Folyamatos finomítás alatt van a fejlesztés, egymással párhuzamosan is futnak a lépések



RUP módszertan

RUP

- IBM **R**ational **U**nified **P**rocess (RUP), 2003
- Átfogó folyamat keretrendszer
- Iparban alaposan tesztelt gyakorlatok szoftver és rendszer fejlesztésre, projektmenedzsmentre

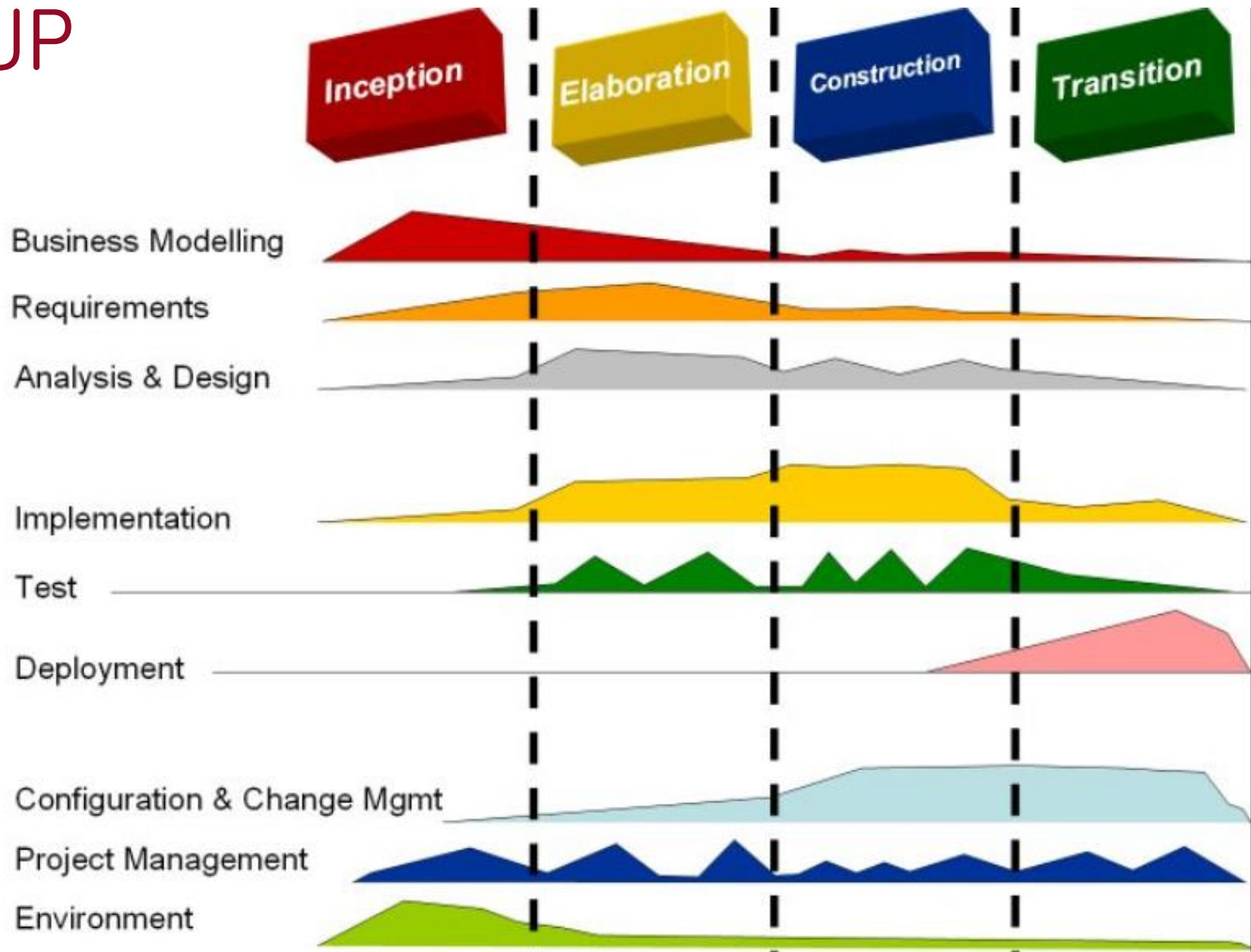
RUP

- ~ „best practice”-ek (legjobb gyakorlatok) átgondolt gyűjteménye
 - > Több ezer projekt alapján
 - > Használjuk a mások által sikerrel alkalmazott módszereket, folyamatokat
 - > Projekt sablonok (felépítés, erőforrások, mérföldkövek, leszállítandók), dokumentumok

RUP

- Iteratív módszer
- Fázisokkal dolgozik

RUP



RUP építőelemek

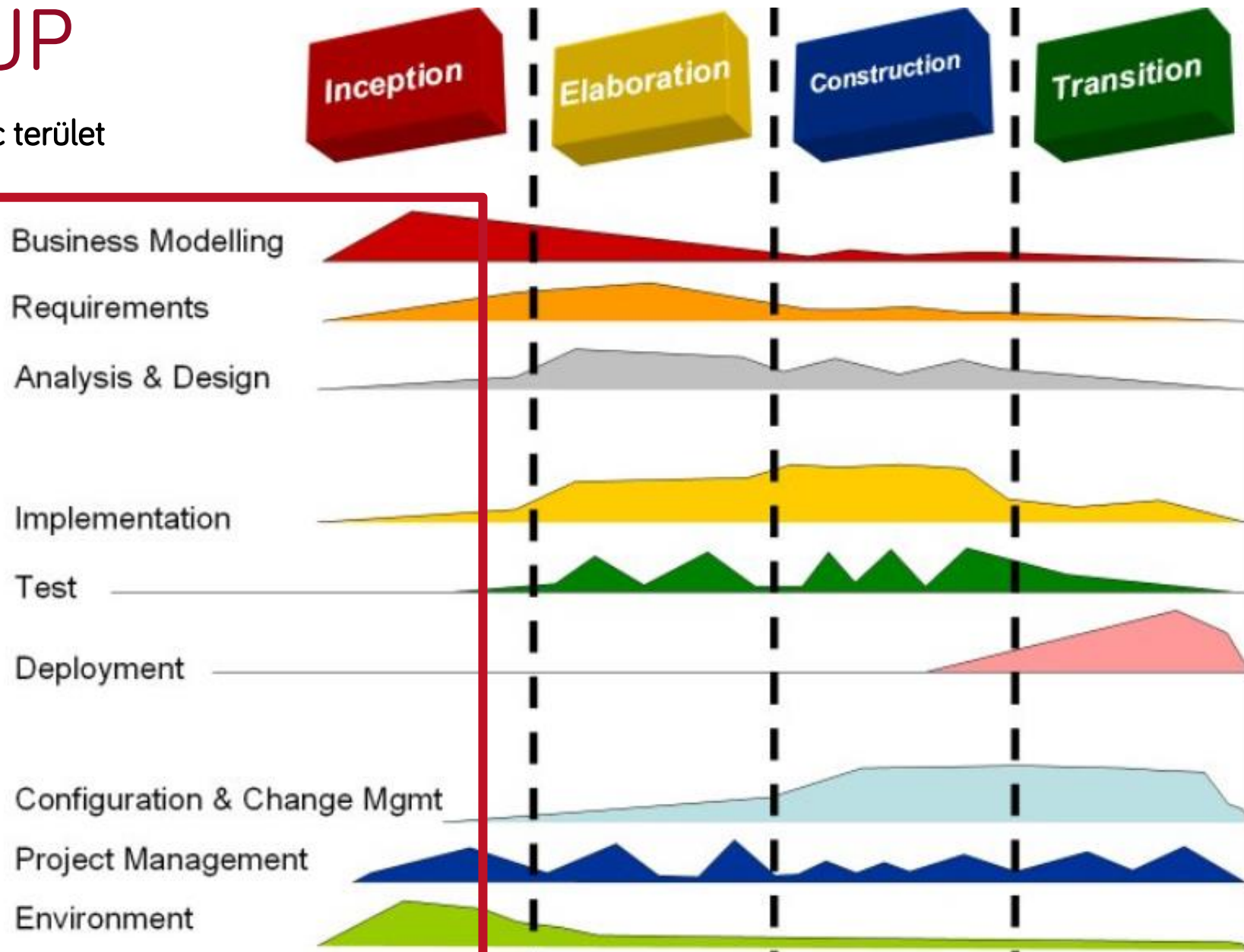
- Szerepkörök (who) – A szerepkör képességet, kompetenciát és felelősséget definiál.
- Eredménytermékek (what) – Az eredménytermék (Work Product) a feladatok eredményeit jelentik, beleértve a teljes folyamat során keletkezett modelleket és dokumentációt

RUP építőelemek

- Feladatok (how) – A feladat egy szerepkörhöz rendelt munkaegységet definiál, melynek használható eredménye van
- Minden iteráción belül a feladatok kilenc területre oszlanak fel:
- Hat „mérnöki” területre
 - > Üzleti modellezés
 - > Követelmények
 - > Elemzés és tervezés
 - > Implementáció
 - > Tesztelés
 - > Üzembe helyezés
- Három támogató területre
 - > Konfiguráció és változáskövetés
 - > Projektmenedzsment
 - > Környezet

RUP

kilenc terület

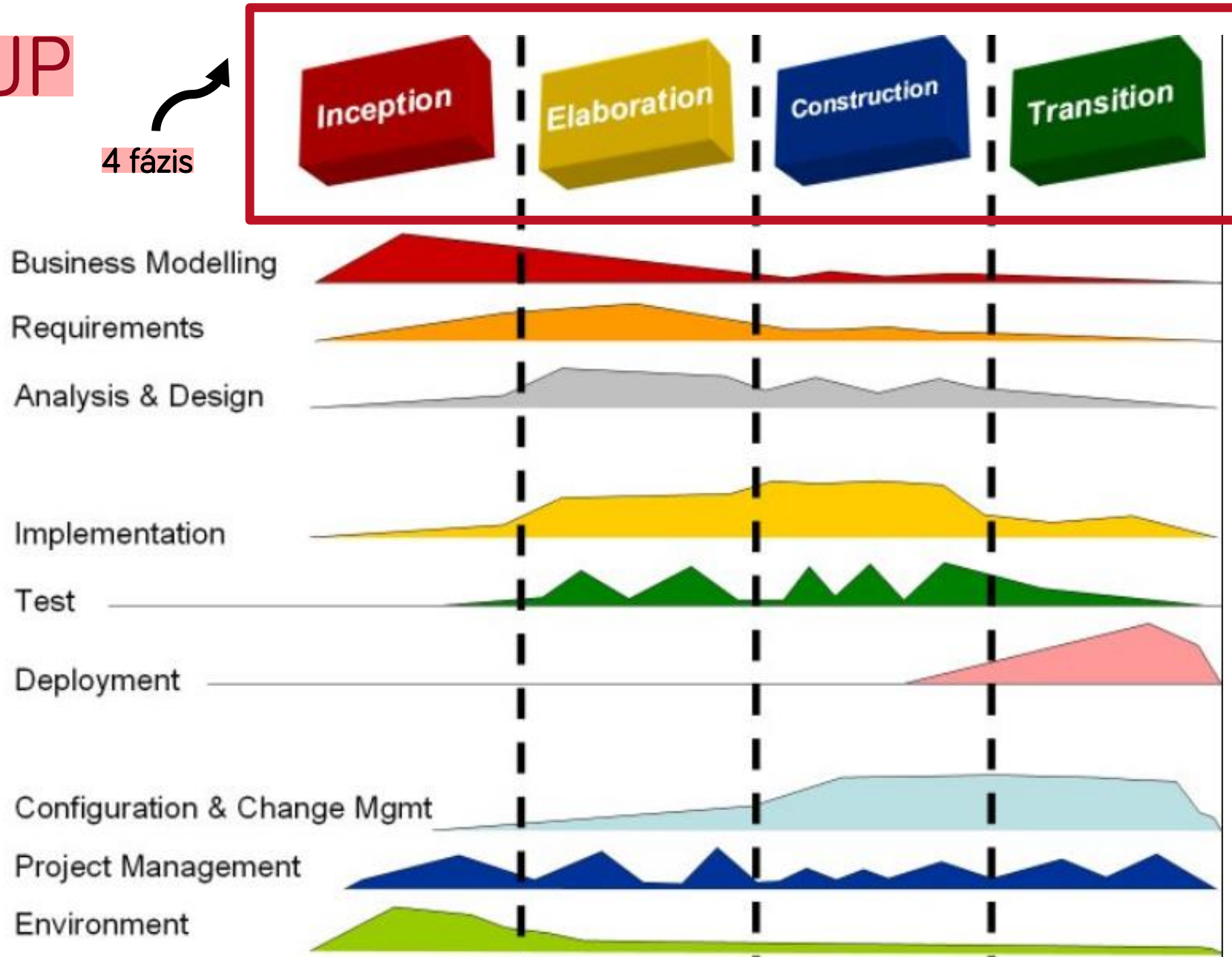


RUP – A projekt élelciklus 4 fázisa

- Az iteratív fejlesztési folyamatot négy fázisra osztja
- Minden fázis egy vagy több iterációt tartalmaz
- Minden fázis alatt minden terület megjelenik, de más mértékben
- Minden teljes új ciklust új generációnak hívnak
 - > Generációnkénti szerződés
 - > Tipikusan néhány hónap hosszúságú

RUP

4 fázis



RUP – Inception (Kezdet)

- A projektötlet kialakítása
- A csapat eldönti, hogy érdemes-e a projektet elindítani

Vagy

- A csapat eldönti, hogy érdemes-e a projektet érdemben folytatni
- A csapat megbebecsüli, hogy a projektnek milyen erőforrás igénye lesz

RUP – Elaboration (Kidolgozás)

- A projekt architektúra és a szükséges erőforrásigény kidolgozása
- A fejlesztők átgondolják a lehetséges alkalmazási területeket, valamint a fejlesztés költségeit

RUP – Construction (Megépítés)

- A projekt ebben a szakaszban kidolgozásra kerül
- A hagyományos fázisok közül itt történik:
 - > A szoftver tervezése
 - UML és egyéb mérnöki tervek, kevesebb szöveges dokumentáció
 - > A szoftver implementációja/fejlesztése
 - > A szoftver tesztelése

RUP – Transition (Átadás)

- A szoftver publikus kiadása, átadása
 - > végső simítások
 - > visszajelzések alapján történő korrekciók

A 6 best practice

1. Develop iteratively (Fejlesszünk iteratívan):
 - > A legjobb minden követelményt előre ismerni, ugyanakkor ez gyakran nem így van.
2. Manage requirements (Kezeljük a követelményeket):
 - > Mindig tartsuk szem előtt, hogy a követelményeket a felhasználók határozzák meg.

A 6 best practice

3. Use components (Használjunk komponenseket): A projekt komponensekre bontása segíti a tesztelést, újrafelhasználást, átlátást, menedzselést.
4. Model visually (Vizuálisan modellezünk): Használjunk diagramokat a fő komponensek, felhasználók, interakciók leírására (szoftver modellezés, UML, DSL).

A 6 best practice

5. Verify quality (Ellenőrizzük a minőséget):

- > A tesztelés mindig kapjon kitüntetett szerepet a projekt minden fázisában.
- > A tesztelési feladat folyamatosan nő a projekt előrehaladtával, de konstans faktornak kell lennie!

A 6 best practice

6. Control changes (Kövessük és ellenőrizzük a változásokat):

- > Számos projektet több különböző csapat fejleszt, gyakran különböző helyszínen, más-más platformok használatával.
- > Folyamatosan gondoskodjunk a változások szinkronizálásáról és verifikálásáról. (Continuous integration).

Köszönöm a figyelmet!

Szoftvertchnológia és -technikák

12. Előadás - Agilis fejlesztés, Scrum



Mik a kihívások a fejlesztésben?

- Követelmények megértése
- Követelmények változása
- Pontatlan becslések / keretek tartása
- Adminisztrációs/kommunikációs overhead
- A vízesés modell nem kezeli jól ezeket a kihívásokat, ezért születtek meg az agilis módszertanok



Agilis módszertanok

- Eleve figyelembe veszik a fenti nehézségeket
- Iteratívan, mindig a prioritásokra fókuszálunk
- Transzparencia: mindig legyen látható, hol tartunk
- Vizsgálódás: rendszeresen vizsgáljuk meg, hogy jó-e az irány
- Adaptálás: szükség esetén változtassunk



Az Agile Manifesto

- Az agilis mozgalom fő dokumentuma
- 2001-ben írta le több neves szoftverguru
- A fontosabbak:
 - > Kent Beck
 - > Alistair Cockburn
 - > Martin Fowler
 - > Robert C. Martin
 - > Ken Schwaber és Jeff Sutherland
- Értékeket és 12 irányelvet rögzítettek.
- <https://agilemanifesto.org/>

Agilis kiáltvány

- 2001. február 11-13. The Lodge at the Snowbird ski resort
- Célravezetőbb alternatívák a szoftverfejlesztés támogatására
- **Értékek:**
 - > Egyének és interakció inkább, mint folyamatok és eszközök
 - > Működő szoftver inkább, mint jól dokumentált szoftver
 - > Felhasználó bevonása inkább, mint szerződések tárgyalása
 - > Reagálás a változásokra inkább, mint egy terv követése



Az agilis fejlesztés 12 pontja

1. A legfontosabb dolog a megrendelő kielégítése úgy, hogy a kezdetektől folyamatosan értékes szoftvert szolgáltatunk.
2. Üdvözljük a változtatási kérelmeket, még a fejlesztés késői szakaszában is. Az agilis folyamatok a megrendelő versenyelőnye érdekében hasznosítják a változtatásokat.
3. Működő szoftver gyakori kiadása (pár hét – pár hónap) minél rövidebb időszakokkal.
4. Az üzleti megrendelőnek és a fejlesztőknek napi szinten együtt kell működniük.



Az agilis fejlesztés 12 pontja

5. A projekteket jól motivált egyénekre kell építeni. Adjuk meg nekik a környezetet és a támogatást, és bízunk bennük, hogy a rájuk bízott feladatot elvégzik.
6. A leghatékonyabb és leghatásosabb módja az információáramlásnak a csapaton belül és azon kívül a személyes beszélgetés.
7. A haladás mértéke a működő szoftver.
8. Az agilis folyamatok a fenntartható fejlesztést támogatják. A megrendelők, a fejlesztők és a végfelhasználók egy állandó sebességet tudnak fenntartani gyakorlatilag végtelen ideig.



Az agilis fejlesztés 12 pontja

9. A technikai tökéletességre és a jó dizájnra való törekvés növeli az agilitást.
10. Az egyszerűség – az el nem végzett munka maximalizálása – alapkövetelmény.
11. A legjobb architektúrát, követelményt és dizájnt önszerveződő csoportok tudják kitermelni.
12. Bizonyos időközönként a csapat megvitatja, hogyan lehetne hatékonyabb és ennek megfelelően hangolja a viselkedését.



Értékek - Kommunikáció

- „What matters most in team software development is communication”
- Minden probléma visszavezethető a kommunikációra
 - > Programozó – Programozó
 - > Programozó – Ügyfél
 - > Manager – Programozó
- Egy cél: minden fejlesztő ugyanúgy lássa a rendszert, ahogy a majdani felhasználók is látni fogják.



Értékek - Egyszerűség

- „What is simplest thing that could work?”
- Az agilis módszer arra fogad, hogy...
- YAGNI
- Az egyszerűség segíti a kommunikációt

Értékek – Visszacsatolás

- „Az optimizmus szakmai ártalom a programozóknál, és a visszajelzés rá a gyógyír.”
- „Elsőre jól?”
 - > Nem tudjuk hogyan
 - > Ha tudjuk, holnap már lehet a jó rossz
 - > Sok és gyors visszacsatolással tudunk közel kerülni a jó megoldáshoz.



Értékek – Bátorság

- Tenni valamit a félelem ellenére
 - > Betartani a YAGNI-t
 - > Refaktorálni
 - > Revertálni
- Vagy nem tenni
- Főleg a többi értéket támogatja
 - > Kimondani jót és rosszat
 - > Eldobni a rossz megoldásokat
 - > Igazi konkrét válaszokat keresni

Értékek – Respect

- Ha nem törődünk egymással, akkor...
- Ha nem törődünk a projekttel, akkor...
- „I am important and so are you”

Elvek

- Humanity
 - > Business and personal needs
- Economics
 - > Somebody has to pay for all this.
- Mutual Benefit
 - > Every activity should benefit all concerned
- Self-Similarity
 - > Copying a structure of a solution to new context
- Improvement
 - > In software development „perfect” is a verb, not an adjective
- Diversity
 - > Conflict comes with diversity
- Reflection
 - > Good teams don't just do their work, they think about “how” and “why”
- Flow
 - > Steady flow of valuable sw
- Opportunity
 - > Problems as opportunity for change
- Redundancy
 - > Redundancy of practices
- Failure
 - > If you can't succeed, fail
- Quality
 - > Quality is not a control variable
- Baby Steps
 - > Many small steps rapidly looks like a leap
- Accepted Responsibility
 - > Responsibility cannot be assigned

Scrum

- Egy konkrét agilis keretrendszer
- Előír szerepköröket és eseményeket, de további technikák is belevehetők, amelyek nem mondanak ellent az alapvetéseinek
- Időkeretes eseményekkel dolgozik, maximális időt ad meg, ez alatt be kell fejeződniük

Követelmények megértése

- Sok résztvevő
 - > Fejlesztő: fejlesztéshez ért, az üzleti területhez nem
 - > Megrendelő: üzletileg érdekelt a szoftverben, de nem feltétlen ért se a szoftverhez, se az üzleti területhez
 - > Domain expert, felhasználó: elvileg ért a doménhez, talán tudja, mi segítené a munkáját, de nem mindig kérdezik meg elégszer, illetve nem is feltétlen tudja elmondani
 - > Megrendelő $\neq$ felhasználó: sokszor nem azonosak, pl. kórházigazgató és ápolónő

Követelmények megértése

- Rossz irányba ment fejlesztés
 - > Frusztráció a fejlesztőknek
 - > Pazarlás és időbeli csúszás a megrendelőnek
 - > Üzleti bukás
- Iteratív fejlesztés
 - > Gyakran validálni: „ez az, amit szeretnél volna?”
 - > Nem kerüli el a félreértéseket, de kordában tarthatóak
 - > Más megközelítést igényel (nem lehet rétegenként haladni, horizontálisan kell felosztani a feladatot)

A követelmények változása

- Még ha meg is értjük egymást, sok minden változhat
 - > Jogi környezet (ld. Uber, AirBnb)
 - > Piaci verseny helyzete
 - > Technológiai fejlődés
- Megrendelőnek kell tudnia, hogy versenyképes-e, amit szeretne
- Nekünk felkészülni az esetleges változásokra
 - > Lehet holnap már mást kér tőlünk
- Erre is az **iteratív** fejlesztés a megoldás

Verifikáció és Validáció

- Verifikáció: a specifikációval összhangban implementálom-e a rendszert?
 - > Jól építem-e meg?
 - > Klasszikusan ezt teszteljük
- Validáció: az üzleti igényeket kielégítő rendszert építek-e?
 - > A jó rendszert építem meg?
 - > Jól lettek definiálva a követelmények?
 - > Felhasználó előtti demóval vagy speciális tesztekkel végezzük

Pontatlan becslések

- A szoftverfejlesztés kreatív alkotómunka
 - > Szobafestés vs. költészet
- Tapasztalat kell a becsléshez
- Így is lehetnek nem várt kihívások, érdemes ráhagyással
- Elnagyolt, változó, félreértett követelmények tovább nehezítik a helyzetet

Pontatlan becslések

- Nehezen tartható keretek!
- Következmények
 - > Több idő = több költség
 - > Ha kifutunk az időből, nem biztos, hogy továbbra is lesznek anyagi erőforrások a fejlesztés fedezésére
 - > Csúszással nemcsak többet kell a megrendelőnek költenie, hanem a korábbi piacra lépésből realizálható haszontól is elesik

Priorizálás

- A fenti problémák priorizálással mérsékelhetők
- Mindig a legfontosabb elemeken dolgozunk
 - > Ami elkészül, az legyen jó és használható...
- Ezek minél előbb elkészülnek, ha valamire végül nem jut pénz/idő, az kisebb jelentőségű dolog
- Kevésbé kerül veszélybe a piacra lépés
- Hogyan?
 - > Azt teljesítsük, amit kérnek, ne próbáljuk túlteljesíteni
 - > Gondolkodjunk előre, de ne túlzottan; inkább hagyjuk nyitva a bővítés lehetőségét, de ne tegyünk sok energiát túl előre tervezésre

A Scrum csapat

- Product Owner (PO): megrendelőt képviseli
 - > Átmenet üzleti és szoftveres világ közt
 - > Érti a követelményeket és közvetíti a fejlesztőknek
 - > Csapat része
- Scrum Master (SM): a Scrum elvek betartását segíti
 - > Nem klasszikus PM
 - > Nem a csapat vezetője
 - > A Scrumban képzett
- Development Team: a fejlesztők szigorú szerepkörök nélkül

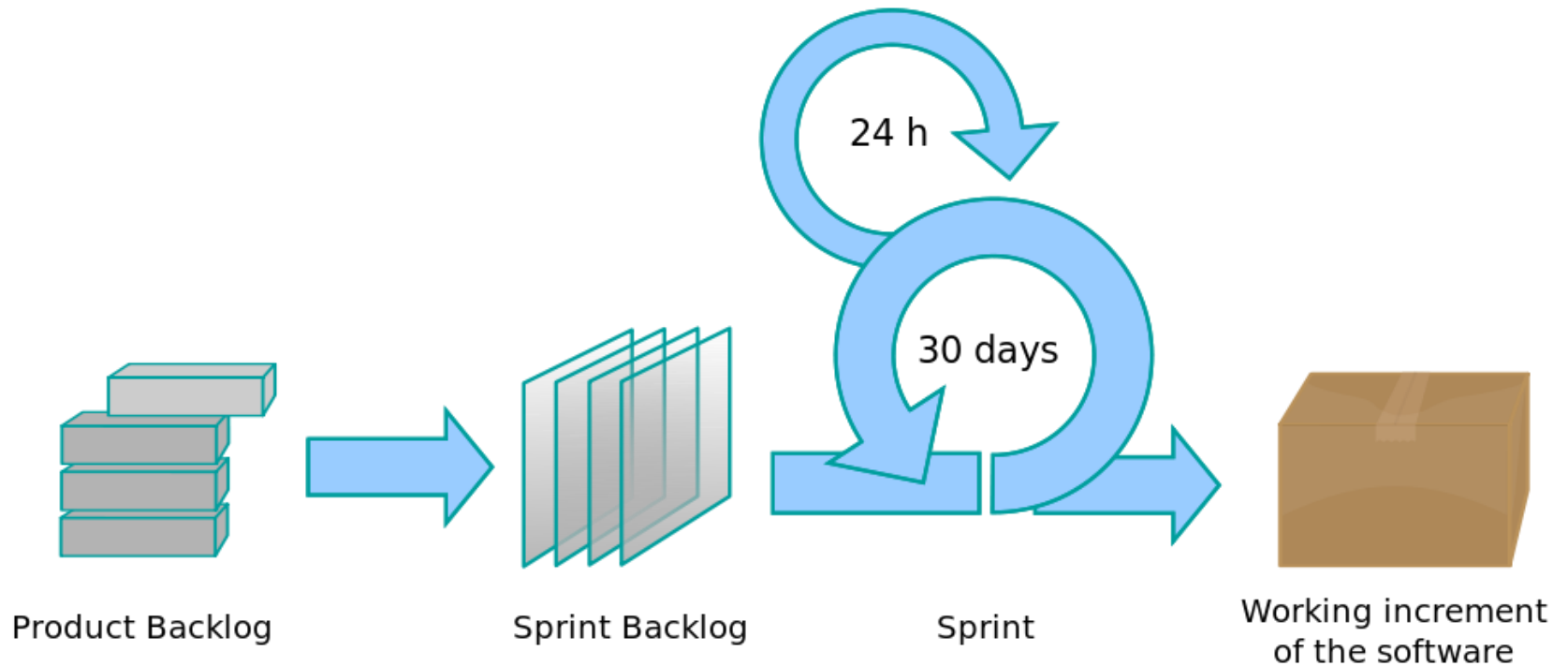
Hajó analógia

- PO: kapitány, kijelöli az irányt
- SM: motivál, koordinál, segít kikerülni az akadályokat
- DT: evezősök
- <https://www.advancedproductdelivery.com/blog/2017/06/20/roles-in-scrum/>

Önszerveződő csapat

- A DT csapat önszerveződő
- Ők tudják a legjobban, hogyan tudnak hatékonyan dolgozni
- Minden taszkért közösen felelnek
- SM, PO nem szól bele

Scrum folyamat



A Scrum események

- Spring Planning (≤ 8 óra)
- Sprint (≤ 1 hó)
- Daily Scrum (≤ 15 perc)
- Sprint Review (≤ 4 óra)
- Sprint Retrospective (≤ 3 óra)

Projektindítás

- Első sprint előtti tervezés
- Product Vision: nagyon magas szintű vízió arról, hogy mi fog készülni
 - > „I would have written you a shorter letter, but I didn't have the time.” (Blaise Pascal)
 - > **Üzleti célok: milyen értéket képvisel a megrendelőnek**
 - > (Gyakorlati célok: fejlesztőként mit kell megcsinálnunk)
- NOT list: egyelőre nem kell vagy bizonytalan
 - > Ez is a fókuszálást, priorizálást szolgálja

Projektindítás

- Definition of Done: fektessük le előre, hogy mikor tekintünk valamit elkészültnek?
 - > Implementáltam
 - > Lefordul
 - > Tesztek futnak
 - > Gitre pusholtam
 - > Többiek átnézték
 - > Többi komponenssel jól integrálódik
 - > Master branchbe merge-elve lett
 - > PO elfogadta
 - > Stb.

Product Backlog

- Definiálni kell az elkészítendő user story-kat
- Hagyományos specifikáció helyett ezeket használjuk
- Prioritás szerint csökkenő sorrendben, ez a backlog
- Minden sprint elején a tetejéről veszünk le annyit, amennyit a sprint alatt elkészítünk → Sprint Backlog

User story

- A követelmények megfogalmazásának eszköze, egy kicsi, tovább nem bontható feature
 - Adott formátuma van
 - > Cím
 - > Leírás
 - > Elfogadási kritériumok
 - > NOT lista
 - > Nemtriviális tesztesetek
 - > Alfeladatok
 - > Story pontok
 - > Becsült idő
 - > Elköltött idő
-
- PO írja meg
- Fejlesztőkre bízva

User story leírása

As [ROLE], I was to achieve [GOAL] for a [REASON]

- Három fő dolog mindig benne van
 - > **Role**: ki számára értékes
 - > **Goal**: funkcionálisan mit tudjon a szoftver (gyakorlati cél)
 - > **Reason**: miért képvisel ez értéket (üzleti cél)

Elfogadási kritériumok

- Nem technológiai, hanem felhasználóközpontú
- De objektíven mérhető kritérium, pl.
 - > A formon szerepel egy név mező
 - > Ebben csak betűk szerepelhetnek, különben hibaüzenet jelenik meg
 - > Van alatta egy mentés gomb
 - > A gombon való kattintás elmenti az adatot az adatbázisban

NOT Lista

- Ami jelenleg nem a scope része a user story-ban
 - > Lehet későbbi user story része
 - > Vagy még nem eldöntött, hogy szükséges-e
- Pl.:
 - > A bemenetet még nem validáljuk
 - > A jelszóemlékeztető funkciónak még nem kell működnie

Nemtriviális tesztesetek

- Emlékeztető, hogy milyen speciális eseteket kellene vizsgálni
- Pl.:
 - > Határidő nem lehet jövőbeli dátum
 - > Kategória nélkül nem menthető el termék

Alfeladatok

- A megrendelőnek lényegtelen
- Az önszerveződő agilis csapat használja
 - > Eldöntik, hogy lehet lépésekre bontani a feladatot,...
 - > ... és felosztják a lépéseket maguk közt
- Teljesen a csapatra van bízva
 - > Lehet alfeladatokat kiosztani, de teljes story-kat is
- Segíti a becslést

User Story példák

- The accounting software must run on a dedicated server
- As an administrator, I want the accounting software not to put extra load of critical systems
- Accounting data must be visualized fast
- As an end-user, I want the data of clients to be accessible in 1 second

A sprintek menete

- Ha a kezdeti tervezés kész, sprinteket kell terveznünk
- A sprintek menete
 1. Sprint planning (≤ 8 óra)
 2. Tényleges fejlesztés napi munkával
 3. Sprint review (≤ 4 óra)
 4. Retrospektív (≤ 3 óra)

Sprint tervezés

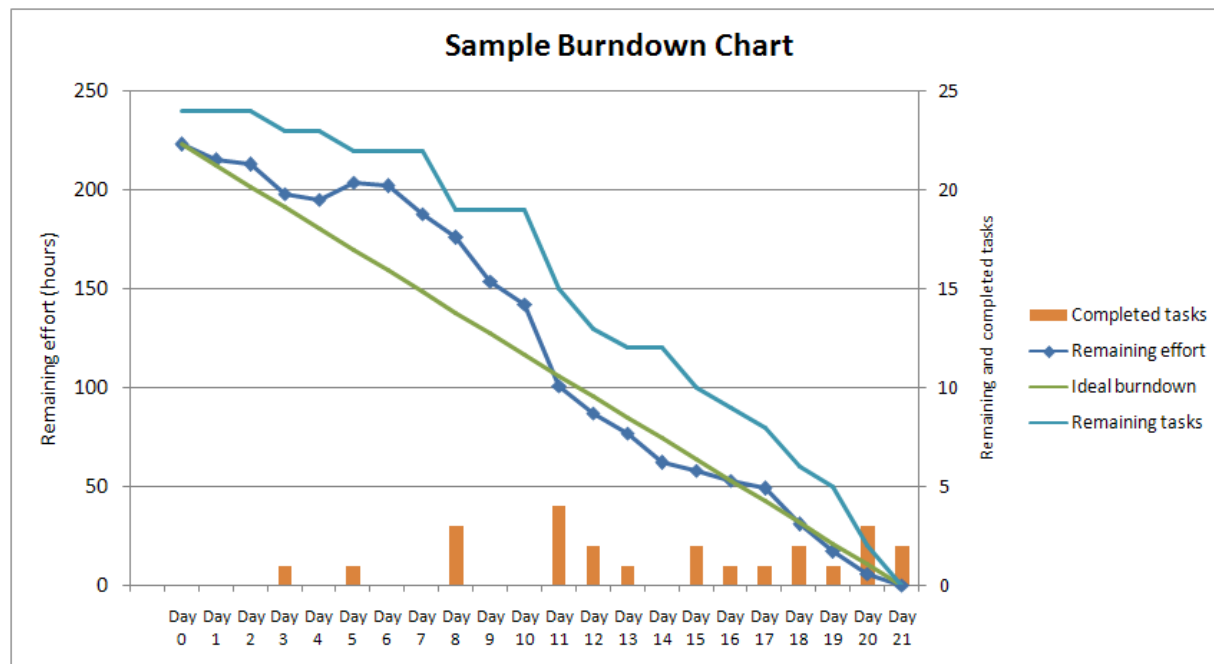
- Párórás **meeting** a sprint elején
- A sprint tipikusan 2-4 hét, erre tervezünk, ideális napokkal és hetekkel (napi 8 óra, heti 5 nap)
 1. A user story-kon megyünk prioritási sorban
 2. Beazonosítjuk az alfeladatokat
 3. Hozzárendelünk egy story pointot
 4. Adunk egy időbecslést
 5. Amikor elég story-t „beáraztunk”, vállalást teszünk

Story Points

- A user story-kat relatív nehézségi sorrend szerint rendezzük, és a közepes ér 5 pontot
- A közepes nehézségre adunk becslést
 - > A többi ehhez relatívan becsüljük, pl. kétszer olyan nehéz, dupla idő
 - > Korábbi tapasztalat segít a becsléseinket egyre pontosabbá tenni
 - > Átlagos story point / sprint teljesítmény kalkulálható
 - > Ez segít a hosszabb távú becslésben

Scrum artifacts

- **Product Backlog:** Termék teendőlistája prioritás szerint rendezve
- **Sprint Backlog:** Sprint teendőlistája
- Burn down chart: napi eredmény-kimutatás



Use Case vs. User Story

- Hasonlóság
 - > Szerep/aktor, cél, elfogadási kritériumok
- Különbségek
 - > Use case általában részletesebb
 - > User story inkább beszélgetés indító
- “A story card is a promise for a conversation.”
- User story ~ feature – eredmény központú
 - > „a marker for what is to be built, fine-grained enough to fit into modern iteration/sprint periods” (A. C.)
- Use case: hogyan reagál/mit csinál a rendszer
 - > a contextual view of what is to be built

Daily Scrum

- Napi szinten is tervezünk a nap elején
- Max. 15 perc, **ténylegesen felállva!**
 - > Nem fotelben elterülve
 - > Nem laptopba bújva
- Mindenki megválaszolja:
 - > Mit végeztem el tegnap?
 - > Mit fogok ma csinálni?
 - > Van-e valami, ami akadályozza a munkámat?

A sprint terv módosulása

- Scope levágható
- Az időbeli hosszt sosem változtatjuk!
- Nincs félkész user story
 - > Kettébontás
 - > Rollback és a következő sprintre marad
- Sürgős user story bejöhet, PO dönti el
- Sprint csak akkor szakítható meg, ha a célja elavulttá vált

“Kész, kész” (done, done)

- Megtervezett
- Lekódolt
- Integrált
- Tesztelt
- A build lefut
- A termék installálódik
- Migrálódik
- A végfelhasználó ellenőrizte és elfogadta
- Megfixált

Demo

- Annak áttekintése, hogy mely munkák készültek el és melyek nem.
- Az elkészült munka bemutatása a terméktulajdonos és a fejlesztésben érdekeltek részére (demo).

Retrospektív

- Cél a munkafolyamataink folyamatos javítása
 - > Mit csináltunk jól, mit kell megtartani?
 - > Mit kéne másképp próbálnunk?
 - > Nem bűnbakok kereséséről szól!
- Kimenet: **végrehajtható akciók**
 - > Legalább egyet fel kell venni a következő sprint backlogjába
 - > **Teszteljünk többet**
 - > Csak getter, setter és konstruktor maradhat teszteletlen

Mit csinál pontosan a SM?

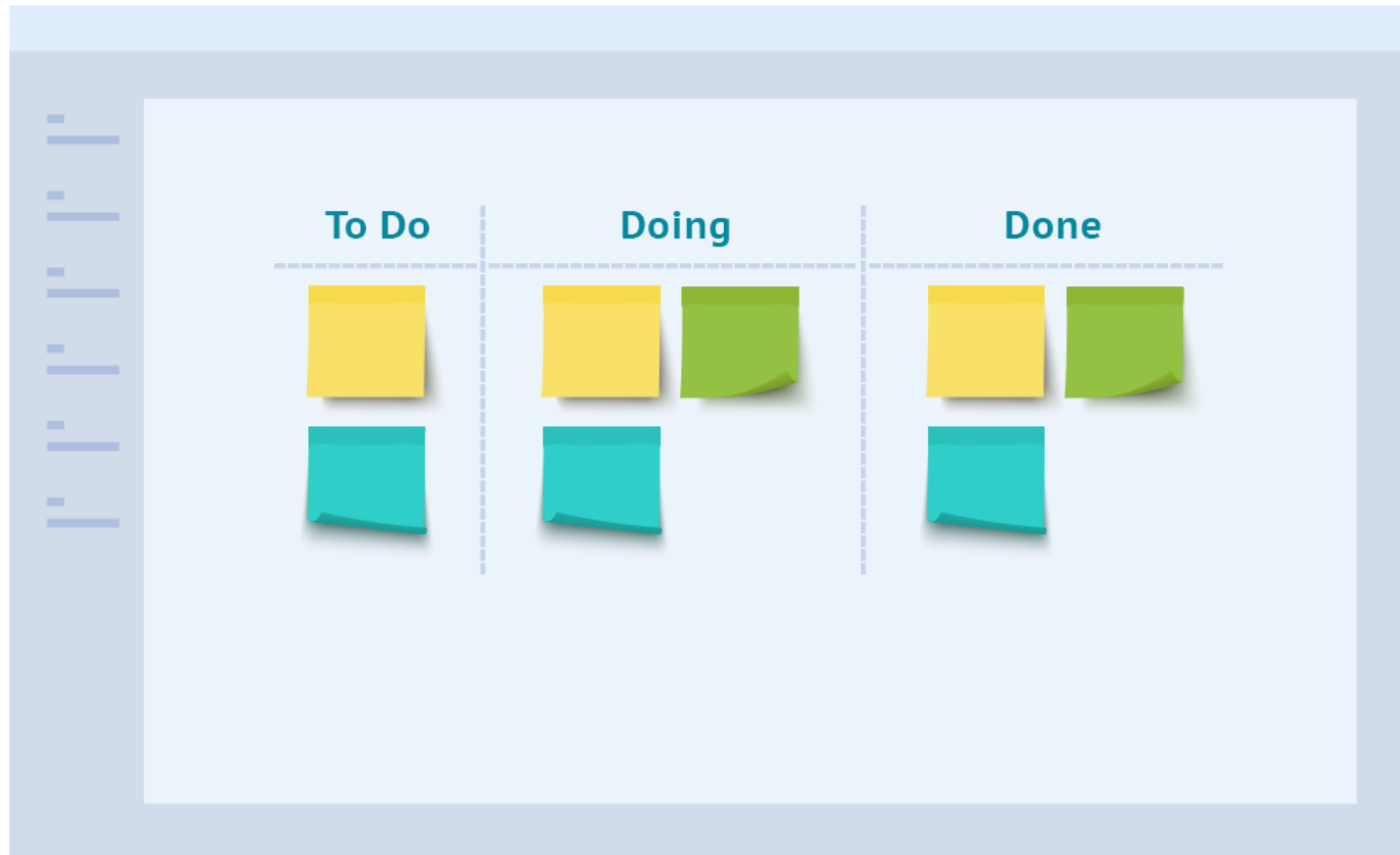
- „Szolgálva vezető”
- Facilitátor
- Coach
- Menedzser
- Mentor
- Oktató
- Problémamegoldó
- Változást segítő

<https://www.scrum.org/resources/blog/8-stances-scrum-master>

Kanban tábla

- Nem a Scrum része, külön technika, de szokták vele használni
- Fizikai tábla oszlopokkal, a sprint alatt a user story-kat és esetleg más taszkokat ezen vezetjük
- Az állapotokat a csapat dönti el, tipikusan a Definition of Done lépései alapján

Kanban



Az eXtreme Programming

- A Scrum mellett egy másik agilis módszer
- Nagy különbség nincs
- Az XP több gyakorlatot tesz kötelezővé
 - > Pair Programming
 - > Test-Driven Development (TDD)
 - > Continuous Integration (CI)
 - > Collective Code Ownership

Tényleg működik ez?

CHAOS RESOLUTION BY AGILE VERSUS WATERFALL

SIZE	METHOD	SUCCESSFUL	CHALLENGED	FAILED
All Size Projects	Agile	39%	52%	9%
	Waterfall	11%	60%	29%
Large Size Projects	Agile	18%	59%	23%
	Waterfall	3%	55%	42%
Medium Size Projects	Agile	27%	62%	11%
	Waterfall	7%	68%	25%
Small Size Projects	Agile	58%	38%	4%
	Waterfall	44%	45%	11%

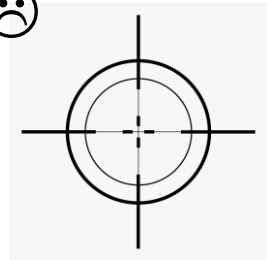
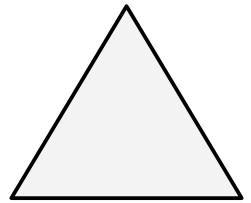
The resolution of all software projects from FY2011-2015 within the new CHAOS database, segmented by the agile process and waterfall method. The total number of software projects is over 10,000

CHAOS konklúzió

- Korábban a sikeres projekt:
 - > büdzsén belül
 - > időben
 - > Tervezett funkcionalitás
- “We found that both **satisfaction** and **value** are greater when the features and functions delivered are much less than originally specified and only meet obvious needs.”
 - 2015 Chaos Report, Standish Group

Összevetés

- Mindegyik módszertannak megvan a maga erőssége, ki van valamire „hegyezve”!
 - > Egyiket sem „könnyű” jól csinálni
 - > Máshol a kockázat... - más a
- Vízesés:
 - > Előre rögzített keretek tartása, tervezés, beavatkozás 😊
 - > Nem biztos, hogy az készül el, amire szükség van ☹️
- Agilis
 - > Biztos, hogy olyan készül el, amire szükség van 😊
 - > Előre nem tudjuk biztosan, hogy mikorra, mennyiért, és azt sem, hogy pontosan mi lesz az ☹️



Köszönöm a figyelmet!

Szoftvertechnológia és -technikák

13. Előadás - DevOps, Continuous integration, Continuous delivery



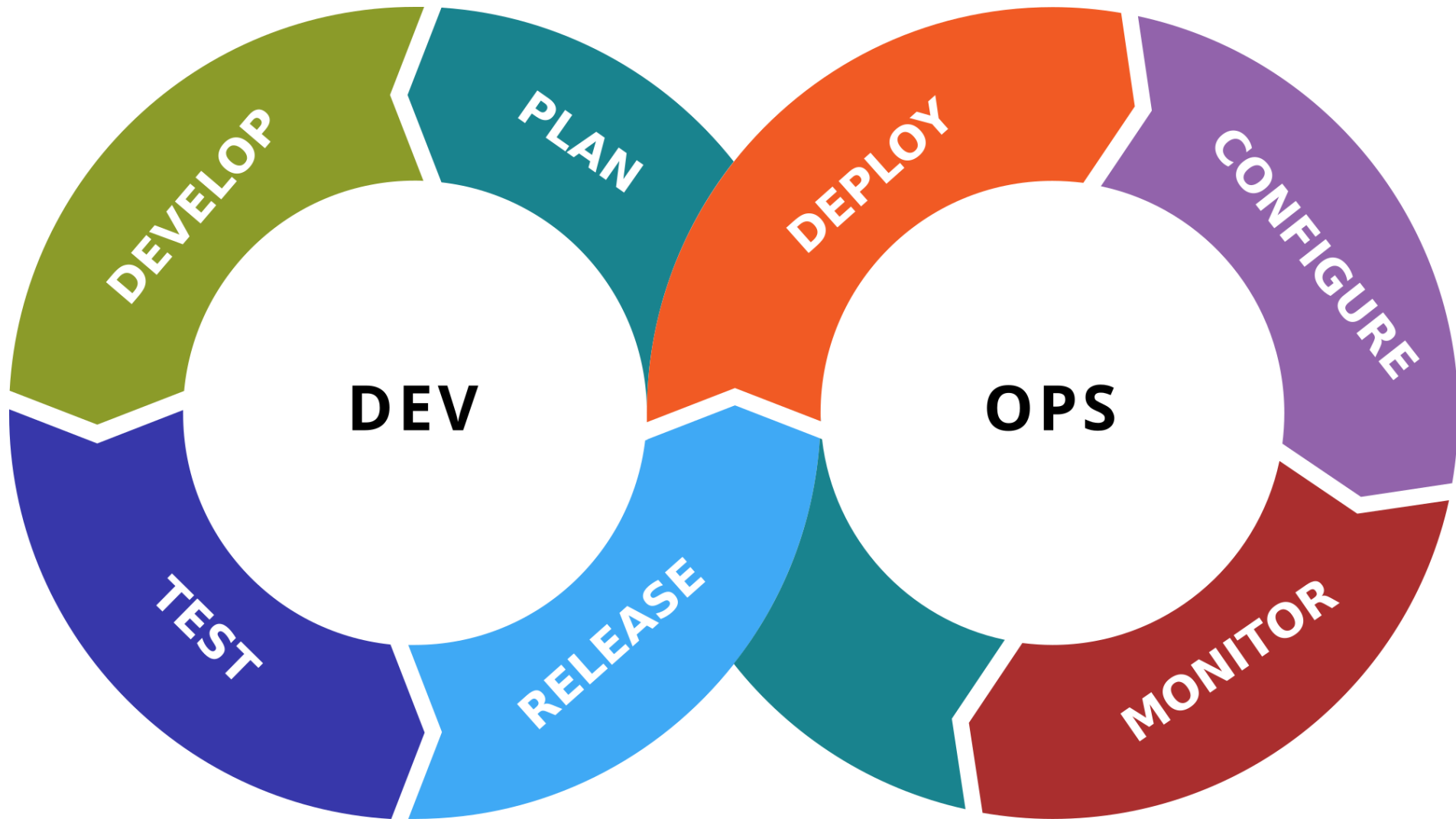
DevOps

Bevezetés

- A szoftver élettartama nem ér véget a fejlesztésnél
- A szoftverfejlesztés csapatjáték
- Hagyományosan a fejlesztők és az üzemeltetők külön csapatokat alkottak
 - > ez a szeparáció csökkenti a kommunikáció hatékonyságát, lassítja az új verziók megjelenését
- Az agilis világban már sokszor nem elegendő



Megoldás



DevOps: Áttekintés

- Software development-ből (Dev) és information-technology operations-ből (Ops) áll
- DevOps egy fejlesztői módszertanok gyűjteménye
- Célja a szoftver fejlesztésének és üzemeltetésének összehozása



DevOps: módszertanok

- Continuous Integration
- Continuous Delivery
- Infrastructure as Code
 - > Pl: Kubernetes
- Monitoring and Logging
- Microservices
- Communication and Collaboration



Continuous Integration

Bevezetés

- Napjaink fontos kérdése az automatizálhatóság
- A szoftverfejlesztés célja sokszor a költséges emberi erőforrás kiváltása
 - > A szoftverfejlesztés emberi erőforrásának kiváltása is
- Nem csupán költségcsökkentés a cél, hanem az esetleges hibalehetőségek csökkentése is



CI elemek

- *Continuous Integration* szerepe és lehetőségei
- *Continuous Integration* eszközök (*Jenkins*)
- *Jenkins* képességek, *script*-elhetőség, *artifact*-ok
- *Lint* v. *Sonar* kódelemző
- Saját monitoring „*Dashboard*”
- *Continuous Delivery*

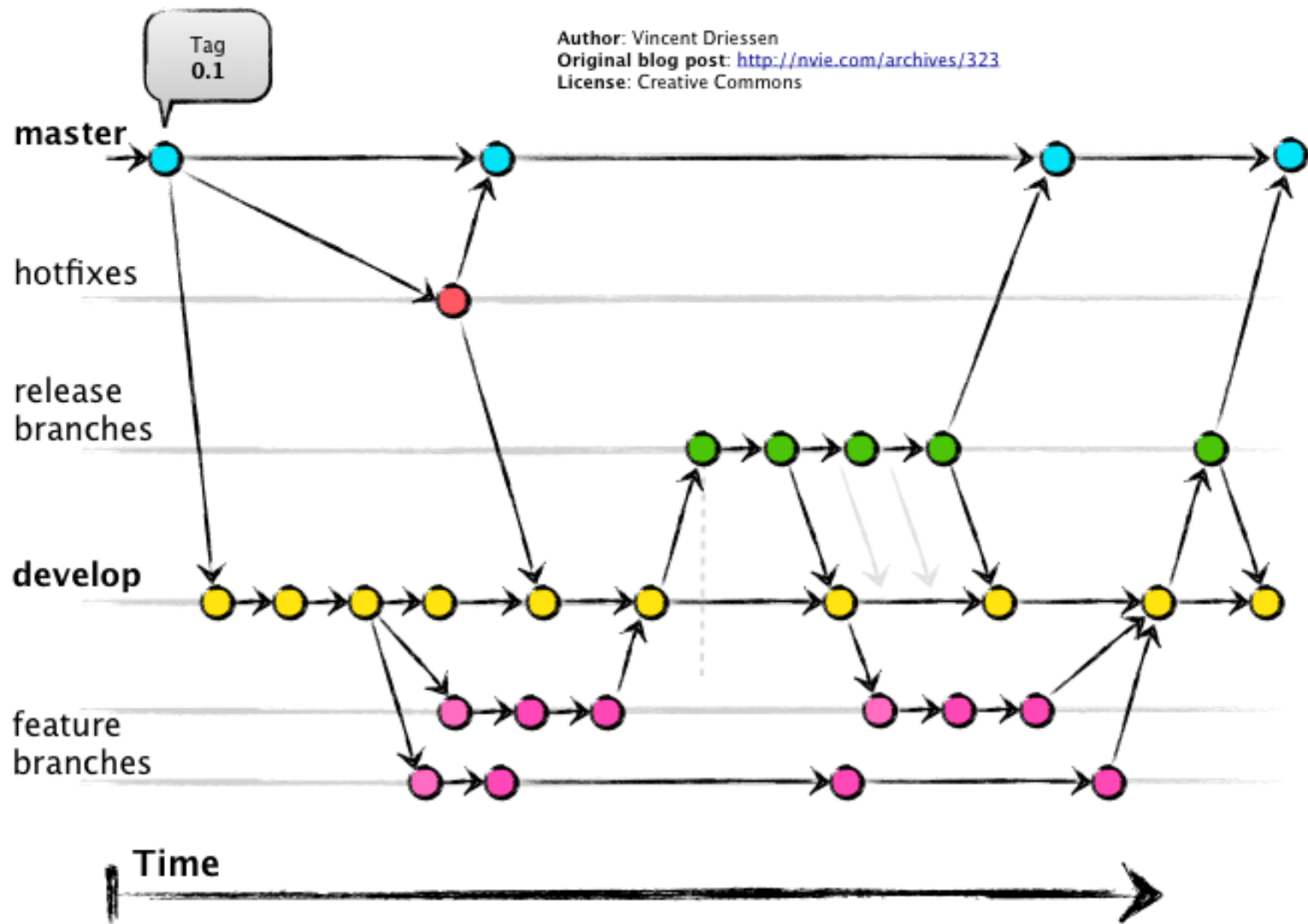


Fejlesztő feladatai

- (*meeting*-eken való részvétel)
- Tesztek írása (specifikáció megfogalmazása programkóddal)
- Funkciók implementálása (specifikáció megvalósítása programkóddal)
- Tesztek futtatása
- Program fordítása
- Telepítések
- Értesítés telepítésekről



Egy alkalmazás időbeni alakulása



Egy alkalmazás verziói időben

- Mergelés előtt célszerű megbizonyosodni, hogy mergelés után az alkalmazás helyesen, specifikáció szerint fog működni
 - Ezeket az ellenőrzéseket lehet kézzel minden merge esetén elvégezni, de célszerű lenne automatizálni
- Ennél jobb ha nem csak merge-enként, hanem push-onként is megbizonyosodunk az alkalmazás helyességéről
 - Kézzel ez sok, ráadásul repetitív munka, ami előbb-utóbb el fog maradni

További problémák

- A fejlesztő ismeri egyedül a folyamatot
 - Fluktuáció esetén a tudás elveszik, átadása másik ember felé költséges és veszteséges lehet
- A folyamat minden egyes eleméhez szaktudás szükséges
 - Fejlesztői (rendszergazdai) ismeretek megléte
- Ha éppen nem elérhető a fejlesztő, nem lesz új *release*
- A fejlesztői (rendszergazdai) erőforrás költséges



Megoldás

Continuous Integration Bevezetése



Mit jelent valójában?

- A rendszerünk, jellemzően naponta 1-2-szer hajtja végre a teljes folyamatot
 - > Integrált
 - Minden változtatás, művelet egy láncként hajtódik végre
 - > Lefordított
 - A kód lefordul és termék keletkezik (például futtatható állomány)
 - > Tesztelt
 - Automatizált tesztek lefutnak
 - > Archivált
 - Verziózott és tárolt, szükség esetén felhasználható
 - > Élesített
 - Feltöltődik abba a célkörnyezetbe, ahol a fejlesztők használhatják



Mikor működik jól?

- Automatizált *build* folyamat
 - > Külön gépen, tehát nem veszi el a fejlesztői gép erőforrását
 - > A fejlesztő tud fejleszteni, míg a folyamat zajlik
- Tegyük a *build*-et öntesztelővé
 - > A sikeres fordítás után fussanak le a tesztek
 - Biztosítja, hogy a szoftver úgy viselkedik, ahogy a fejlesztő elvárja
- Mindenki commitoljon sűrűn
 - > Naponta párszor (1-2)
 - Így hamar kiderül, ha valami nem működik



Mikor működik jól?

- A *build* művelet legyen gyors
 - > Ha nem gyors, nem segíti a munkánk
 - Ha hiba van az integrációban, könnyen azonosíthatjuk
- Független gépen tesztelés
 - > „Nálam megy” hiba elkerülése
 - > Hamar detektálhatjuk, ha a fejlesztői környezetem eltér a tesztkörnyezettől
- Tegyük könnyen elérhetővé a legutóbbi stabil *build*-ek eredményeit
 - > Ezáltal a mindenkori legfrissebb stabil verzió felhasználhatóvá válik
 - > Akár a menedzser is felhasználhatja



Mikor működik jól?

- Mindenki láthatja a *build*-ek eredményeit
 - > Könnyen felderíthető, ha egy *build* sikertelen volt és az is, hogy mi okozta a hibát
- Automatikus telepítés
 - > A legtöbb CI rendszer a engedélyezi szkriptek futtatását
 - A sikeres *build* folyamat után
 - > Éles rendszerekbe célszerűen nem automatizáltan telepítünk
 - Rengeteg commit, nem mindig kész release
 - Vannak ellenpéldák is



Használatának előnyei

- Integrációs *bug*-ok hamar kiderülnek
 - > Gyors visszajelzés, a projekt nem kerül rossz útra (sokáig)
 - Ezzel pénzt és időt takaríthatunk meg
- A *release*-k előtti utolsó utáni pillanatbeli brutális *merge*-eket elkerüljük
 - > Mert rendszeresen használjuk a CI-t lépésről lépésre
- Jelzés ha elbukik egy teszt / bug keletkezik
 - > A rendszeres CI használat miatt könnyen visszaállhatunk egy kevés változtatással ezelőtti változatra
- A mindenkori legfrissebb stabil verzió állandó rendelkezésre állása
- A gyakori *push*-ok akaratlanul is kényszerítik a fejlesztőt az aprólékos fejlesztésre
 - > Nem hoznak létre túl nagy modulokat (nem is fér bele az időbe)
- A projekt mérőszámai generálhatóak a *CI*-ből
 - > Nem szükséges a menedzsernek grafikonokat rajzolni az előrehaladásról



Használatának költségei

- Ki kell mozdulnunk a komfortzónából
 - > Megszokott módszereinket háttérbe szorítani
 - Fokozatosan, hirtelen mindent lehetetlen
- Fegyelmezettség
 - > A csapat minden tagjának be kell tartania
- Üzemeltetés és konfigurálás
 - > Széles spektrumon mozog, a minimum költség ~0



Szerepe

- Felelősség levétele a fejlesztő válláról
 - > Több idő marad a fejlesztésre
- Monitorozhatóság biztosítása
 - > Több projekt állapotának párhuzamos követése
 - > A fejlesztés mélyebb (részletesebb) követhetősége
- Azonnali visszajelzés
 - > A fejlesztőnek
 - > A megadott csapattagoknak
 - > A főnökségnek
 - > A megrendelőnek



CI további szerepei

- Egy élesíthető megoldás bármely pillanatban
 - > A mindenkori legújabb *stabil* verzió
- A projekt fejlődésének követése, rögzítése
- Bug felderítés egy független gépen azonnal
- Mindezt úgy valósítja meg, hogy közben nem szól bele az alkalmazás életciklusába



CI gépi feladatai

- (összetett) *Build* folyamatok automatizálása
 - > Sorrendiség megadásával
- Egy „tisztá” készüléken futtatás
 - > A fejlesztő hibázhat (pl. nem minden kerül a verziókezelőbe)
 - > Lokális beállítások / erőforrások a fejlesztői gépen megfelelőek, azonban lehet, hogy a céleszközön kimaradt
 - > Biztosíték a működő szoftverre
- További funkciók csatolása
 - > Automatizált tesztek futtatása
 - > Kódelemzés

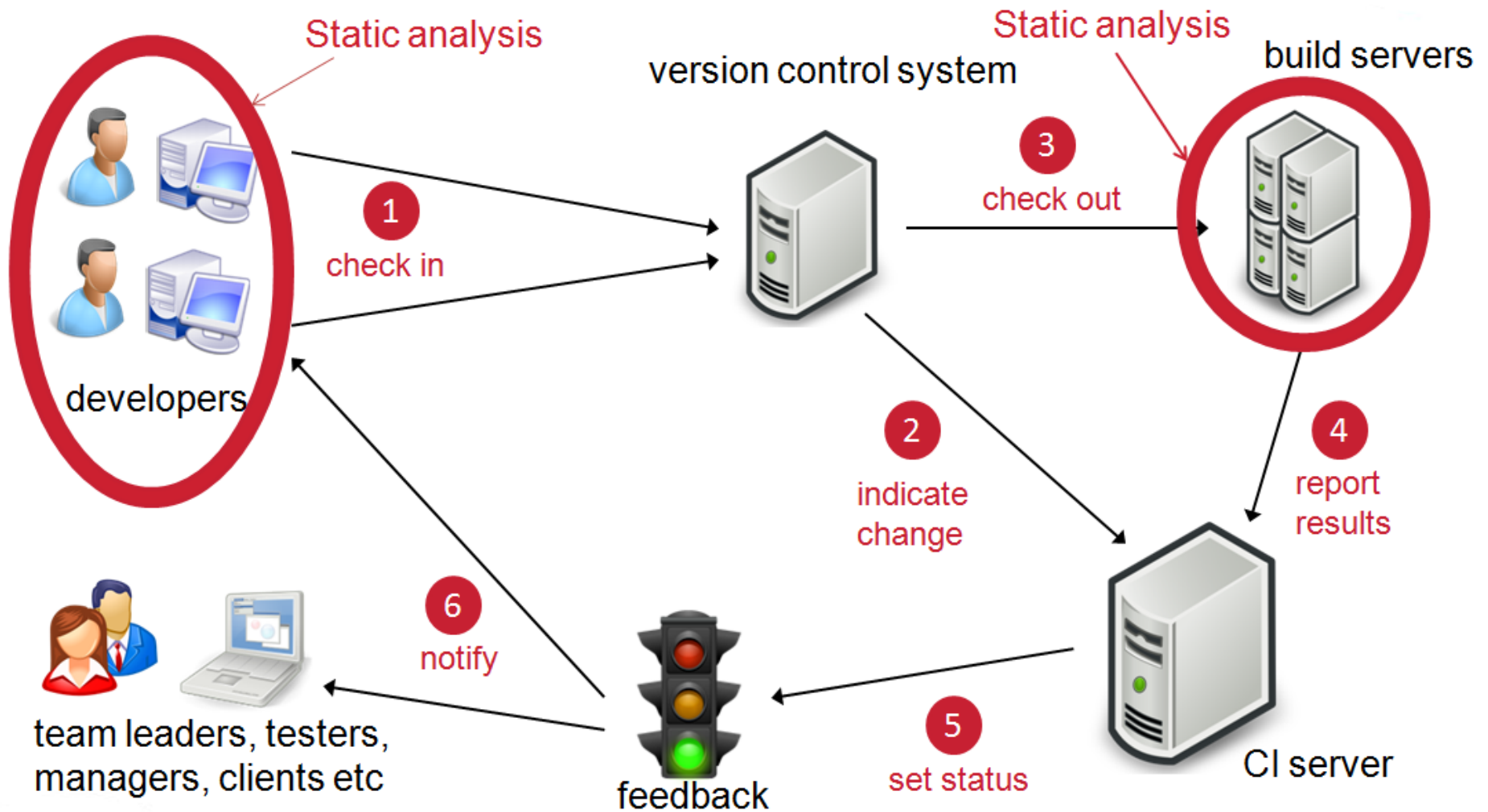


Continuous Integration lehetőségei

- Integráció verziókezelőkkel
 - > Értesülés *push*-ról
- Teszt riportok generálása
- *Push*-olás különféle *artifact repository*-kba
 - > Minden build kimentének automatikus mentése
- Élesítés (deployment) éles- és tesztkörnyezetekbe
- A projekten dolgozók értesítése
- Egyéb tool-okkal való integrálás
 - > Pl build rendszerek, mint ant, maven, gradle
- És sok egyéb



Egy tipikus CI rendszer



CI célja

Általánosan tehát a cél az, hogy a céges folyamatok, melyek emberi interakciót is igényelnek, sorrendiséget követelnek meg, egyetlen nagy, összefüggő, teljesen automatizált flow-vá váljanak.



A Continuous Integration eszközök



Szűkebb értelemben...

- *Continuous Integration* eszköz esetén jellemzően egy-egy *CI*-t megvalósító megoldásra asszociálhatunk
 - > Ez egy szoftver, ami integrál
 - > Ez egy szerver, ami elérhető
- Ez nem egy rossz megközelítés, de valóban csak ennyiből áll?
- Önmagában is megold mindent?



Tágabb értelemben...

- Természetesen nem
- Nem is lenne értelme
 - > Fontos, hogy könnyen illeszthető legyen
 - Jelenlegi folyamatainkhoz
 - Jelenlegi eszköztárunkhoz
- Ezért amikor egy CI rendszerről beszélünk, szokás minden komponensét beleértenünk
 - > Mivel egy futási lánc hajtódik végre, ez jogos is, hiszen bármelyik elem kimaradása a teljes futás hibáját okozhatja
 - Kis hibatűrés beállítható



Continuous Integration eszközei

- Verziókezelő
 - > Innen szedi ki a forráskódot a speciális CI eszköz
- Maga a *CI*-t megvalósító komponens (aki futtatja a build scripteket)
- Adatbázis
- Kódelemző
- Az *artifactory*
- Az alkalmazásszerver
- A ticket kezelő rendszer
- És az eszköz használói
 - > A fejlesztő, a menedzser
 - > A CI szakember (rendszergazda)
 - > *Stakeholder*-ek: vezetőség, megrendelők



CI választáskor összehasonlítási szempontok

Saját host-olás	SaaS megoldás
Testreszabhatóság, bővíthetőség	Egyszerűség, átláthatóság
Védett szoftver	Open Source szoftver
Continuous Integration a fókuszban	Continuous Delivery is fontos szempont



CI választás szempontjai

- Elterjedtség
 - > Közösség és fejlesztői támogatás
- Kompatibilitás
 - > Támogatott nyelvek
 - > Támogatott platformok
- Árazás, licenszelés
 - > A hostolás is költséges
- *Repository*-k támogatása
- Értesítési módok

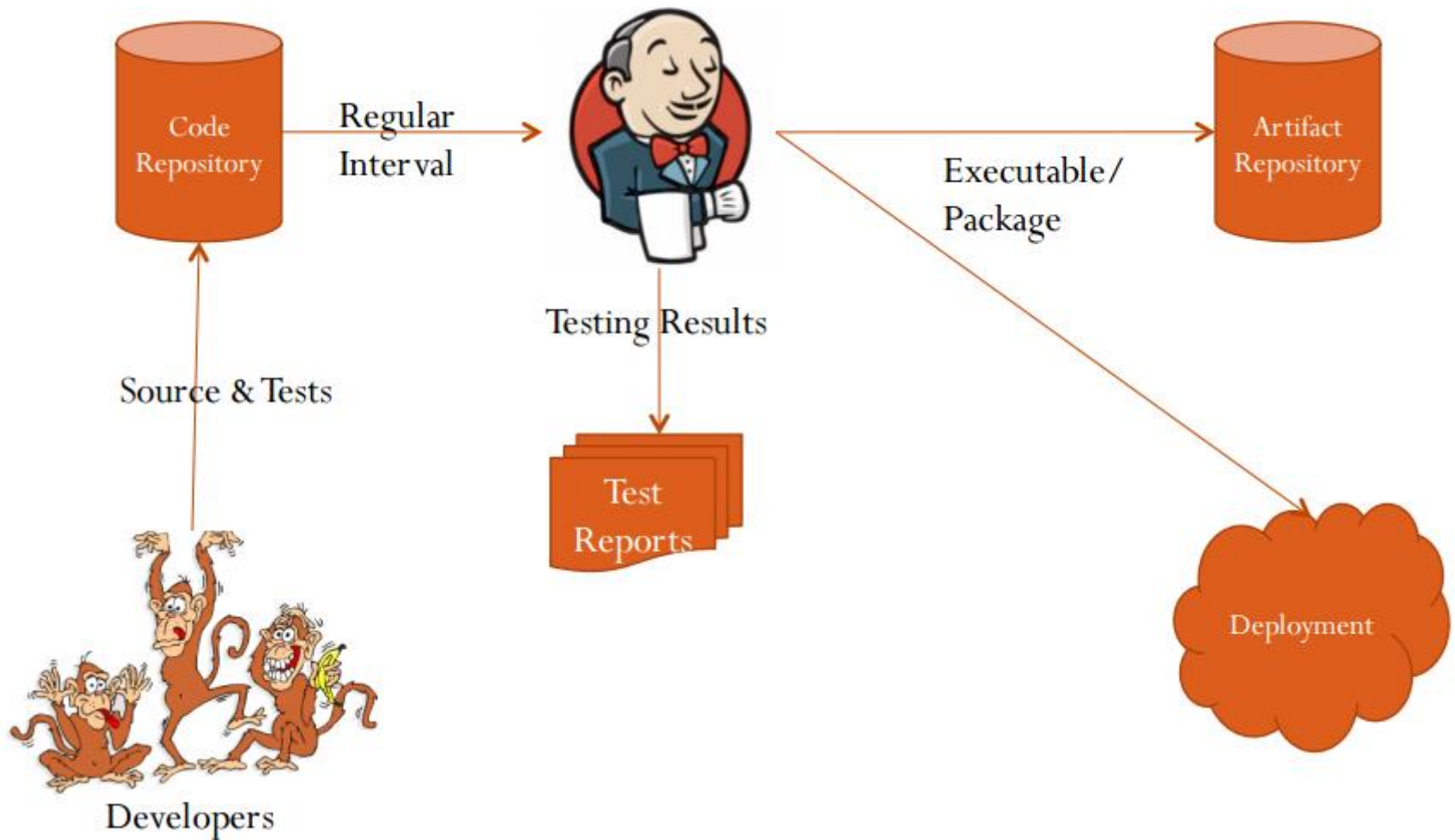


Népszerűbb CI eszközök

- *Jenkins*
- *CodeShip (Cloud alapú)*
- *(Hudson)*
- *Travis CI (Cloud alapú)*
- *TeamCity*
- *Circle CI*



CI eszköz működése



Jenkins



Jellemzői

- Egyik legelterjedtebb megoldás
 - > Ingyenes
 - > Elérhető *számos platformra*
 - > *Minden ismertebb repository támogatása*
- Egyszerű telepítés
 - > *Plug 'n play* jellegű
 - Elérhető platformspecifikus telepítők és WAR formájában is
 - > *Standalone Tomcat* szervert magától telepít (*default port.* 8080)
- Gazdag funkcionalitás, testreszabhatóság
- REST interface a vezérléshez
- Bővíthetőség
 - > 1000+ *plugin*
 - > Ebből sok integrációt biztosít más (pl. *ticket* kezelő *JIRA*) rendszerekhez



Hozzáférés szabályozás

- A hozzáférés felhasználói fiókhoz kötött
 - > Saját beépített megoldás
 - > Delegálható *servlet* konténerhez is
 - > *LDAP*
 - > Számos egyéb megoldás plugin-okkal
- Felhasználónként állítható jogosultságok
 - > Projektenként (adott projektre jellemző)
 - > Műveletenként (általánosan jellemző)



Projektek kezelése

- Eltérő tulajdonságokkal rendelkeznek
- Eltérő konfiguráció
- Eltérő technológia
- Más célszerver
- ...
- Ezek miatt fontos, hogy elszigetelve legyenek a projektek
 - > Egy projektet érdemes egy CI eszközön használni csak
- Azonban nem megkötés, hogy egy cég csak egy(féle) CI eszközt használjon
 - > Nagy cégeknél lehetetlen is, túlterhelt lenne



Item

- Önálló és szeparált egység
- A CI szerver kifejezetten alkalmas sok projekt együttes kezelésére
 - > A projektek egyesével konfigurálhatóak
- A *Jenkins*-ben a projektek kezelőit *Item*-eknek nevezzük
 - > Projektenként egy *Item*-et hozunk létre jellemzően
 - Vannak kivételek
- Az egyes *Item*-ek tartalmazzák a projektre jellemző beállításokat



Item jellemzői

- Új *item* létrehozásánál választhatunk, hogy milyen típusú projektet szeretnénk létrehozni, ezzel gyorsítva a konfigurációt
 - > *Maven project*
 - *Maven 2, 3* projektek esetén
 - > *Freestyle project*
 - Bármilyen verziókezelő bármilyen *build* rendszerrel konfigurálható, akkor ajánlott használni, ha a többi opció nem megfelelő (több konfiguráció)
 - > *External Job*
 - Érdemes használni, ha külső automatizálási rendszerrel rendelkezünk és ez a rendszer végzi az automatizálás feladatát. A *Jenkins* ekkor *dashboard* jellegű funkciókat lát el.
 - > *Multiconfiguration Job*
 - Összetett, több komponensből (például különböző konfigurációval rendelkező tesztszerverek) álló, esetleg több technológiát támogató, több *build* rendszerre épülő megoldásoknál ajánlott.
 - > Meglévő projekt klónozása
 - Létező és vagy megegyező, vagy nagyon hasonló beállítások esetén



Item funkciói

- Figyeli a projekthez tartozó *repository*-t (adott *branch* – *master* – javasolt)
 - > *Git*
 - > *Svn*
 - > *Mercurial*
 - > ...
- *Build*
 - > Kódelemzés (opcionális)
 - > Tesztek futtatása és kiértékelése (opcionális)
 - > Eredménytermék archiválása (opcionális)
- *Post-build akciók*
 - > Pl. APK eltárolása, e-mail küldés, stb.
- *Deploy (opcionális)*
- *Feedback (opcionális)*



Új Item beállításai (részlet)

Build

Invoke Gradle script

- ☐ Invoke Gradle
- ☒ Use Gradle Wrapper
- Make gradlew executable ☒

From Root Build Script Dir ☒

Build step description

The build command itself

Switches

Tasks

clean build

Root Build script

\${workspace}

Build File

Specify Gradle build file to run. Also, [some environment variables are available to the build script](#)

Force GRADLE_USER_HOME to use workspace ☐



Törölés

Add build step ▼

Post-build Actions

Archive the artifacts

Files to archive



Haladó...

Törölés

Add post-build action ▼

Save

Alkalmaz



Scriptelhetőség

- A folyamat testreszabásához széleskörű eszköztárat kínál a *Jenkins*
- A folyamat részeként futtathatóak *scriptek* is
 - > *Build* előtt és/vagy után
 - > *Batch, Shell*
 - > A rendszer számára elérhető (pl. parancssorból futtatható) erőforrások hozzáférhetőek
 - > Példák
 - fájlok másolására
 - fájlok módosítására
- *Pluginek* segítségével szinte a végtelenség tárháza válik elérhetővé



Artifactok kezelése

- A *build* folyamat eredménytermékei
- Felhasználási lehetőségek
 - > Feltölthetjük (saját) *artifactory*nkba (pl. *Maven* projektnél)
 - > Felhasználhatjuk más projektünkben függőségként
 - > Feltölthetjük alkalmazásszerverre
 - *WAR*-t telepítünk *Tomcat* szerverre
 - *APK*-t menthetünk el vagy telepíthetünk
 - Stb.



Más item buildjének indítása

- Tegyük fel, hogy másik projektünk használja függőségként egy *artifact*unk
 - > Egyszerűség kedvéért lokális *JAR*-ként tartalmazva azt
- Az *artifact* buildelésénél szeretnénk újrafordítani automatikusan a másik *item*ünk, de már az új *artifact*tal
- FONTOS!!!
 - > Ha valamilyen meglévő *interface*-t változtatunk / megszüntetünk, az hibához vezet!
 - > Akkor használjuk, ha *bug*ot javítottunk például



Saját monitoring dashboard



Mire jutottunk eddig?

- Tudjuk fordítani projektjeink egy független gépen
 - > Ezzel elértük, hogy minden beállítás és erőforrás rendelkezésre áll (pl. *properties* fájlok, adatbázis, *maven* függőségek)
 - > Kivédjük vele a *cache*-elés okozta inkonzisztenciákat is
- Automatikus kódelemzés történik a forráskódon
 - > Ezzel is javítva a kódminőséget, illetve betartatva a kódolási konvenciókat
- Ezeket a folyamatokat sorosítva, egy flow-ban végezzük



Probléma

- Nem szeretnénk a projektek eredményeit külön nézeteken követni, ha egyen is követhető
 - > Például a kódminőséget
 - > A fordítás sikerességét
 - > Teljesítménytesztek eredményét
- Jellemzően egy cég nem csak egy projektet visz
 - > Több projekt párhuzamos követése a cél
 - Szükséges a vezetőknek, akik több projektért is felelnek
 - Motiválhatja a projektcsapatokat, ha látják a többiek előrehaladását is

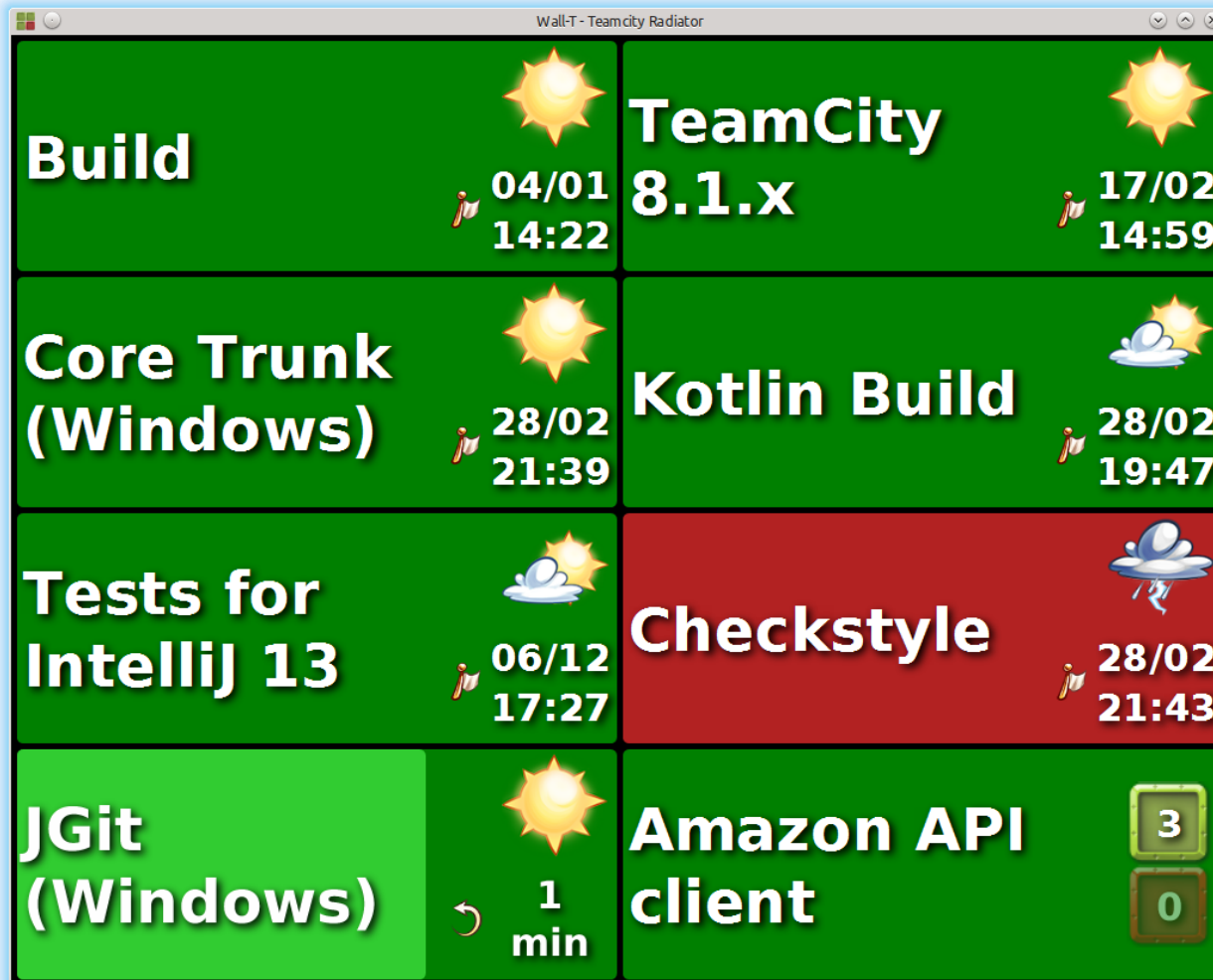


Megoldás

- Saját monitoring *dashboard* készítése
 - > Használható a *Jenkins* beépített megoldása
 - Saját nézetek projektre szűréssel, megjelenítendő adatok választásával is létrehozhatóak
 - > Az imént említett problémákra megoldás
 - > *Jenkins*ből *plugin*ek segítségével támogatott
 - > Külön rendszerek is alkalmasak rá
 - A háttérben *REST* hívásokkal, melyeket a *plugin* elfed
 - > Ma már számos kész megoldás közül választhatunk



Konkrét példa



Travis CI





- Online megoldás
 - > Telepítés nem szükséges saját szerverre
- Ingyenes verzió
 - > Megkötések betartásával
- *Repositoryk* támogatása
 - > Csak GitHub
- Integrálható a népszerű szerverekre
 - > Pl. *Heroku*
- Nyelvek támogatása
 - > Az elterjedt nyelvek támogatása (25 féle)
- Testreszabhatóság
 - > Egy speciális *.travis.yml* fájlba leírható
 - > Erősen korlátos lehetőségek



Travis CI helyi konfigurálása

- A projekt jellemzőit tartalmazó fájlt hozzuk létre:
 - > **.travis.yml*, ahol *** a projekt neve *sneak-case* formában
 - > Ez a fájl leírja
 - A projekt típusát
 - Verzióját
 - Itt specifikálhatjuk a build előtti és utáni feladatokat
 - Esetleges CD beállításokat



Travis CI konkrét YAML példa (Android)

```
language: android
android:
  components:
    - build-tools-23.0.1
    - android-23
    - android-21
    - add-on
    - extra
    - platform-tools
    - tools
    - extra-google-googleplayservices
    - extra-google-m2repository
    - extra-android-m2repository
    - addon-google_apis-google-19
    - sys-img-armeabi-v7a-android-21

deploy:
  provider: releases
  apikey: $GITHUB_KEY
  file: app/build/outputs/apk/app-release.apk
  skip_cleanup: true
  on:
    tags: true

notifications:
  email:
    recipients:
      - myemail@address.com
    onsuccess: always
    onfailure: always
```



Continuous Delivery

Hol tartunk?

- Tudjuk fordítani projektjeink egy független gépen
 - > Ezzel elértük, hogy minden beállítás és erőforrás működik (kivédjük vele a cache-elés okozta hibalehetőségeket)
- Automatikus kódelemzés történik a forráskódon
 - > Ezzel is javítva a kódminőséget, illetve betartatva a kódolási konvenciókat
- Tudjuk monitorozni egy elegáns, letisztult nézeten a projektjeink előrehaladást
- Ezeket a folyamatokat sorosítva, egy flow-ban végezzük

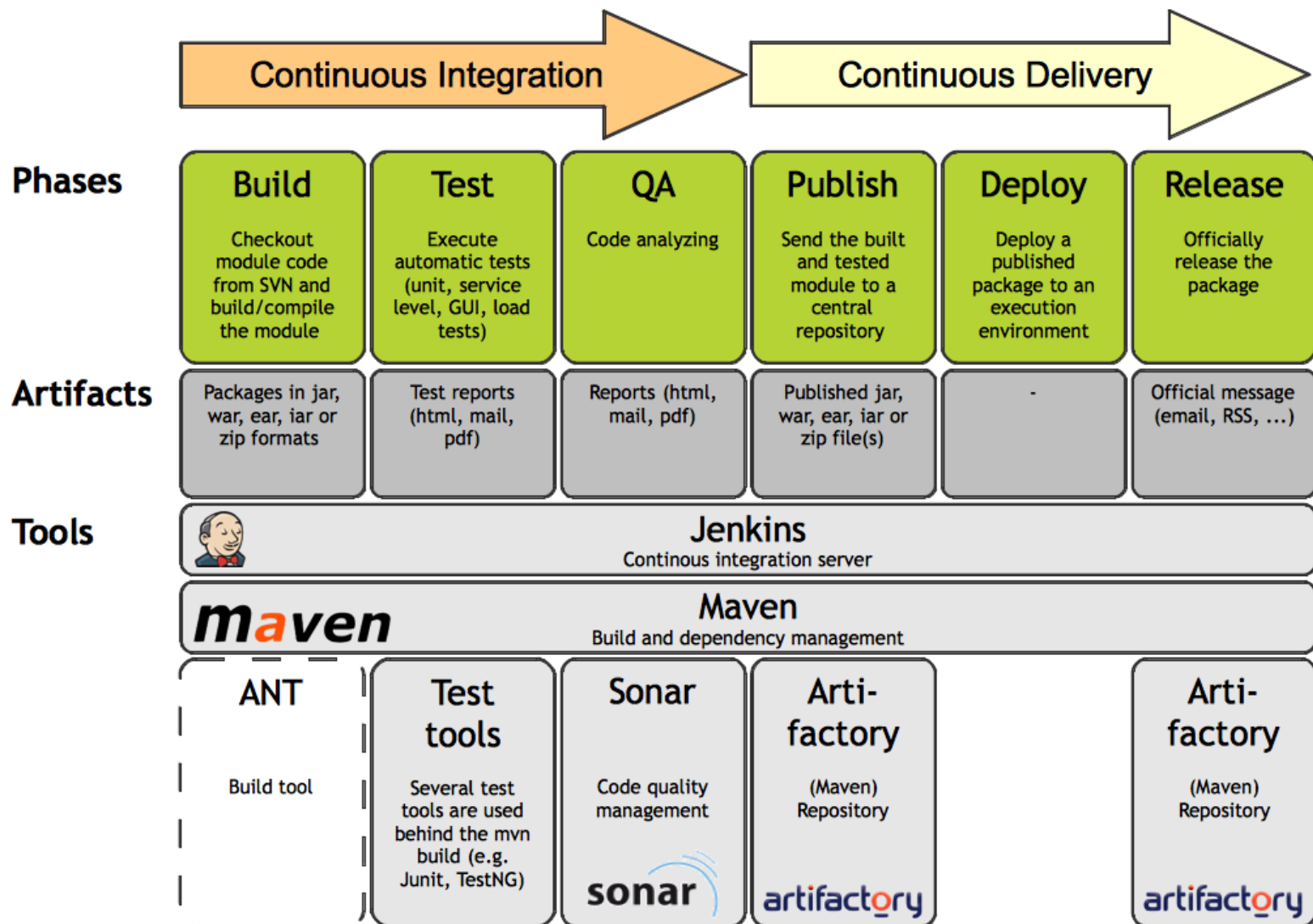


Mit érhetünk még el?

- A *flow (chain)* korábbi pontjainak stabil kimenetét szeretnénk felhasználni
 - > Korábban: a fejlesztő lefordította a saját gépén a kódot, felmásolta egy szerverre, esetleg újraindította, manuálisan tesztelte
 - > Ha nem volt tesztrendszer, az itt felmerült hibák az éles rendszer ideiglenes leállítását okozták
- Ezen is szeretnénk javítani!



CI+CD, a teljes folyamat



Mikor használjuk a teljes folyamatot?

- **Teszt szervereken nyugodtan**
 - > Hiszen ez egy speciális tesztelő kör számára hozzáférhető
 - > Ha bugos, akkor se veszít üzleti presztízséből a cég
- **Az éles szerveren / kliensen ne**
 - > Új verzióról értesíteni kell a felhasználókat, amint összeszedetten célszerű
 - Napi több frissítés már zavaró
 - Túl sok változást nehéz feldolgozni
 - > Egy kiadás jellemzően néhány hetes fejlesztési funkciókat tartalmaz
 - Amit már teszteltek a tesztelők és a megrendelők
 - > Ezt mindig kézzel csináljuk és alaposan ellenőrizzük, amint lehetséges



Több szerver kezelése

- Nap közben a *frontend*esek kértek az új részhez egy változtatást
 - > A backendes megcsinálja, majd *pushol* egy local nevű *branch*re
 - > A *Jenkins* érzékeli, majd futtatja a teszteket, stb.
 - > Amennyiben minden sikeres, telepíti is a saját teszt szerverünkre
- Miért nem probléma ez?
 - > Ha bármi tönkremegy, ez a sajátunk, nem egy éles rendszer, javítjuk
 - > Ráadásul ez tényleg gyorsan kell, akár óránként többször is, ügyfél úgysem látja



Több szerver kezelése

- Érdeemes még egy *remote test* nevű *branch*-et is készítenünk
 - > Ennek a célja, hogy itt kezeljük az ügyfél teszt szerverét
 - > A *Jenkins*-ünk (vagy az ügyfélé) figyeli ezt a *branch*-et
 - > *Push* után, ha minden sikeres, kiteszi oda is
- Miért nem probléma ez?
 - > Továbbra sem éles rendszer (nem számít a felhasználóknak, ha történik valami rossz)
- Miért hasznos?
 - > Az ügyfél kér egy gyors változtatást, mert eszébe jutott, hogy valami jobban működhet és ki szeretné próbálni
 - > Amennyiben nem biztosít ilyen teszt környezetet, jobb hiányában tesztelheti a mi teszt szerverünkön is
 - De tudjuk, hogy ez nem erre a célra van, gyakori változtatásokkal, nem vállaljuk ezért a felelősséget!



Források

- <https://www.jfrog.com/confluence/display/RTF/Build+Integration>
- <http://www.quora.com/What-is-the-best-continuous-integration-deployment-server>
- <https://jenkins-ci.org/>



Köszönöm a figyelmet!